Kubernetes networking

05

1. Cluster networking

2. Internal traffic

3. External traffic

1. Cluster networking
2. Internal traffic
3. External traffic

1. Cluster networking
2. Internal traffic
3. **External traffic**

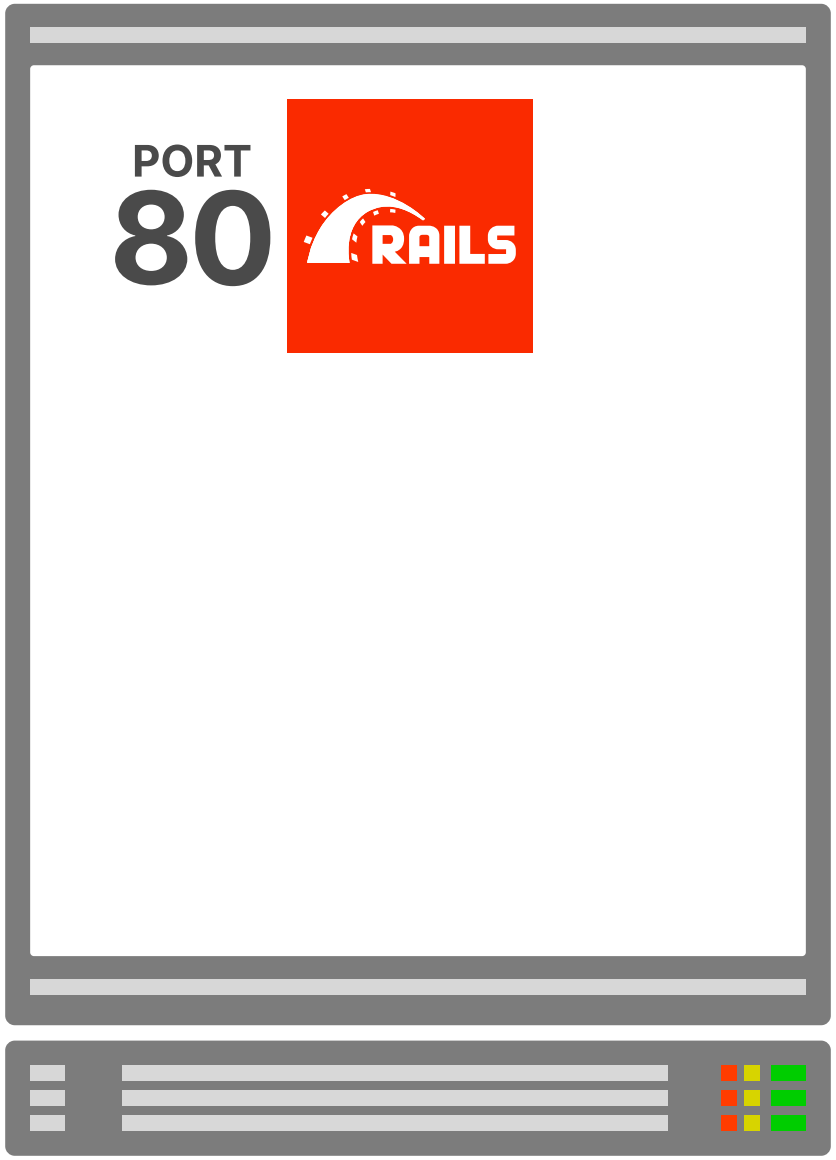
Kubernetes networking

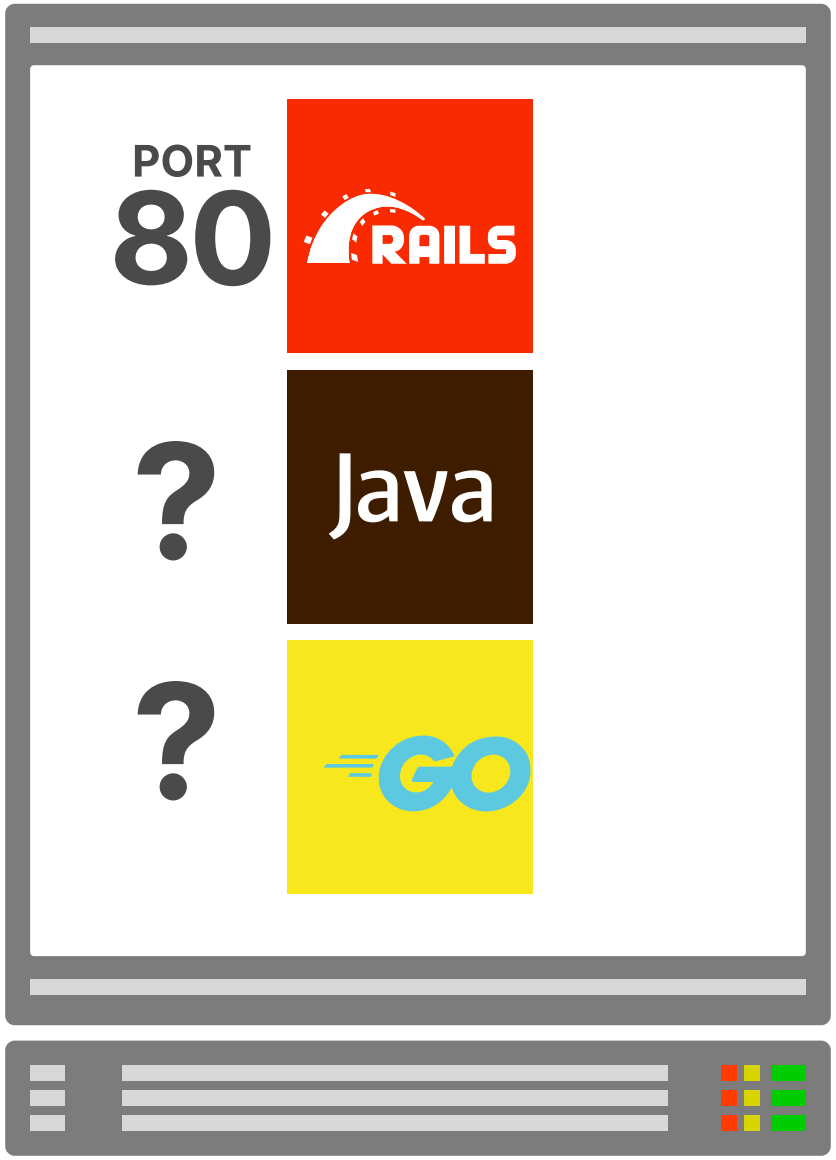
1. Cluster networking 
2. Internal traffic 
3. External traffic

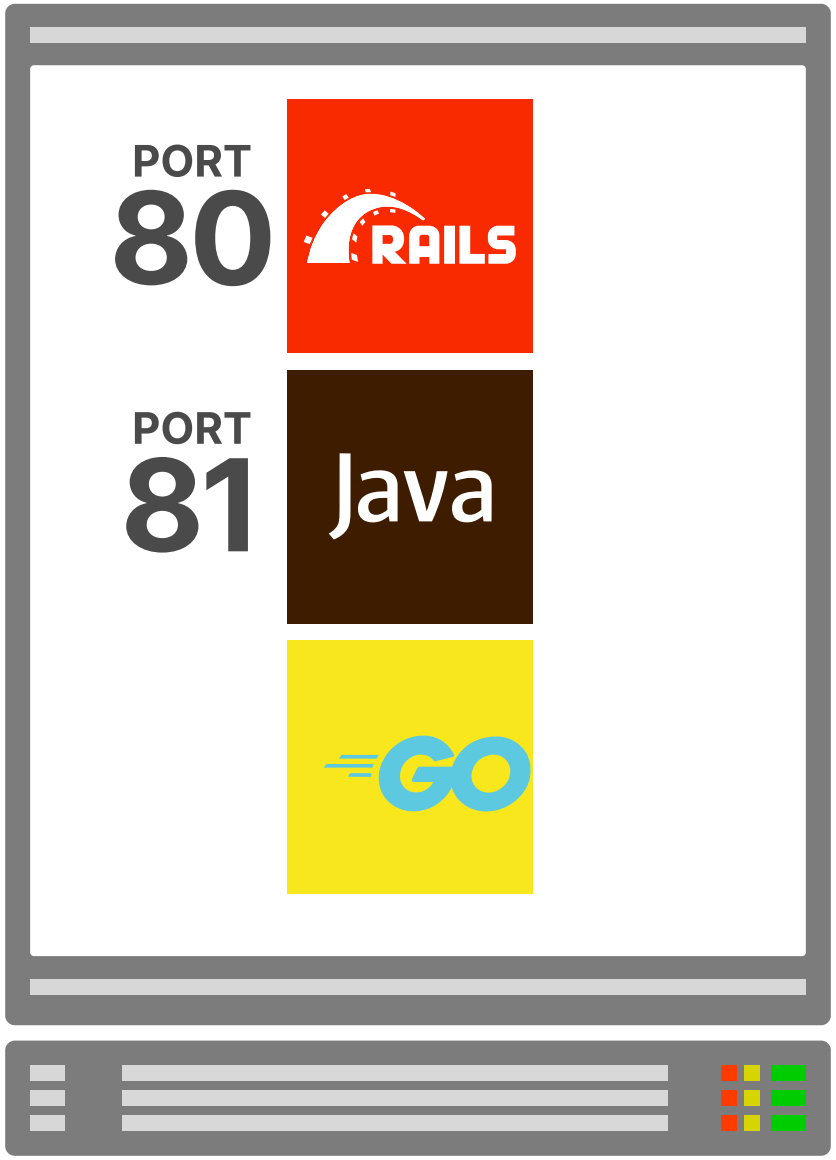
Cluster networking

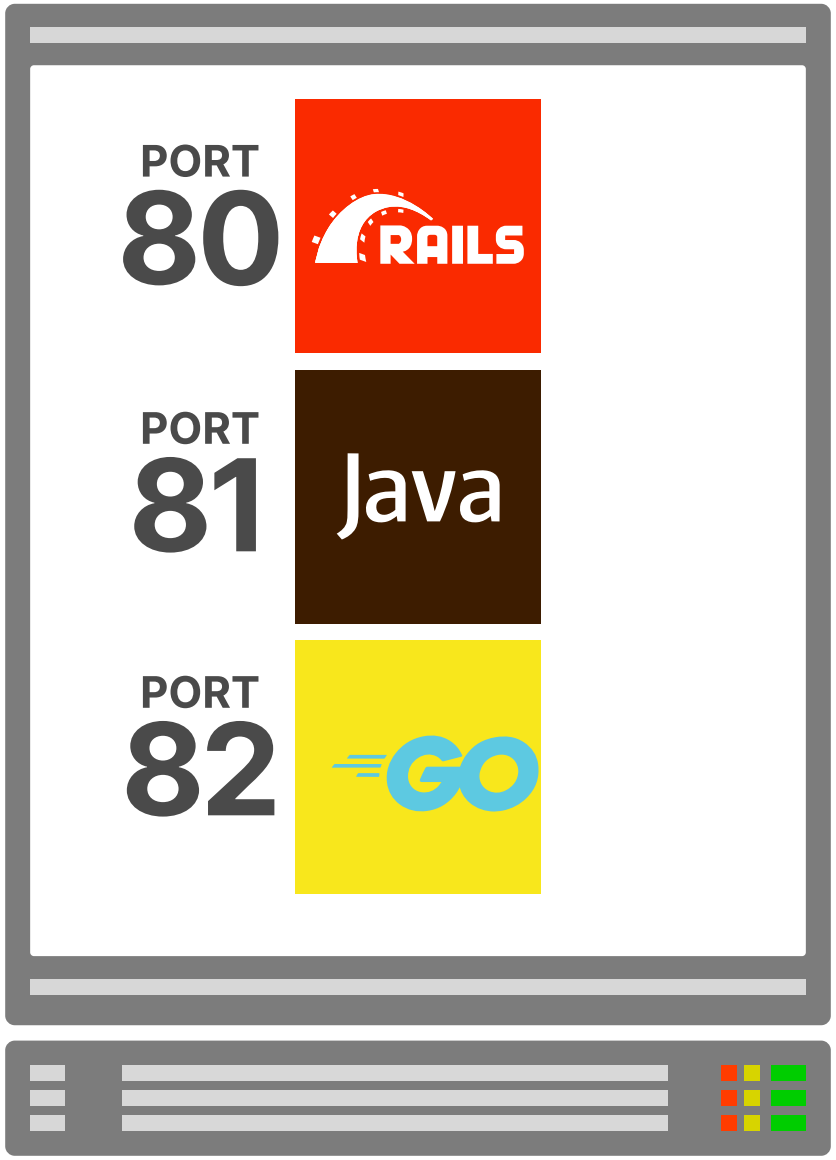
01

Running multiple webservers

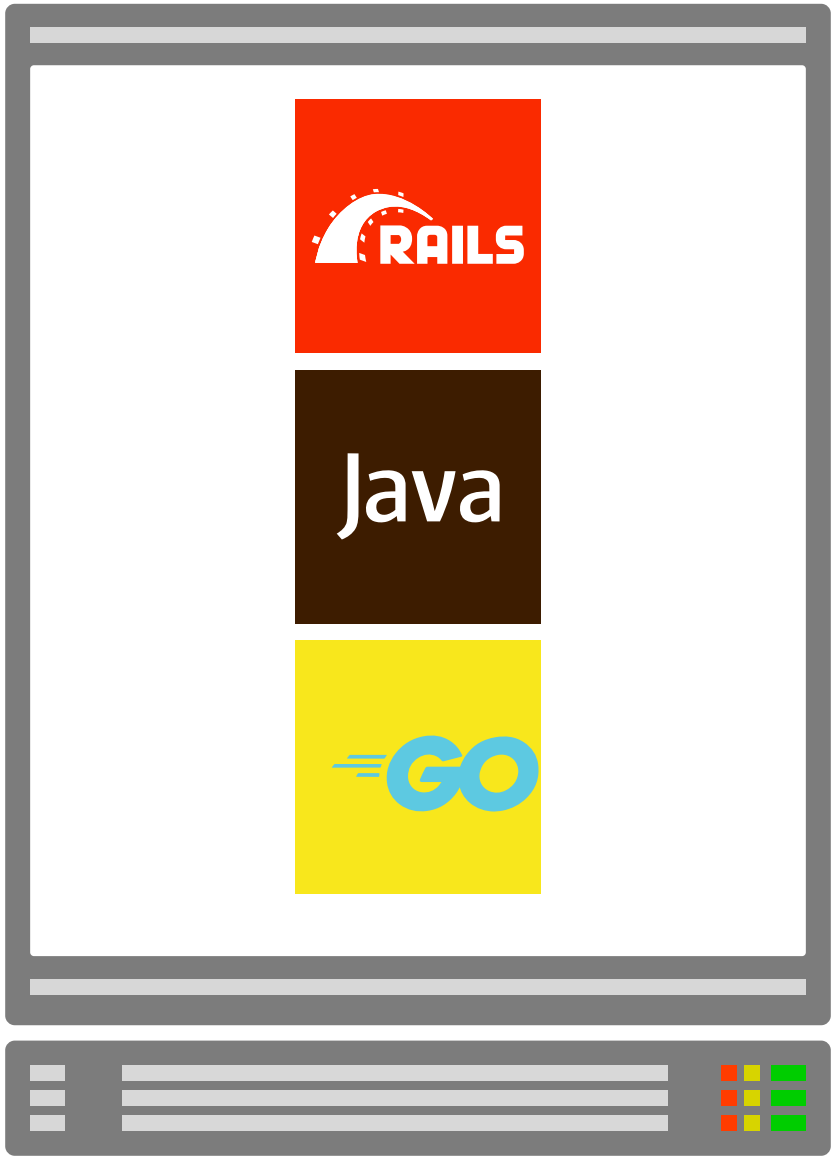

PORT
80
RAILS


PORT
80
RAILS
?
Java
?
GO

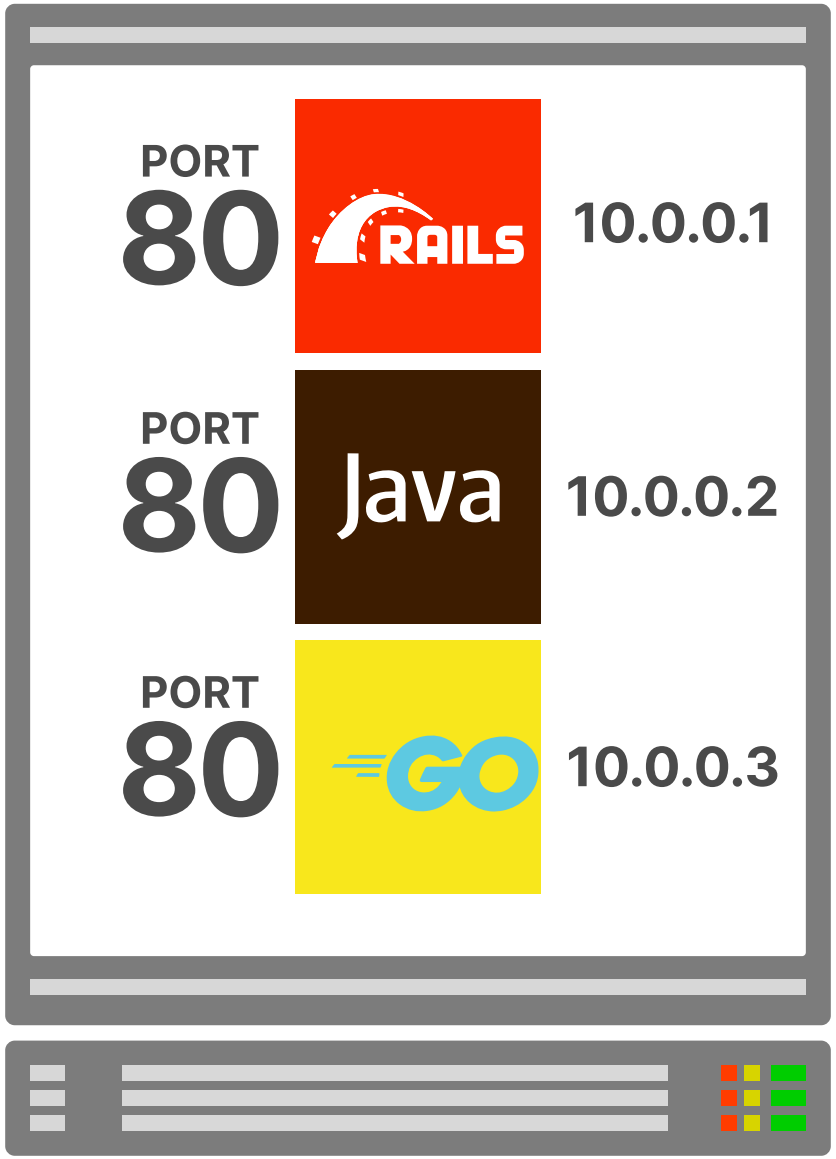

PORT
80
RAILS
PORT
81
Java
GO



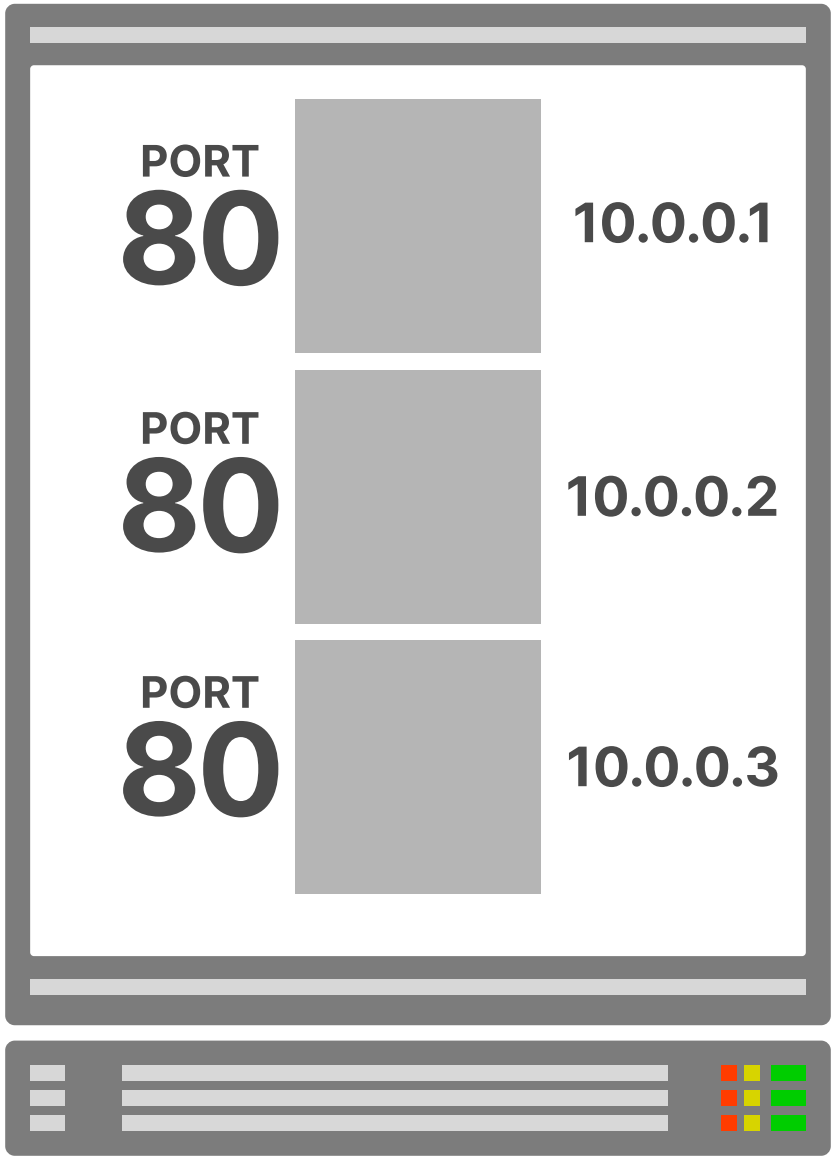
192.168.0.3



192.168.0.3

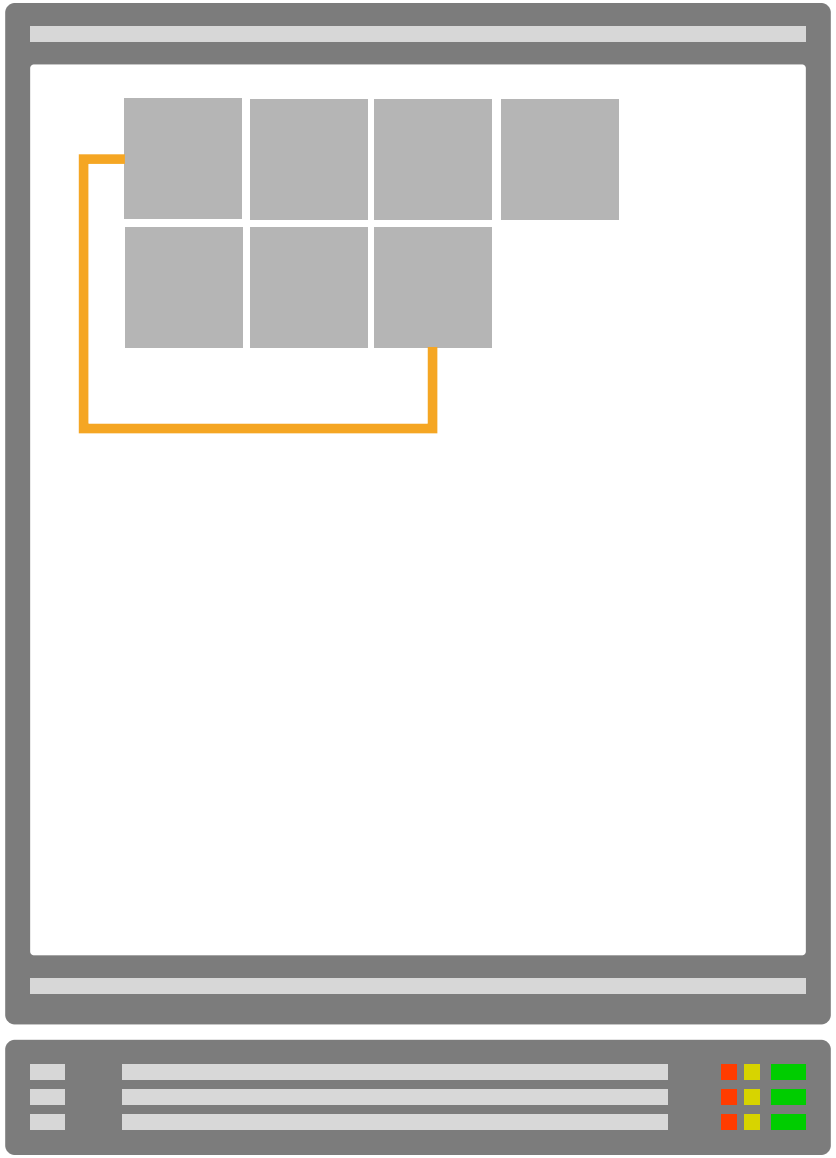


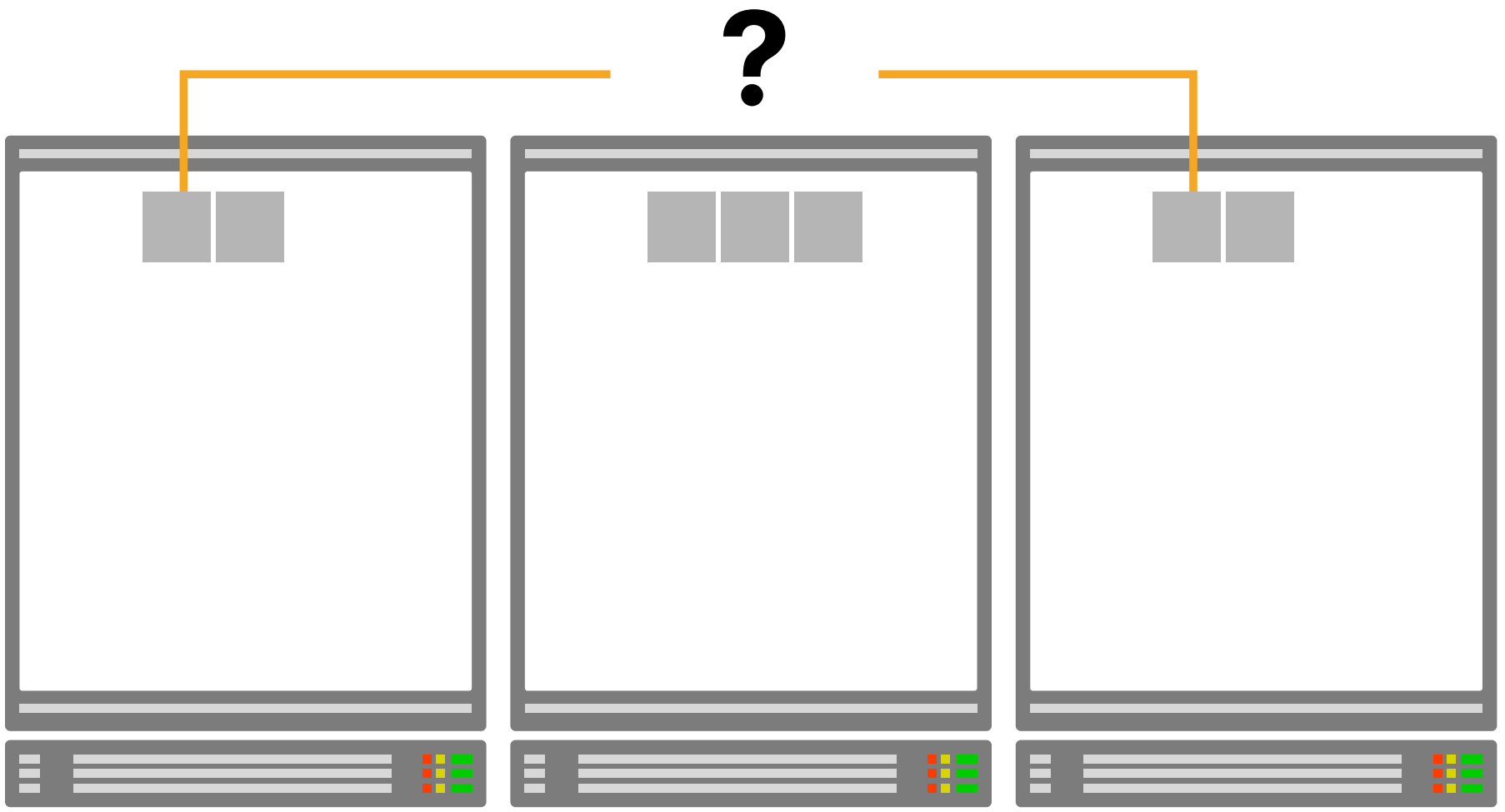
192.168.0.3



192.168.0.3

Networking challenge



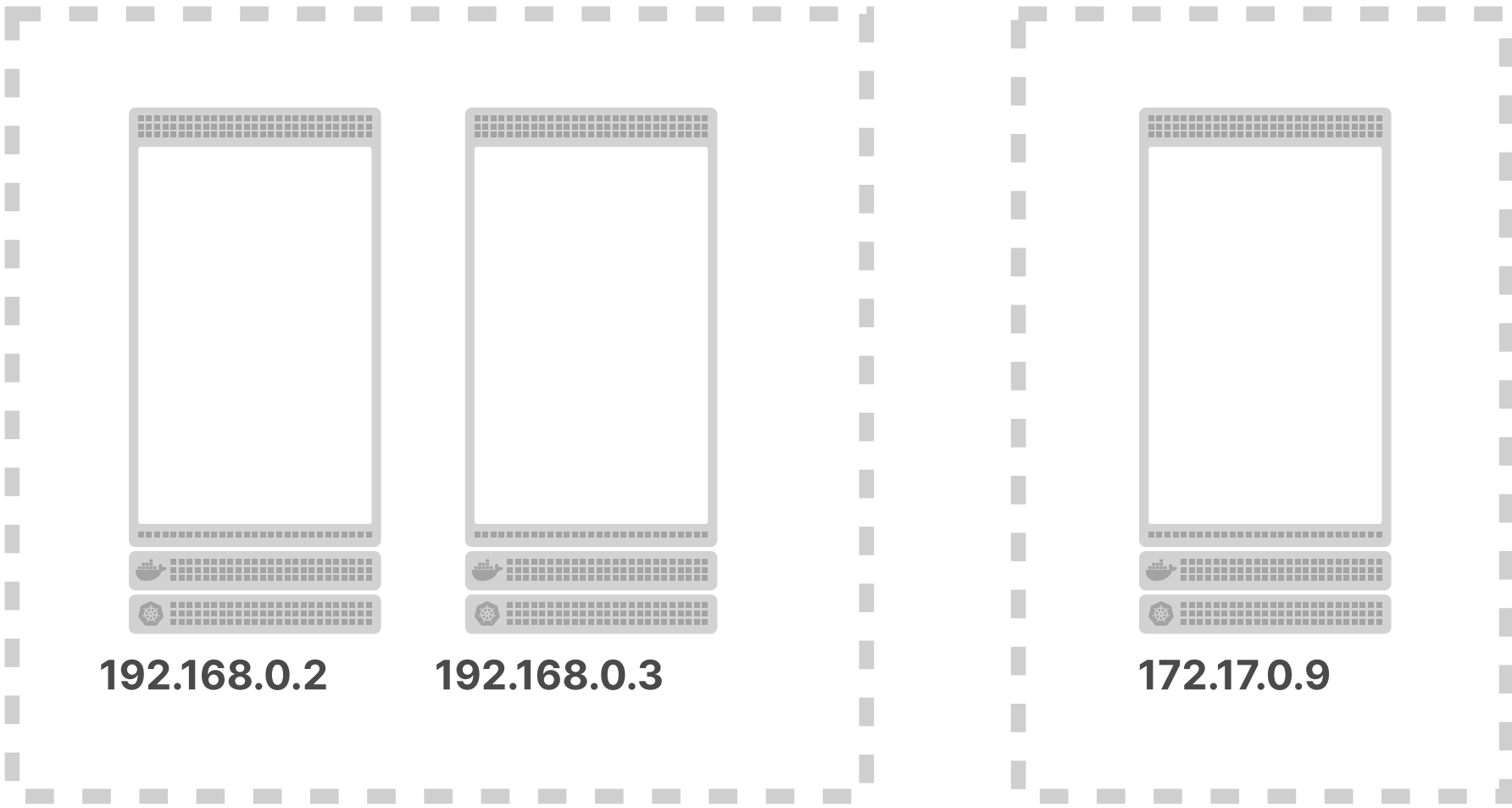


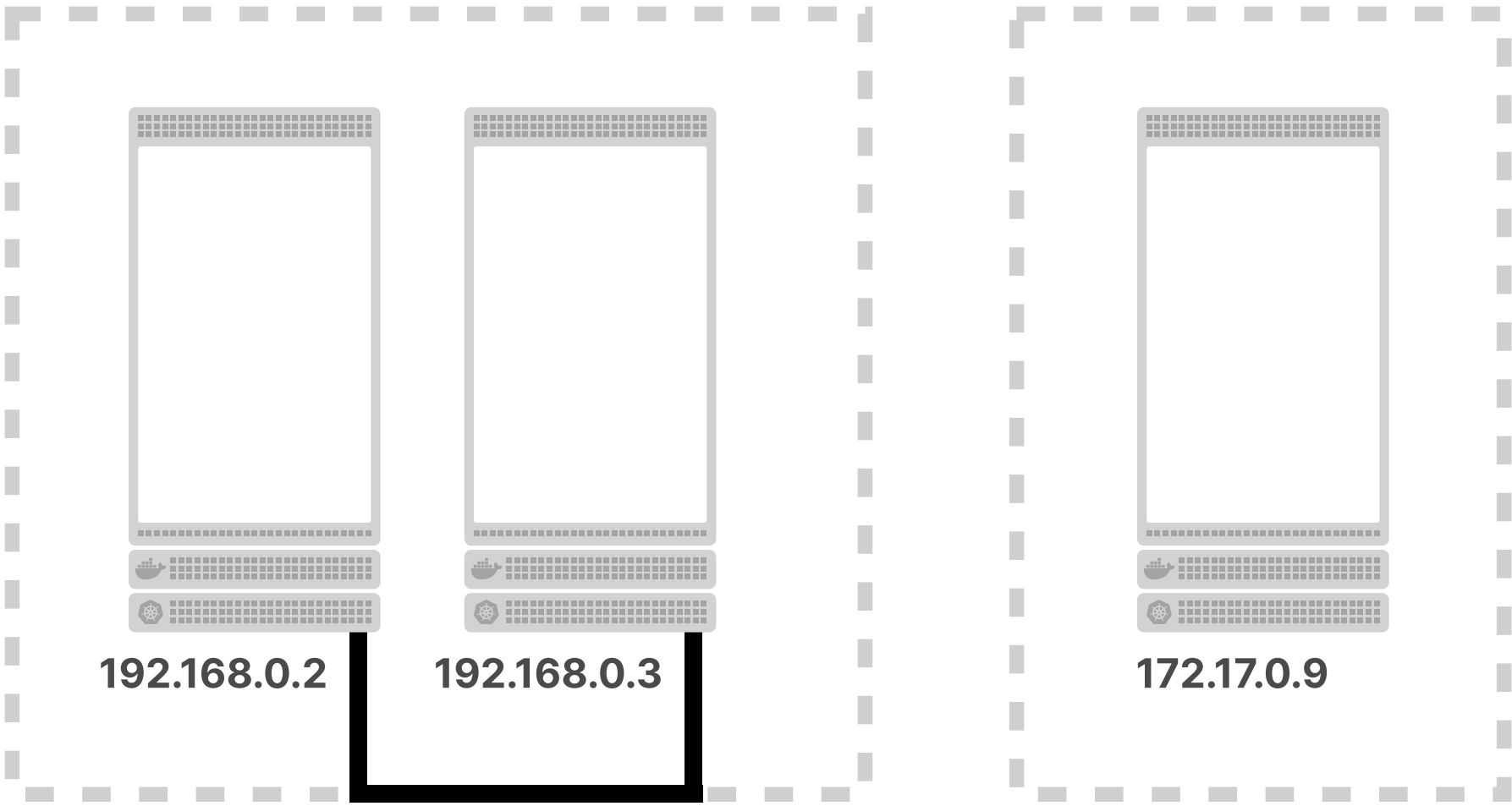
Networking recap

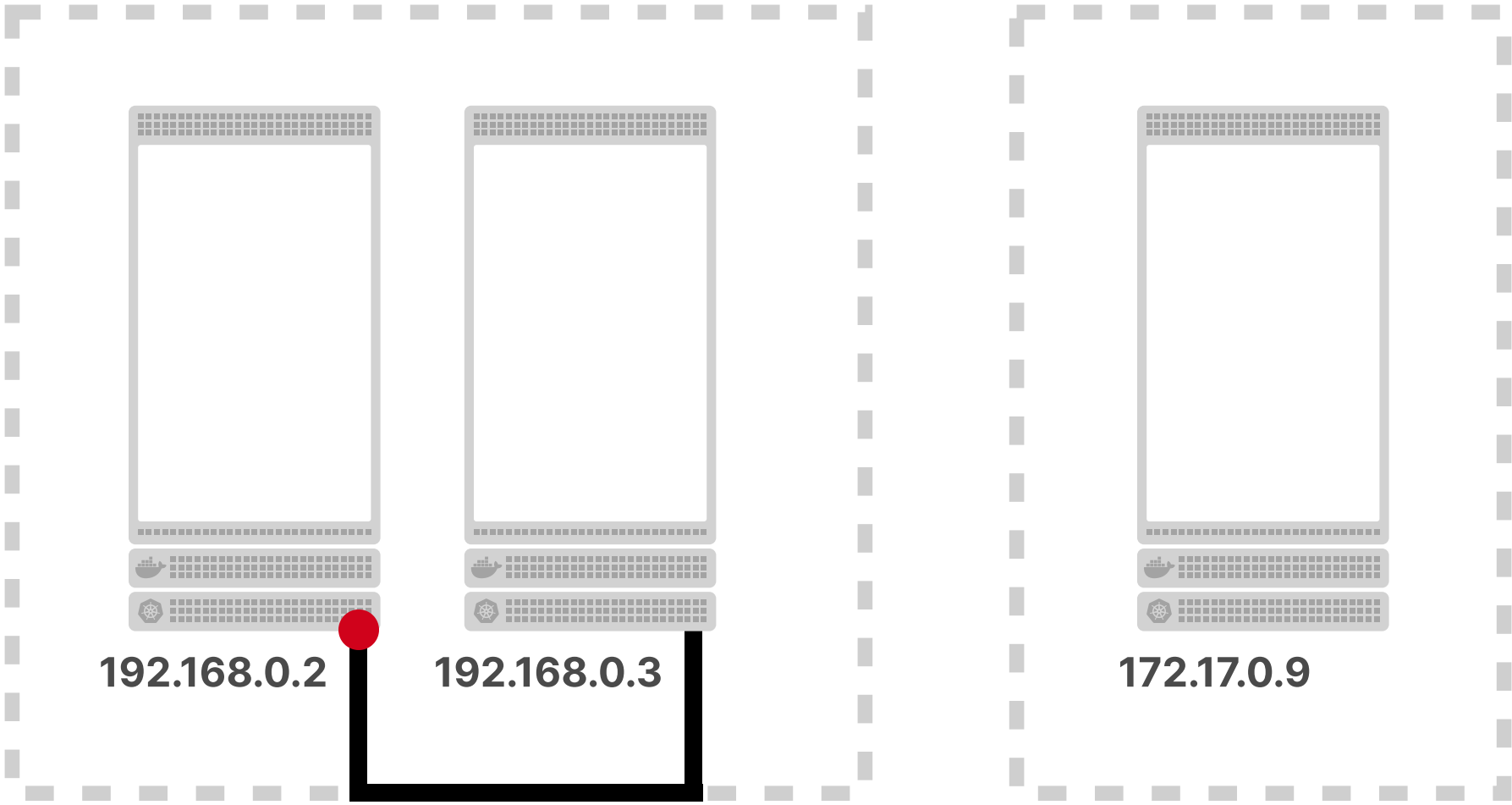
When the IP address is in the local network, send the message to receiver

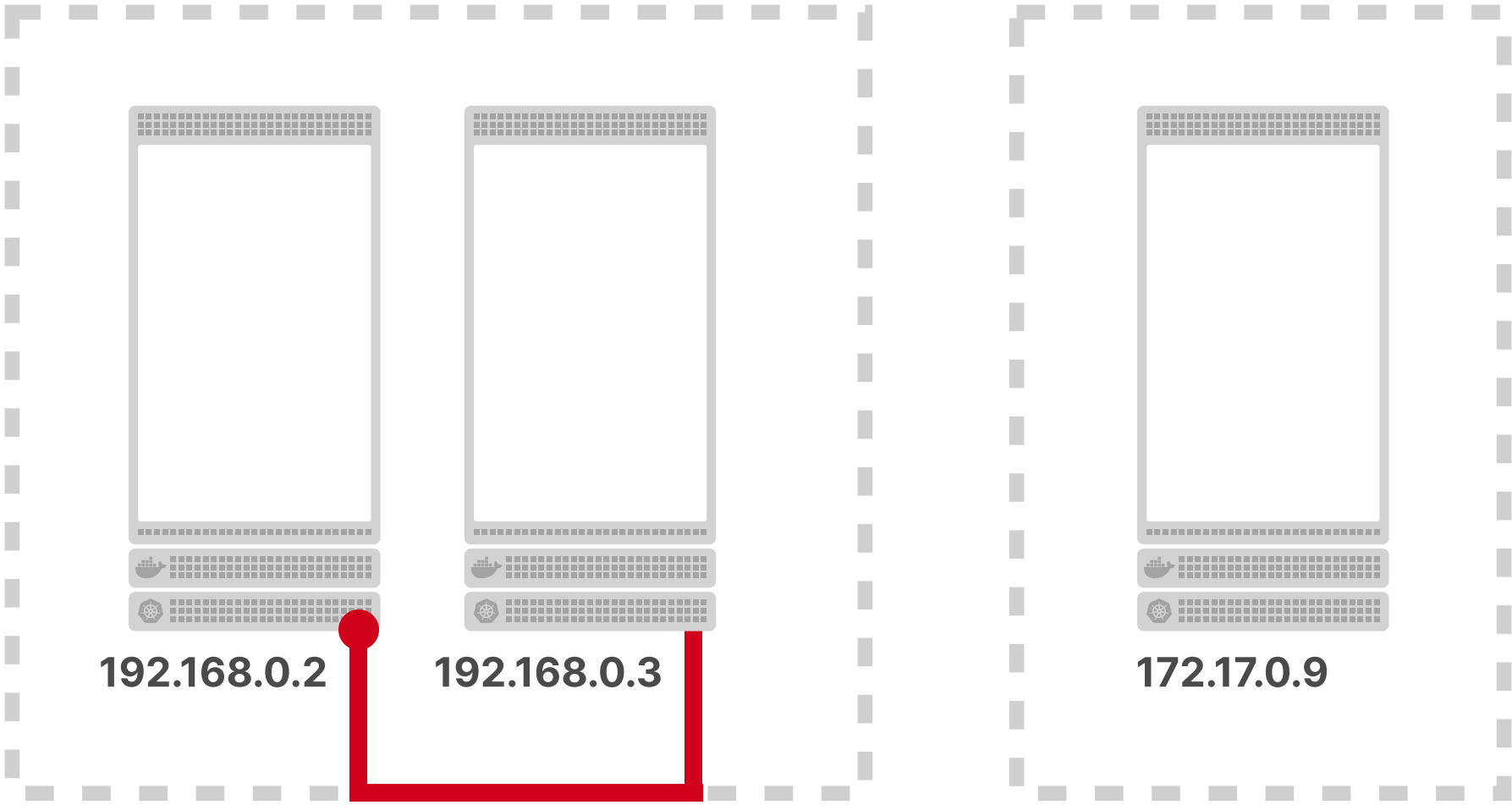
---

Networking recap


192.168.0.2
192.168.0.3
172.17.0.9


192.168.0.2
192.168.0.3
172.17.0.9

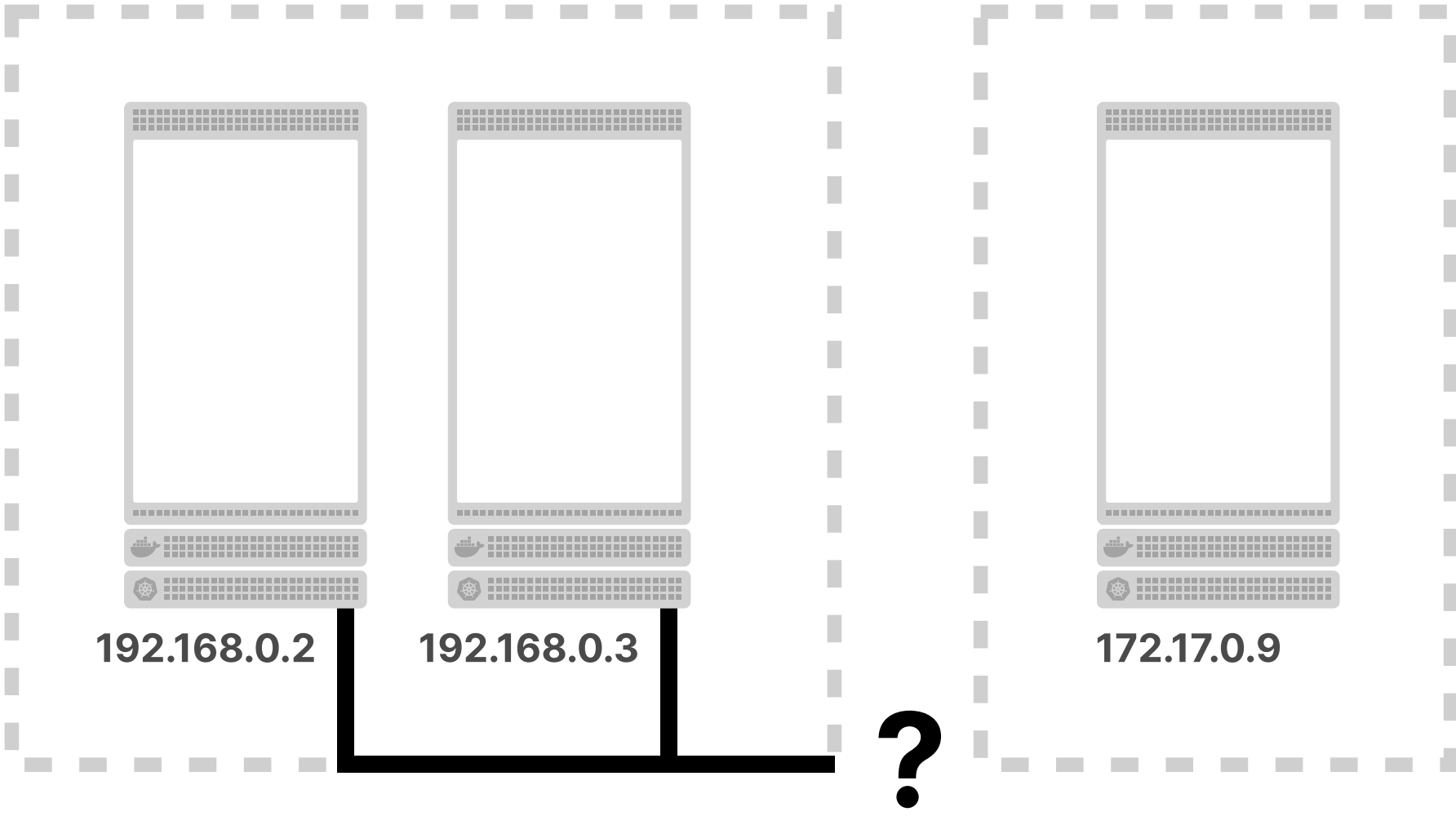

192.168.0.2
192.168.0.3
172.17.0.9

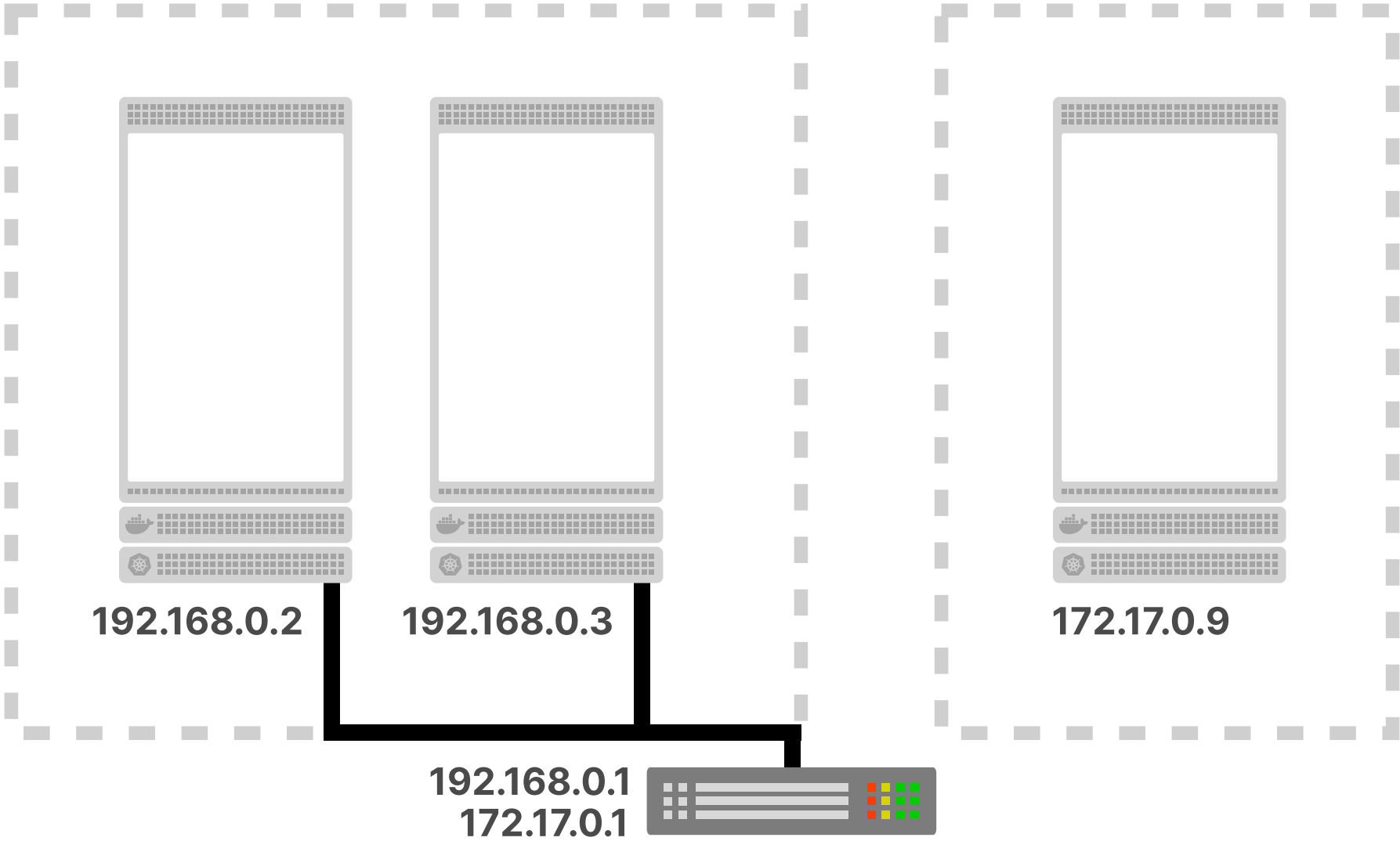

192.168.0.2
192.168.0.3
172.17.0.9

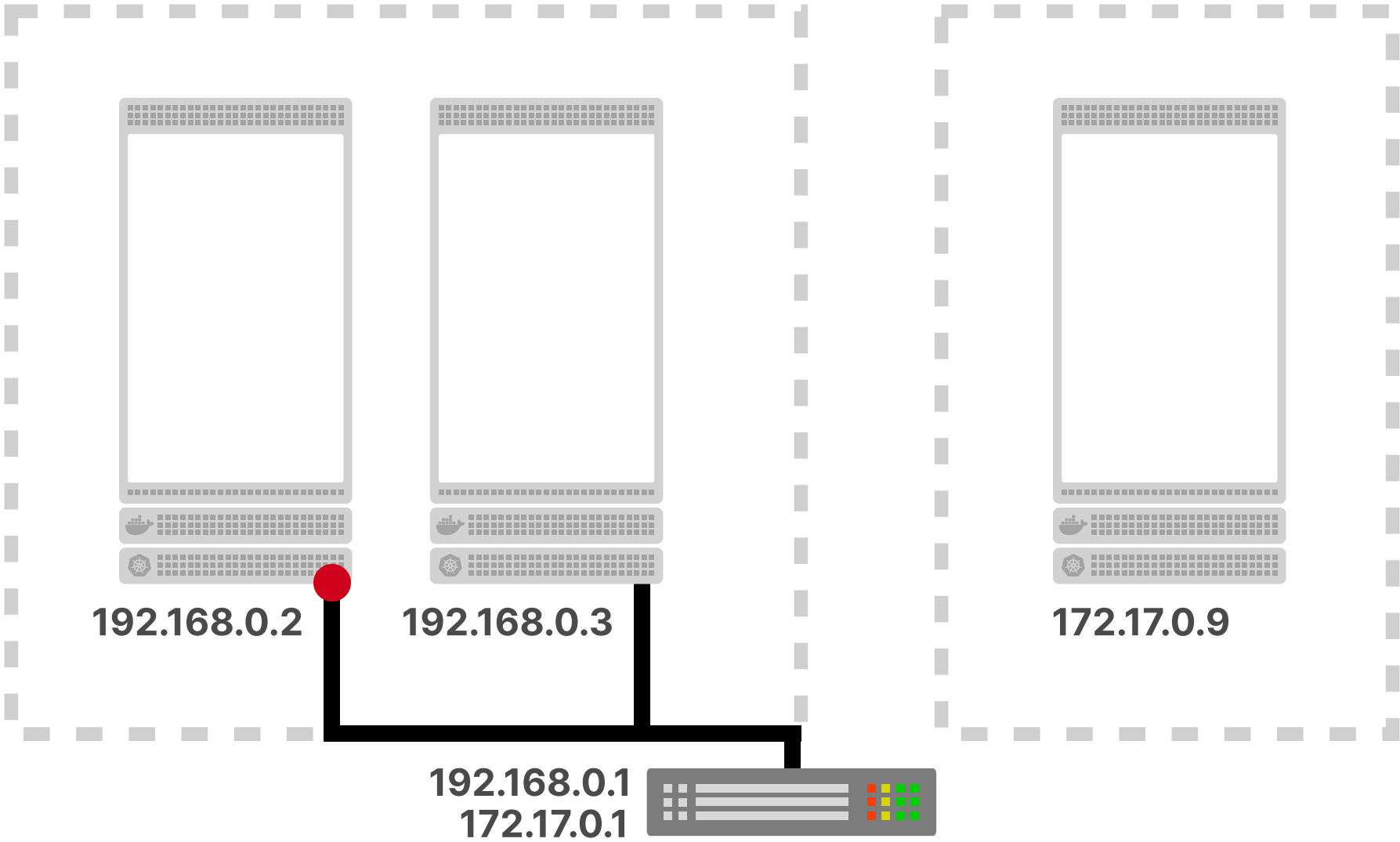
When the IP address is not in the local network, send the message to default gateway

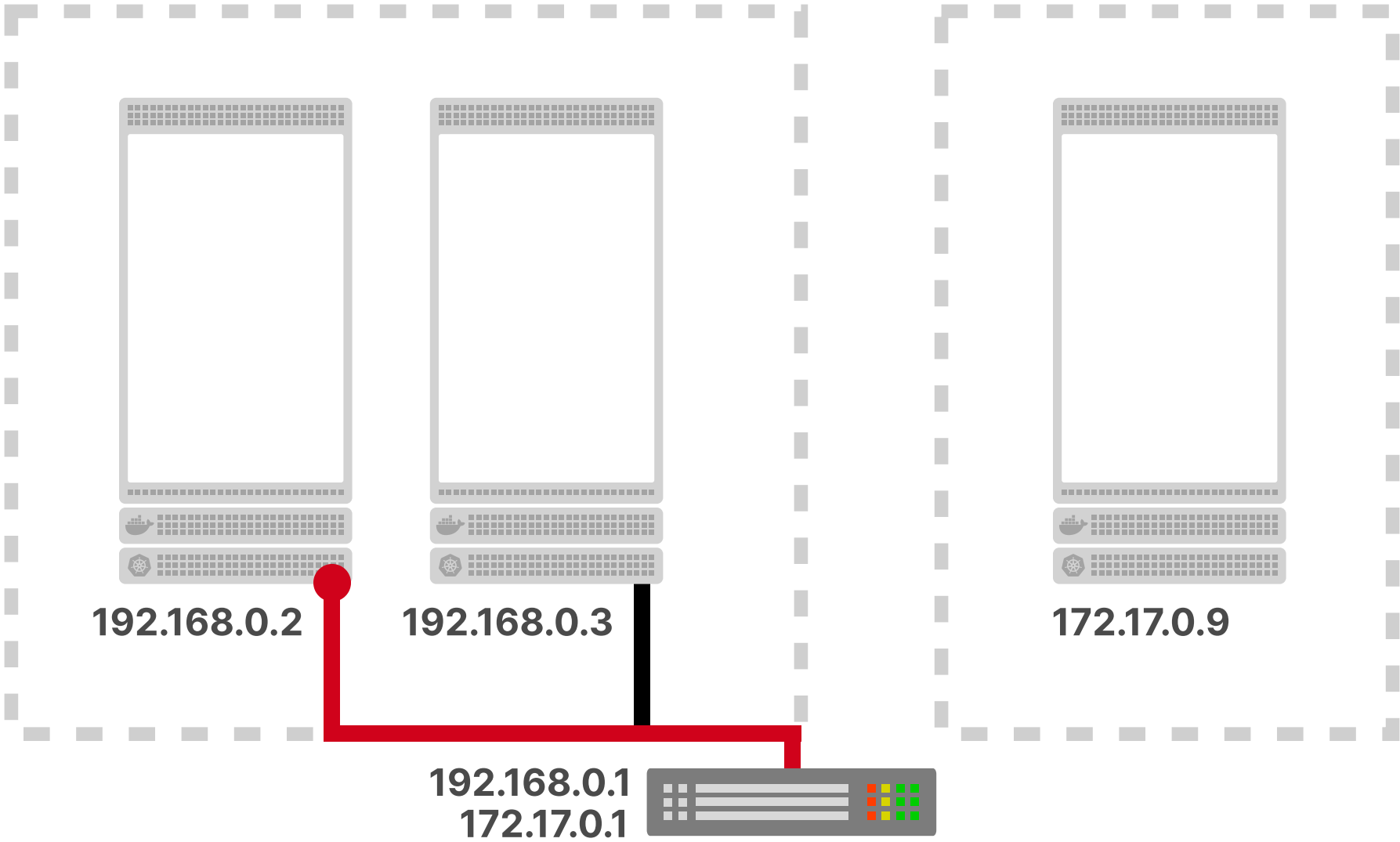
---

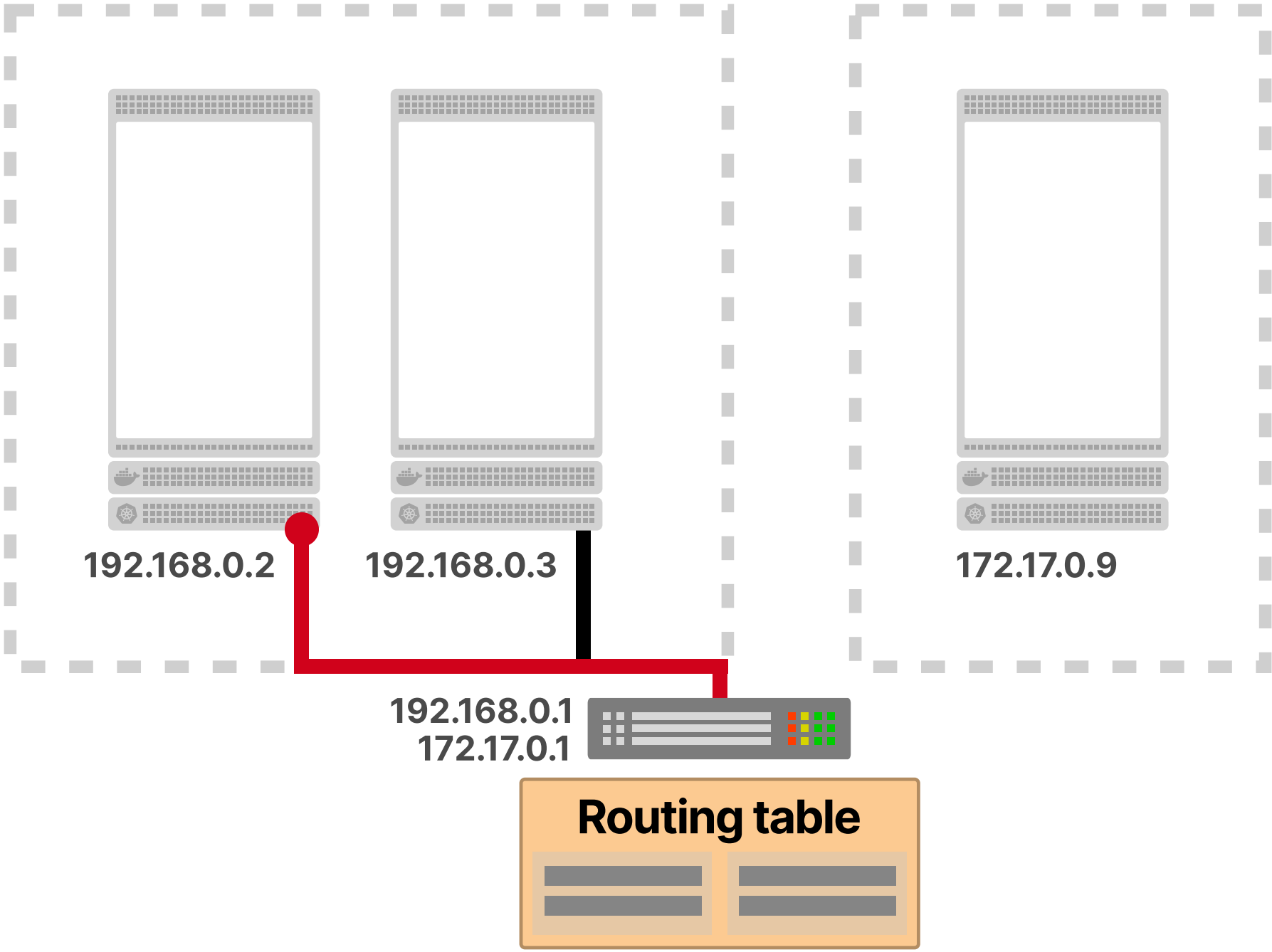
Networking recap

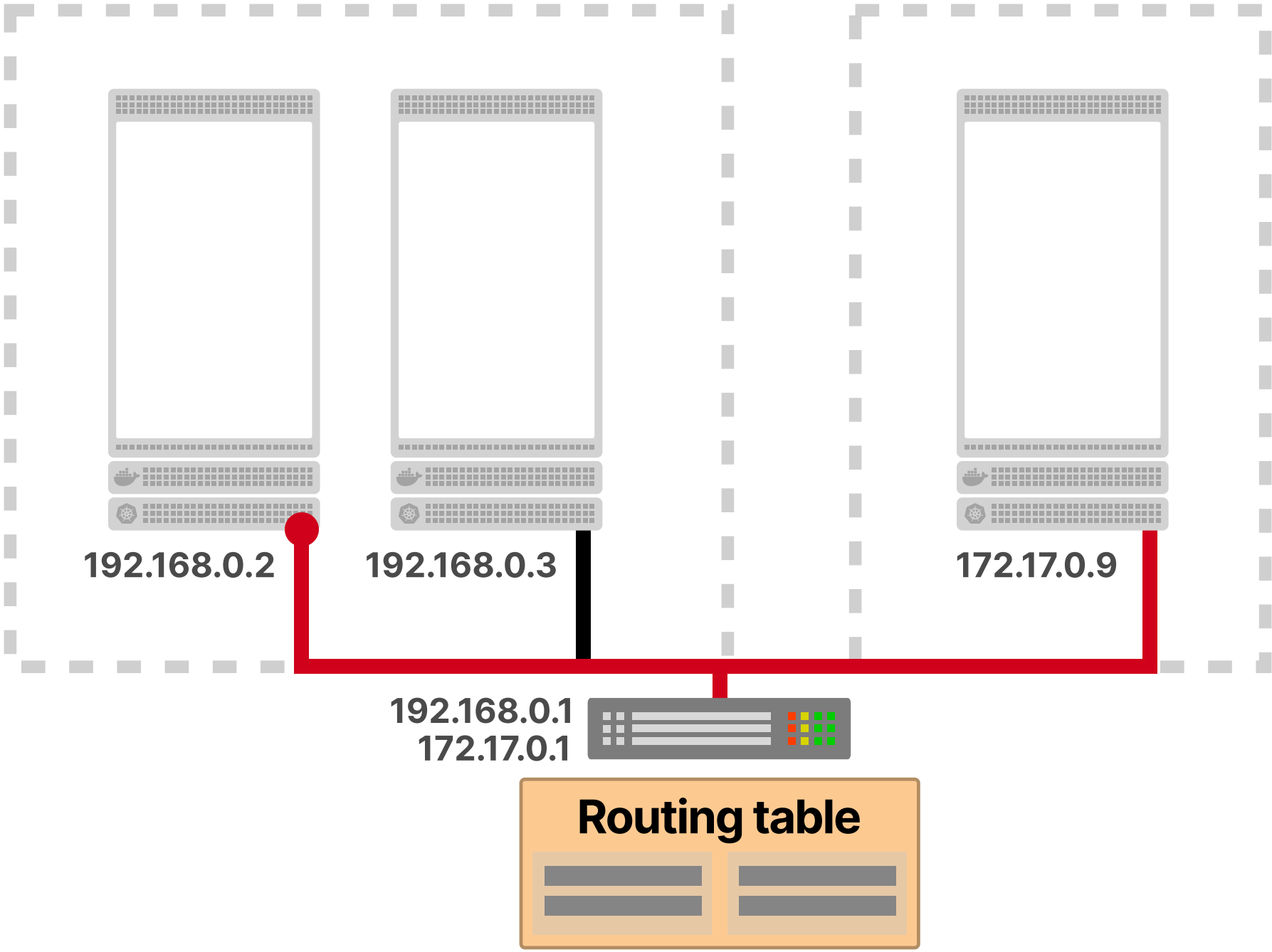

192.168.0.2
192.168.0.3
172.17.0.9
?


192.168.0.2
192.168.0.3
172.17.0.9
192.168.0.1
172.17.0.1


192.168.0.2
192.168.0.3
172.17.0.9
192.168.0.1
172.17.0.1


192.168.0.2
192.168.0.3
172.17.0.9
192.168.0.1
172.17.0.1


192.168.0.2
192.168.0.3
172.17.0.9
192.168.0.1
172.17.0.1
Routing table


192.168.0.2
192.168.0.3
172.17.0.9
192.168.0.1
172.17.0.1
Routing table

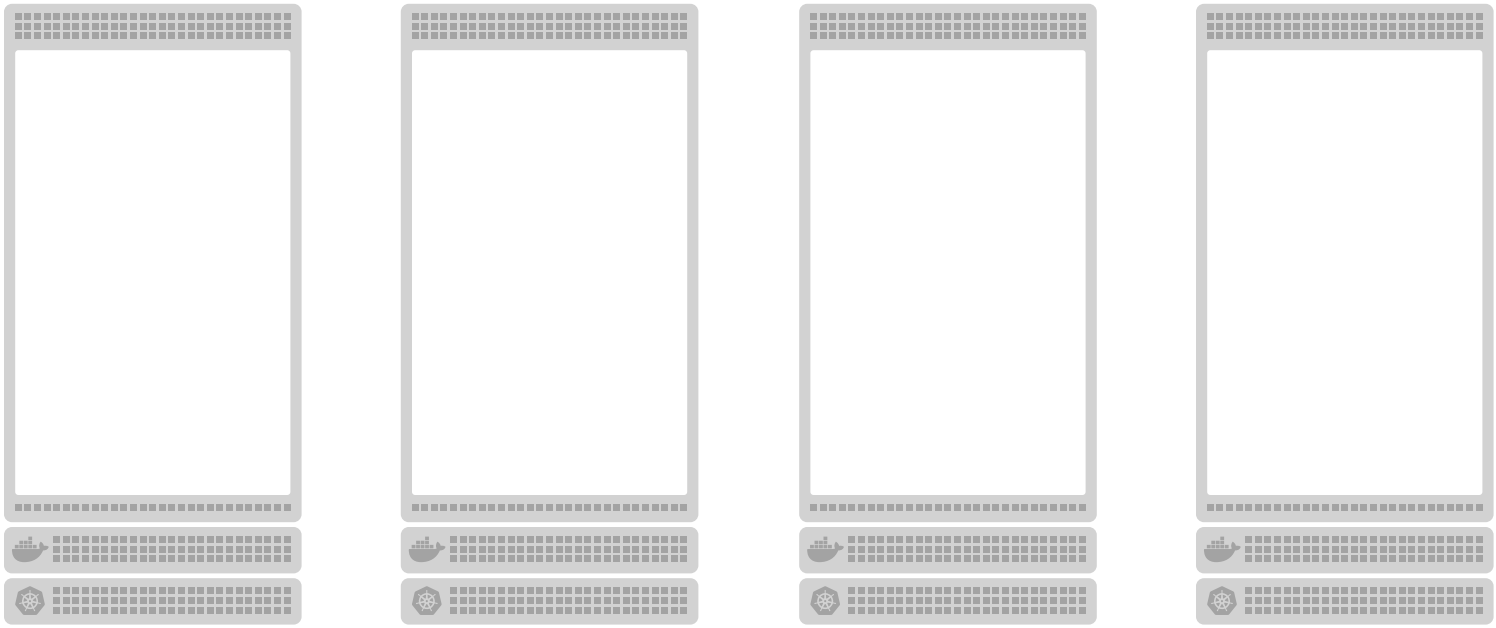
Networking implementation

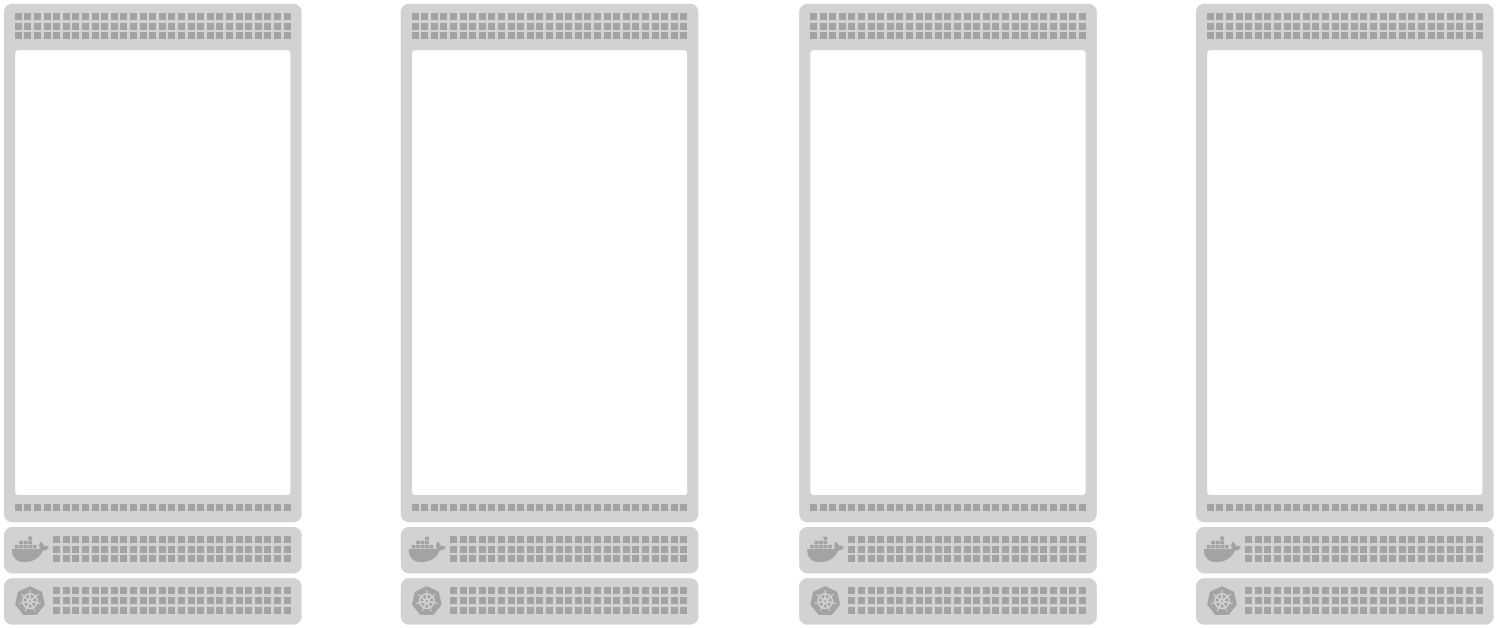
All Pods can communicate with all other Pods in the network without NAT

01

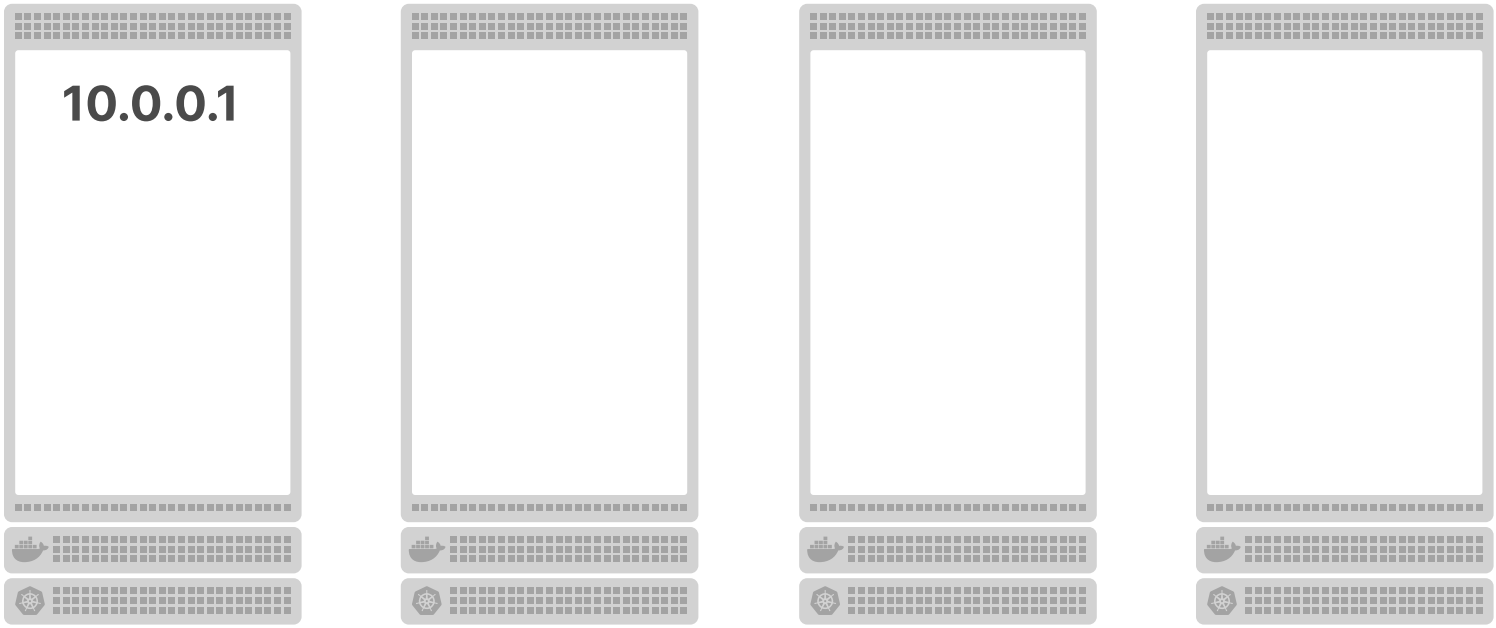
Process on a Node (e.g. kubelet) can communicate with all Pods on that Node

02

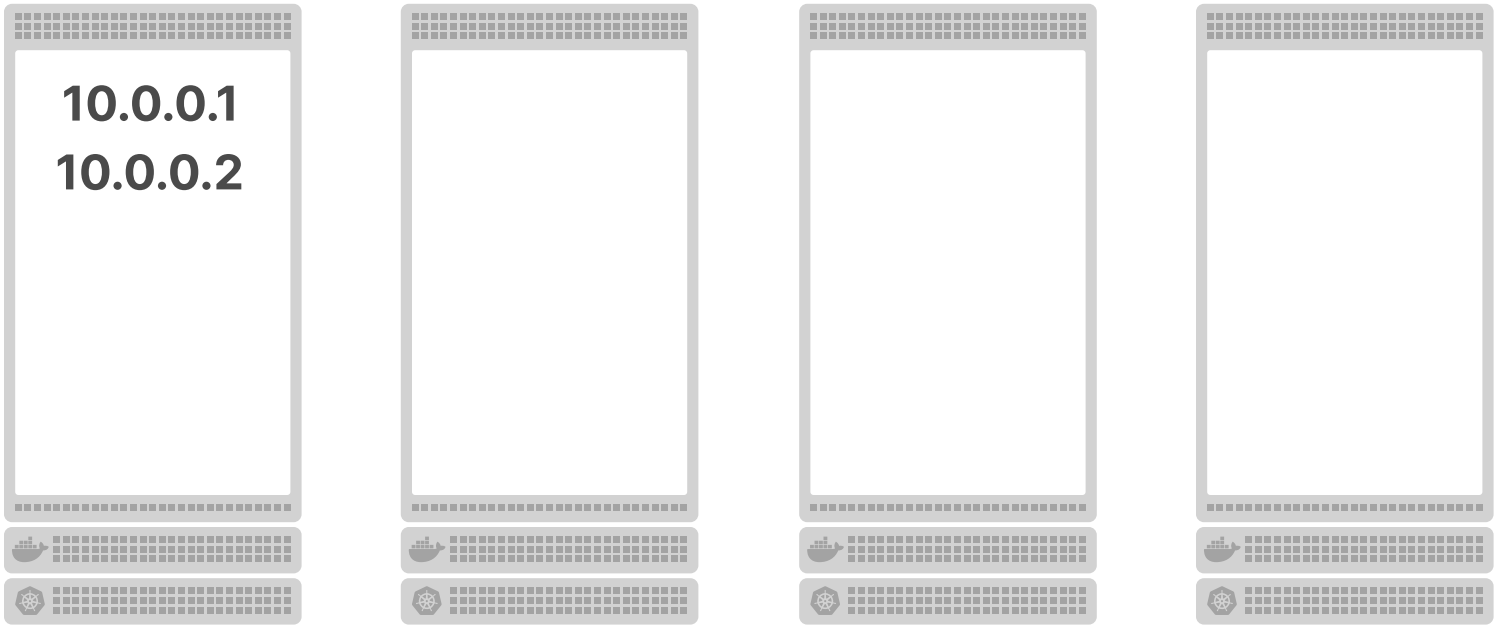


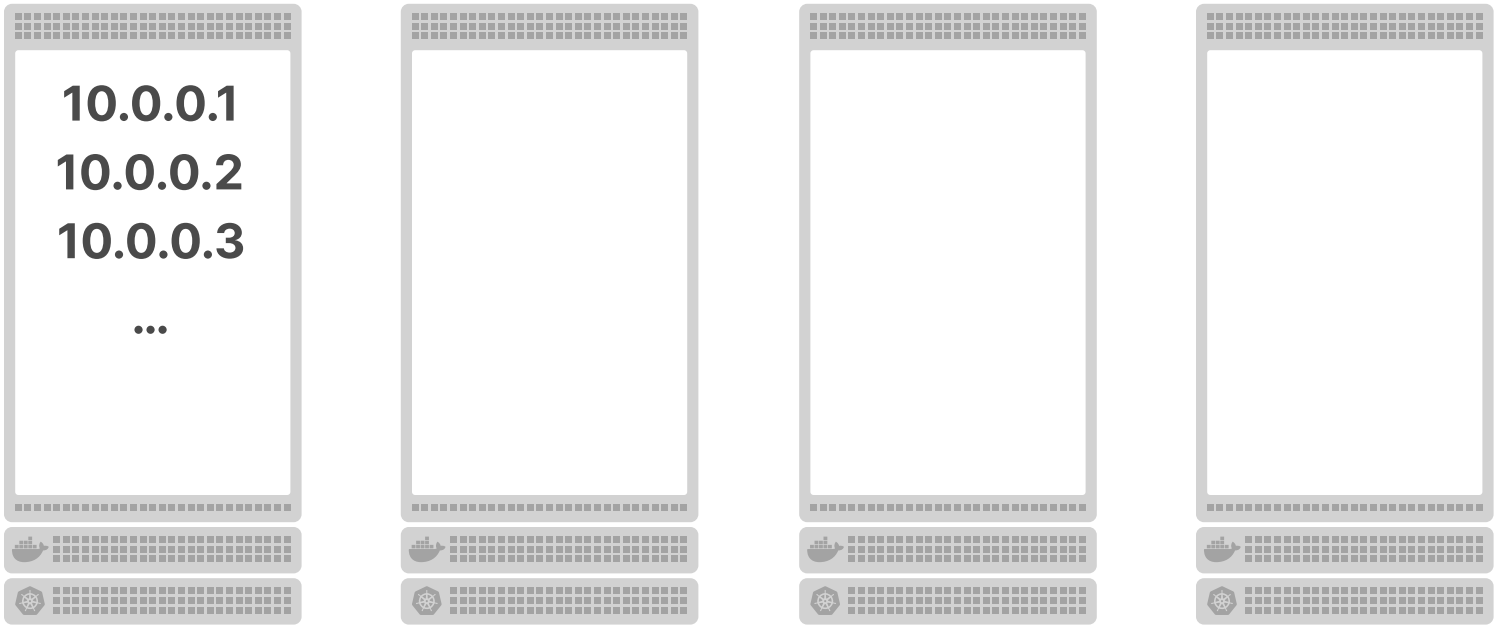


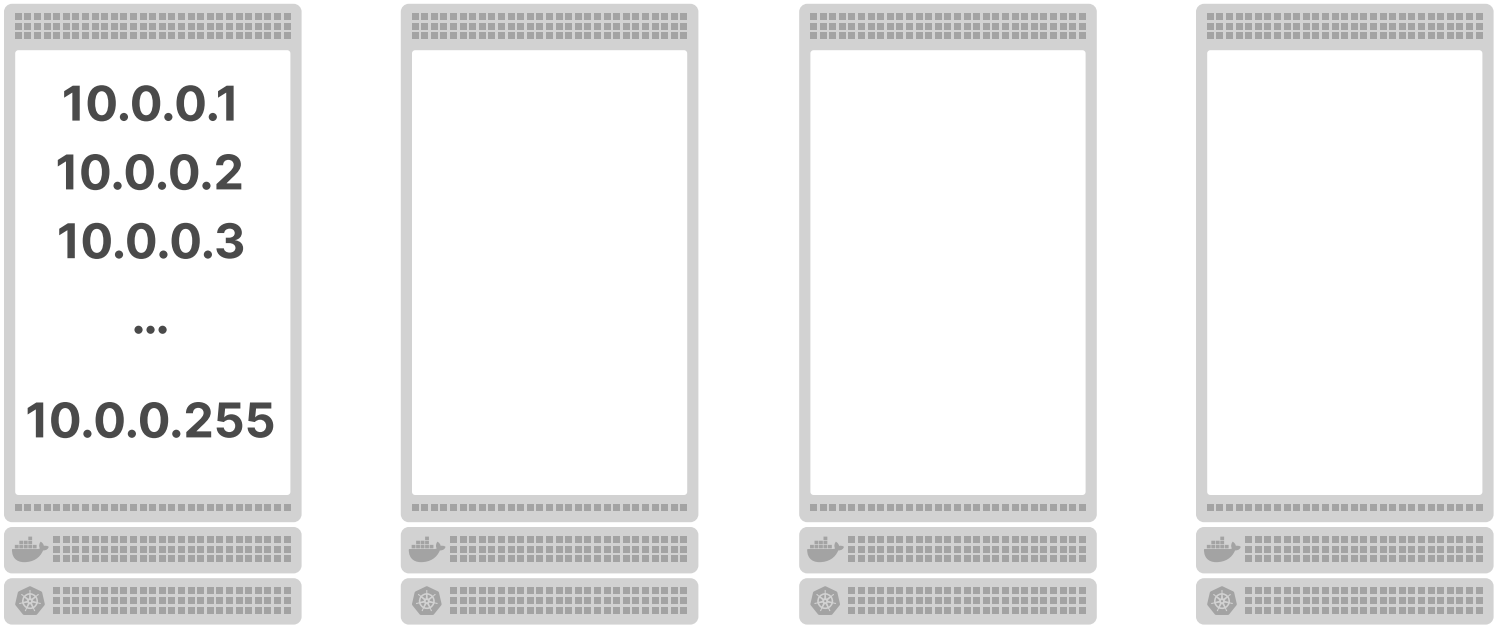
subnet 1



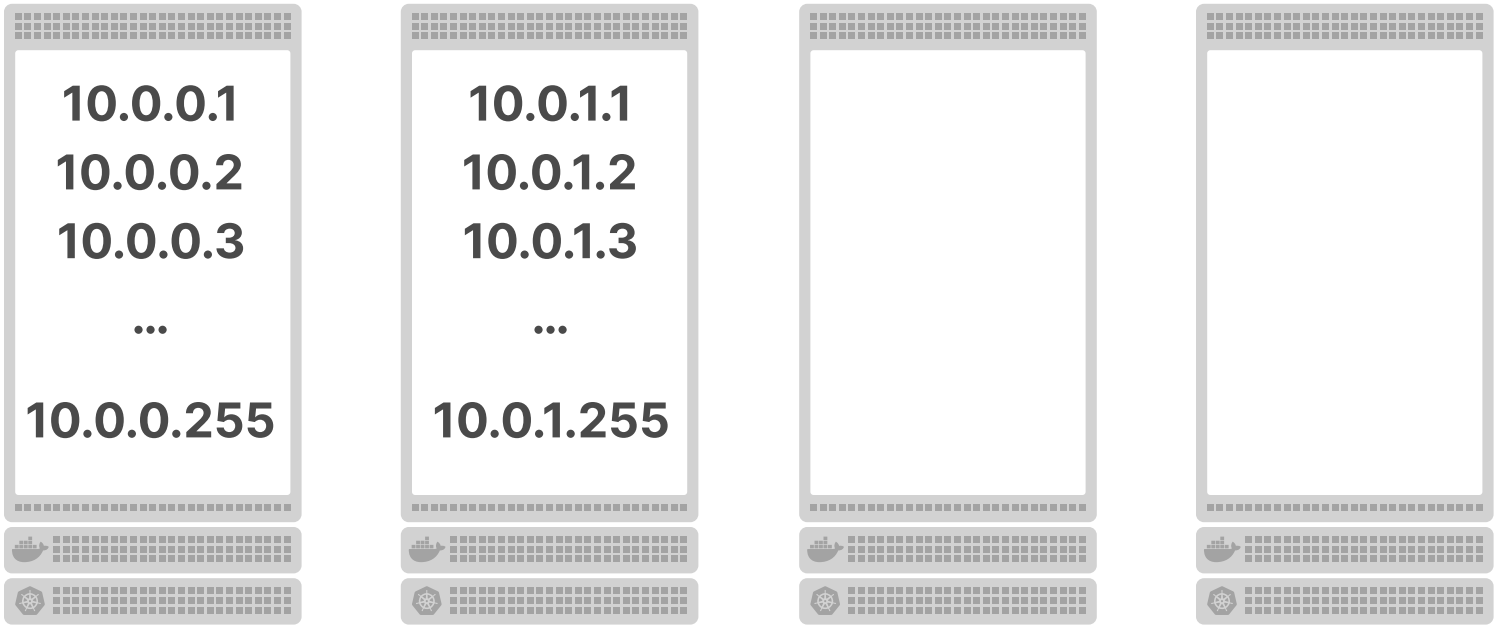
subnet 1

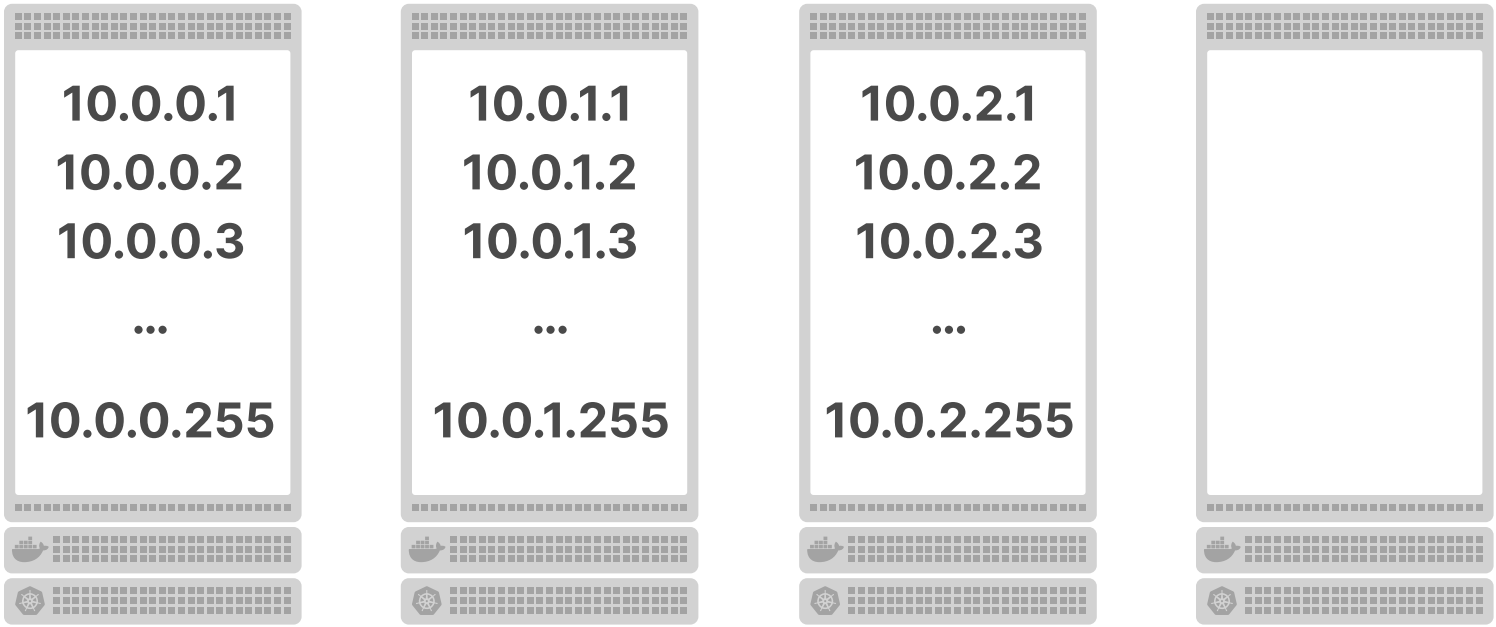

10.0.0.1
10.0.0.2
subnet 1

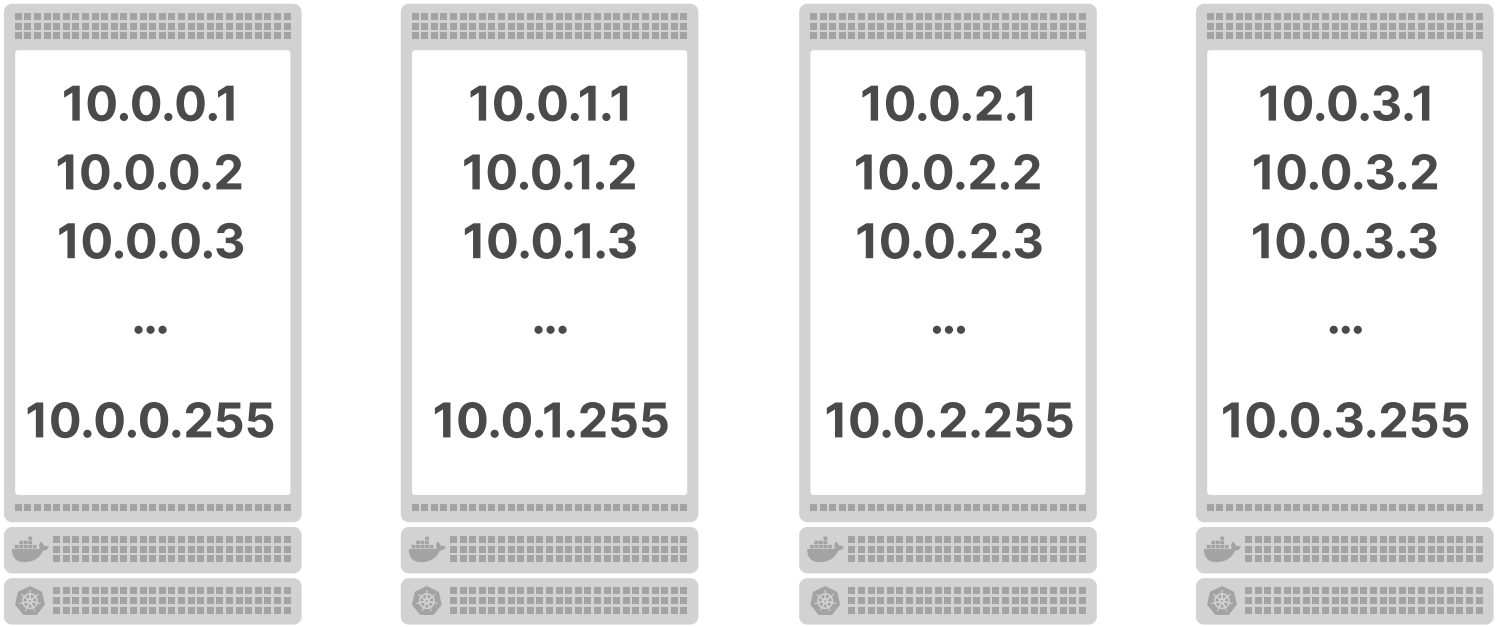

10.0.0.1
10.0.0.2
10.0.0.3
...
subnet 1

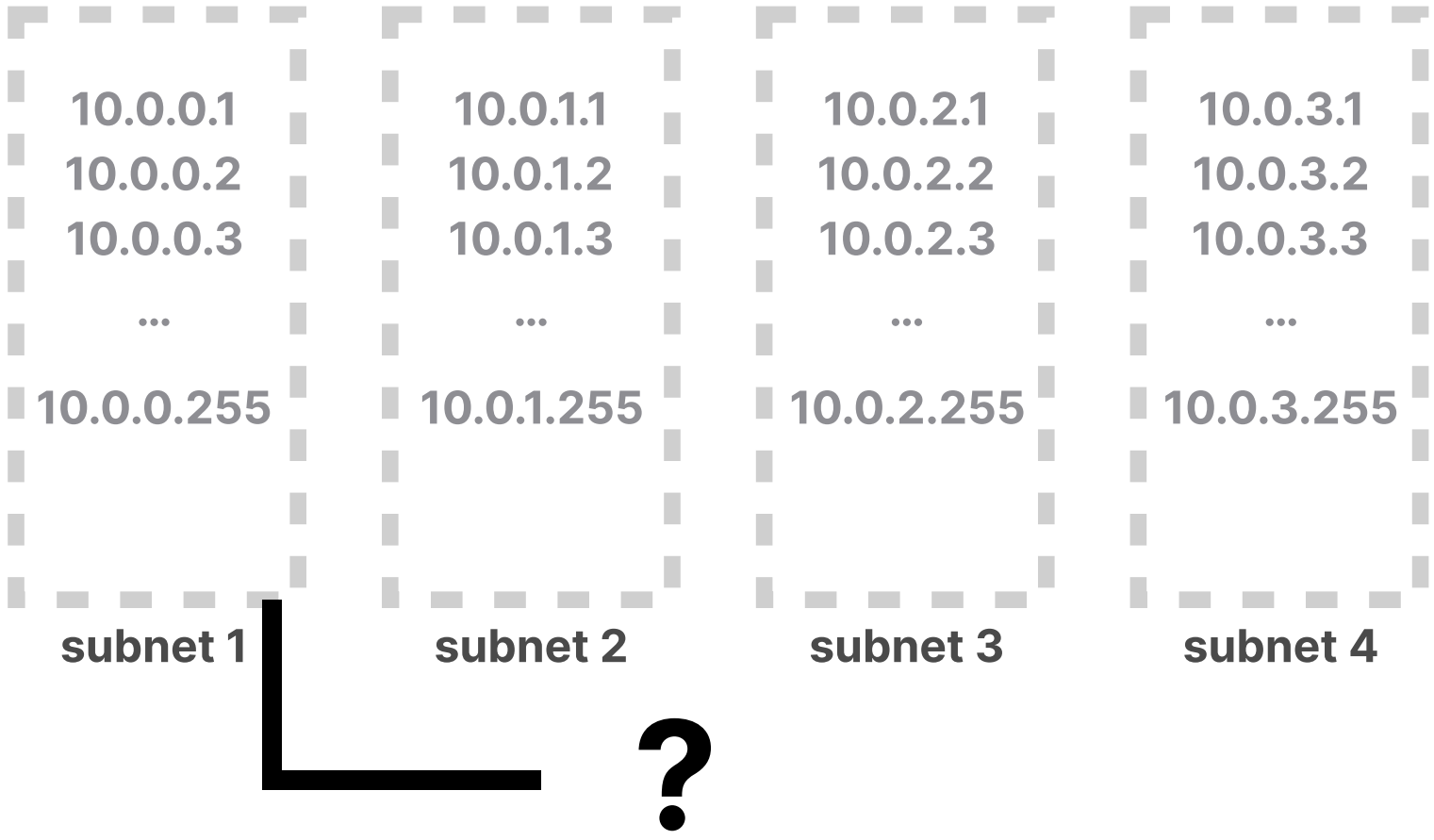


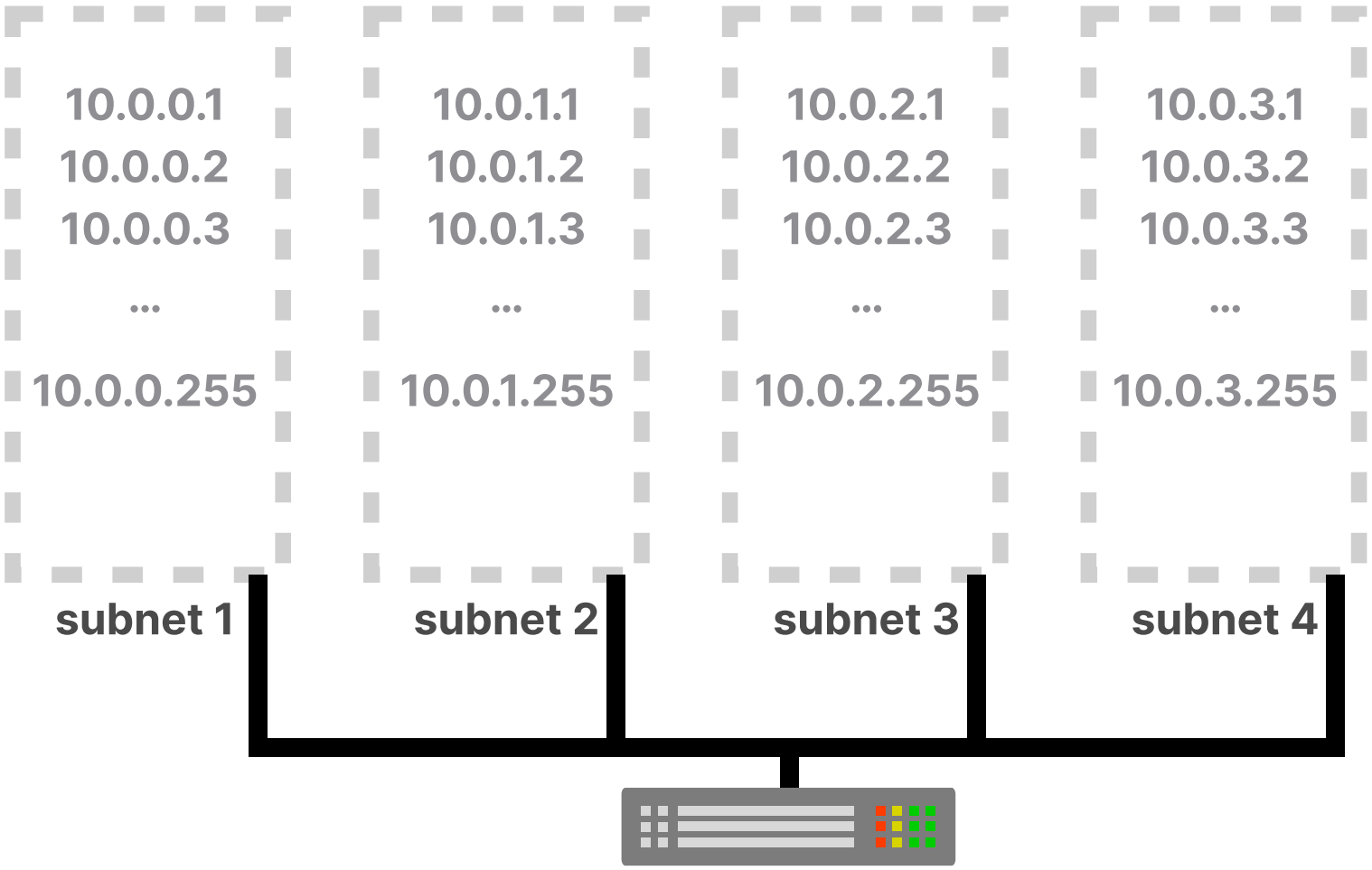
subnet 1

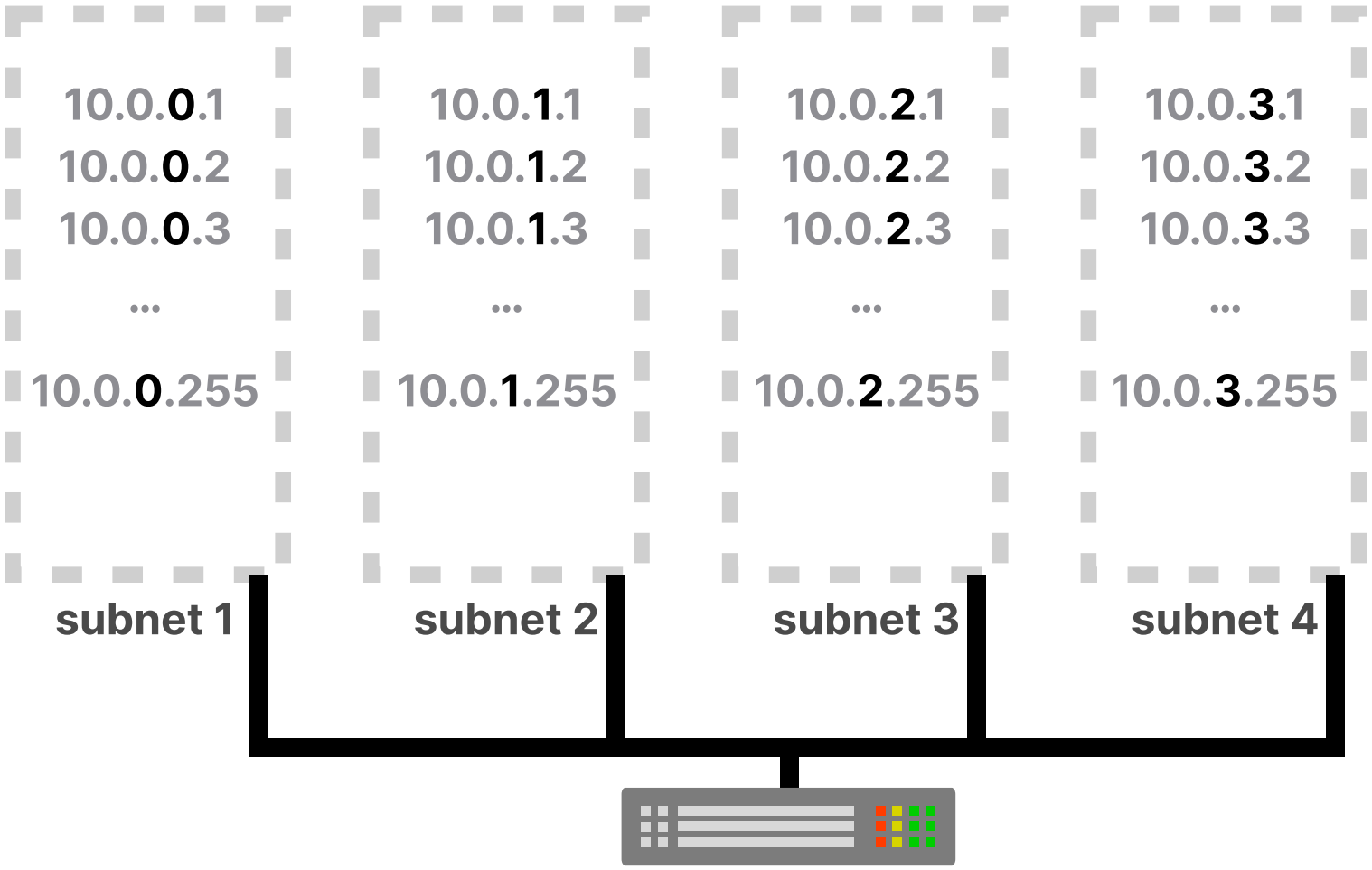

10.0.0.1
10.0.0.2
10.0.0.3
...
10.0.0.255
10.0.1.1
10.0.1.2
10.0.1.3
...
10.0.1.255
subnet 1
subnet 2

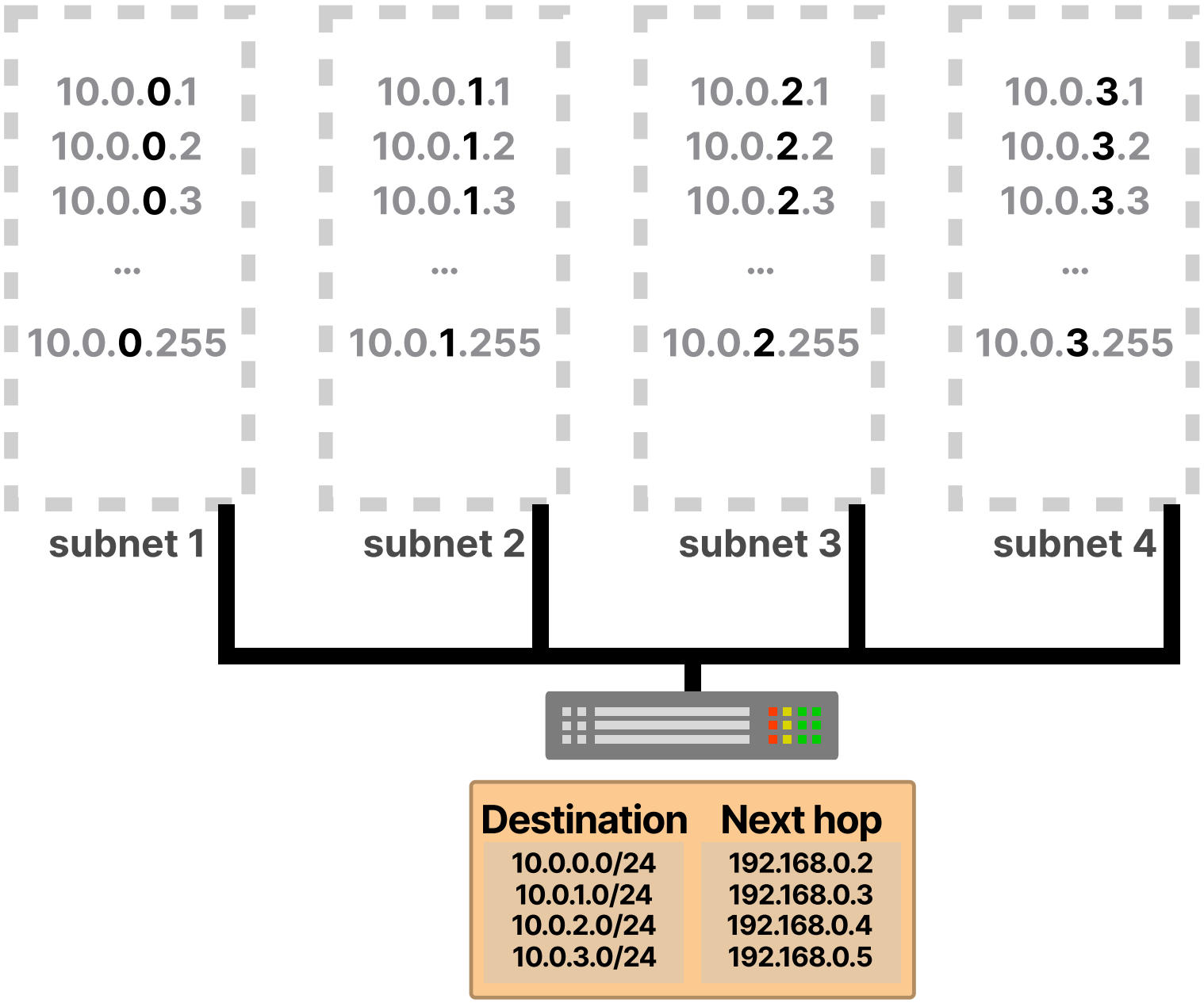

10.0.0.1
10.0.0.2
10.0.0.3
...
10.0.0.255
10.0.1.1
10.0.1.2
10.0.1.3
...
10.0.1.255
10.0.2.1
10.0.2.2
10.0.2.3
...
10.0.2.255
subnet 1
subnet 2
subnet 3

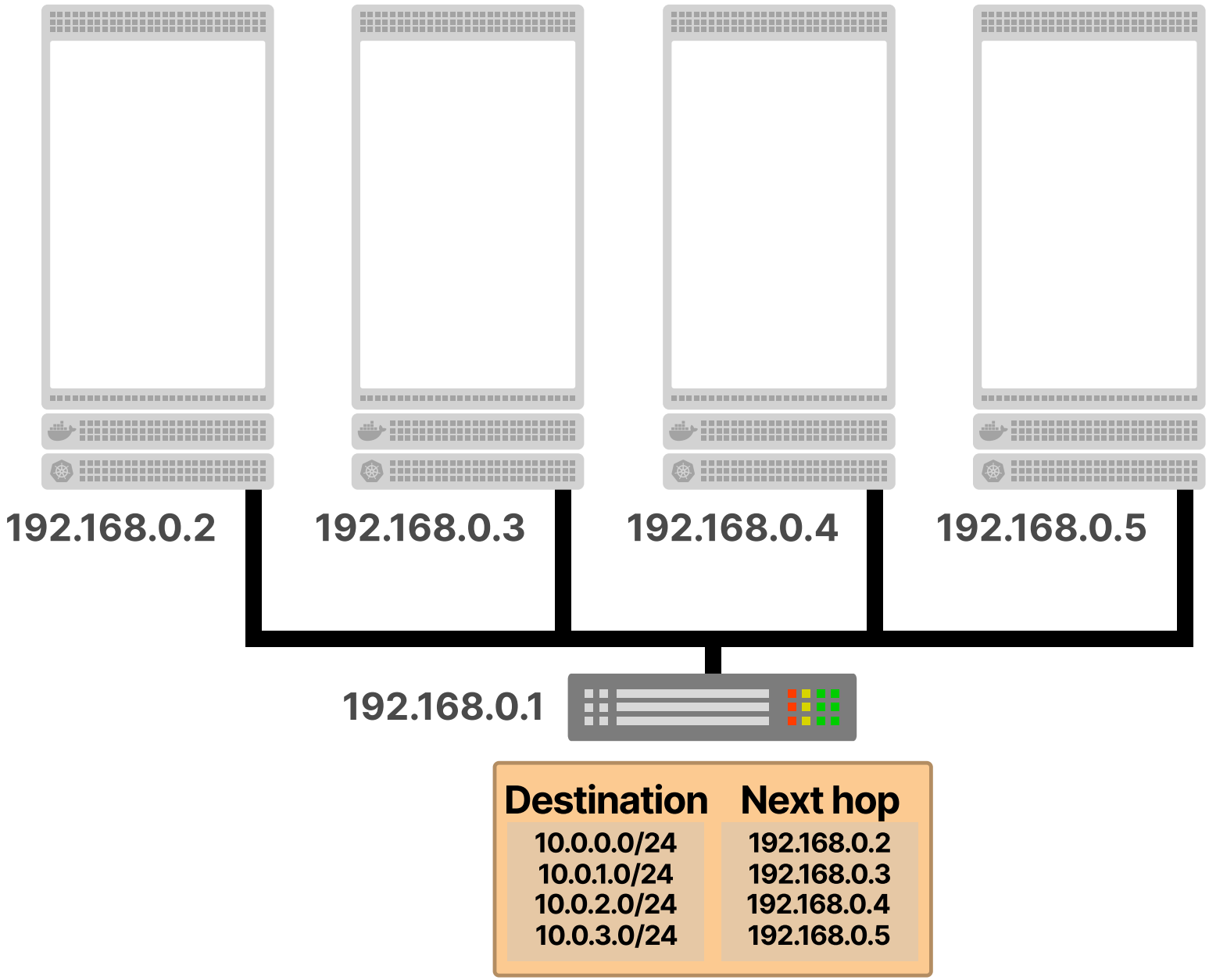

10.0.0.1
10.0.0.2
10.0.0.3
...
10.0.0.255
10.0.1.1
10.0.1.2
10.0.1.3
...
10.0.1.255
10.0.2.1
10.0.2.2
10.0.2.3
...
10.0.2.255
10.0.3.1
10.0.3.2
10.0.3.3
...
10.0.3.255
subnet 1
subnet 2
subnet 3
subnet 4

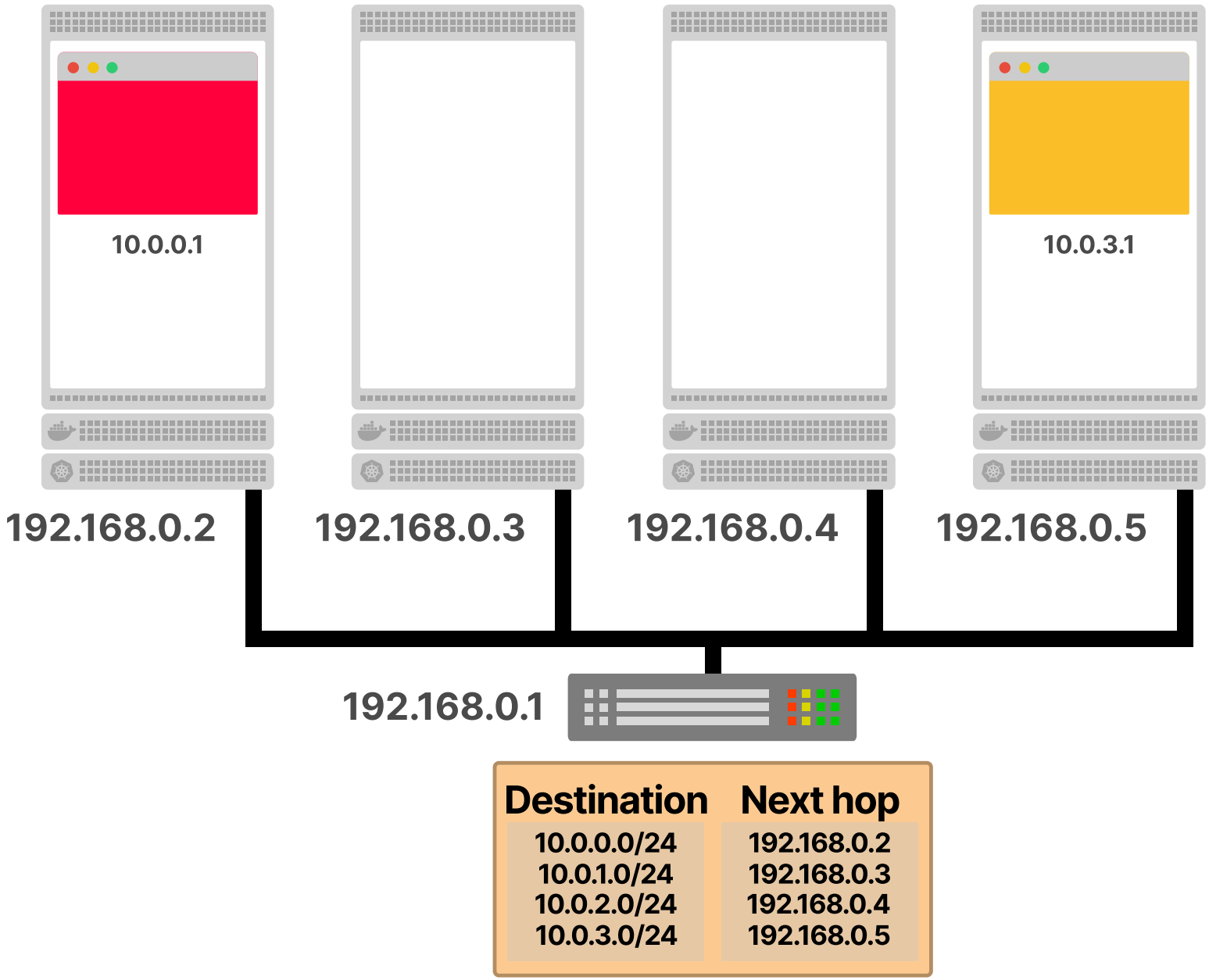

10.0.0.1
10.0.0.2
10.0.0.3
...
10.0.0.255
10.0.1.1
10.0.1.2
10.0.1.3
...
10.0.1.255
10.0.2.1
10.0.2.2
10.0.2.3
...
10.0.2.255
10.0.3.1
10.0.3.2
10.0.3.3
...
10.0.3.255
subnet 1
subnet 2
subnet 3
subnet 4
?


10.0.0.1
10.0.0.2
10.0.0.3
...
10.0.0.255
10.0.1.1
10.0.1.2
10.0.1.3
...
10.0.1.255
10.0.2.1
10.0.2.2
10.0.2.3
...
10.0.2.255
10.0.3.1
10.0.3.2
10.0.3.3
...
10.0.3.255
subnet 1
subnet 2
subnet 3
subnet 4


10.0.0.1
10.0.0.2
10.0.0.3
...
10.0.0.255
10.0.1.1
10.0.1.2
10.0.1.3
...
10.0.1.255
10.0.2.1
10.0.2.2
10.0.2.3
...
10.0.2.255
10.0.3.1
10.0.3.2
10.0.3.3
...
10.0.3.255
subnet 1
subnet 2
subnet 3
subnet 4

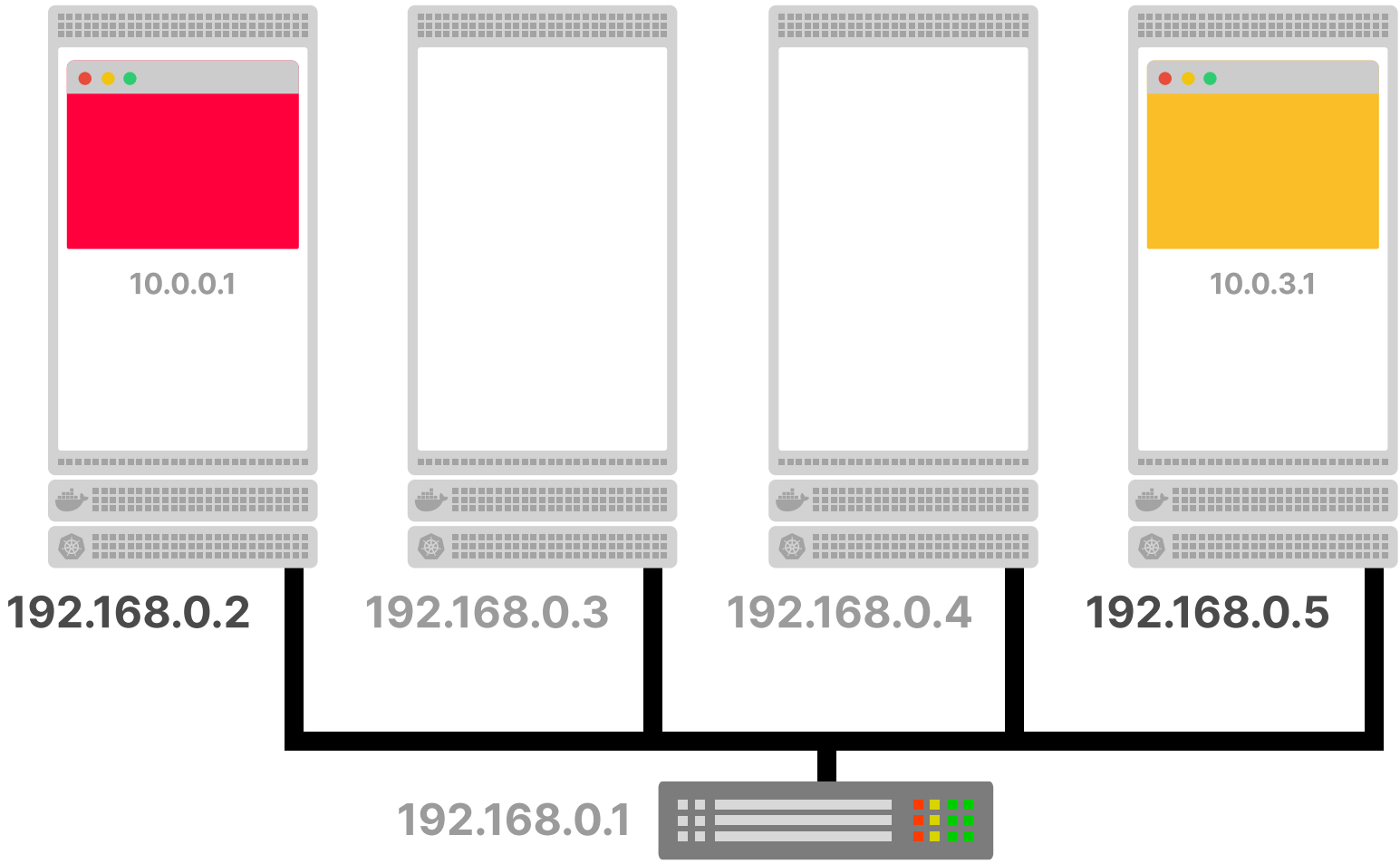

10.0.0.1
10.0.0.2
10.0.0.3
...
10.0.0.255
10.0.1.1
10.0.1.2
10.0.1.3
...
10.0.1.255
10.0.2.1
10.0.2.2
10.0.2.3
...
10.0.2.255
10.0.3.1
10.0.3.2
10.0.3.3
...
10.0.3.255
subnet 1
subnet 2
subnet 3
subnet 4
Destination
Next hop
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5


192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5
192.168.0.1
Destination
Next hop
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5


10.0.0.1
10.0.3.1
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5
192.168.0.1
Destination
Next hop
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5

Node to node communication

01

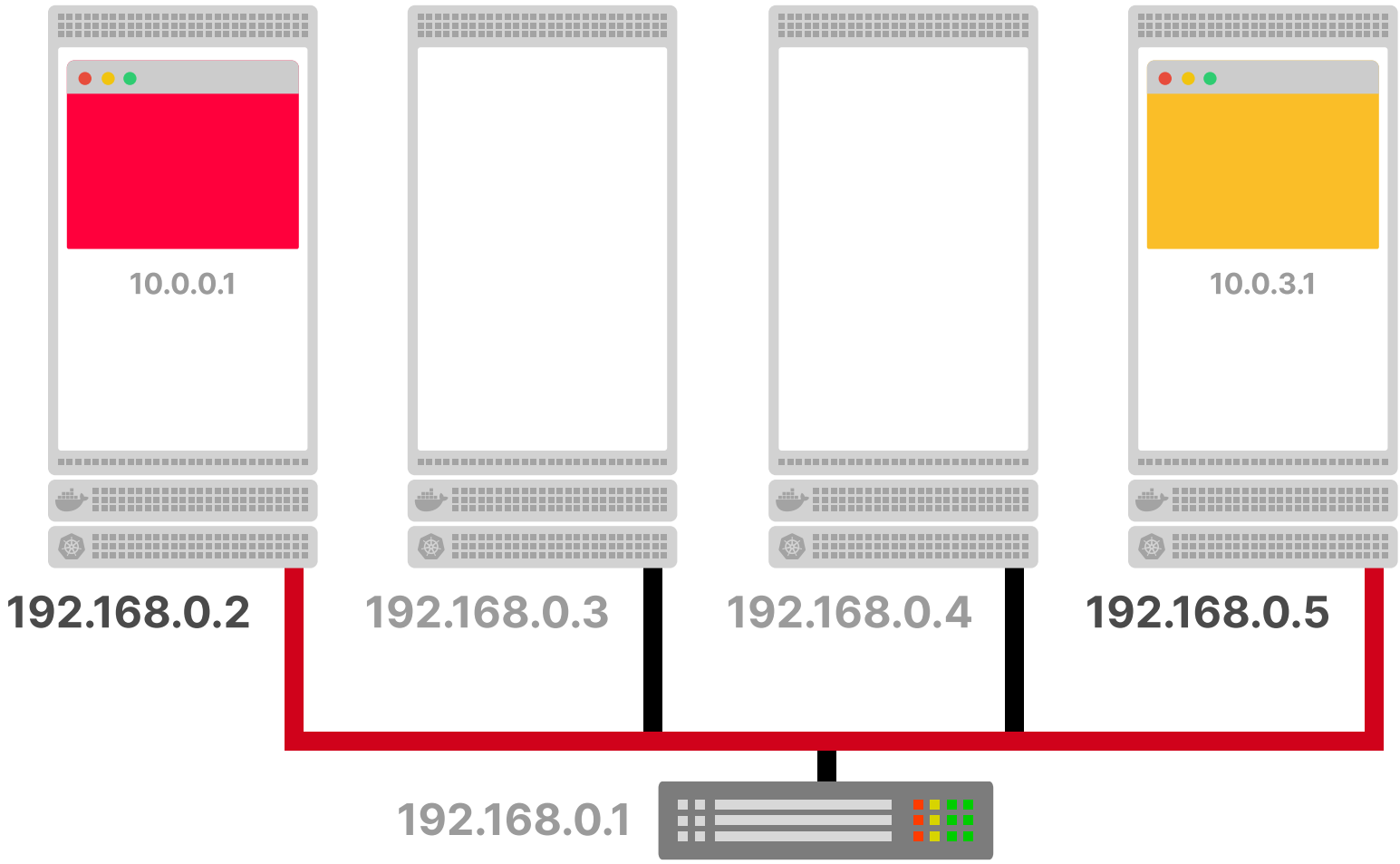


FROM:

Node1 (192.168.0.2)

TO:

Node4 (192.168.0.5)



FROM:

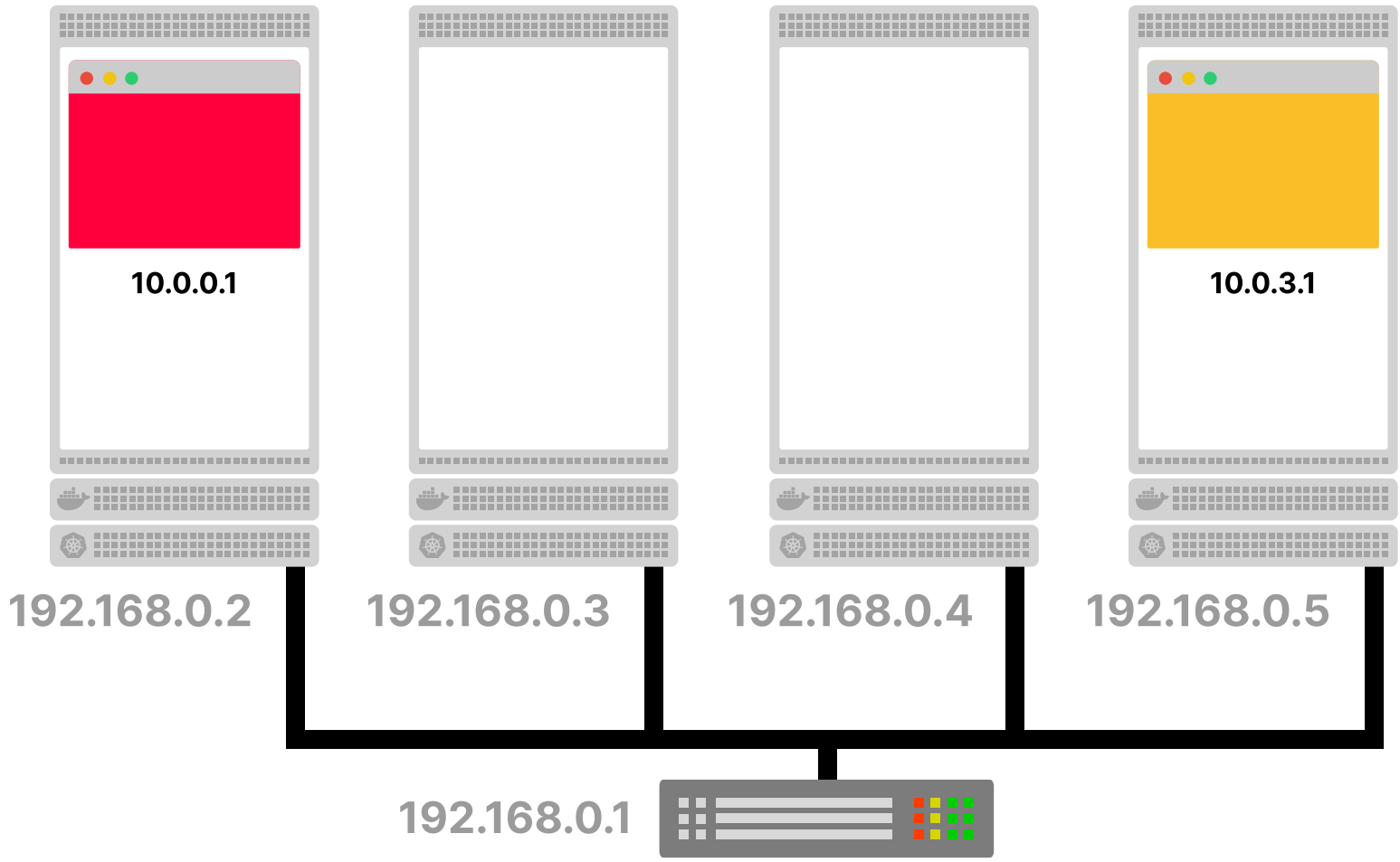
Node1 (192.168.0.2)

TO:

Node4 (192.168.0.5)

Pod to Pod communication

02

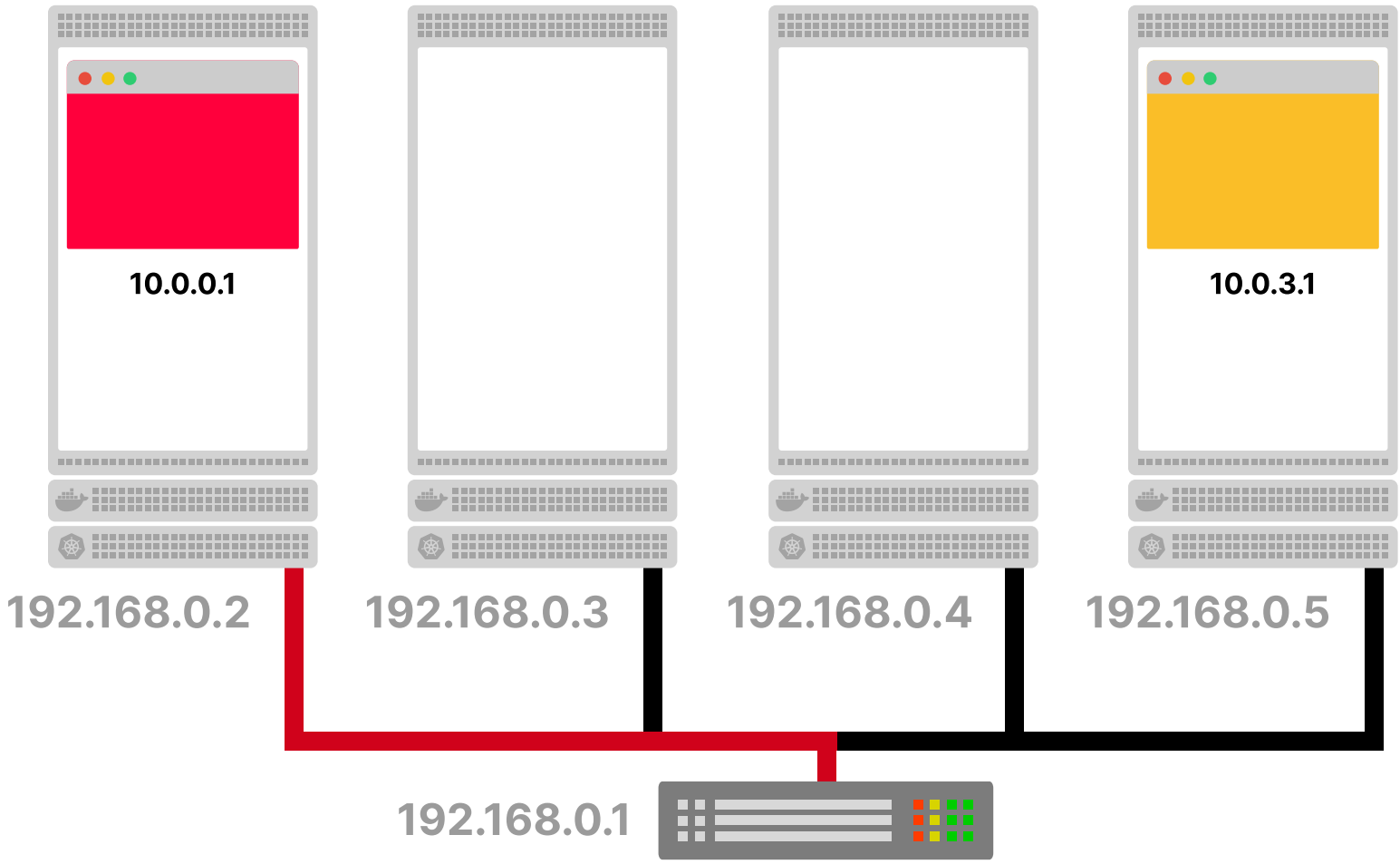


FROM:

Pod (10.0.0.1)

TO:

Pod (10.0.3.1)

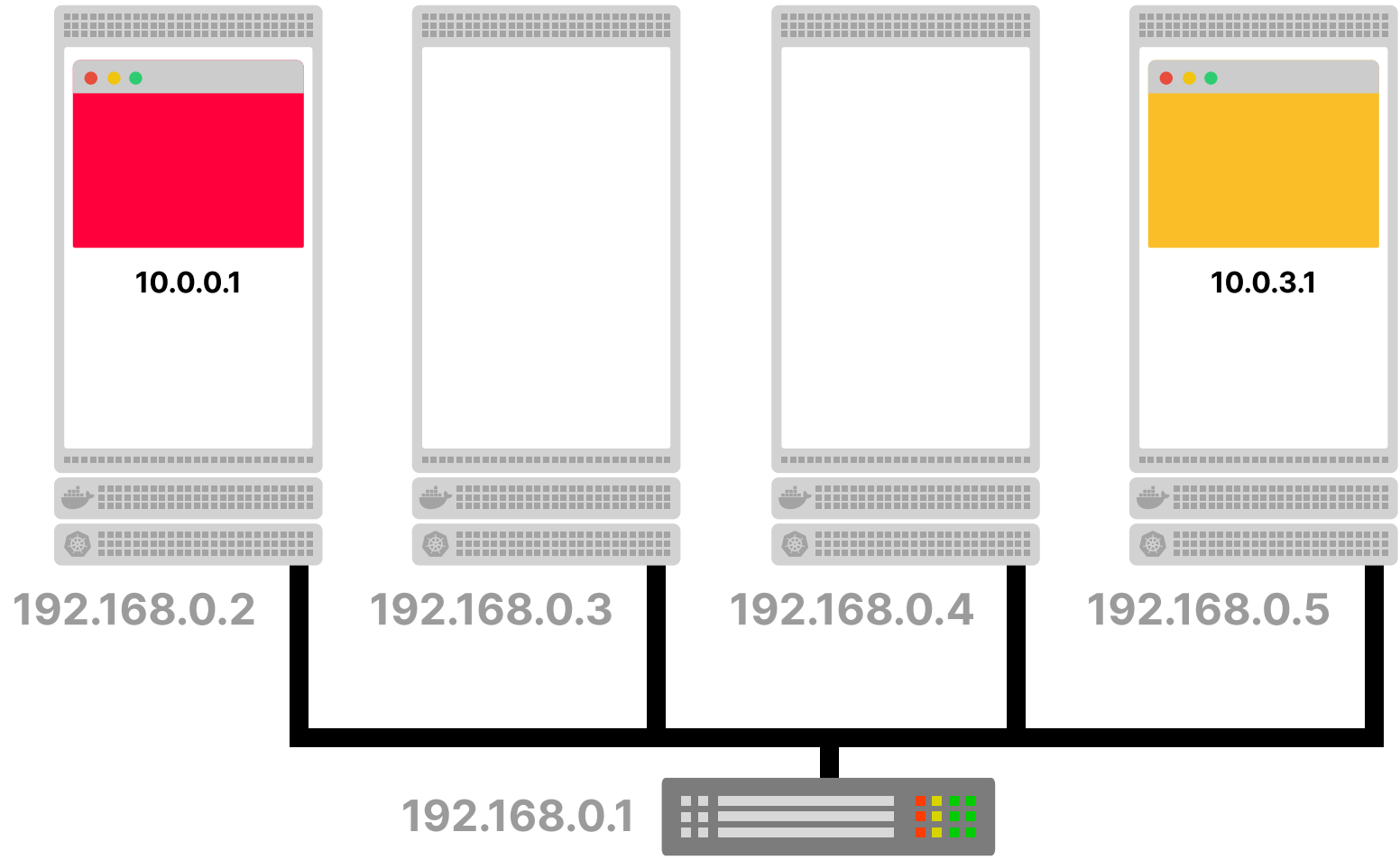


FROM:

Pod (10.0.0.1)

TO:

Pod (10.0.3.1)



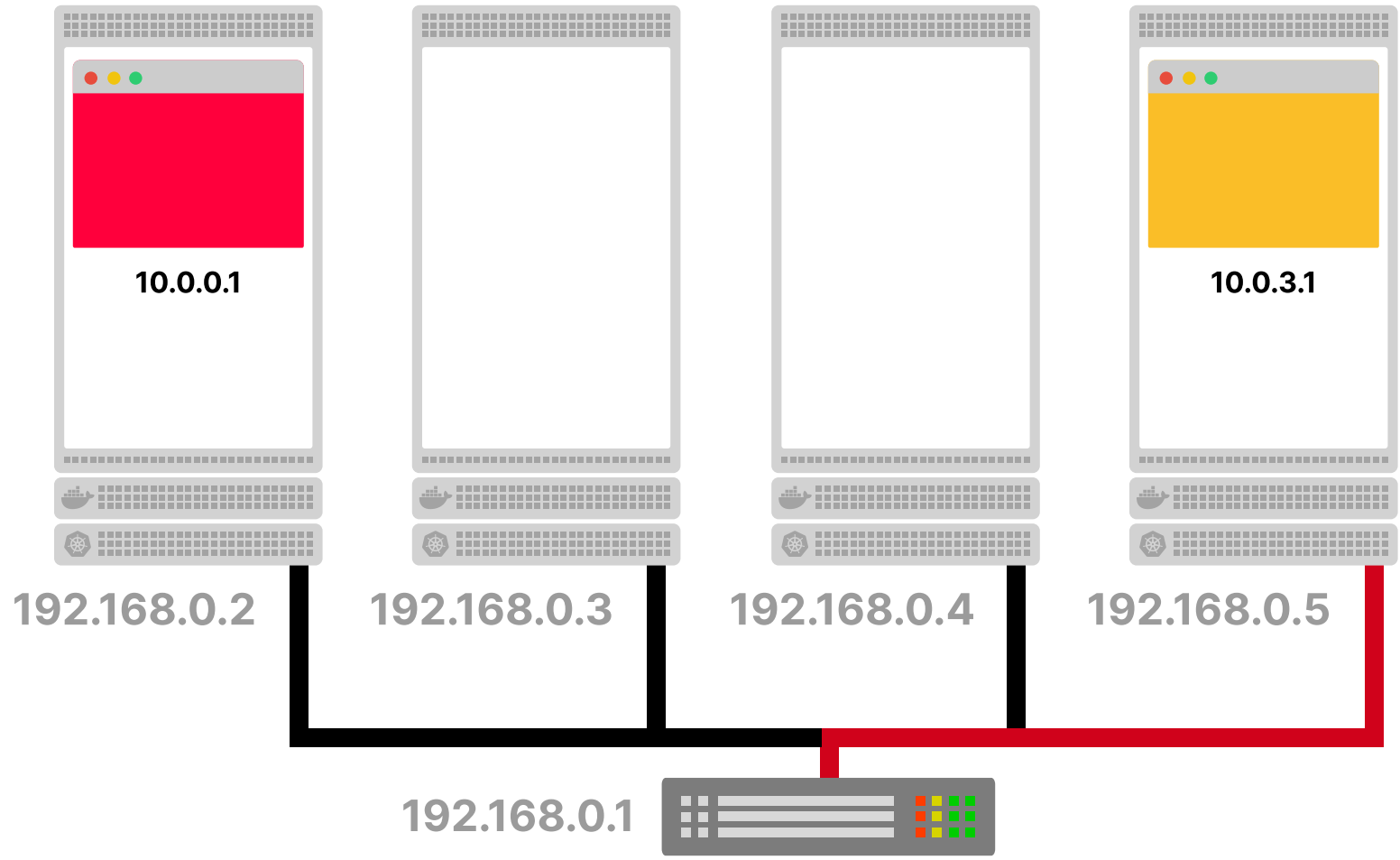
FROM:

Pod ■ (10.0.0.1)

TO:

Pod ■ (10.0.3.1)

Destination	Next hop
10.0.0.0/24	192.168.0.2
10.0.1.0/24	192.168.0.3
10.0.2.0/24	192.168.0.4
10.0.3.0/24	192.168.0.5



FROM:

Pod (10.0.0.1)

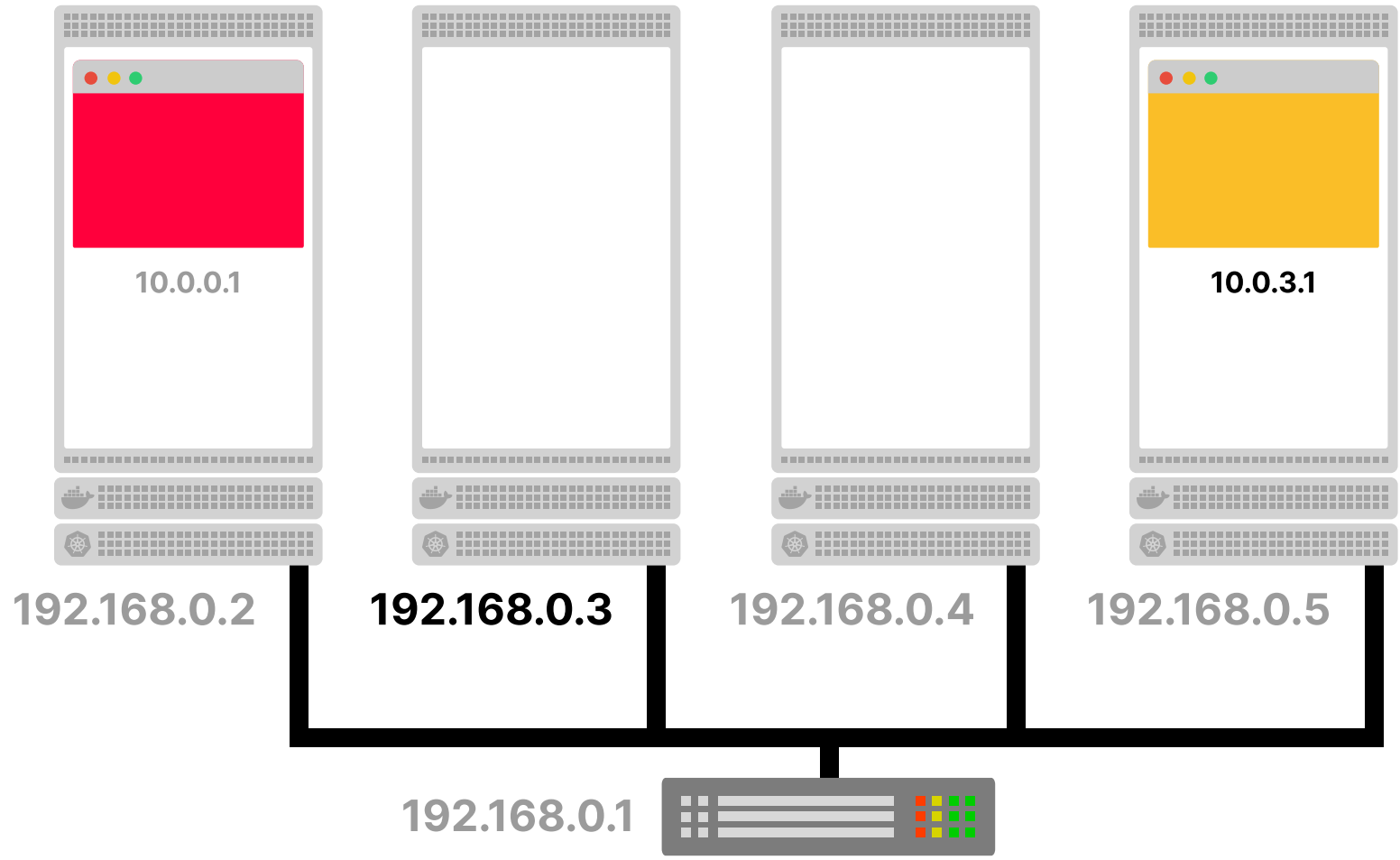
TO:

Pod (10.0.3.1)

Destination	Next hop
10.0.0.0/24	192.168.0.2
10.0.1.0/24	192.168.0.3
10.0.2.0/24	192.168.0.4
10.0.3.0/24	192.168.0.5

Node to Pod communication

03

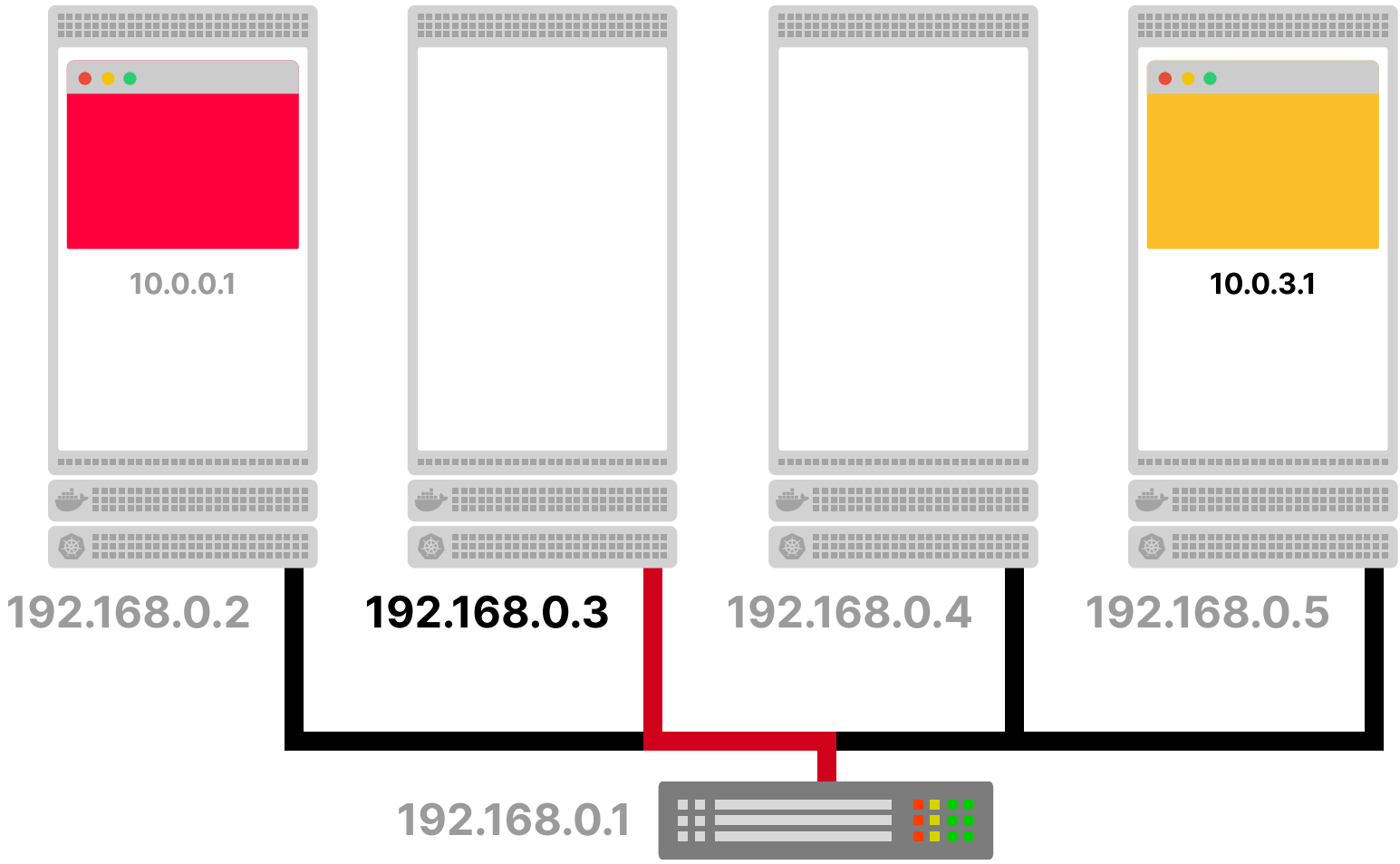


FROM:

Node2 (192.168.0.3)

TO:

Pod ■ (10.0.3.1)

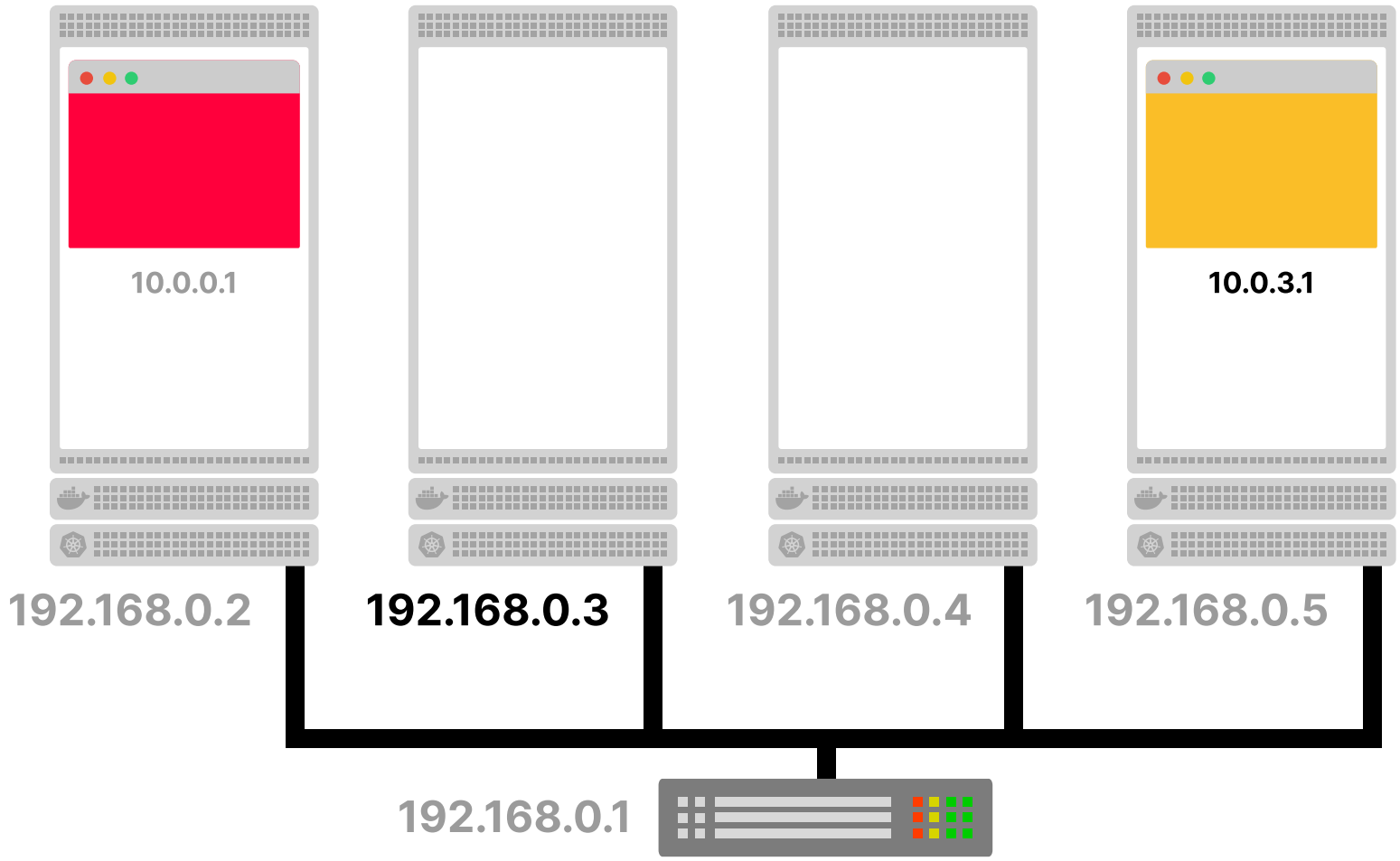


FROM:

Node2 (192.168.0.3)

TO:

Pod ■ (10.0.3.1)



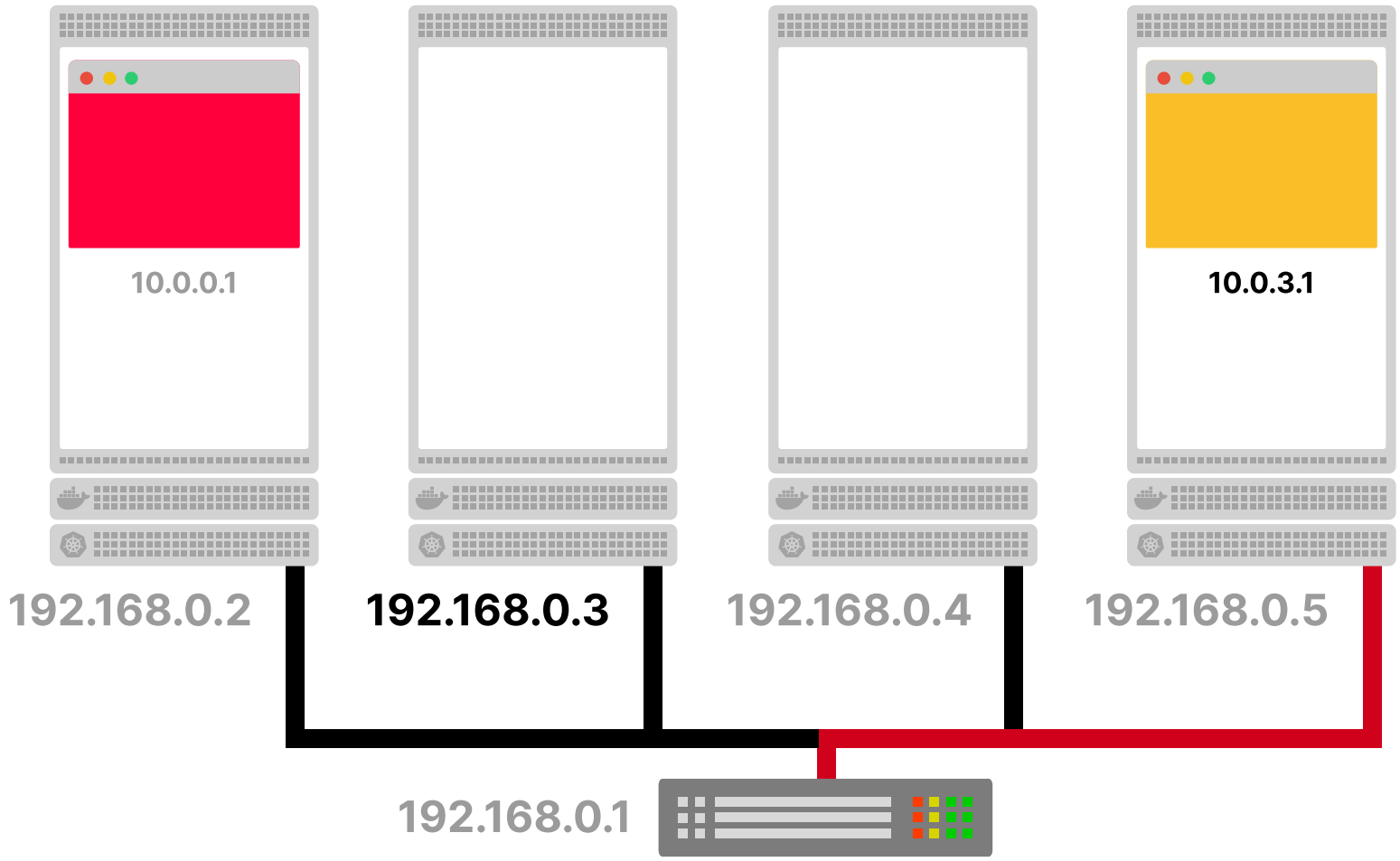
FROM:

Node2 (192.168.0.3)

TO:

Pod (10.0.3.1)

Destination	Next hop
10.0.0.0/24	192.168.0.2
10.0.1.0/24	192.168.0.3
10.0.2.0/24	192.168.0.4
10.0.3.0/24	192.168.0.5



FROM:

Node2 (192.168.0.3)

TO:

Pod (10.0.3.1)

Destination	Next hop
10.0.0.0/24	192.168.0.2
10.0.1.0/24	192.168.0.3
10.0.2.0/24	192.168.0.4
10.0.3.0/24	192.168.0.5

Networking implementation

All Pods can communicate with all other Pods in the network without NAT

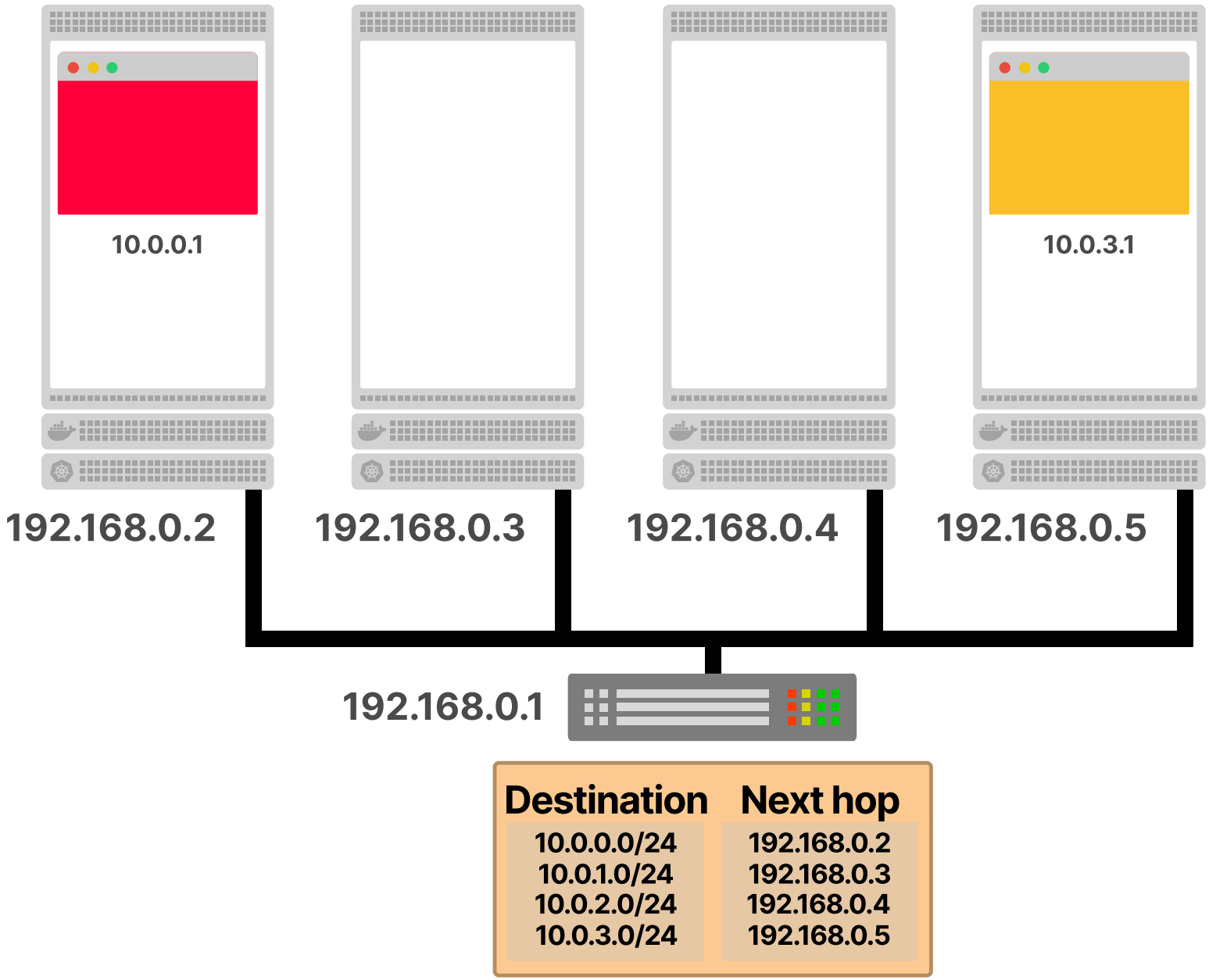


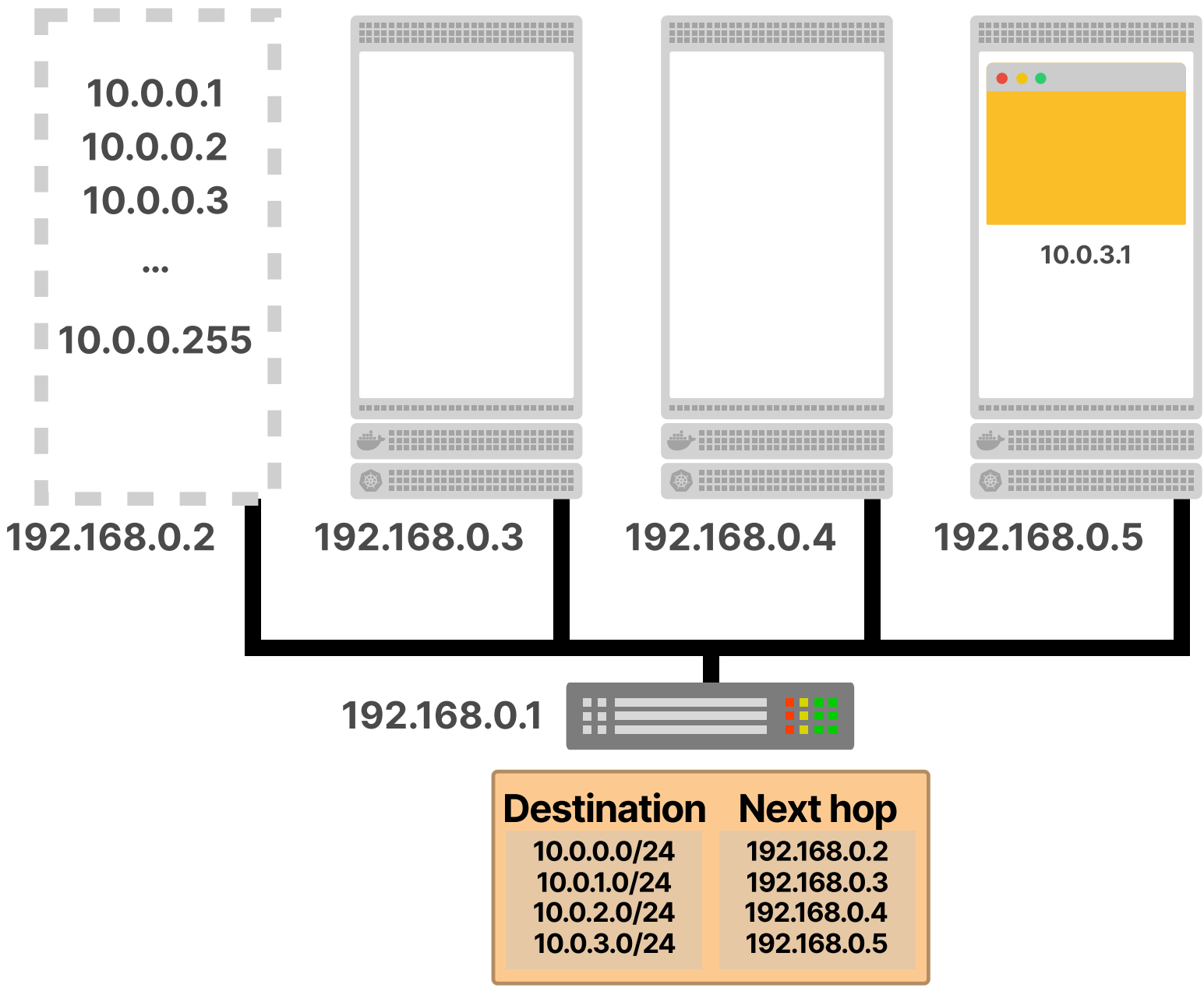
Process on a Node (e.g. kubelet) can communicate with all Pods on that Node

---



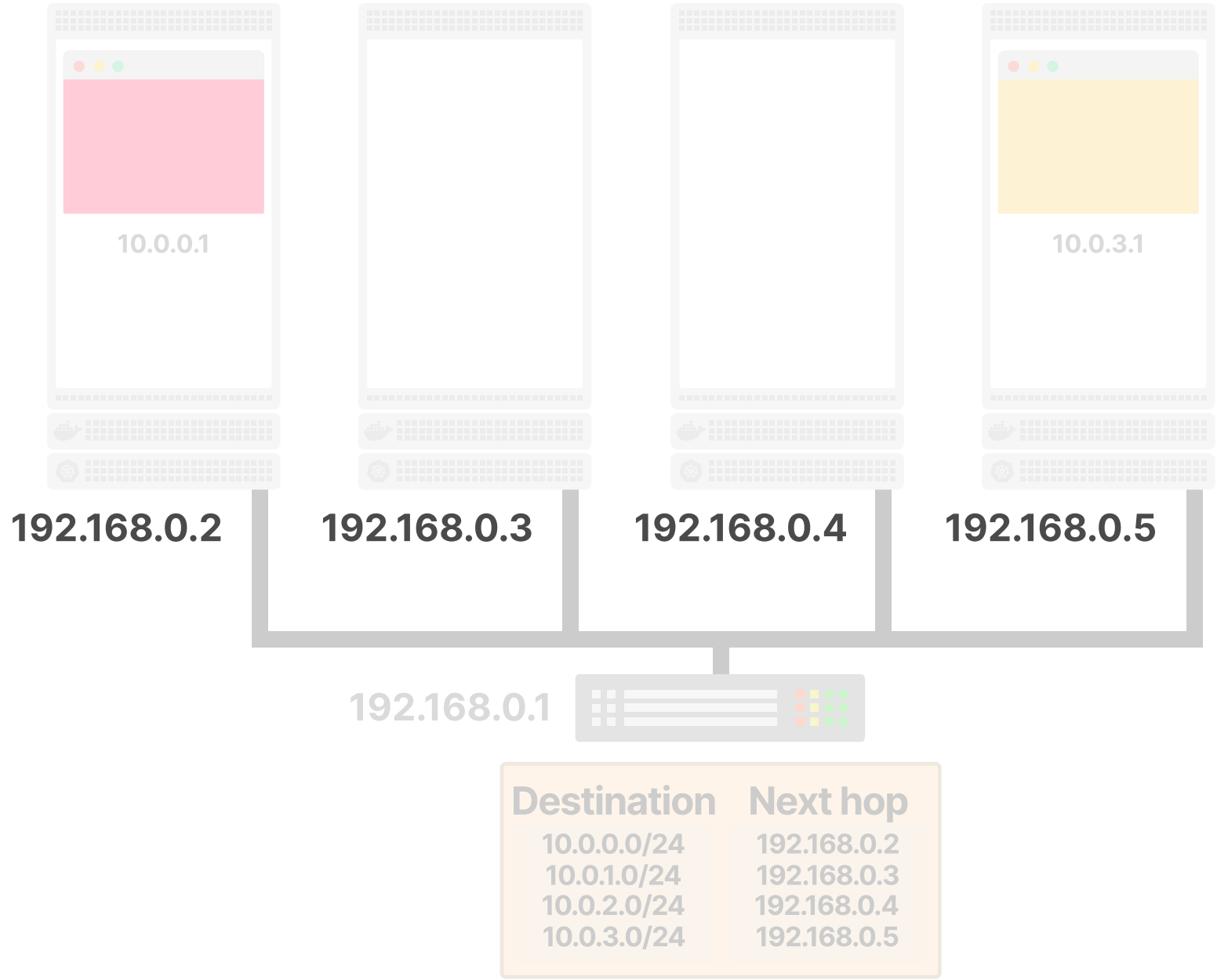
Cluster limits


10.0.0.1
10.0.3.1
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5
192.168.0.1
Destination
Next hop
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5


10.0.0.1
10.0.0.2
10.0.0.3
...
10.0.0.255
10.0.3.1
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5
192.168.0.1
Destination
Next hop
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5

254 Pods per node

Cluster limits


10.0.0.1
10.0.3.1
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5
192.168.0.1
Destination
Next hop
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5

254 Nodes

Cluster limits

Designing larger clusters

Larger network

Encryption

Firewalls and physical network

Designing larger clusters

Larger network

Encryption

Firewalls and physical network

Larger network

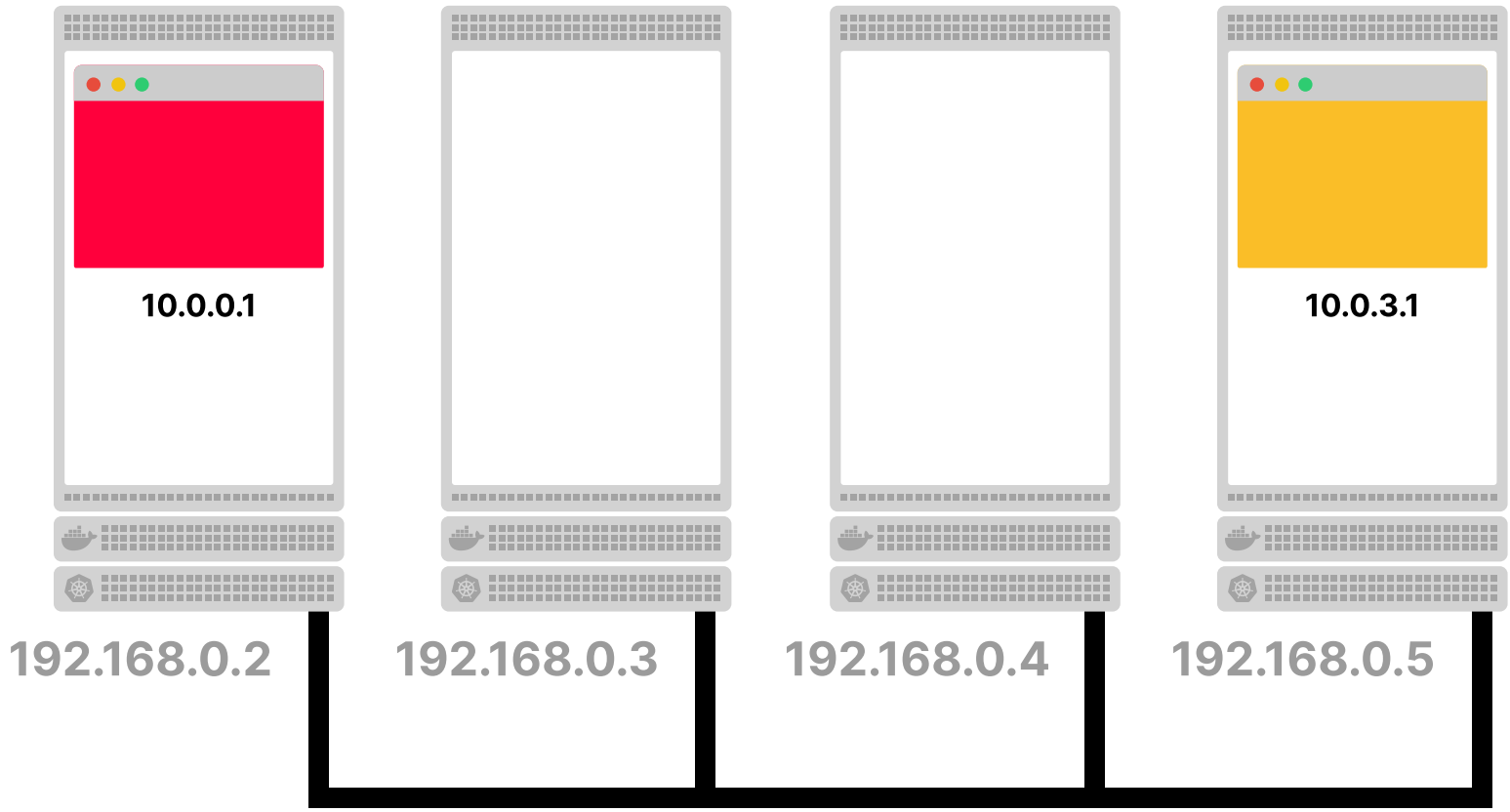
Encryption

Firewalls and physical network


flannel


weave net

Cluster network (2)



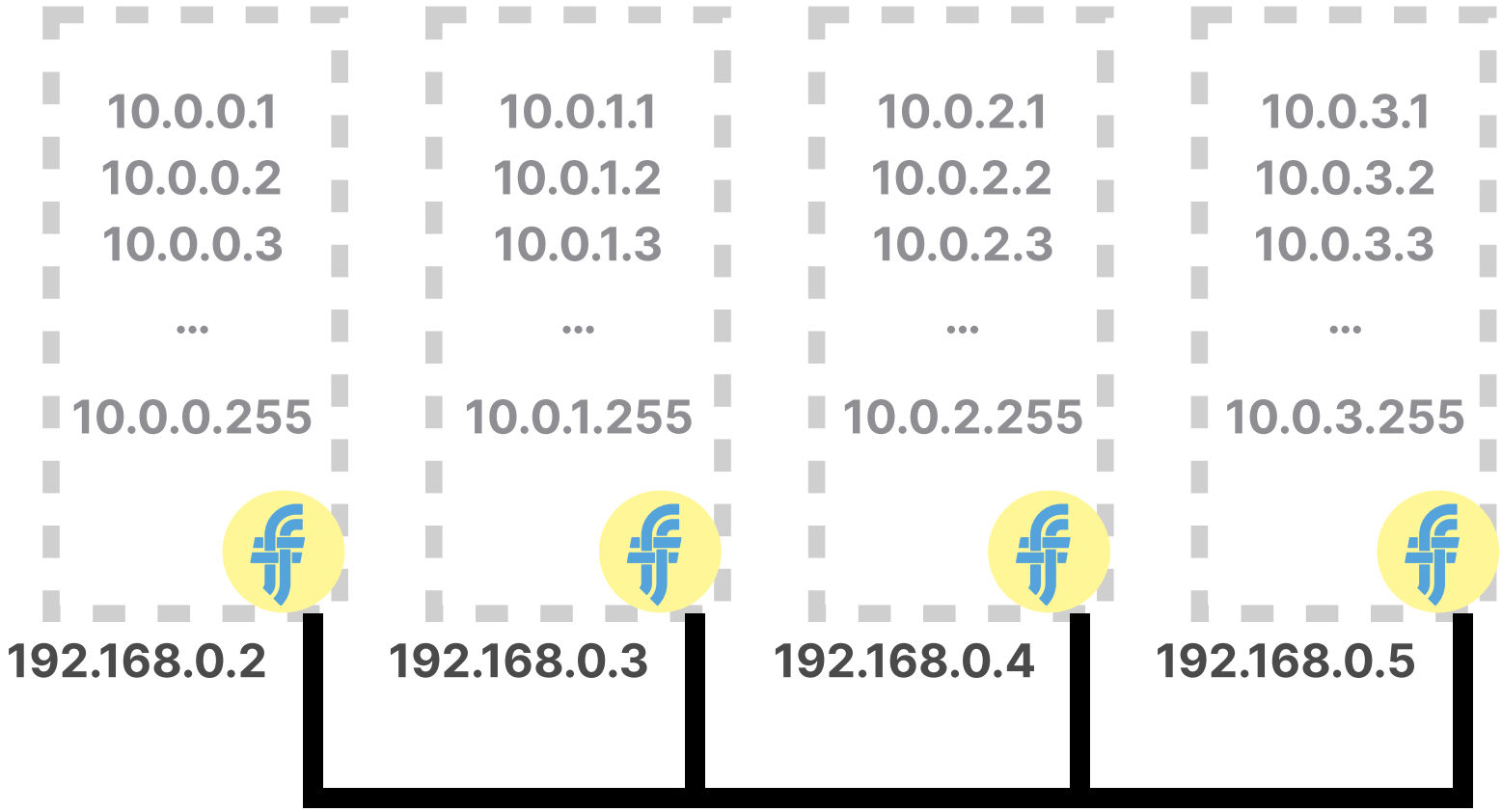
FROM:

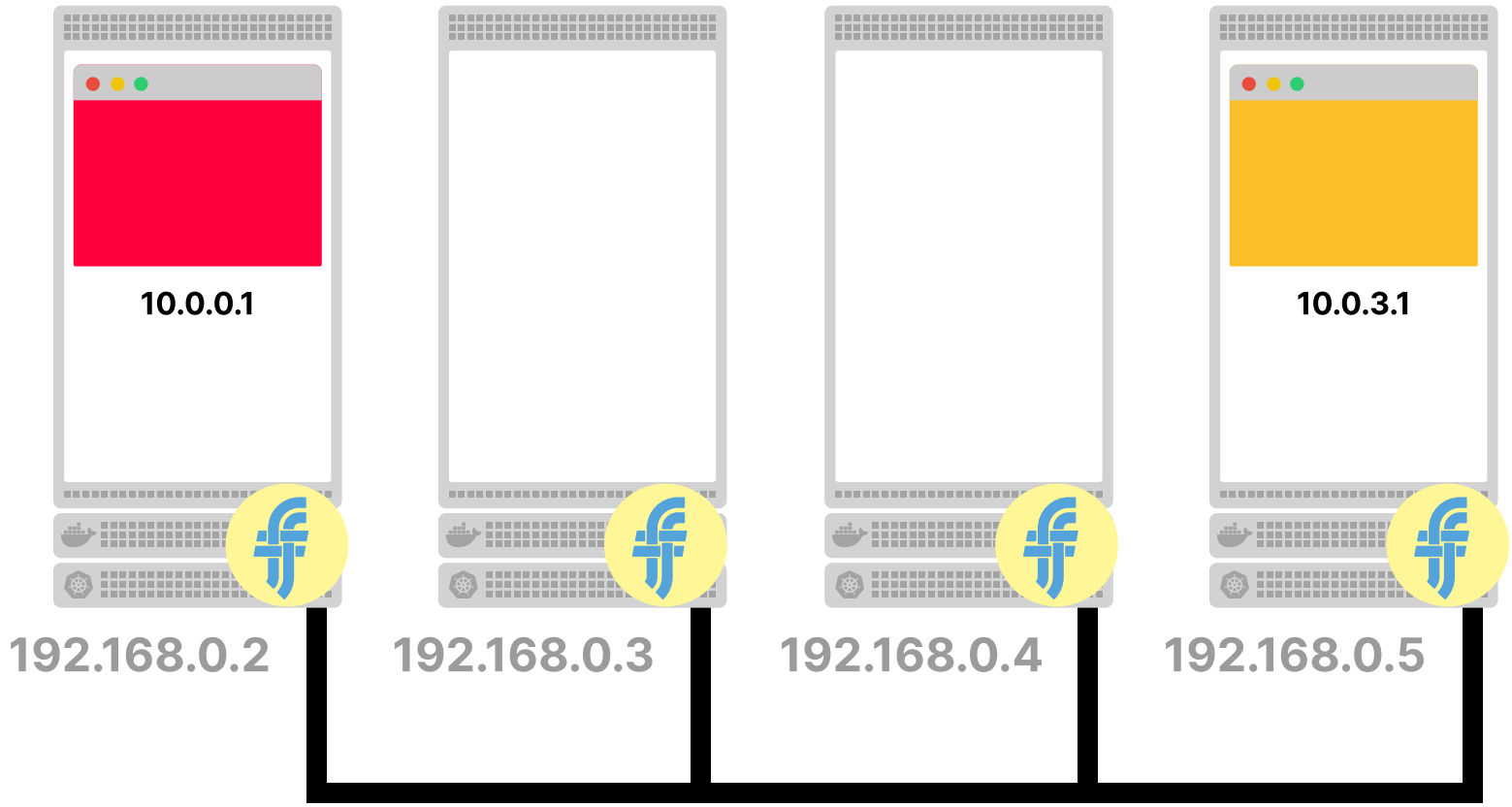
Pod ■ (10.0.0.1)

TO:

Pod ■ (10.0.3.1)

?


10.0.0.1
10.0.0.2
10.0.0.3
...
10.0.0.255
10.0.1.1
10.0.1.2
10.0.1.3
...
10.0.1.255
10.0.2.1
10.0.2.2
10.0.2.3
...
10.0.2.255
10.0.3.1
10.0.3.2
10.0.3.3
...
10.0.3.255
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5



FROM:

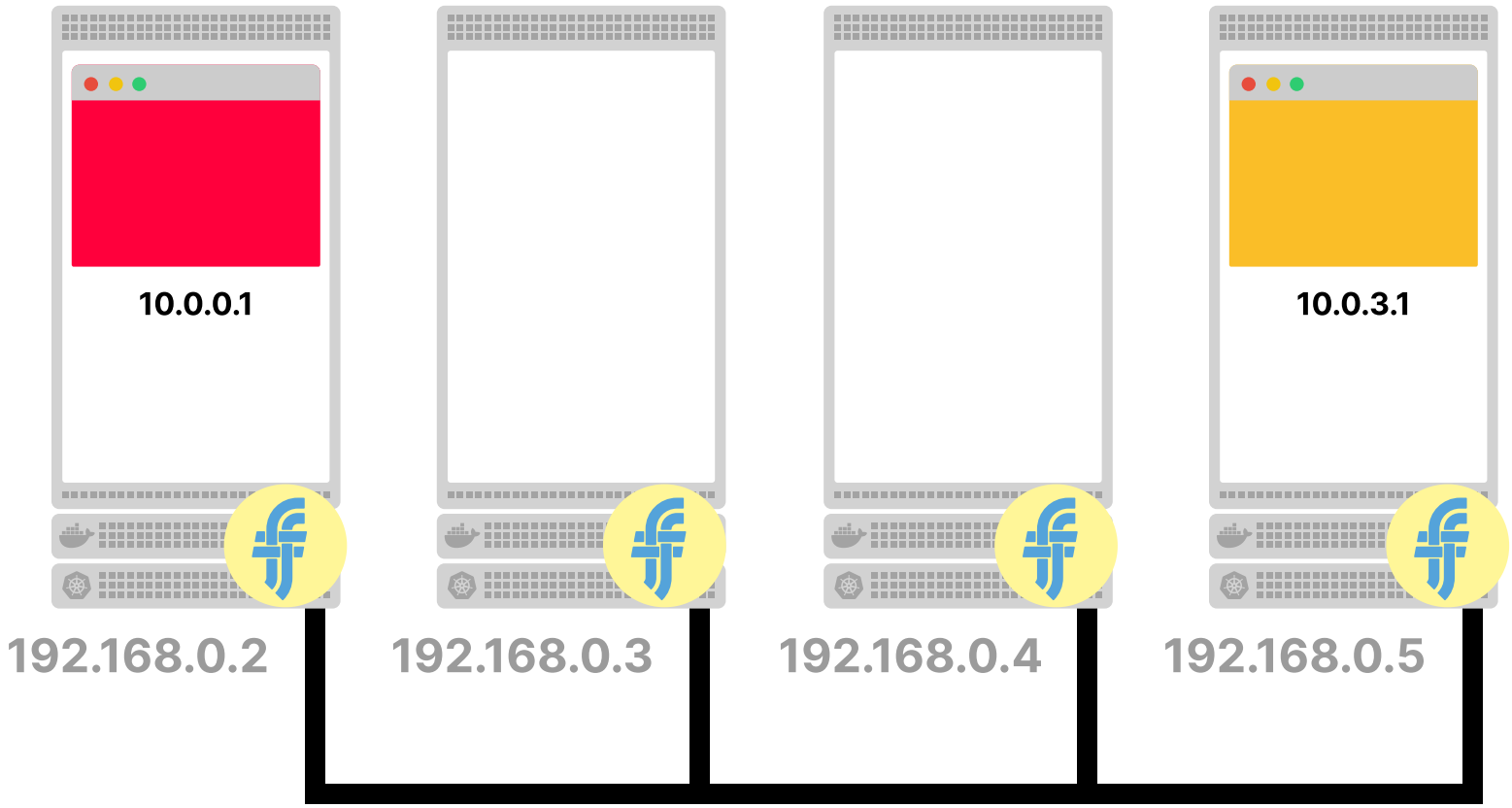
Pod ■ **(10.0.0.1)**

TO:

Pod ■ **(10.0.3.1)**

FROM: Pod1
TO: Pod2

Payload

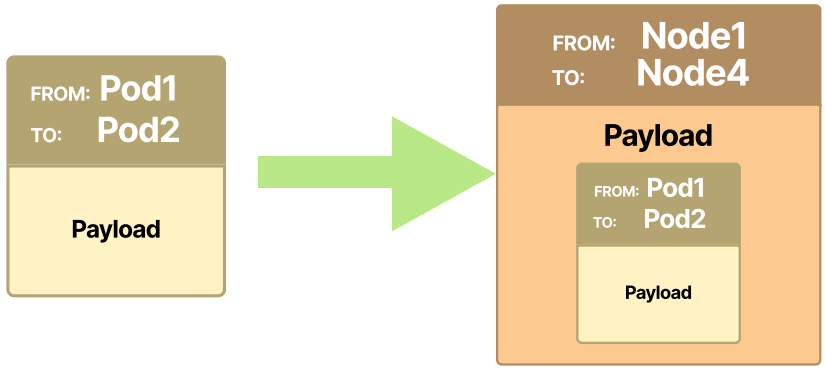


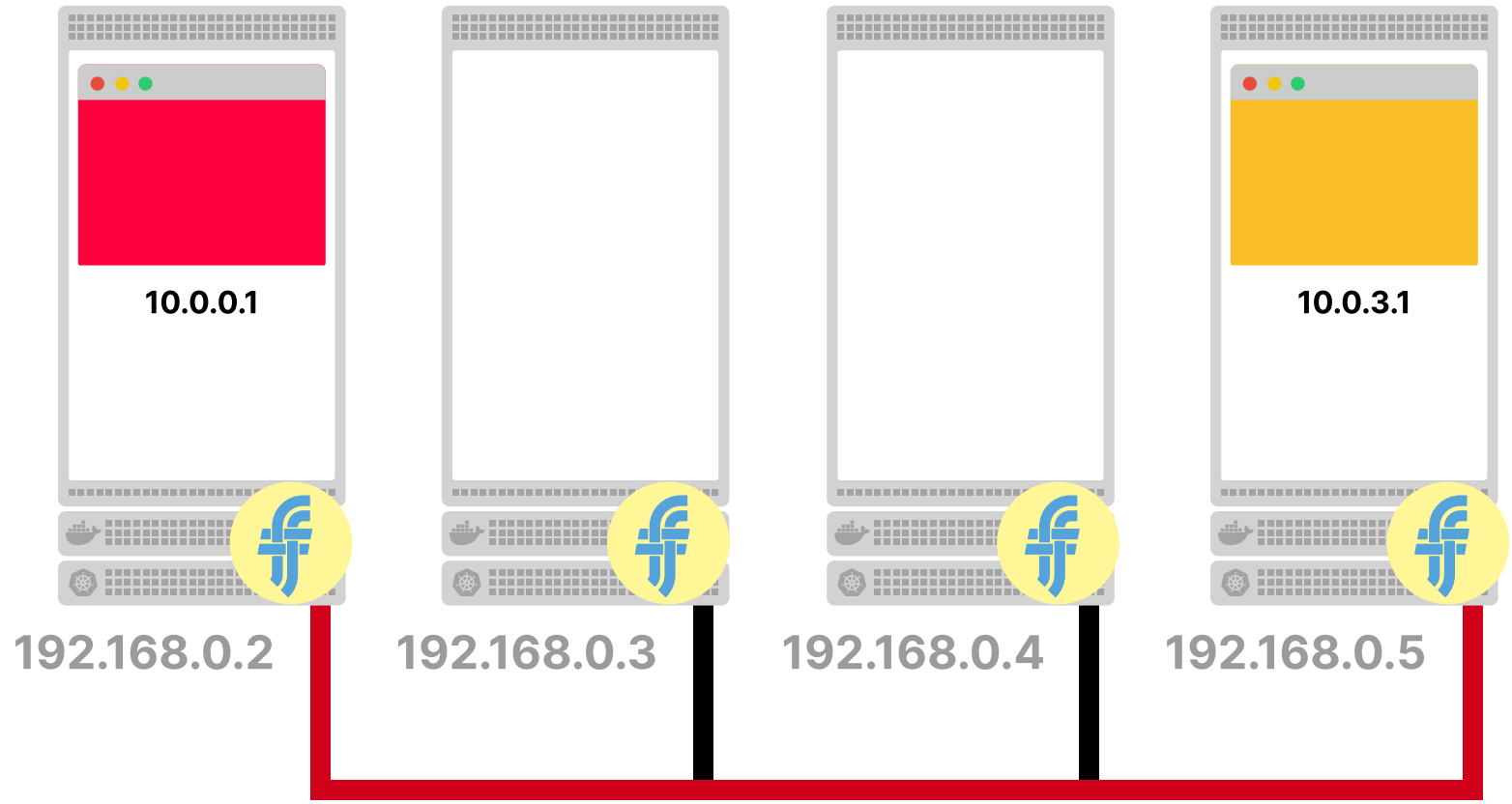
FROM:

Pod ■ **(10.0.0.1)**

TO:

Pod ■ **(10.0.3.1)**



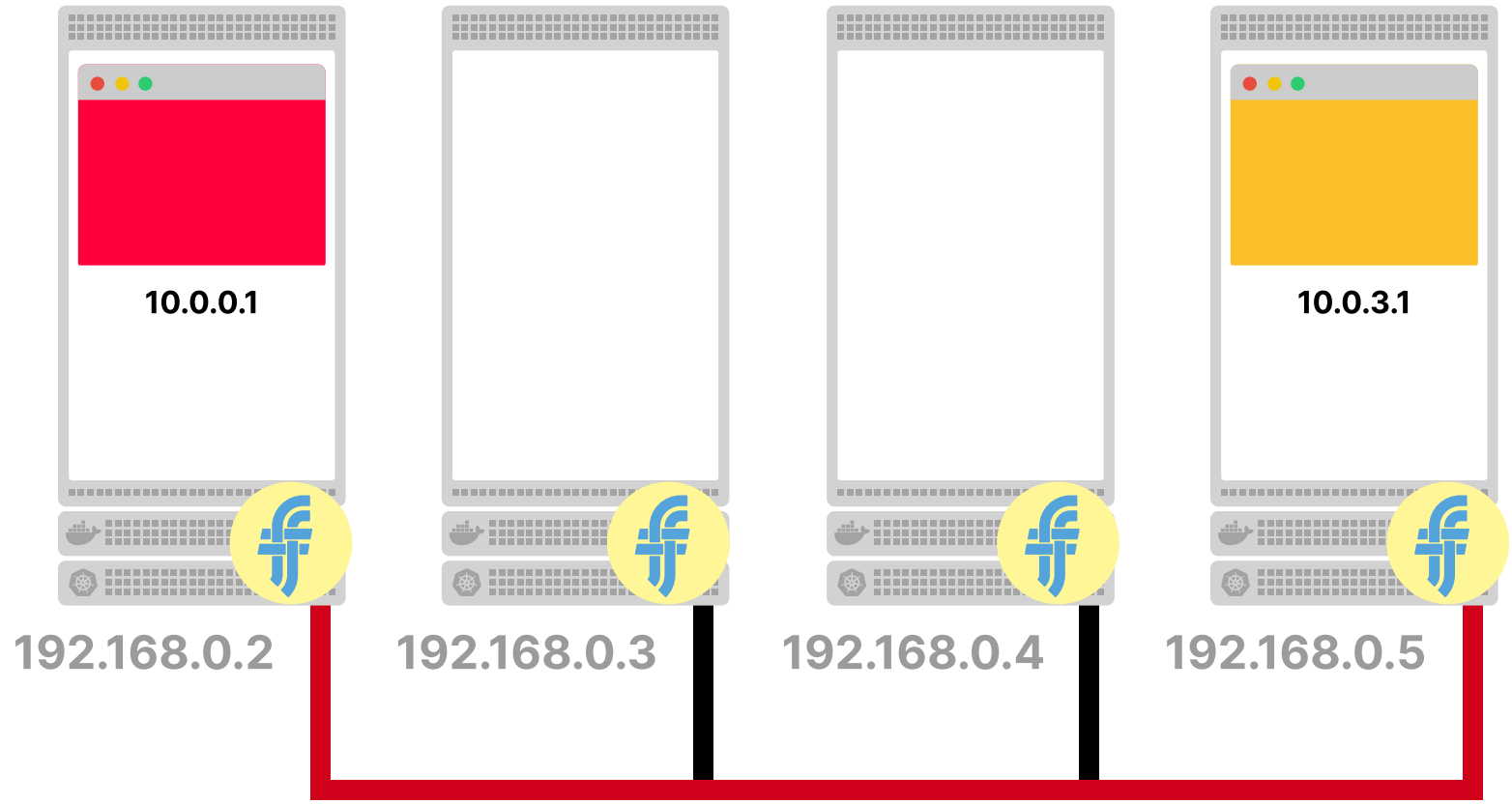


FROM:

Pod (10.0.0.1)

TO:

Pod (10.0.3.1)

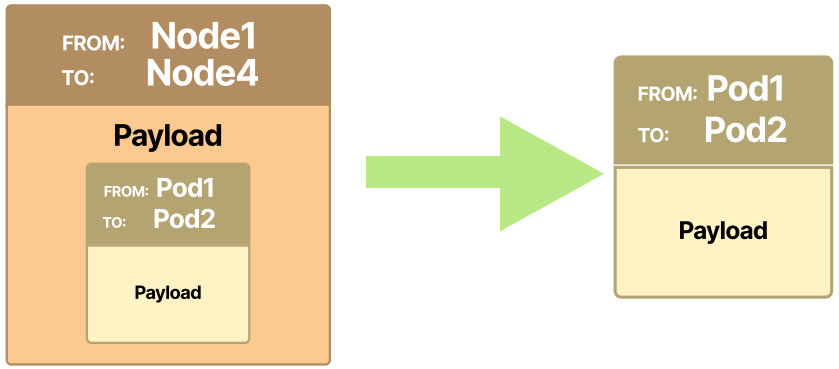


FROM:

Pod ■ **(10.0.0.1)**

TO:

Pod ■ **(10.0.3.1)**



Overlay trade-offs

Overlay trade-offs

Dynamic gateway

Software Defined Network

Nested network

Simpler setup

Overlay trade-offs

Dynamic gateway

Software Defined Network

Nested network

Simpler setup

Overlay trade-offs

Dynamic gateway

Software Defined Network

Nested network

Simpler setup

Overlay trade-offs

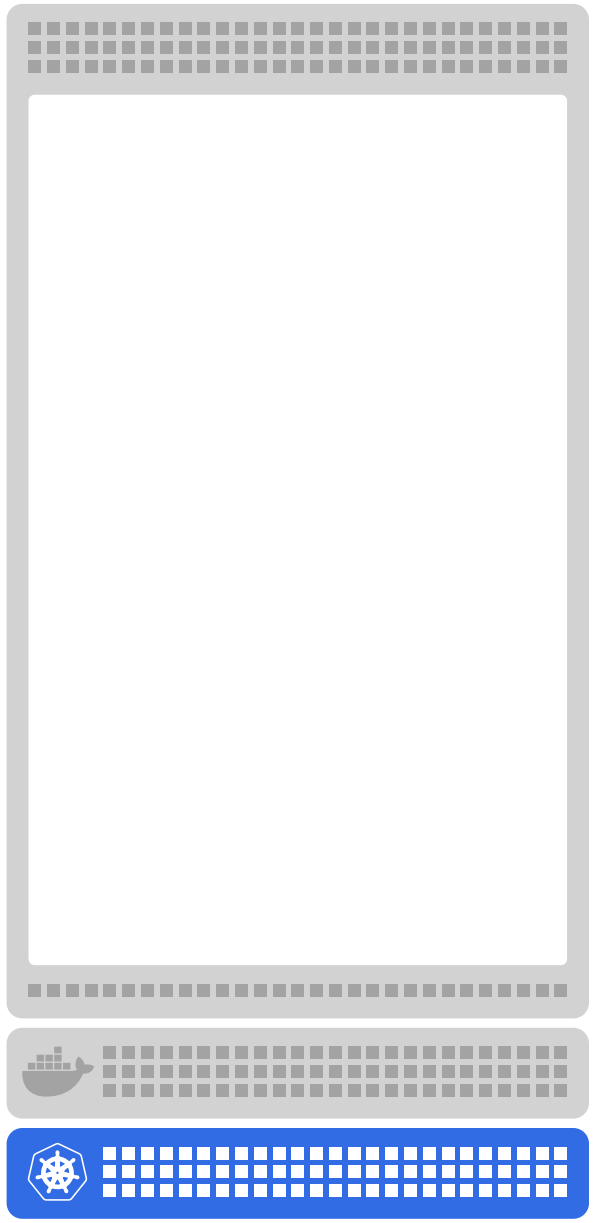
Dynamic gateway

Software Defined Network

Nested network

Simpler setup

Assigning IP addresses



Assigning IP addresses

1. Create the container
2. Attach it to the network
3. Attach volumes

Assigning IP addresses

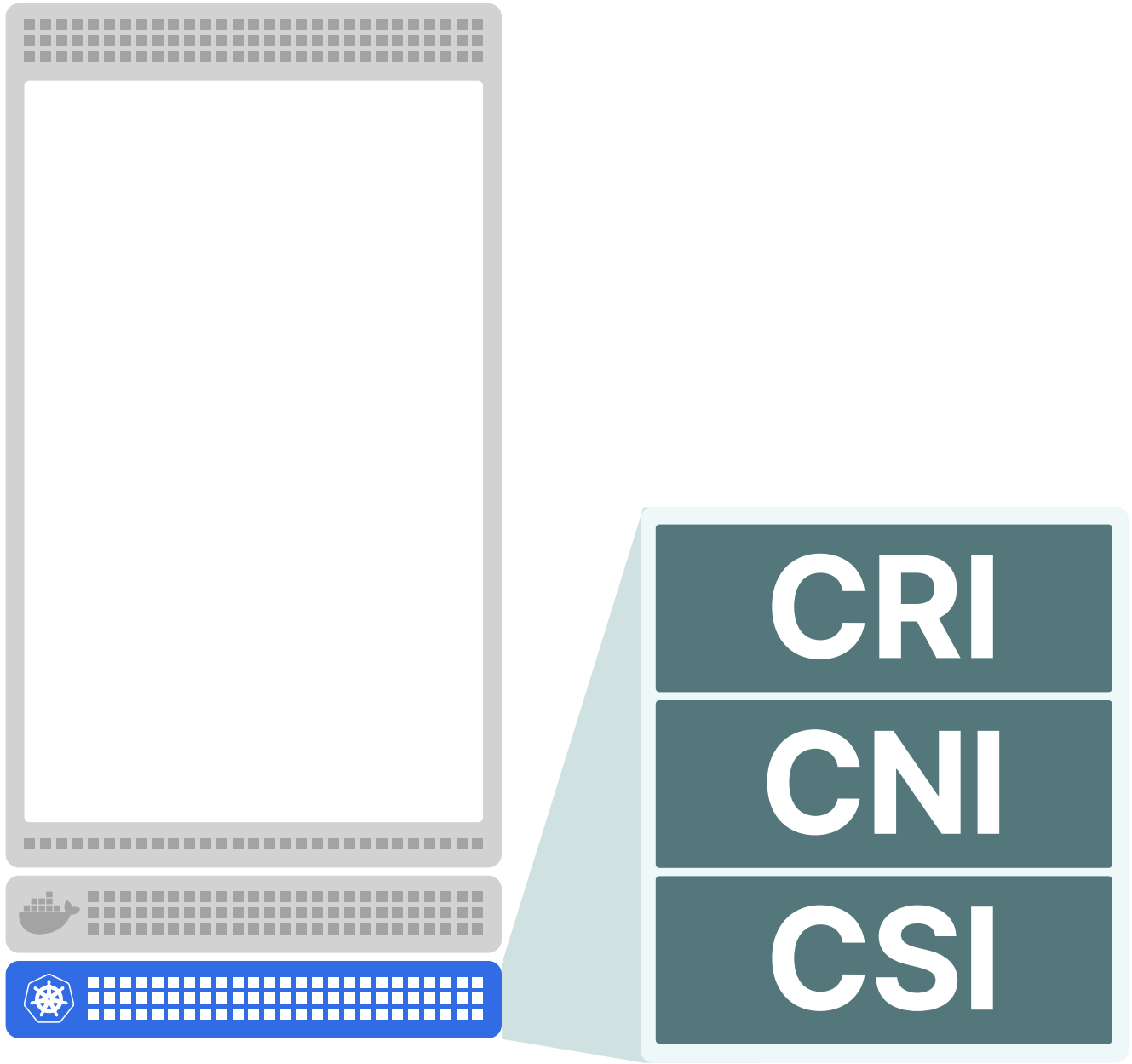
1. Create the container
2. **Attach it to the network**
3. Attach volumes

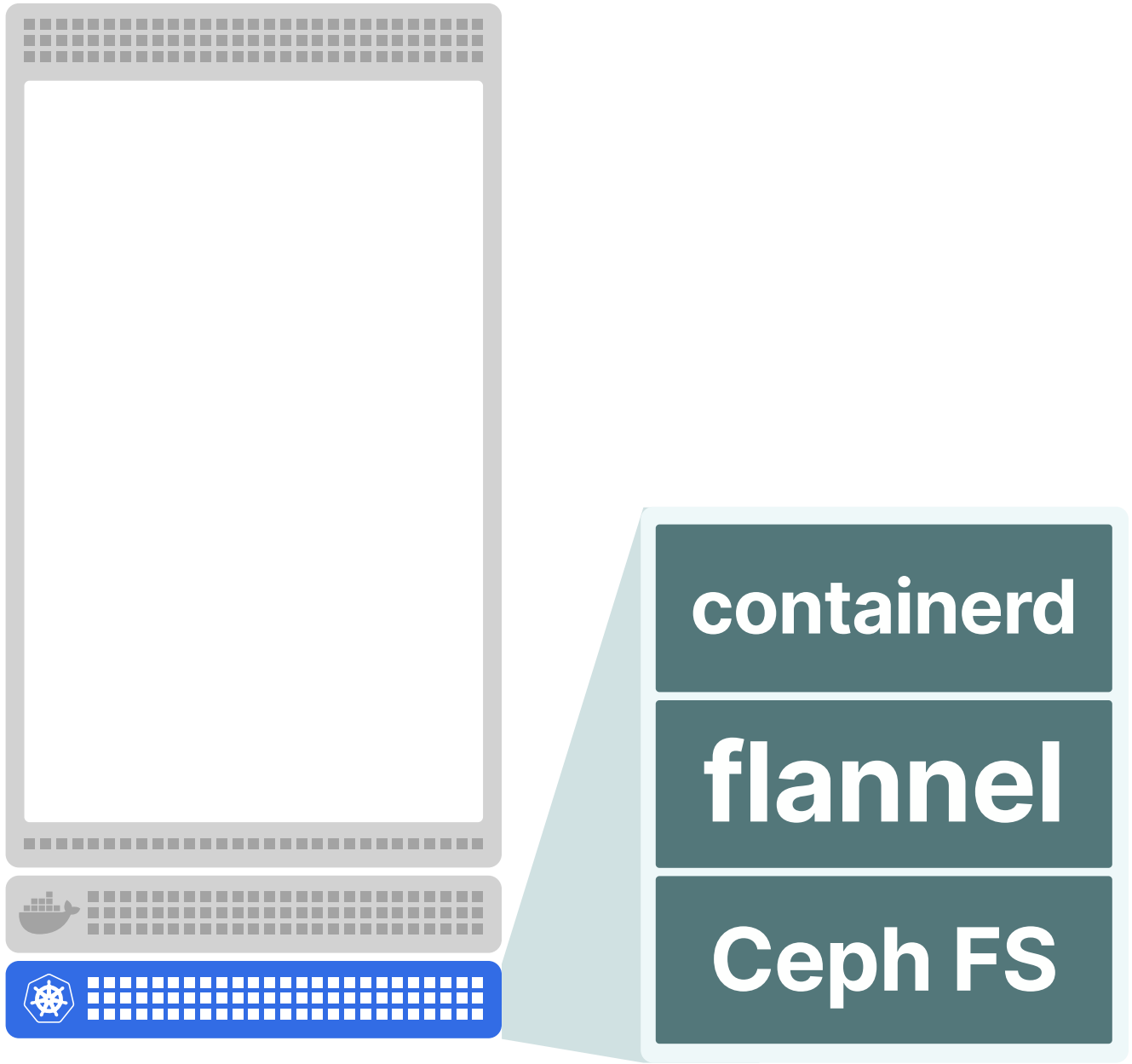
Assigning IP addresses

1. Create the container
2. Attach it to the network
3. **Attach volumes**

Assigning IP addresses

1. Container Runtime Interface
2. Container Network Interface
3. Container Storage Interface


CRI
CNI
CSI


containerd
flannel
Ceph FS

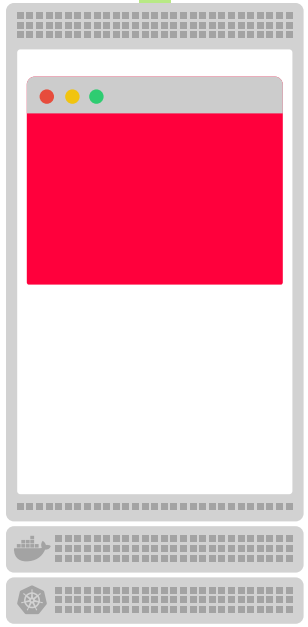
kubelet keeps the master node up to date

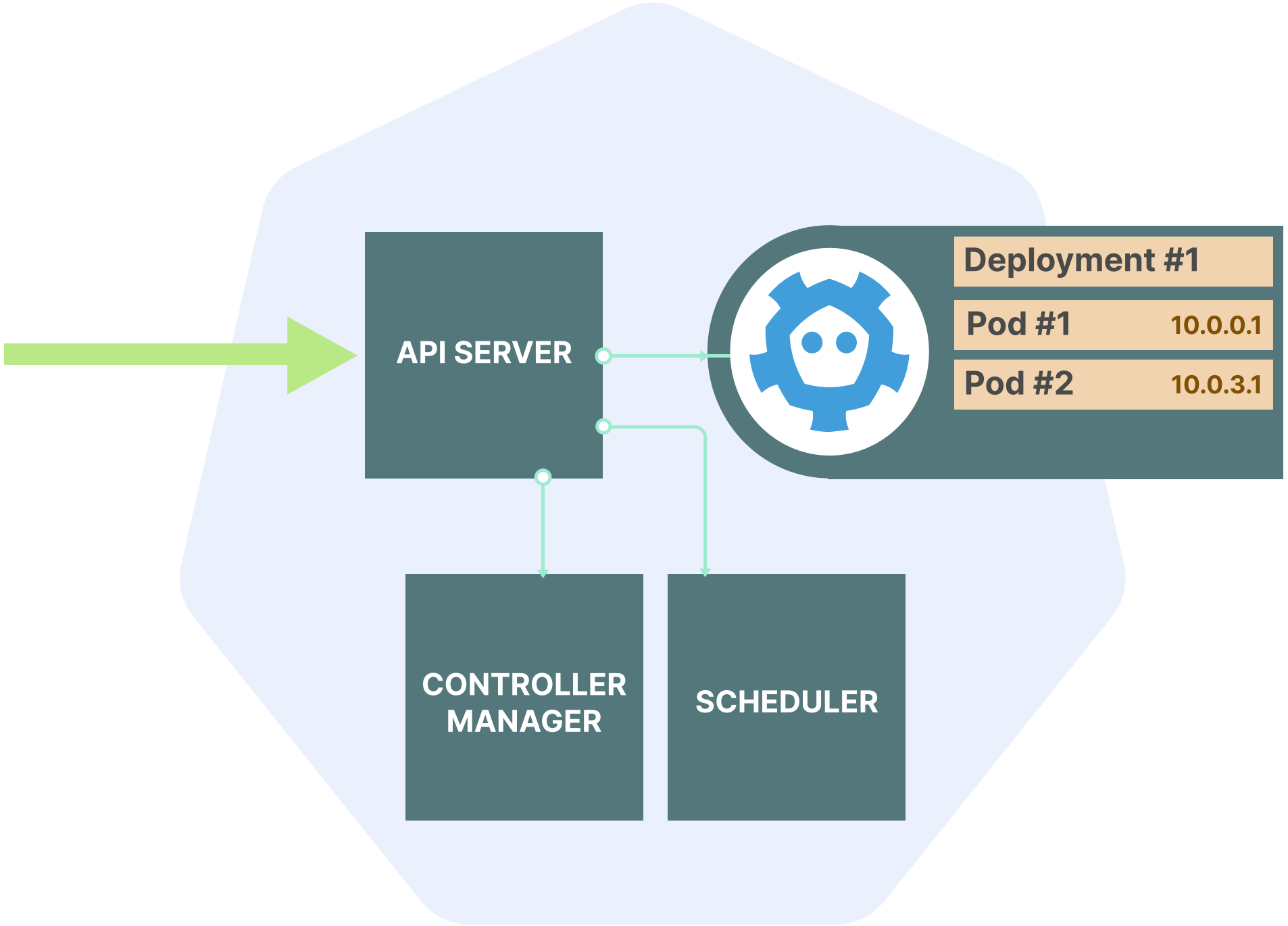
Assigning IP addresses

Create a Deployment with 2 replicas





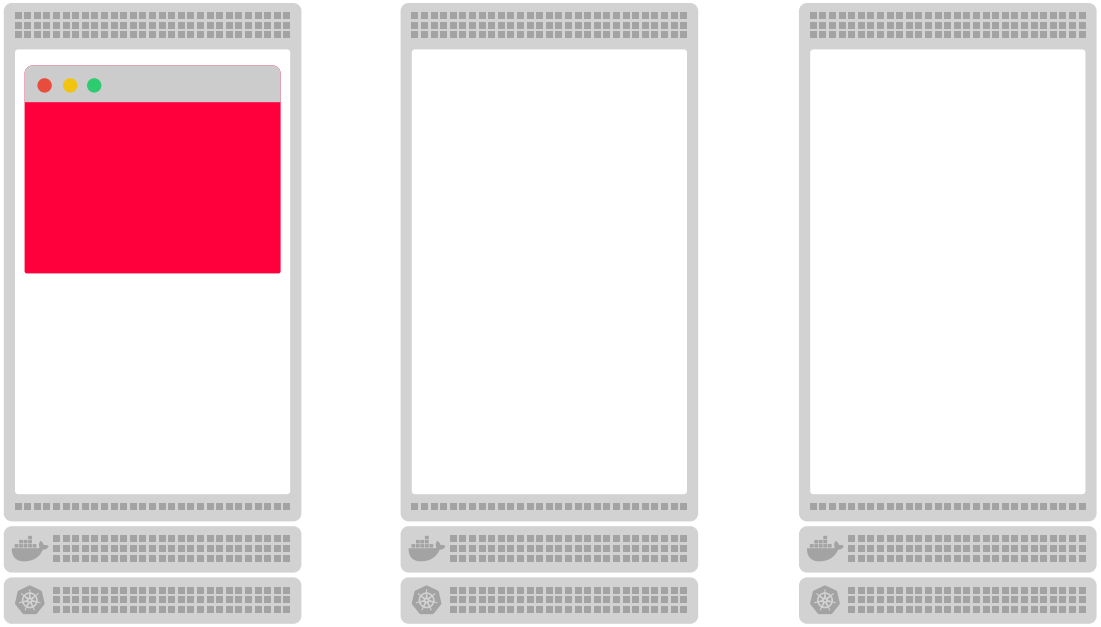



API SERVER
Deployment #1
Pod #1
10.0.0.1
Pod #2
10.0.3.1
CONTROLLER MANAGER
SCHEDULER

podIP in practice

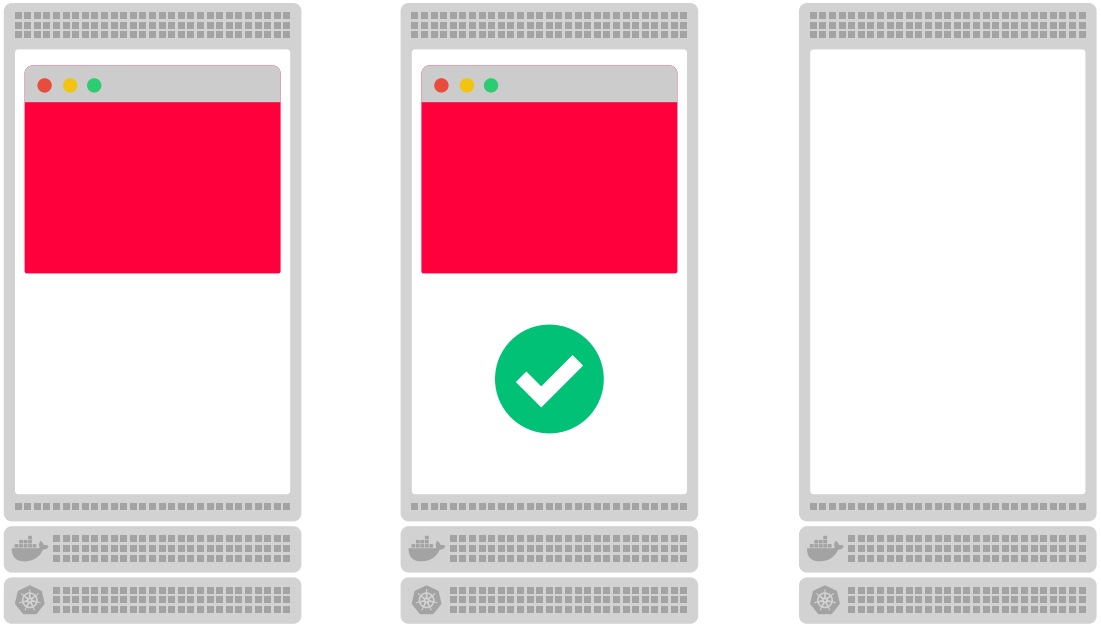


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1



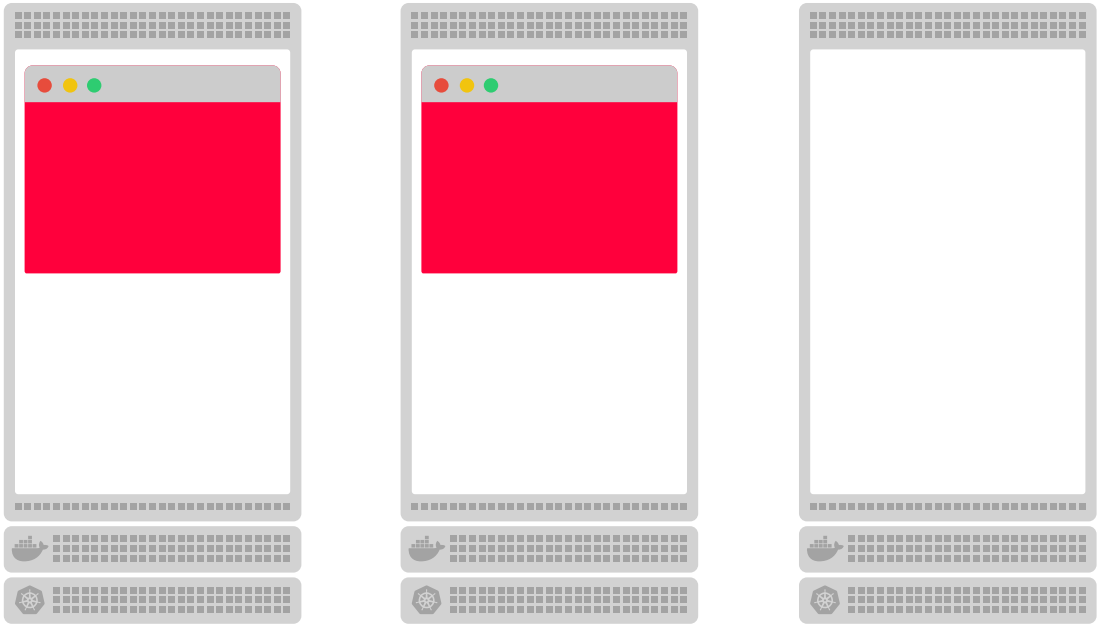


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1



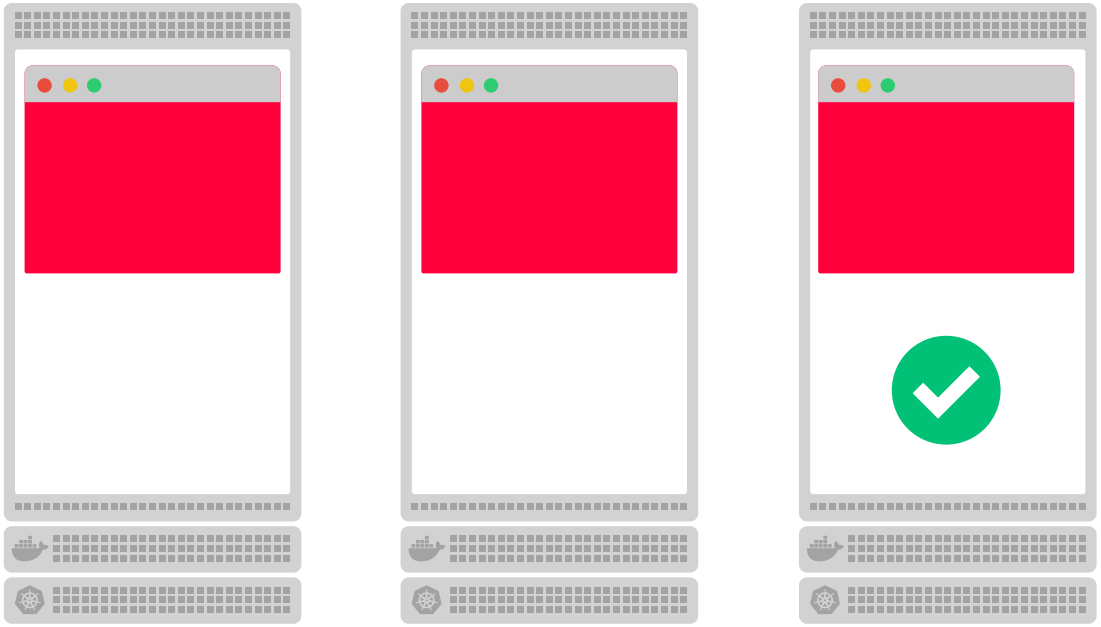


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1



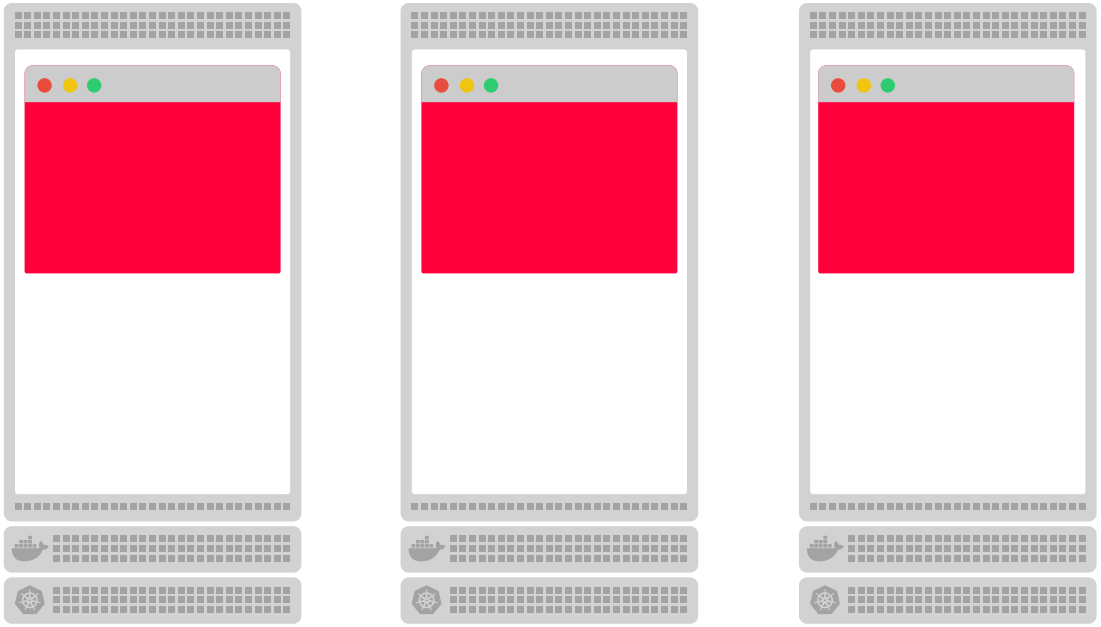


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1



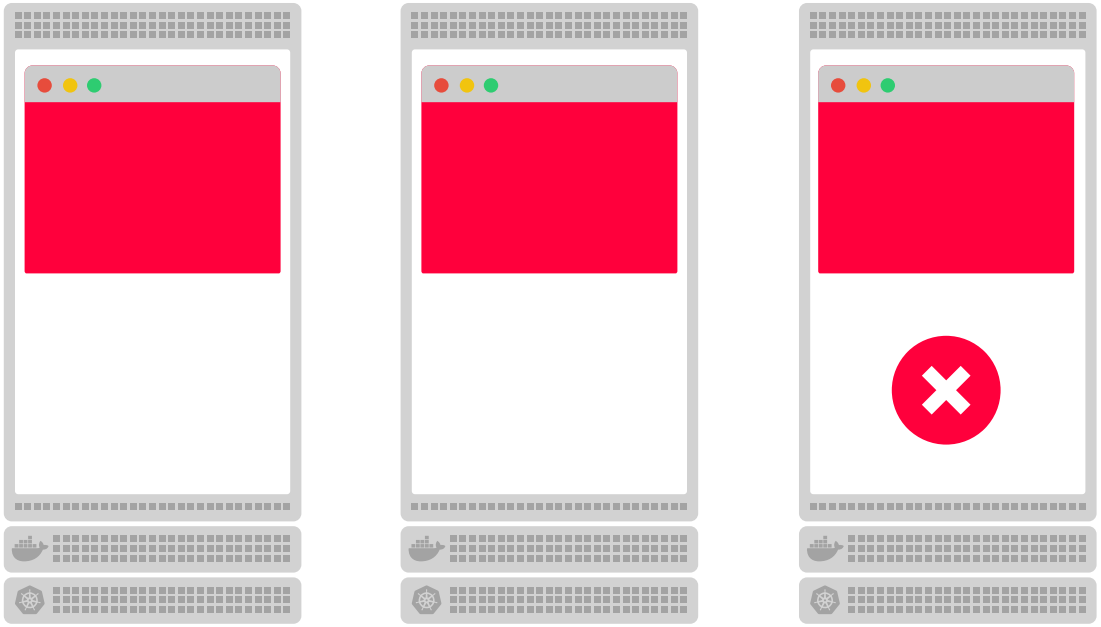


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1



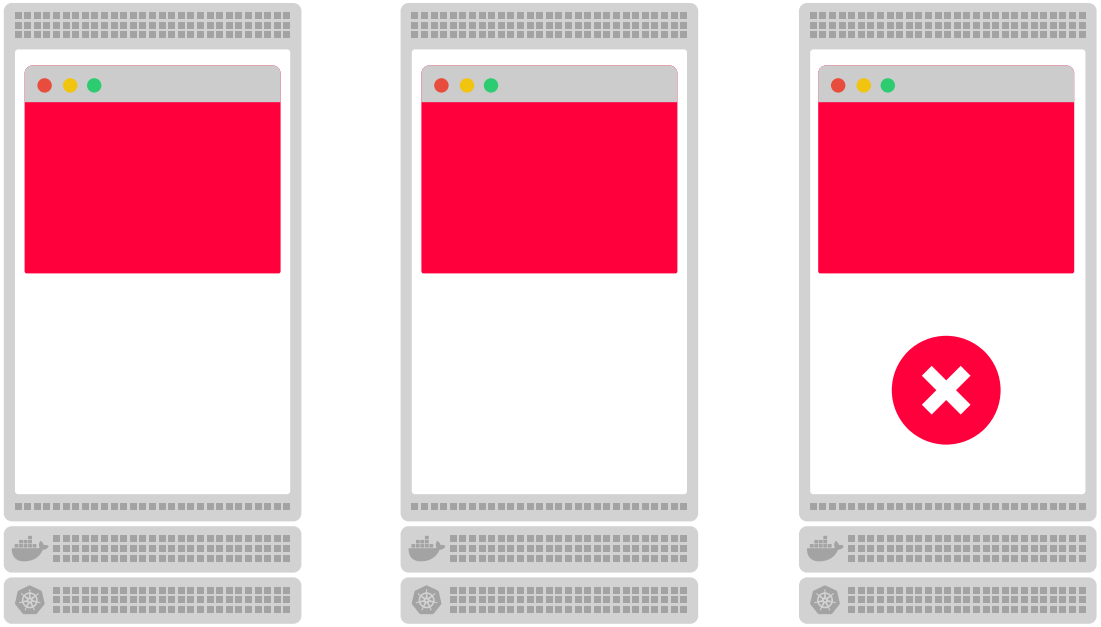


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1



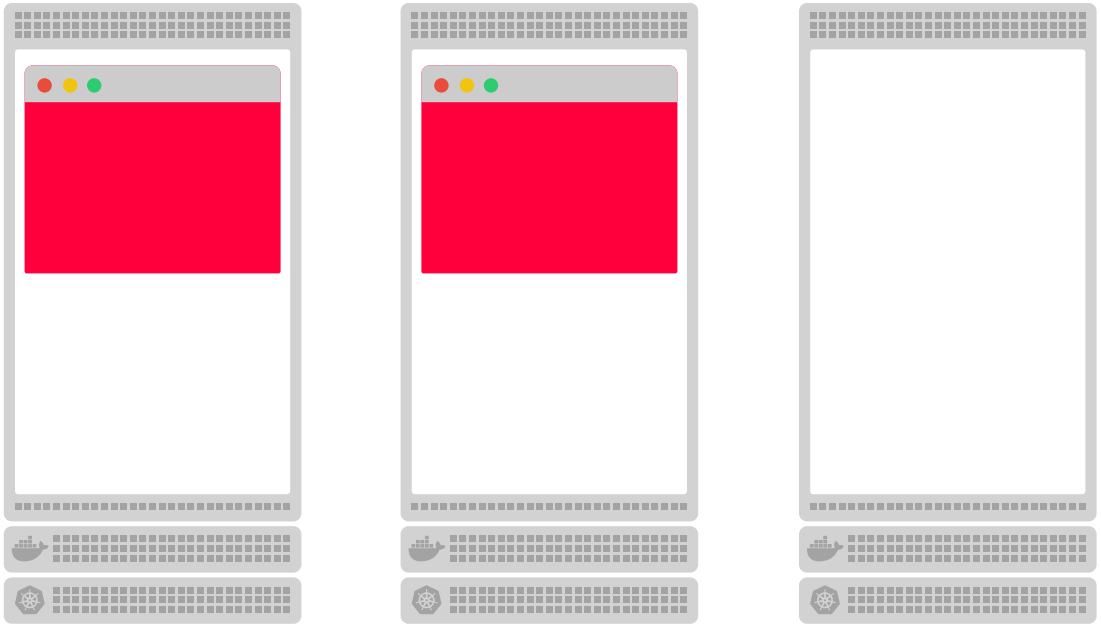


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1





Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1



Recap

Cluster networking

Recap — Cluster networking

1. IP address for every Pod

2. CNI attaches container to the network

3. podIP is stored in etcd

4. Several cluster network implementations

Recap — Cluster networking

1. IP address for every Pod

2. CNI attaches container to the network

3. podIP is stored in etcd

4. Several cluster network implementations

Recap — Cluster networking

1. IP address for every Pod

2. CNI attaches container to the network

3. podIP is stored in etcd

4. Several cluster network implementations

Recap — Cluster networking

1. IP address for every Pod

2. CNI attaches container to the network

3. podIP is stored in etcd

4. Several cluster network implementations

Recap — Cluster networking

5. Every Pod can talk to every Pod

6. Agents on a Node can talk to Pods on that Node

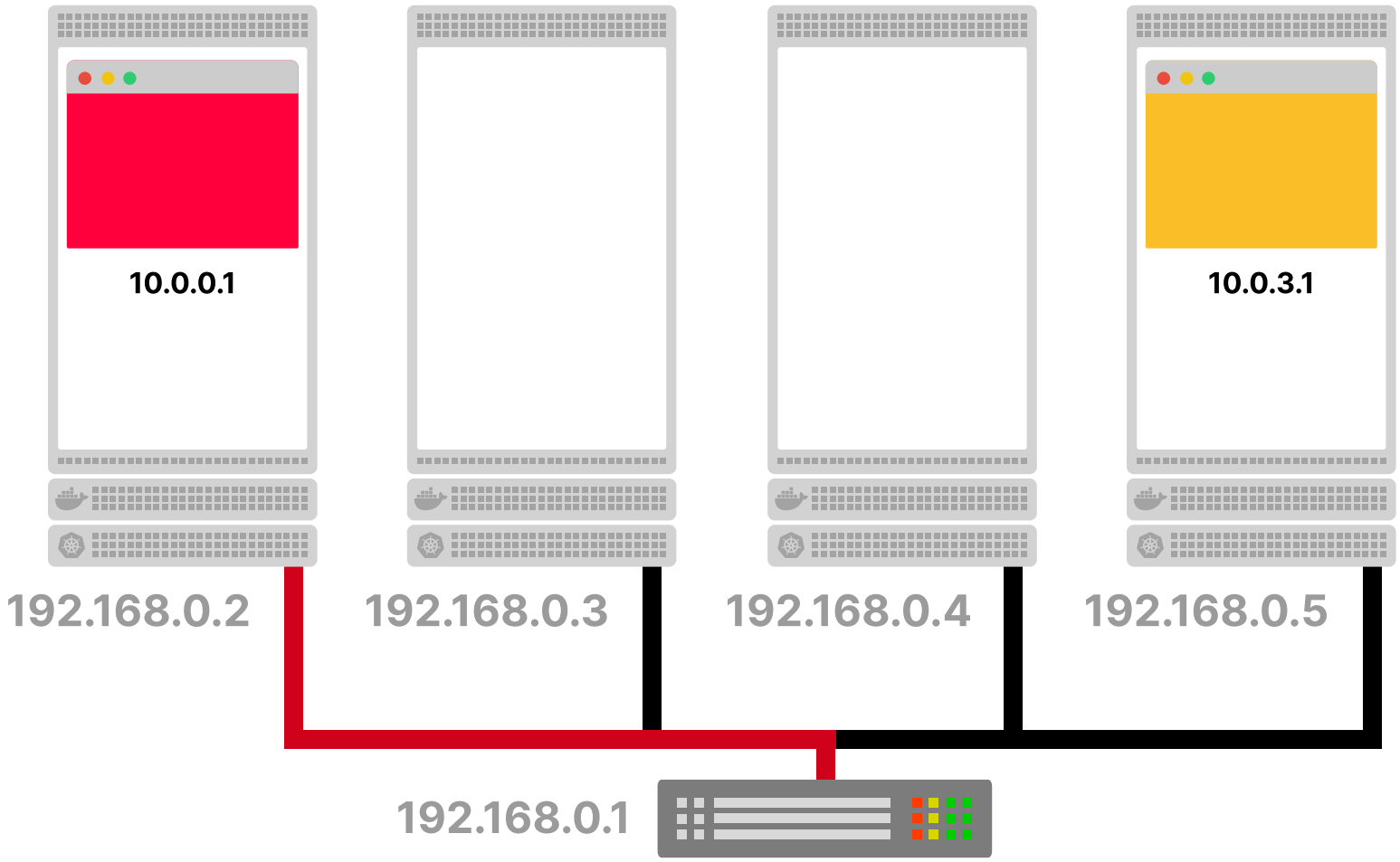
Recap — Cluster networking

5. Every Pod can talk to every Pod

6. Agents on a Node can talk to Pods on that Node

Internal traffic

02

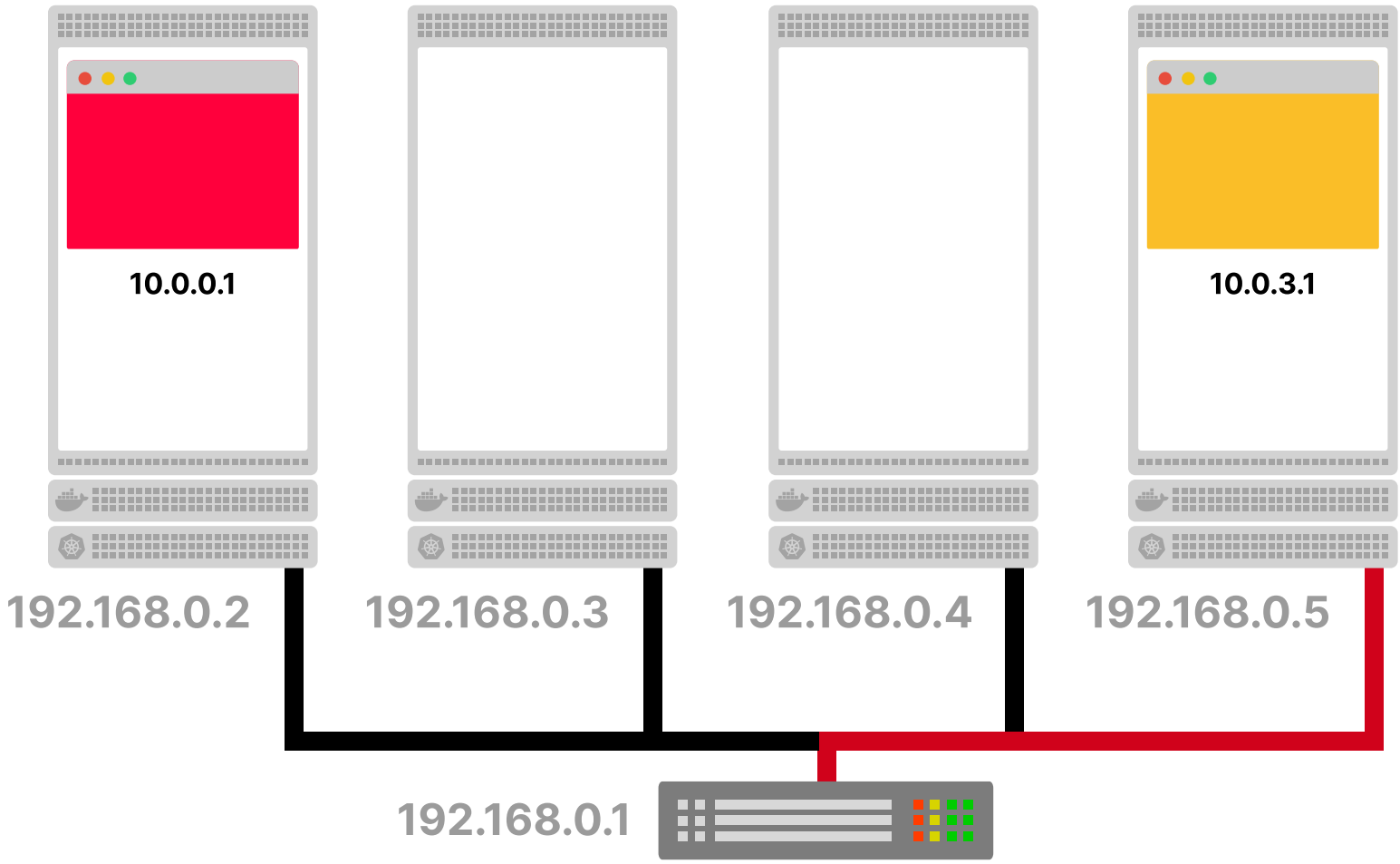


FROM:

Pod ■ (10.0.0.1)

TO:

Pod ■ (10.0.3.1)



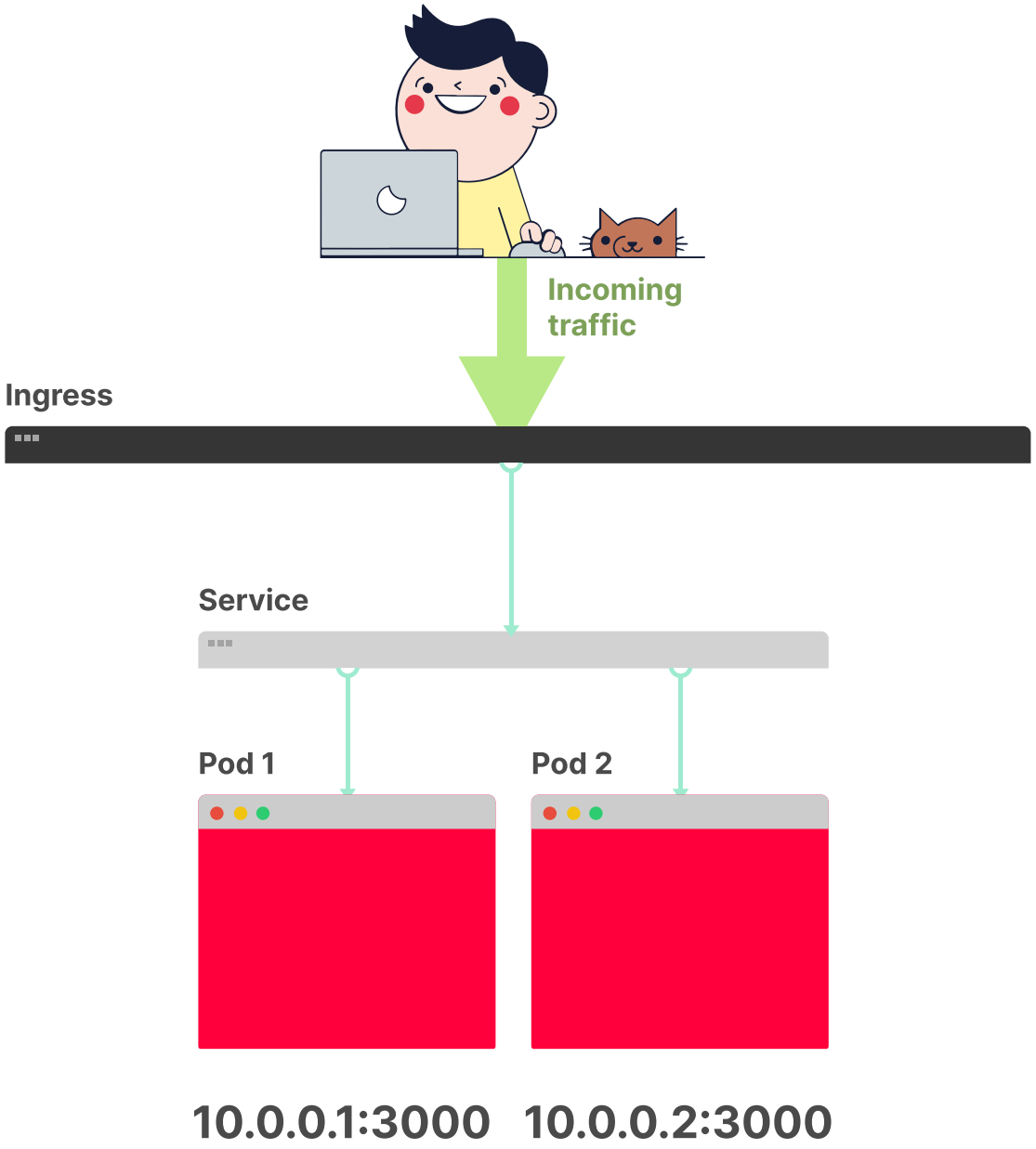
FROM:

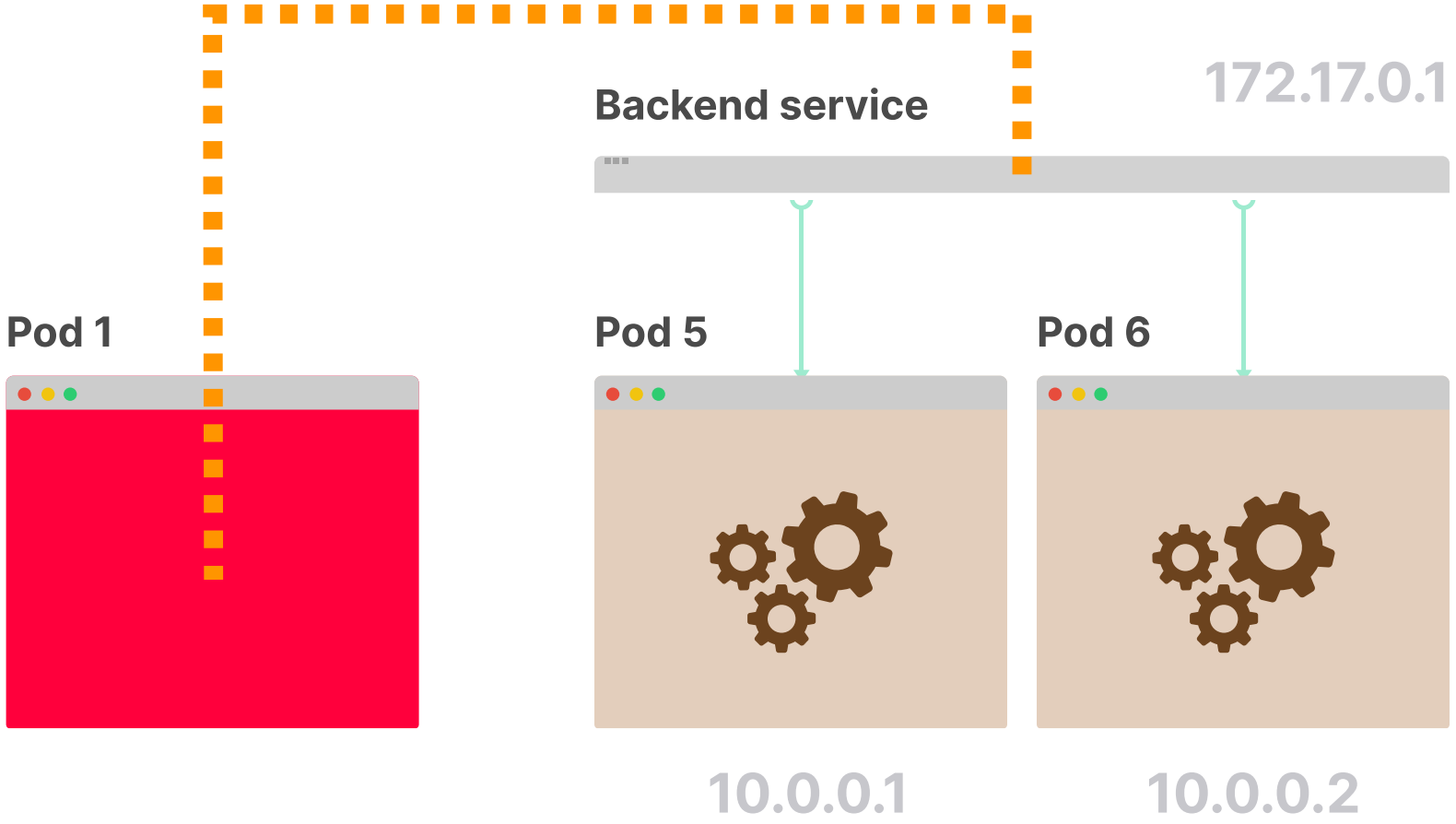
Pod (10.0.0.1)

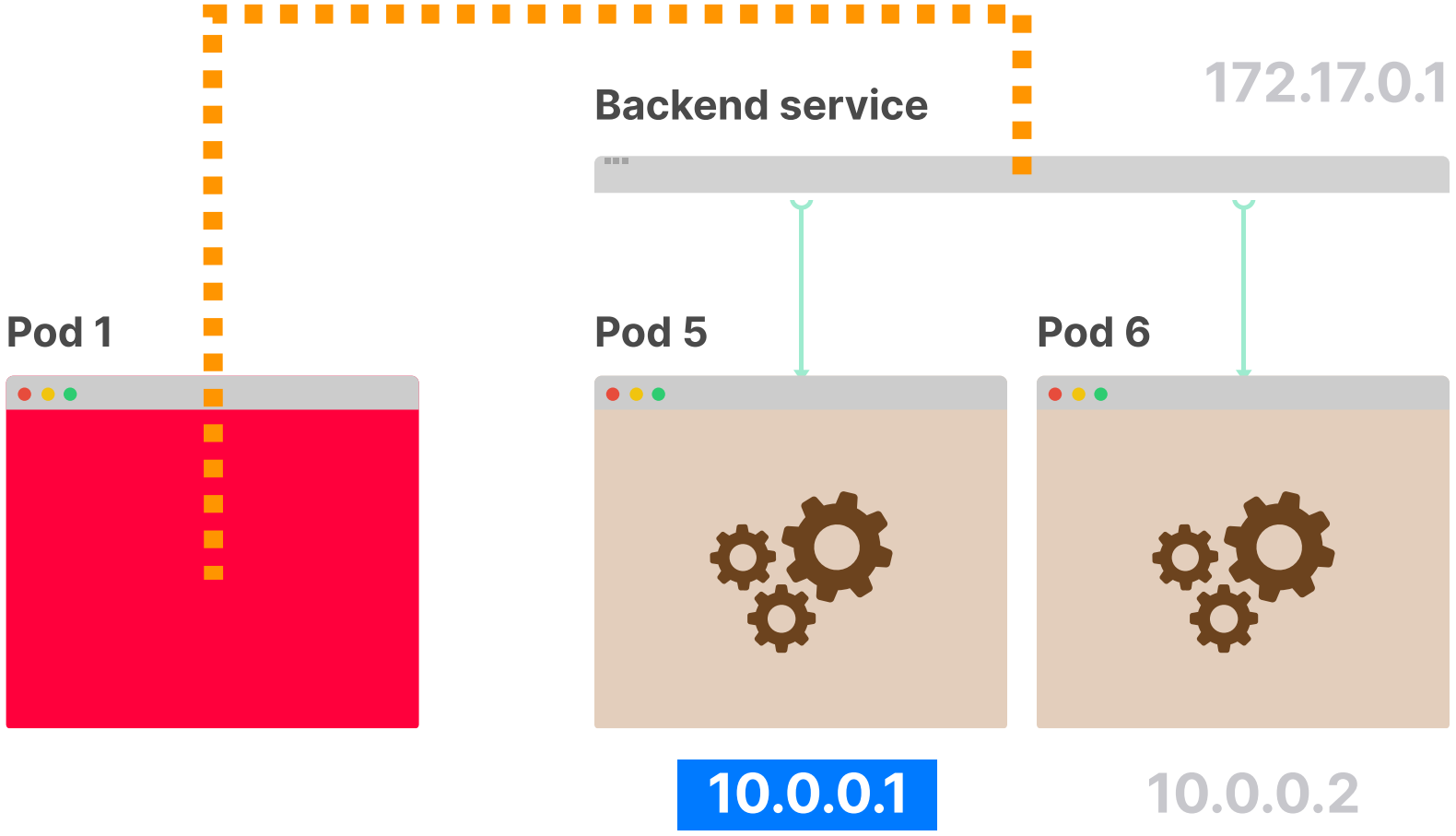
TO:

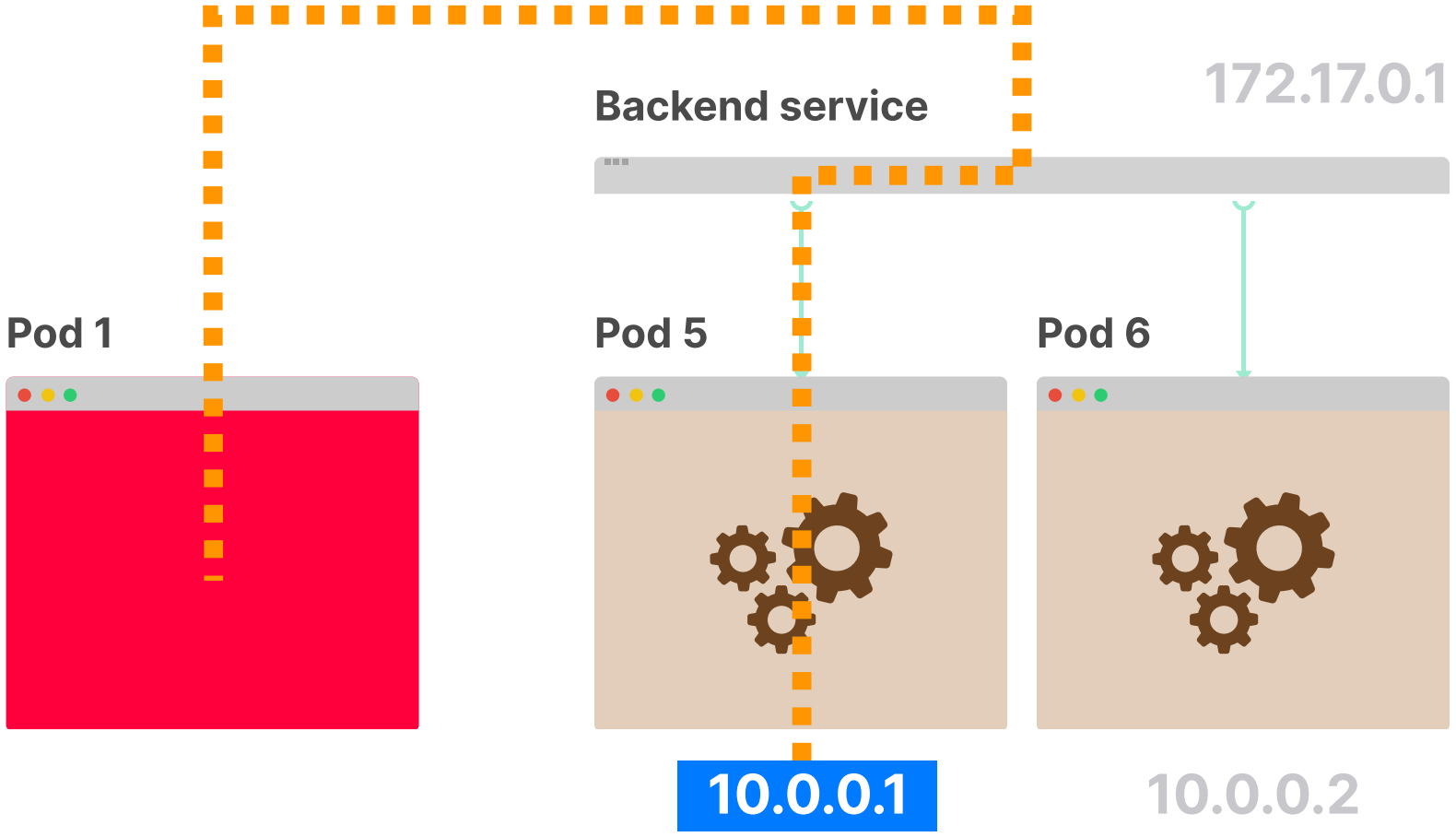
Pod (10.0.3.1)

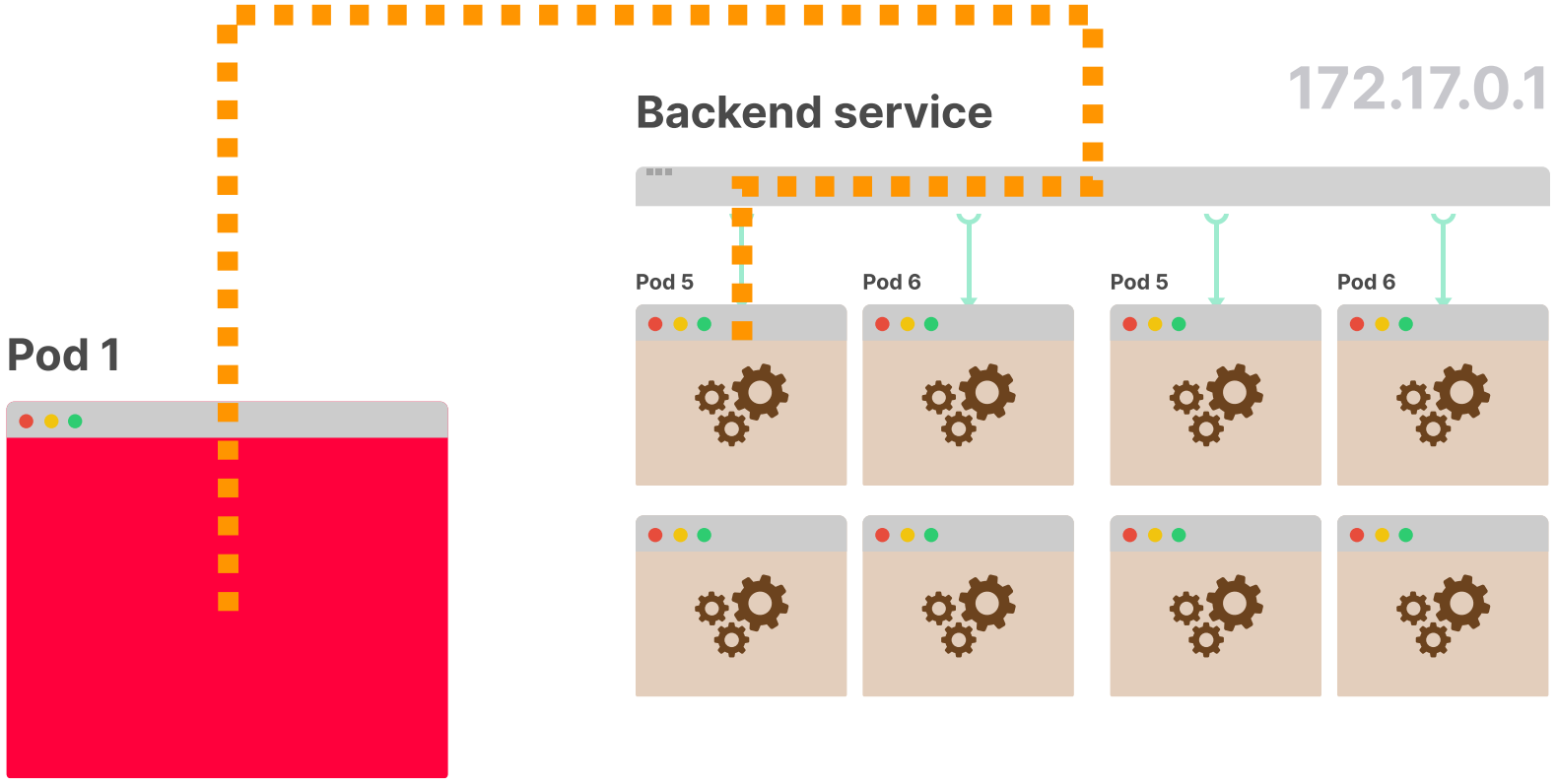
Services


Incoming traffic
Ingress
Service
Pod 1
Pod 2
10.0.0.1:3000
10.0.0.2:3000

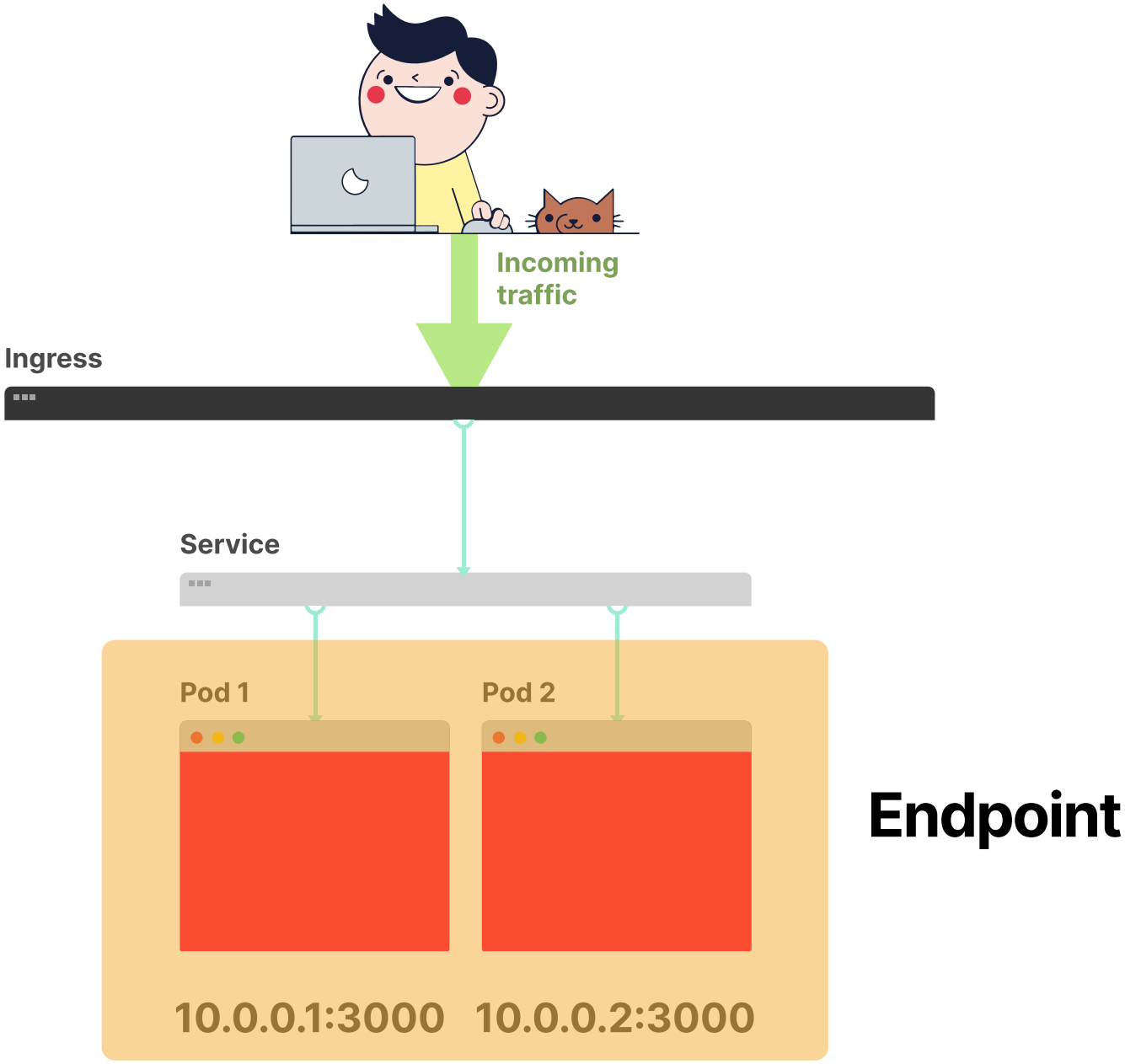

Backend service
172.17.0.1
Pod 1
Pod 5
Pod 6
10.0.0.1
10.0.0.2


Backend service
172.17.0.1
Pod 1
Pod 5
Pod 6
10.0.0.1
10.0.0.2

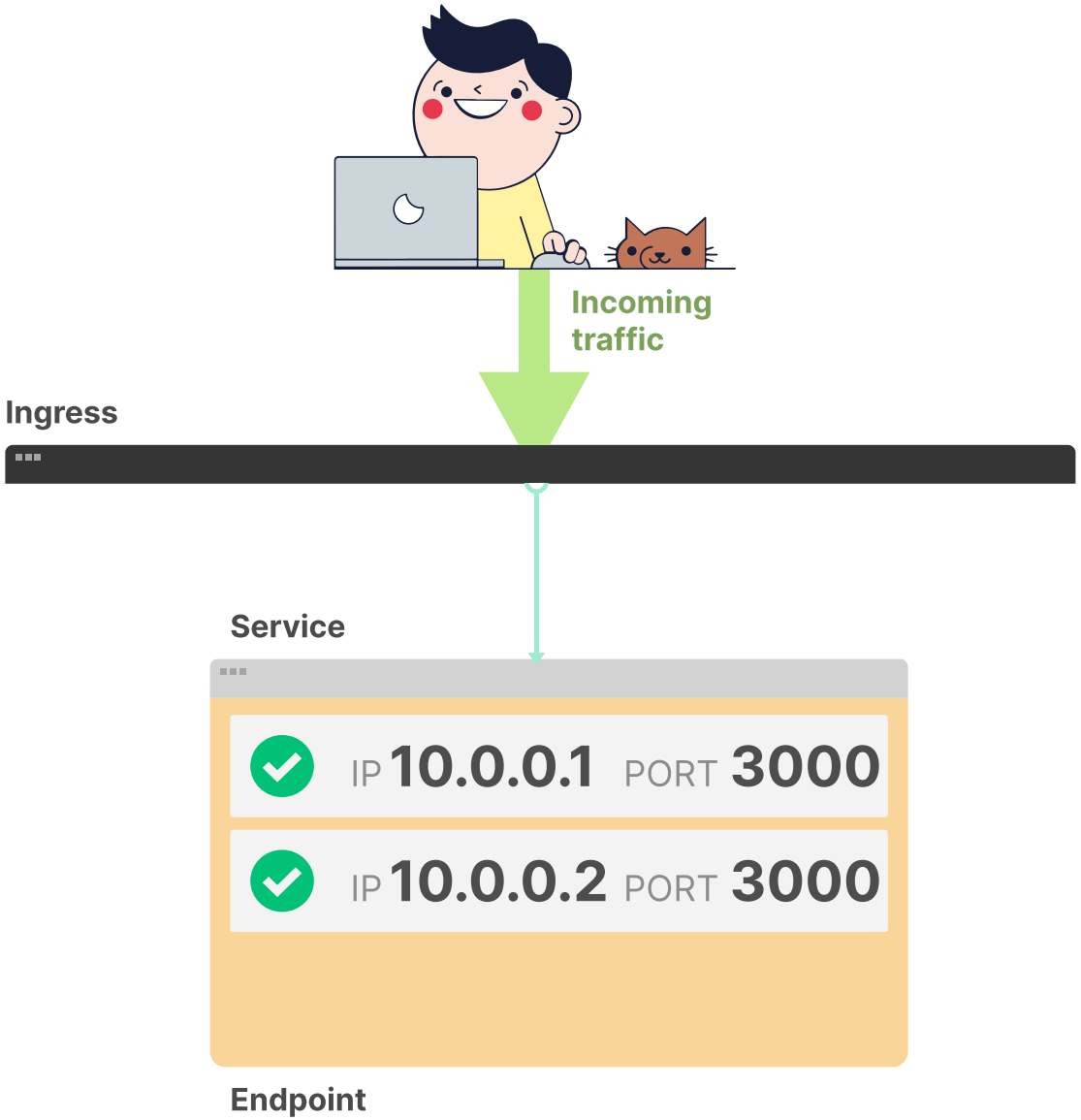

Backend service
172.17.0.1
Pod 1
Pod 5
Pod 6
10.0.0.1
10.0.0.2

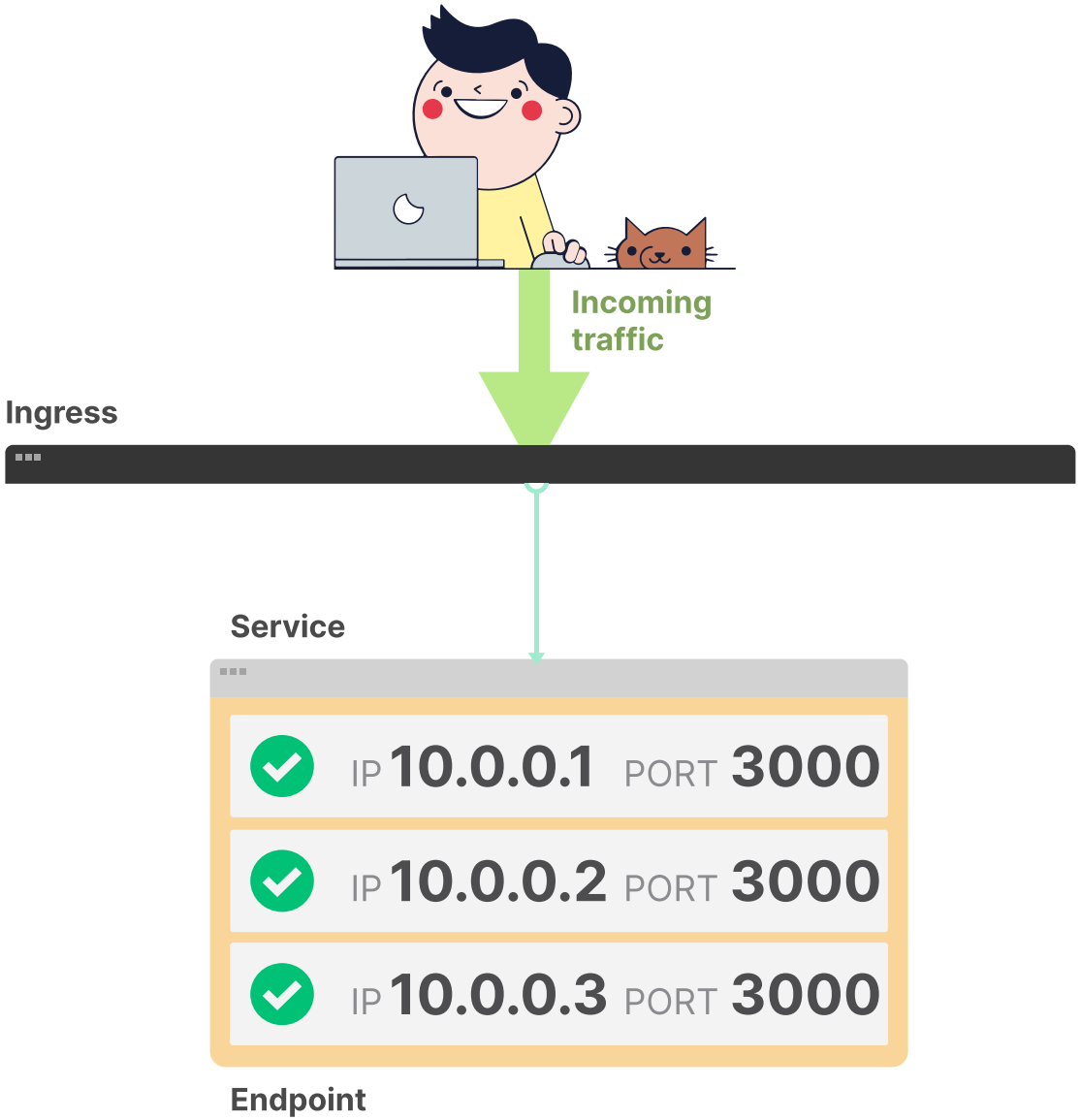

Backend service
172.17.0.1
Pod 1
Pod 5
Pod 6
Pod 5
Pod 6

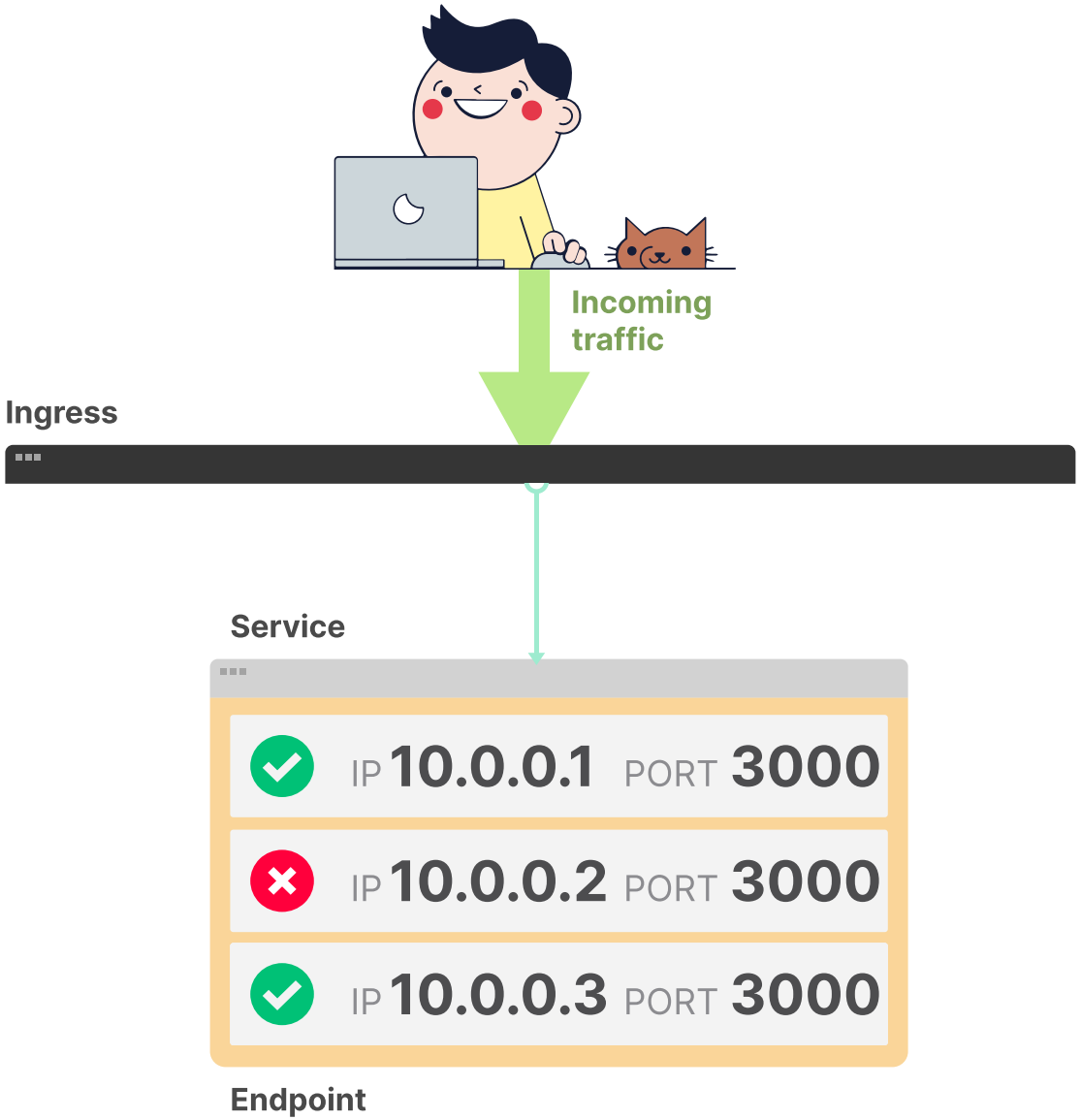
Service as a list

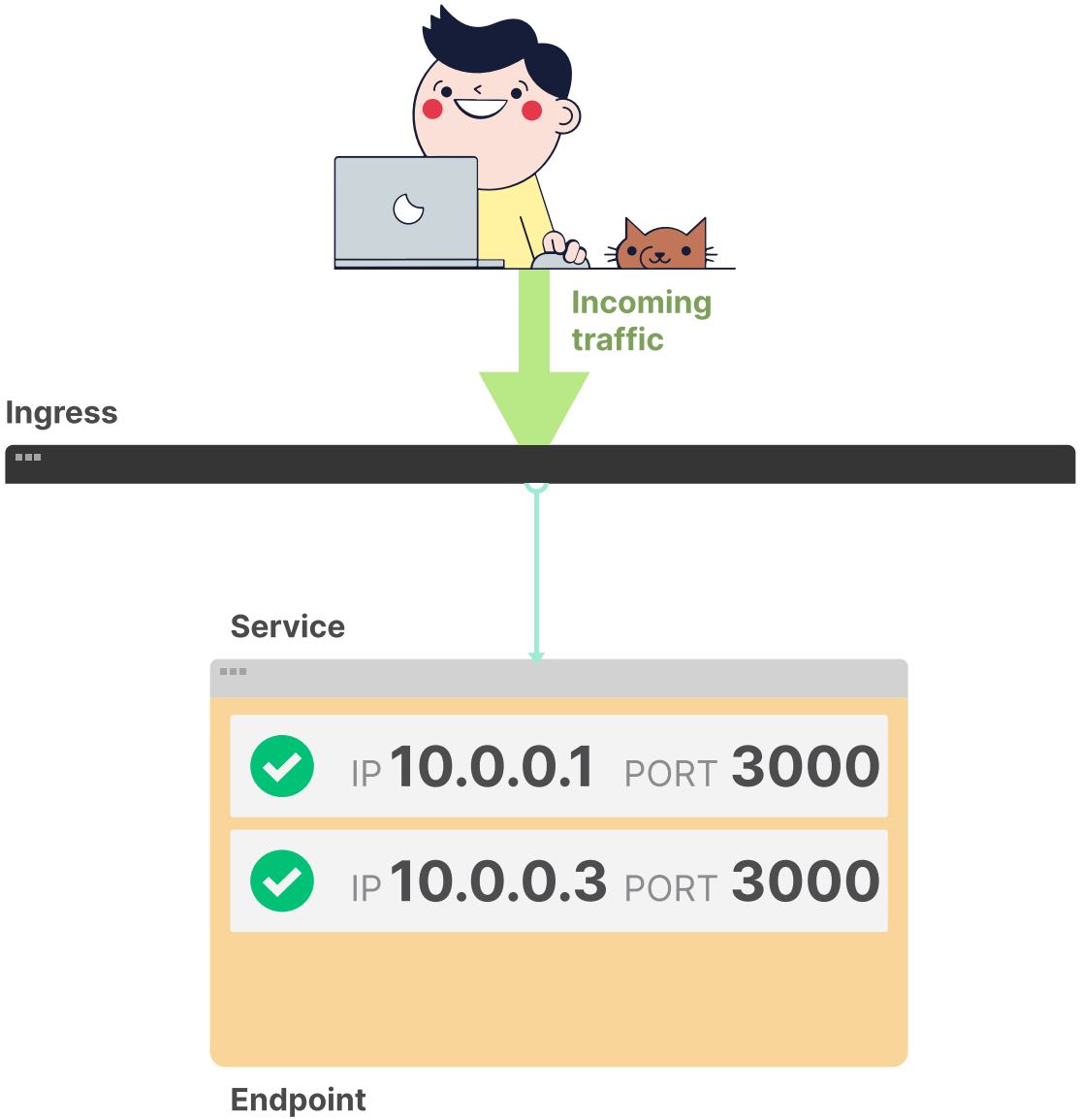

Incoming
traffic
Ingress
Service
Pod 1
Pod 2
10.0.0.1:3000
10.0.0.2:3000

Endpoint


Incoming
traffic
Ingress
Service
IP 10.0.0.1 PORT 3000
IP 10.0.0.2 PORT 3000
Endpoint


Incoming
traffic
Ingress
Service
IP 10.0.0.1 PORT 3000
IP 10.0.0.2 PORT 3000
IP 10.0.0.3 PORT 3000
Endpoint


Incoming
traffic
Ingress
Service
IP 10.0.0.1 PORT 3000
IP 10.0.0.2 PORT 3000
IP 10.0.0.3 PORT 3000
Endpoint


Incoming
traffic
Ingress
Service
IP 10.0.0.1 PORT 3000
IP 10.0.0.3 PORT 3000
Endpoint

Pod name	Status	Node	IP address
Pod 1	RUNNING	worker1	10.0.1.1
Pod 2	RUNNING	worker1	10.0.1.2
Pod 3	RUNNING	worker2	10.0.2.1
Pod 4	RUNNING	worker2	10.0.2.2

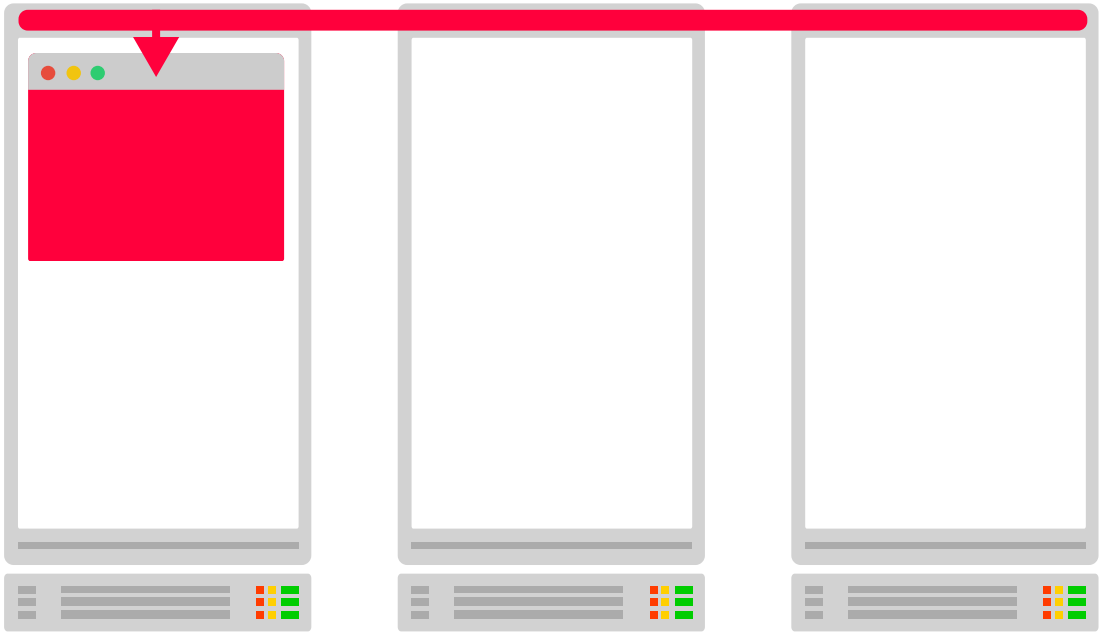
Pod name	Status	Node	IP address
Pod 1	RUNNING	worker1	10.0.1.1
Pod 2	RUNNING	worker1	10.0.1.2
Pod 3	RUNNING	worker2	10.0.2.1
Pod 4	RUNNING	worker2	10.0.2.2

Service name	Endpoints
Service 1	10.0.1.1:3000, 10.0.1.2:3000, 10.0.2.1:3000
Service 2	10.0.2.2:8080

Services in practice

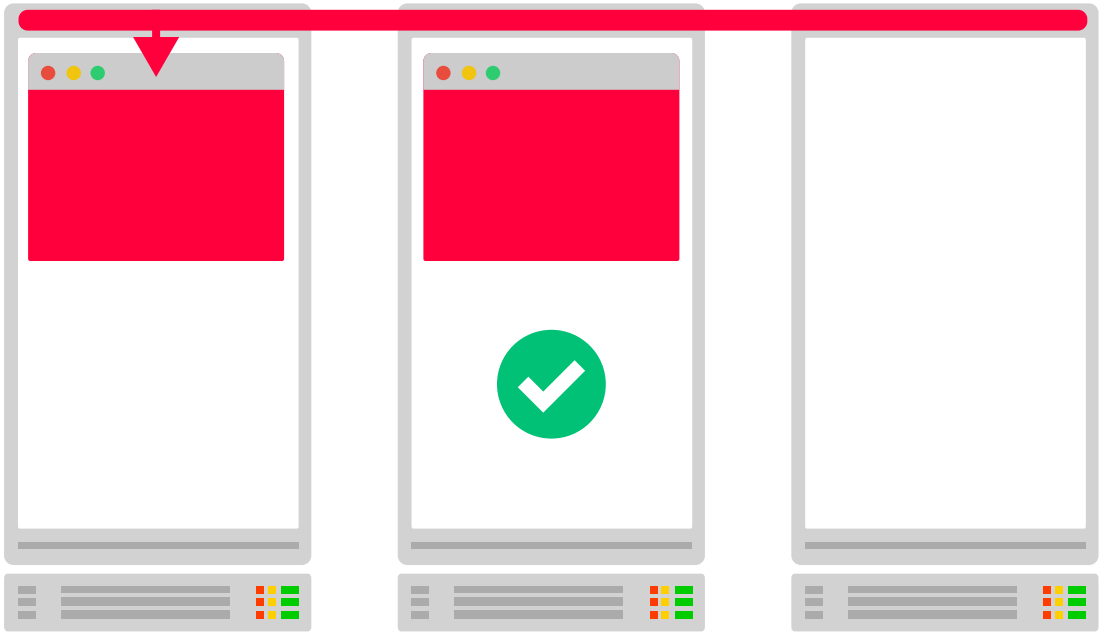


Pod name	Status	Node	podIP	Service name	Endpoints
Pod 1	RUNNING	worker1	10.0.0.1	Red	10.0.0.1:3000





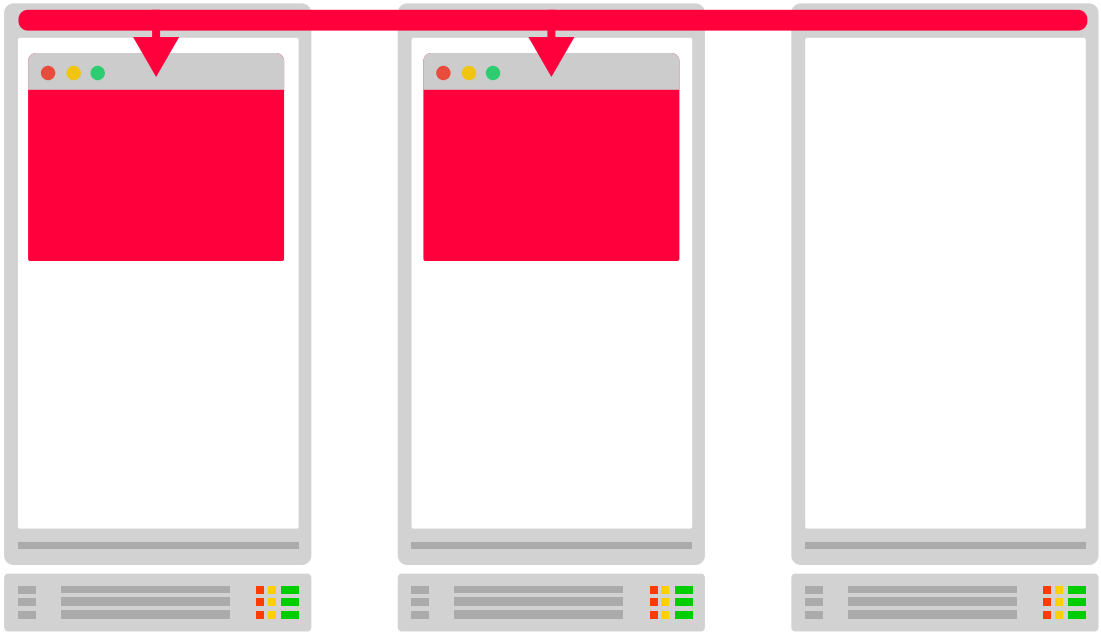
Pod name	Status	Node	podIP	Service name	Endpoints
Pod 1	RUNNING	worker1	10.0.0.1	Red	10.0.0.1:3000





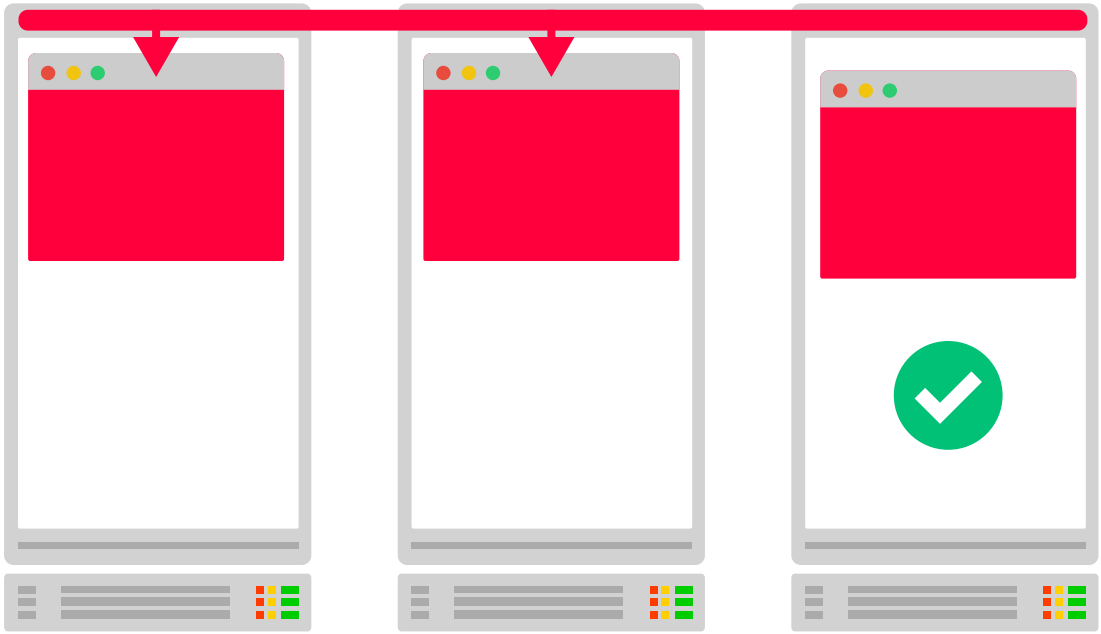
Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	Endpoints
Red	10.0.0.1:3000,10.0.1.1:3000





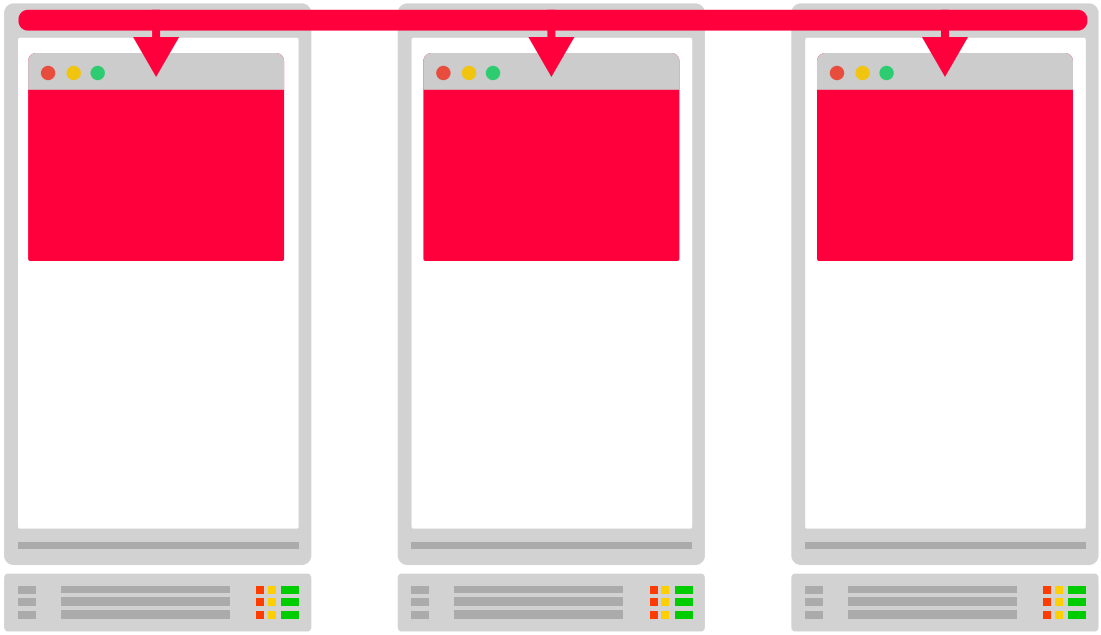
Pod name	Status	Node	podIP	Service name	Endpoints
Pod 1	RUNNING	worker1	10.0.0.1	Red	10.0.0.1:3000,10.0.1.1:3000
Pod 2	RUNNING	worker2	10.0.1.1		





Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1

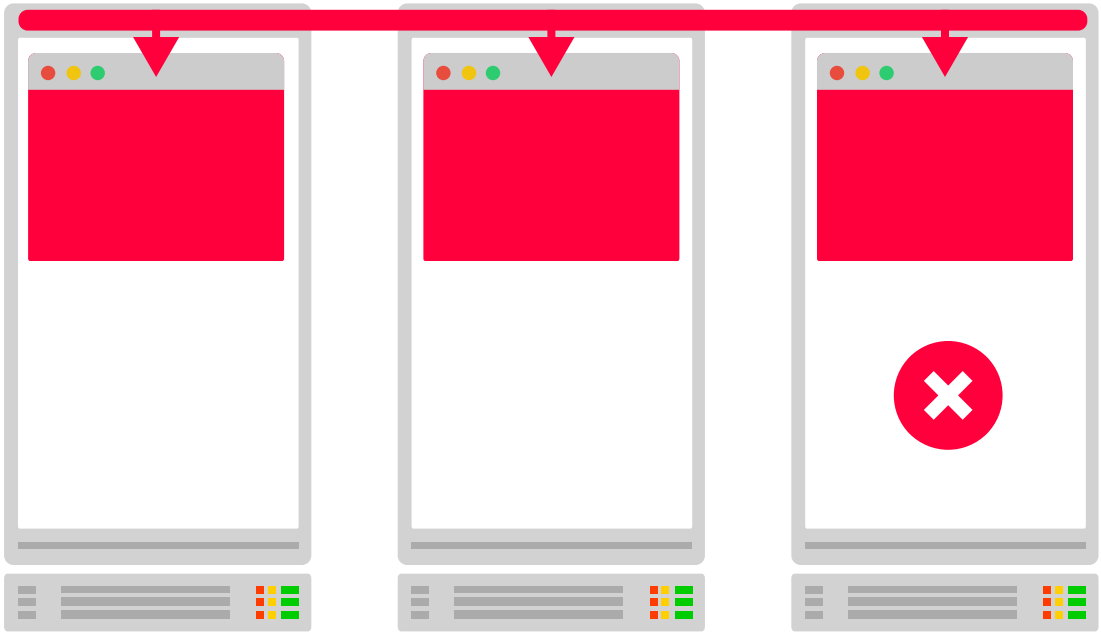
Service name	Endpoints
Red	10.0.0.1:3000,10.0.1.1:3000,10.0.2.1:3000





Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1

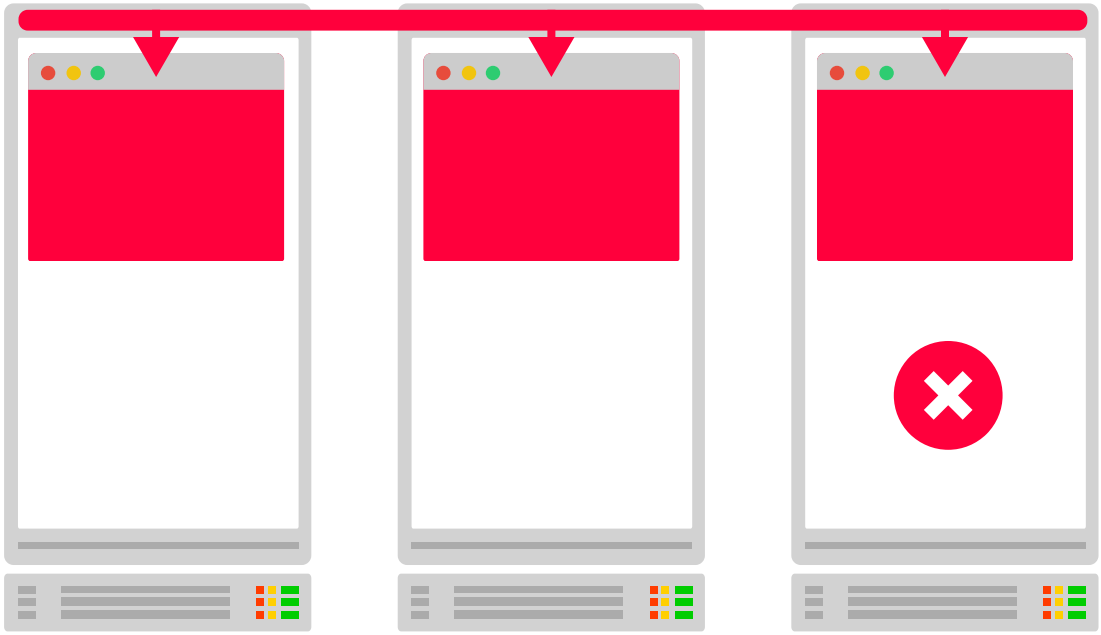
Service name	Endpoints
Red	10.0.0.1:3000,10.0.1.1:3000,10.0.2.1:3000





Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1

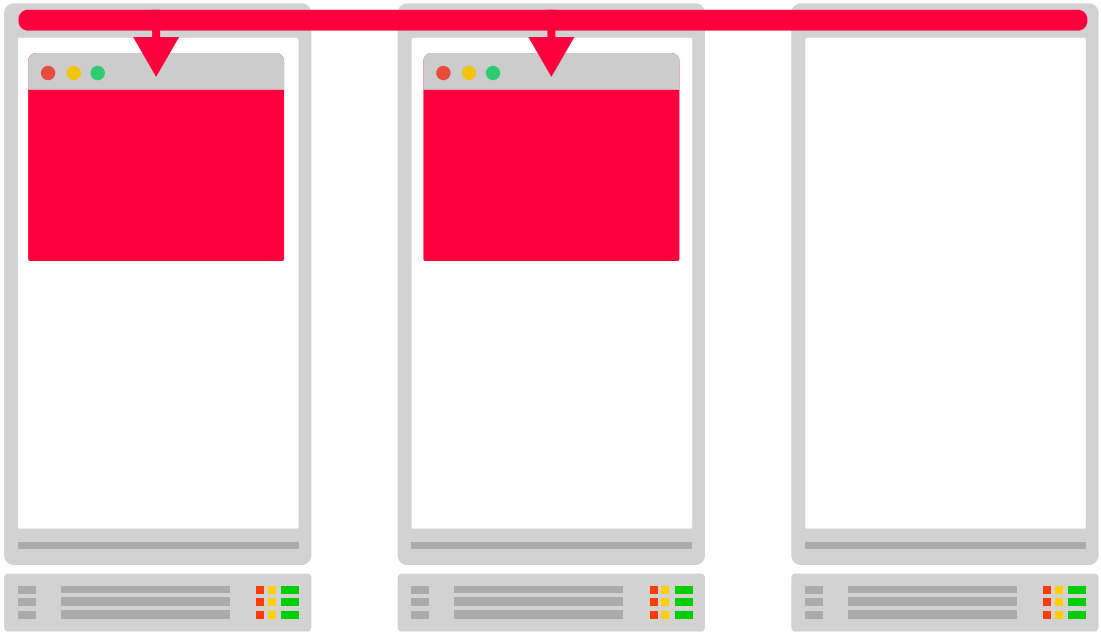
Service name	Endpoints
Red	10.0.0.1:3000,10.0.1.1:3000,10.0.2.1:3000



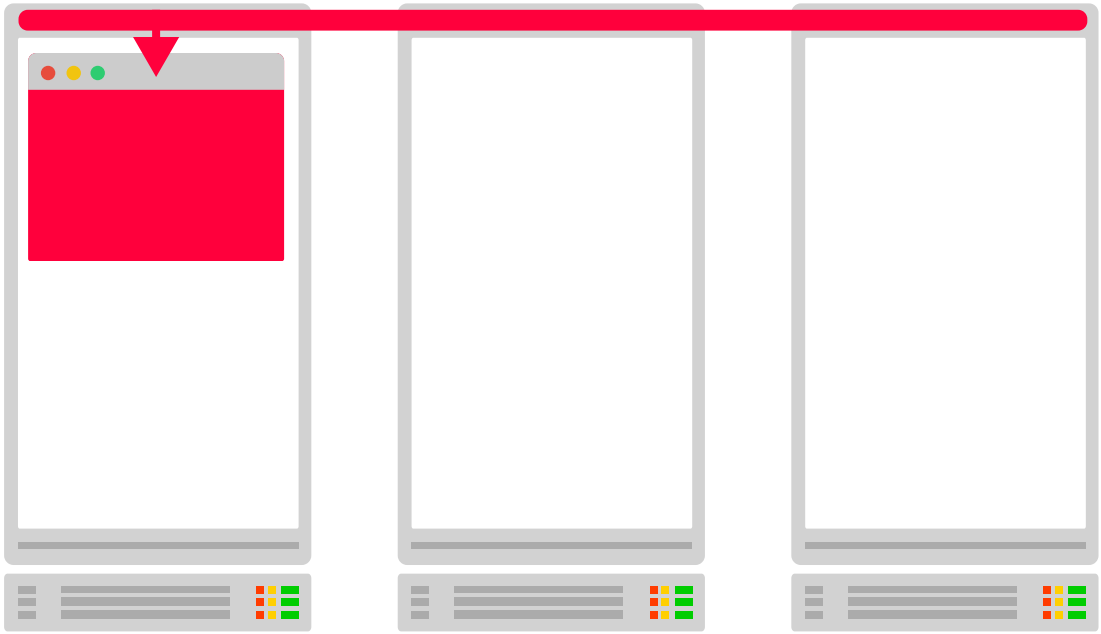


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	Endpoints
Red	10.0.0.1:3000,10.0.1.1:3000



How are Services implemented?



What is a service?

Load balancer

What is a service?

Load balancer

What is a service?

Load balancer

Process listening to connections

What is a service?

Load balancer ✗

Process listening to connections ✗

What is a service?

~~Load balancer~~

~~Process listening to connections~~

At least is has an IP address

What is a service?

Load balancer ✗

Process listening to connections ✗

At least is has an IP address ✗

What is a service?

~~Load balancer~~ ❌

~~Process listening to connections~~ ❌

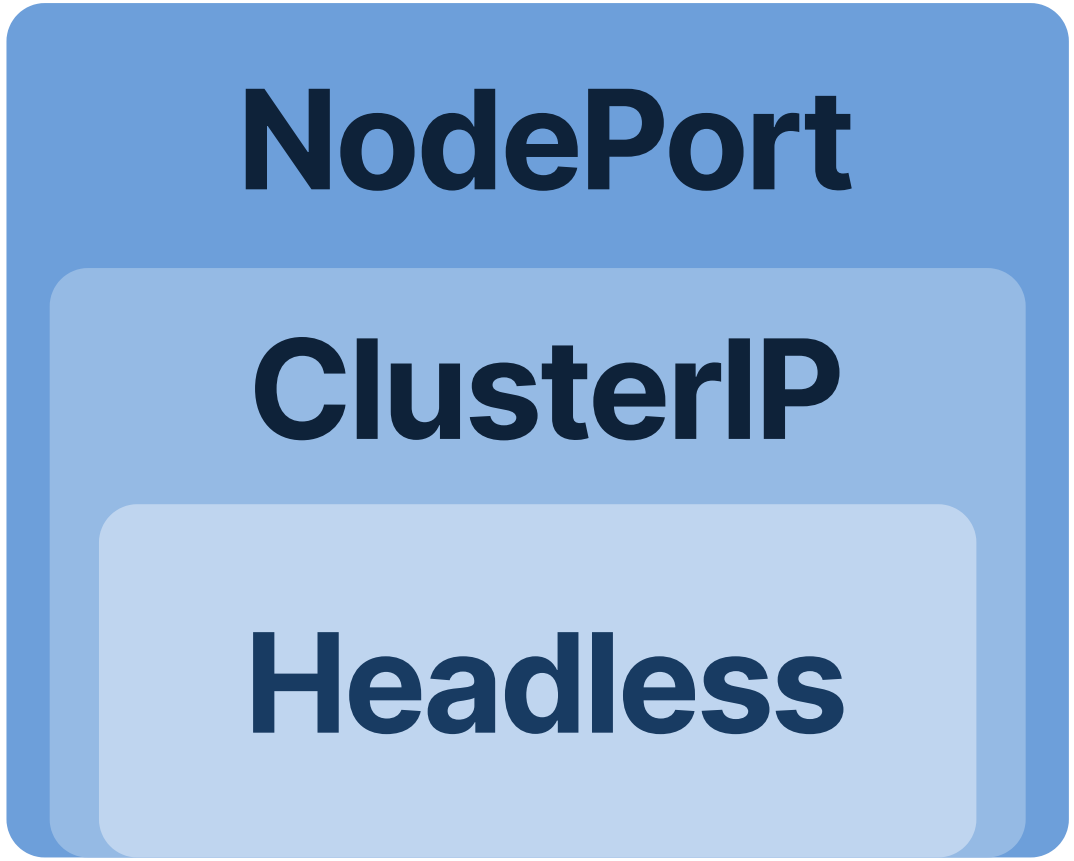
At least is has an IP address ❗

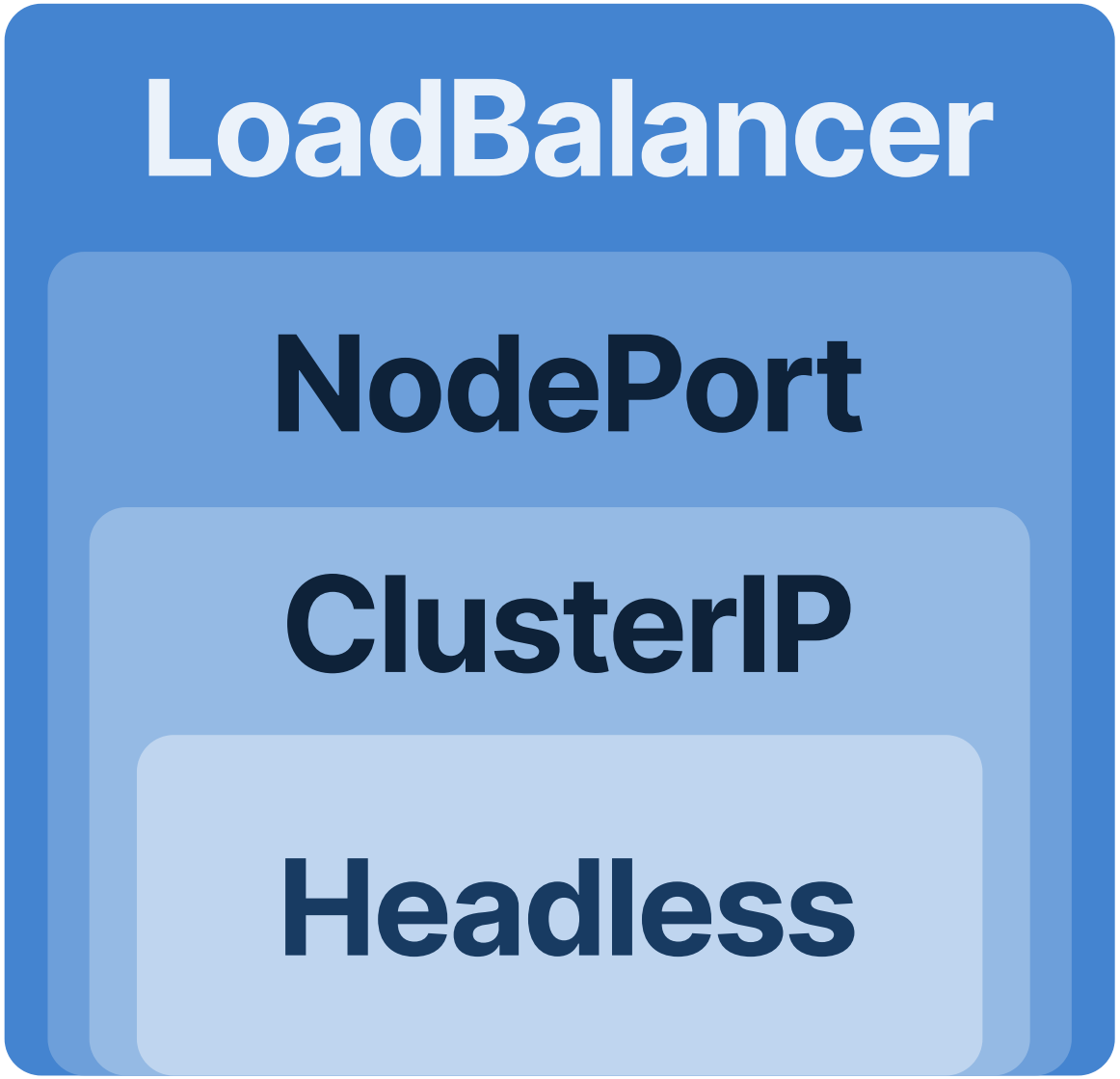
Services don’t exist

4 Services

Headless


ClusterIP
Headless


NodePort
ClusterIP
Headless


LoadBalancer
NodePort
ClusterIP
Headless

Headless Service

Headless Service

No IP address

Headless Service

No IP address

List of endpoints

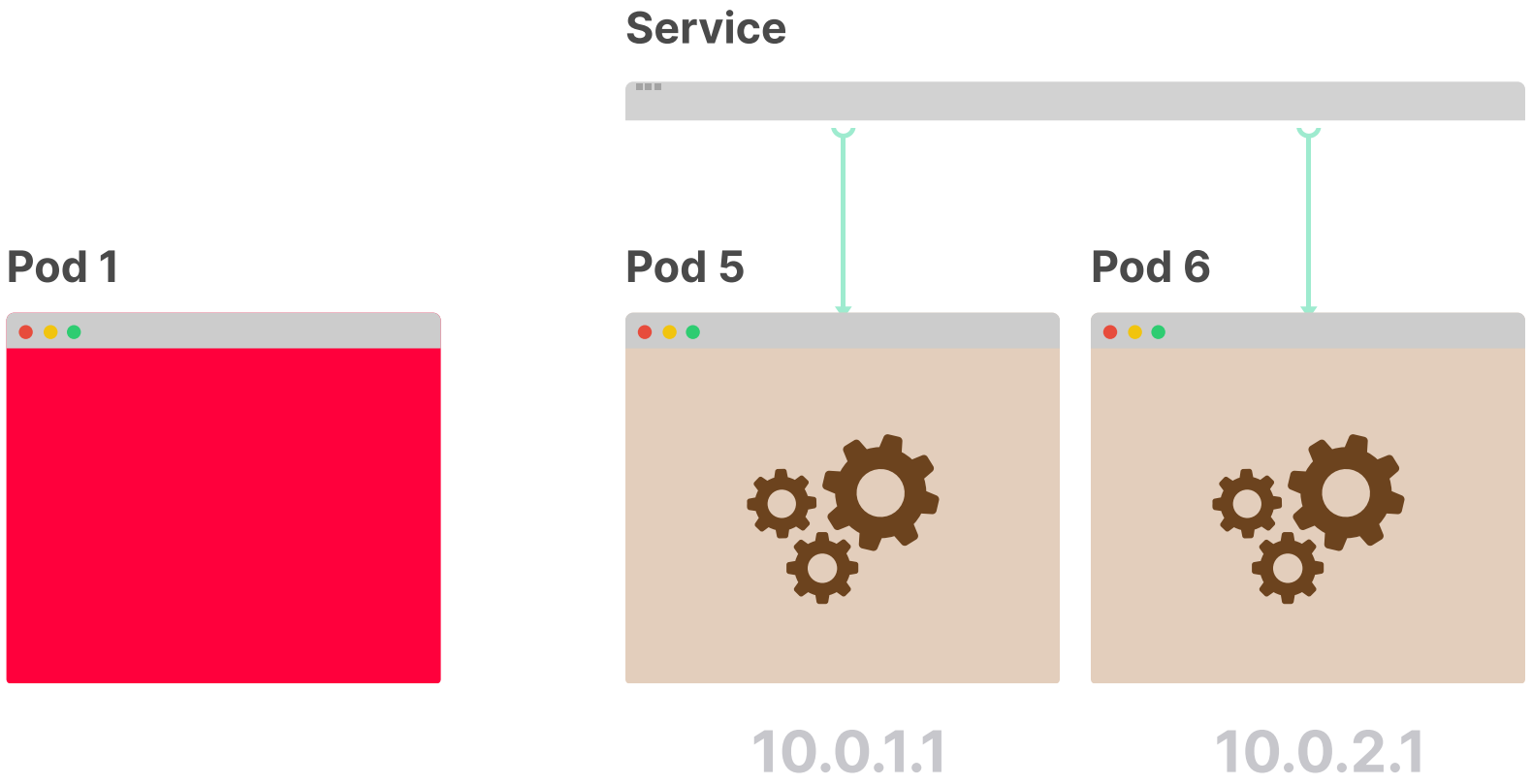
Headless Service

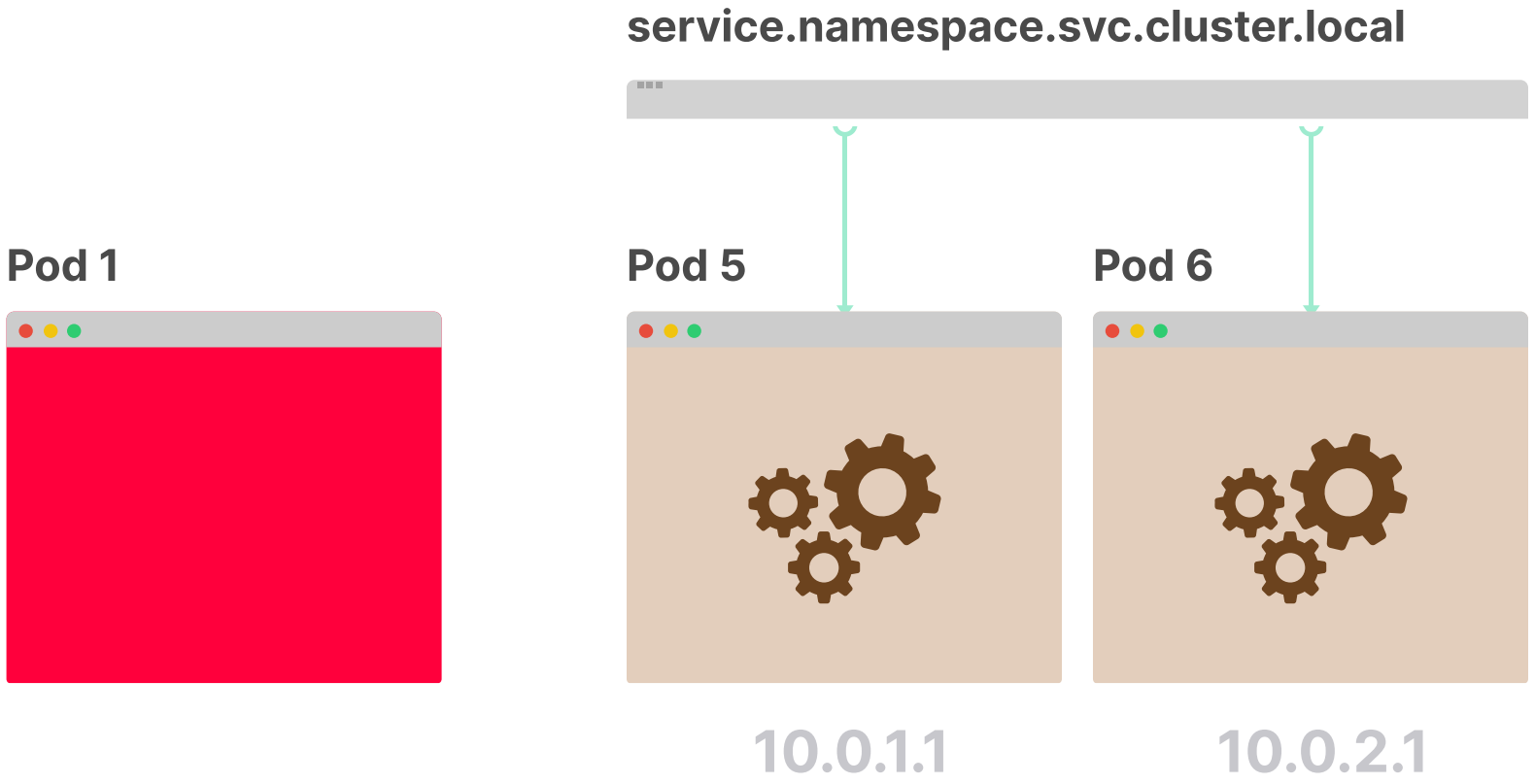
No IP address

List of endpoints

Client-side load balancing

Headless Service in practice

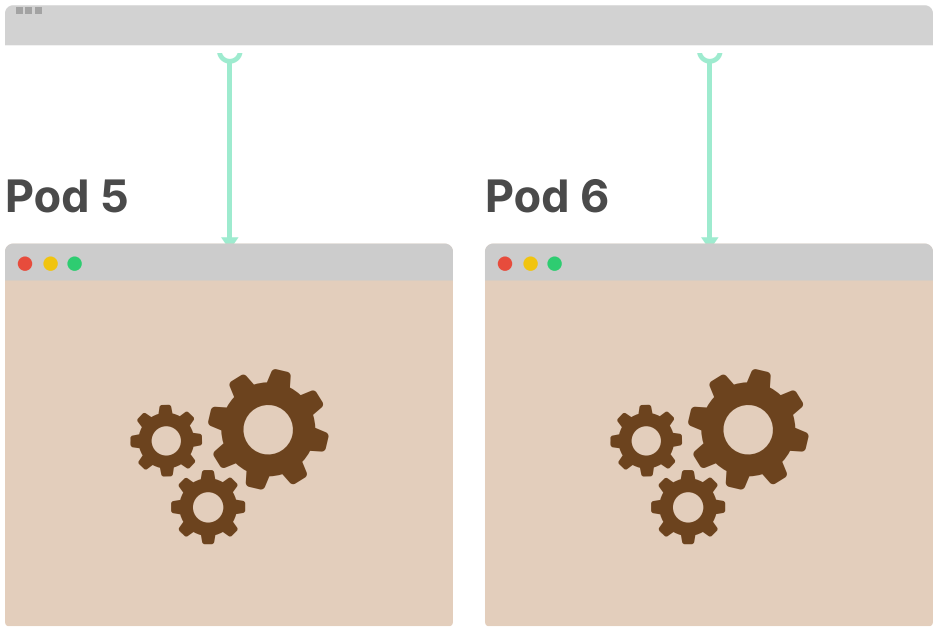

Service
Pod 1
Pod 5
Pod 6
10.0.1.1
10.0.2.1


service.namespace.svc.cluster.local
Pod 1
Pod 5
Pod 6
10.0.1.1
10.0.2.1



CoreDNS

service.namespace.svc.cluster.local



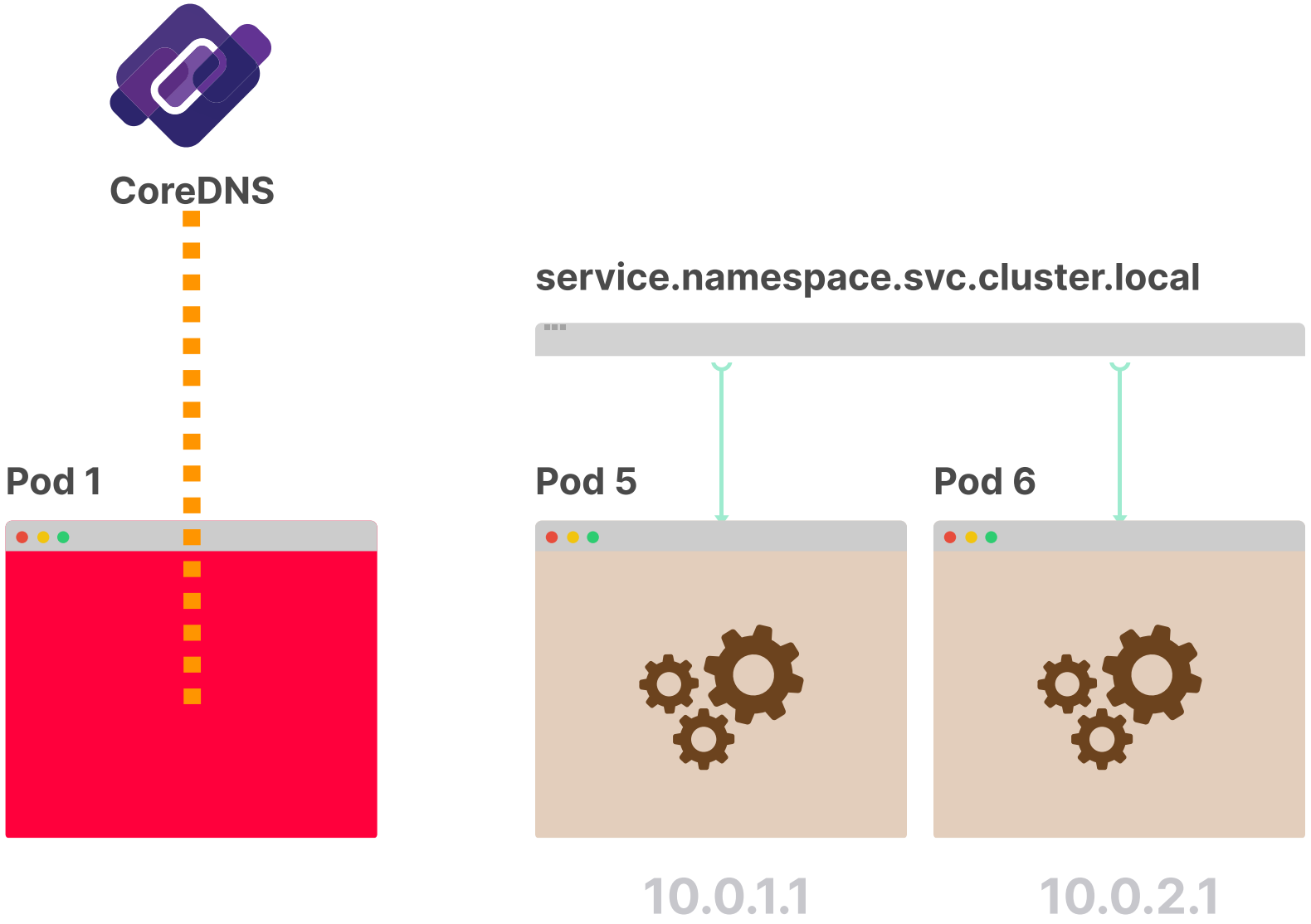
Pod 1

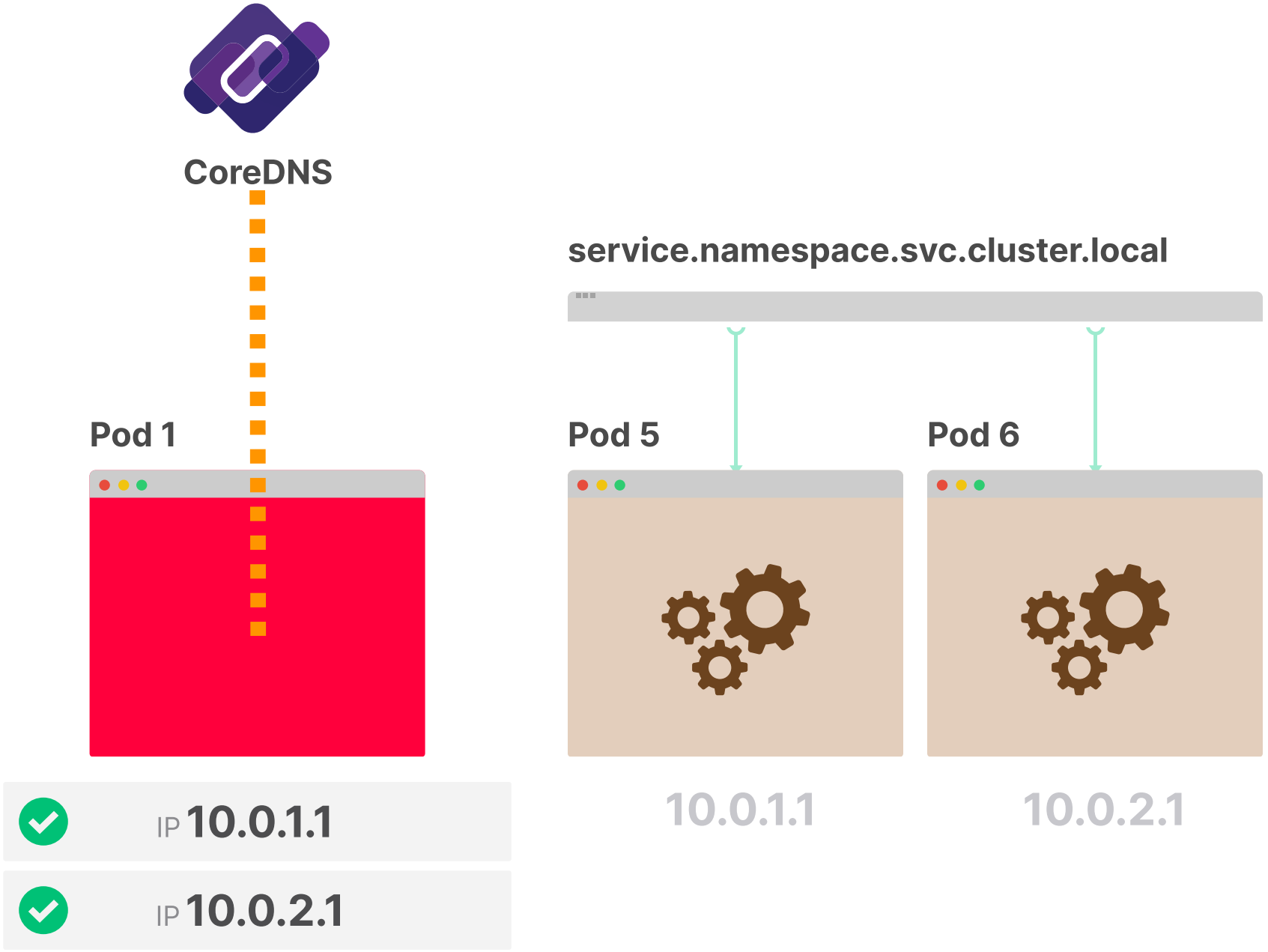
Pod 5

Pod 6

10.0.1.1

10.0.2.1


CoreDNS
service.namespace.svc.cluster.local
Pod 1
Pod 5
Pod 6
10.0.1.1
10.0.2.1


CoreDNS
service.namespace.svc.cluster.local
Pod 1
Pod 5
Pod 6
IP 10.0.1.1
IP 10.0.2.1
10.0.1.1
10.0.2.1



CoreDNS

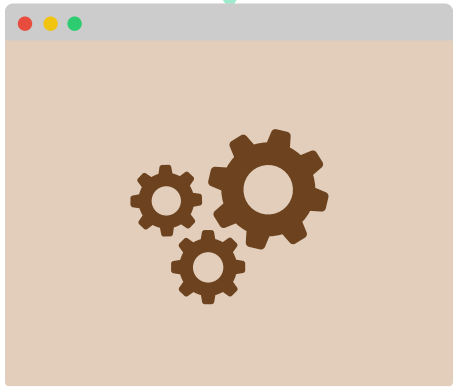
service.namespace.svc.cluster.local

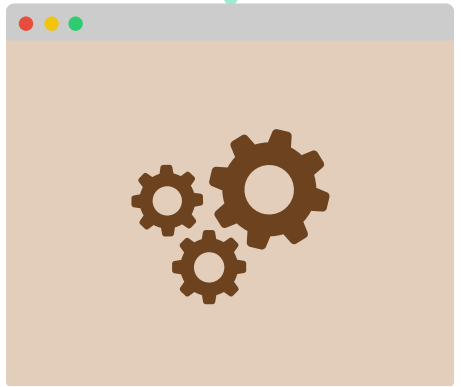
Pod 1

Pod 5

Pod 6





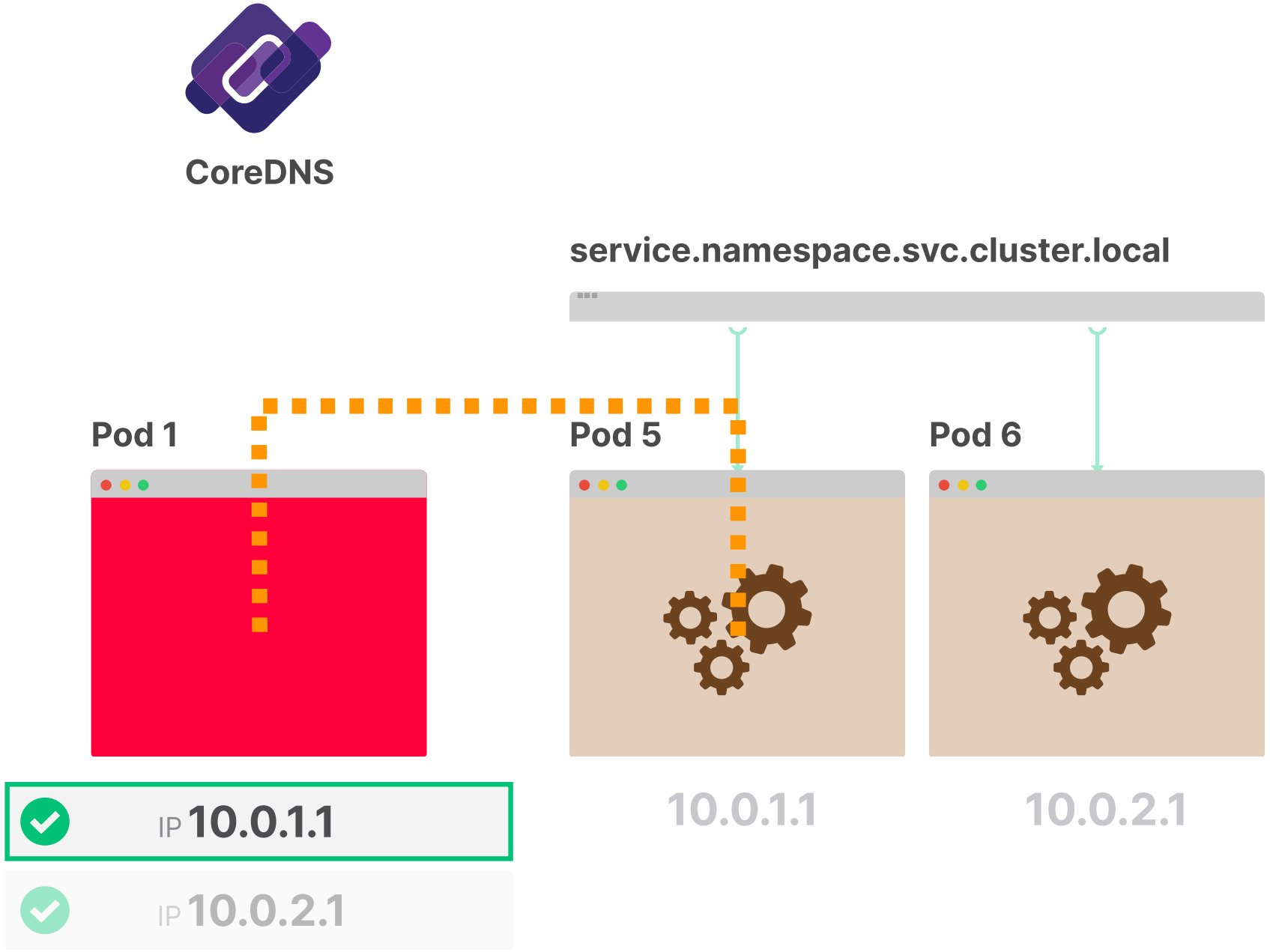


IP 10.0.1.1

IP 10.0.2.1

10.0.1.1

10.0.2.1


CoreDNS
service.namespace.svc.cluster.local
Pod 1
Pod 5
Pod 6
IP 10.0.1.1
IP 10.0.2.1
10.0.1.1
10.0.2.1

CoreDNS

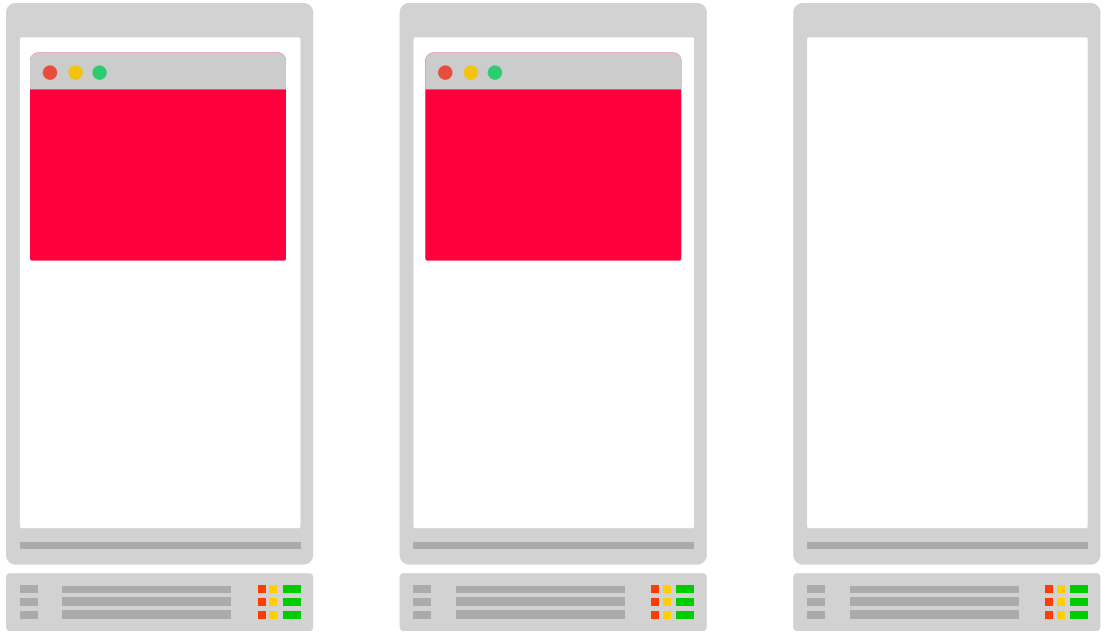


Service name	IP addresses



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	Endpoints





Service name	IP addresses

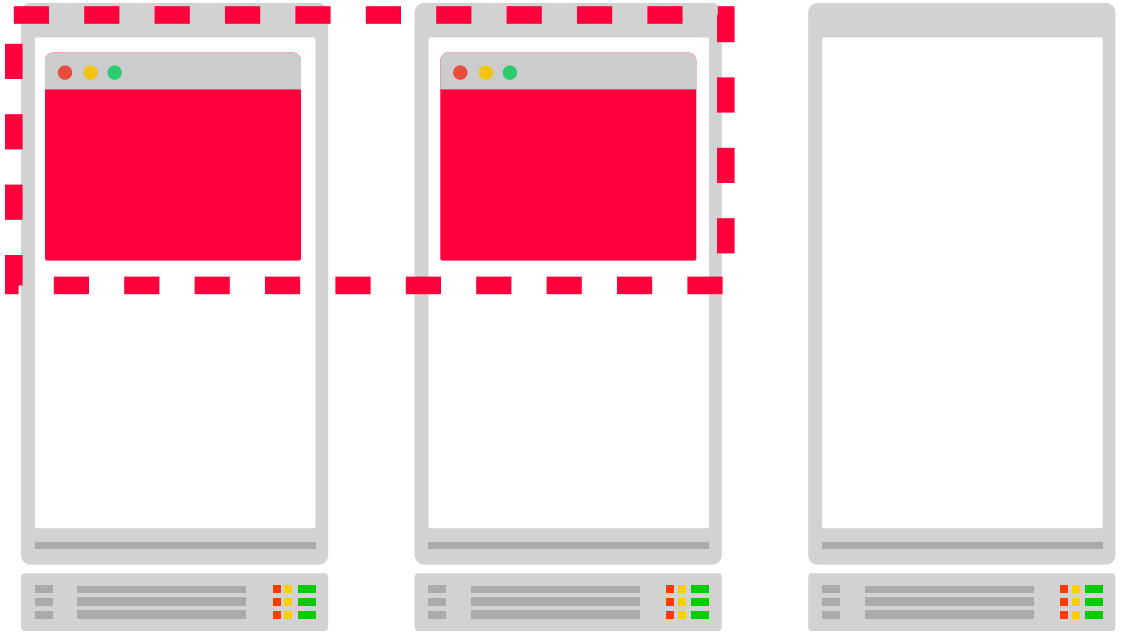


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	Endpoints

SERVICE

Headless





Service name	IP addresses

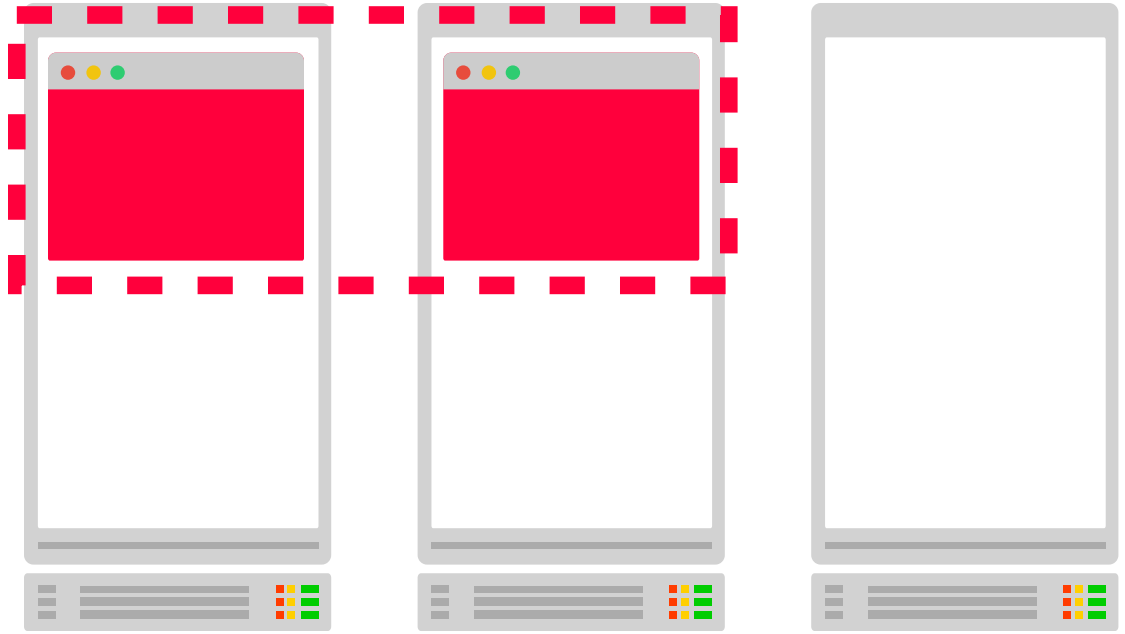


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

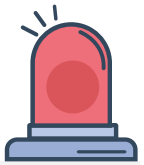
Service name	Endpoints
Red	10.0.0.1:3000,10.0.1.1:3000

SERVICE

Headless







Service name	IP addresses

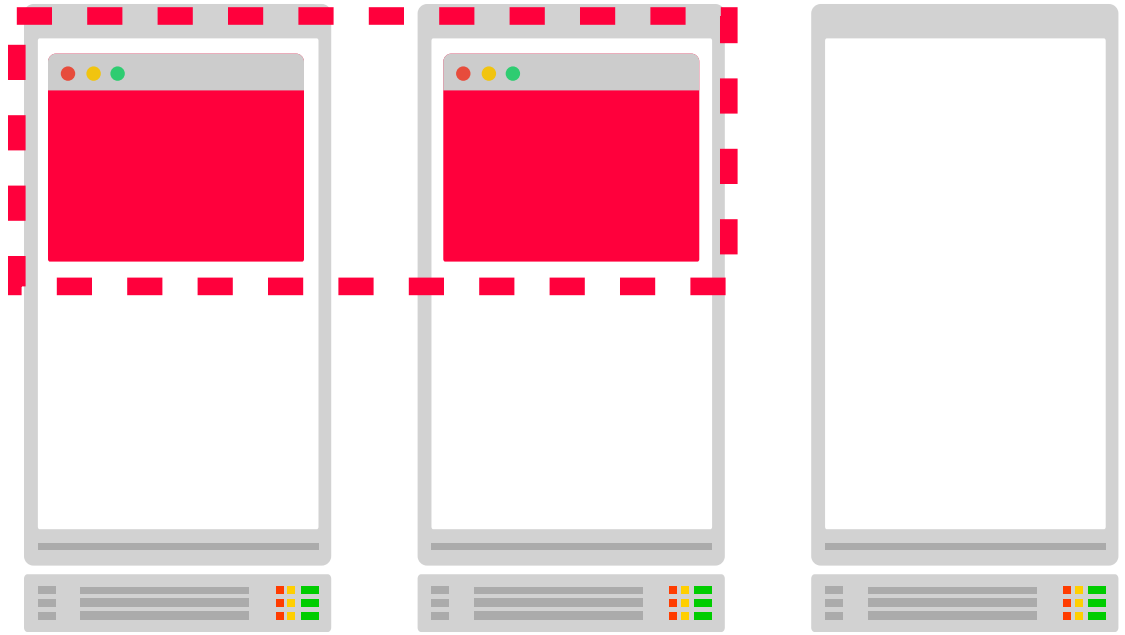


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	Endpoints
Red	10.0.0.1:3000,10.0.1.1:3000

SERVICE

Headless





Service name	IP addresses
red.namespace.svc.cluster.local	10.0.0.1,10.0.1.1

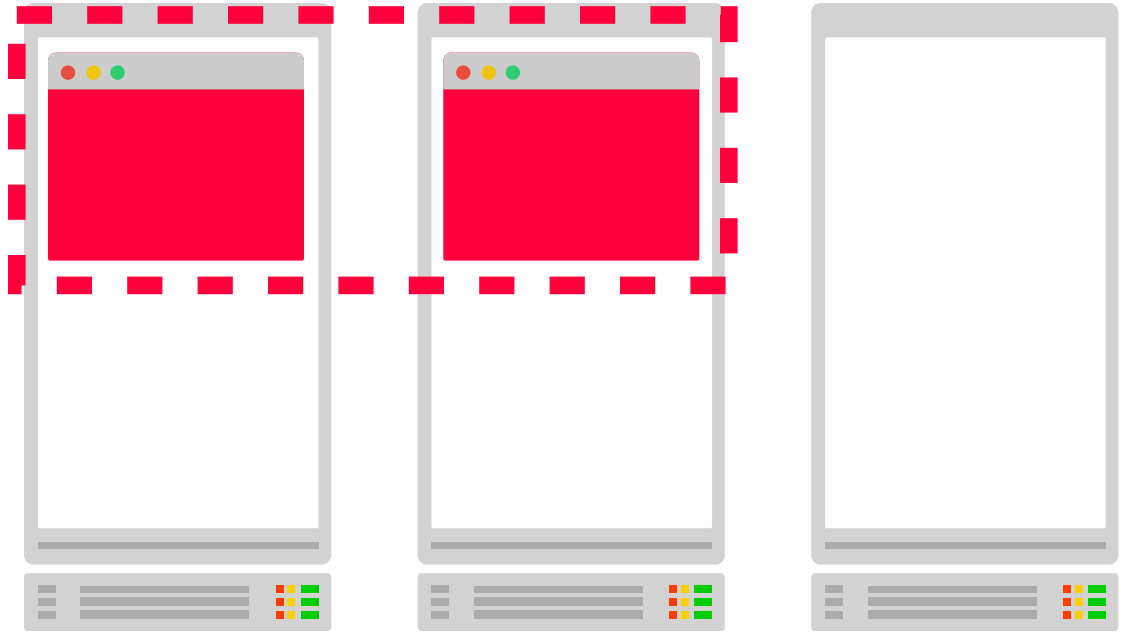


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	Endpoints
Red	10.0.0.1:3000,10.0.1.1:3000

SERVICE

Headless



Headless Service & DNS caveats

Headless Service & DNS caveats

Cache indefinitely

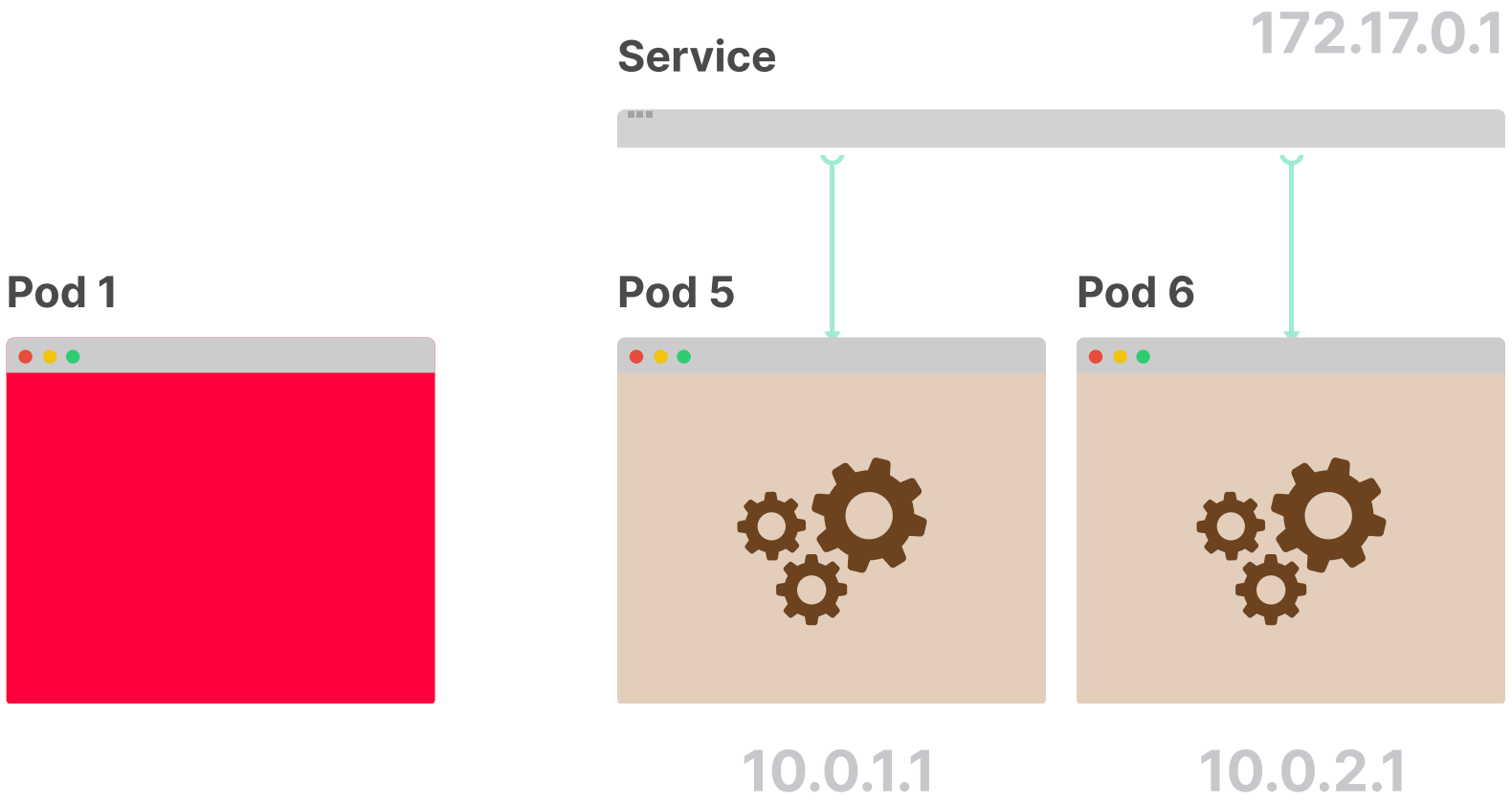
Not respecting TTLs

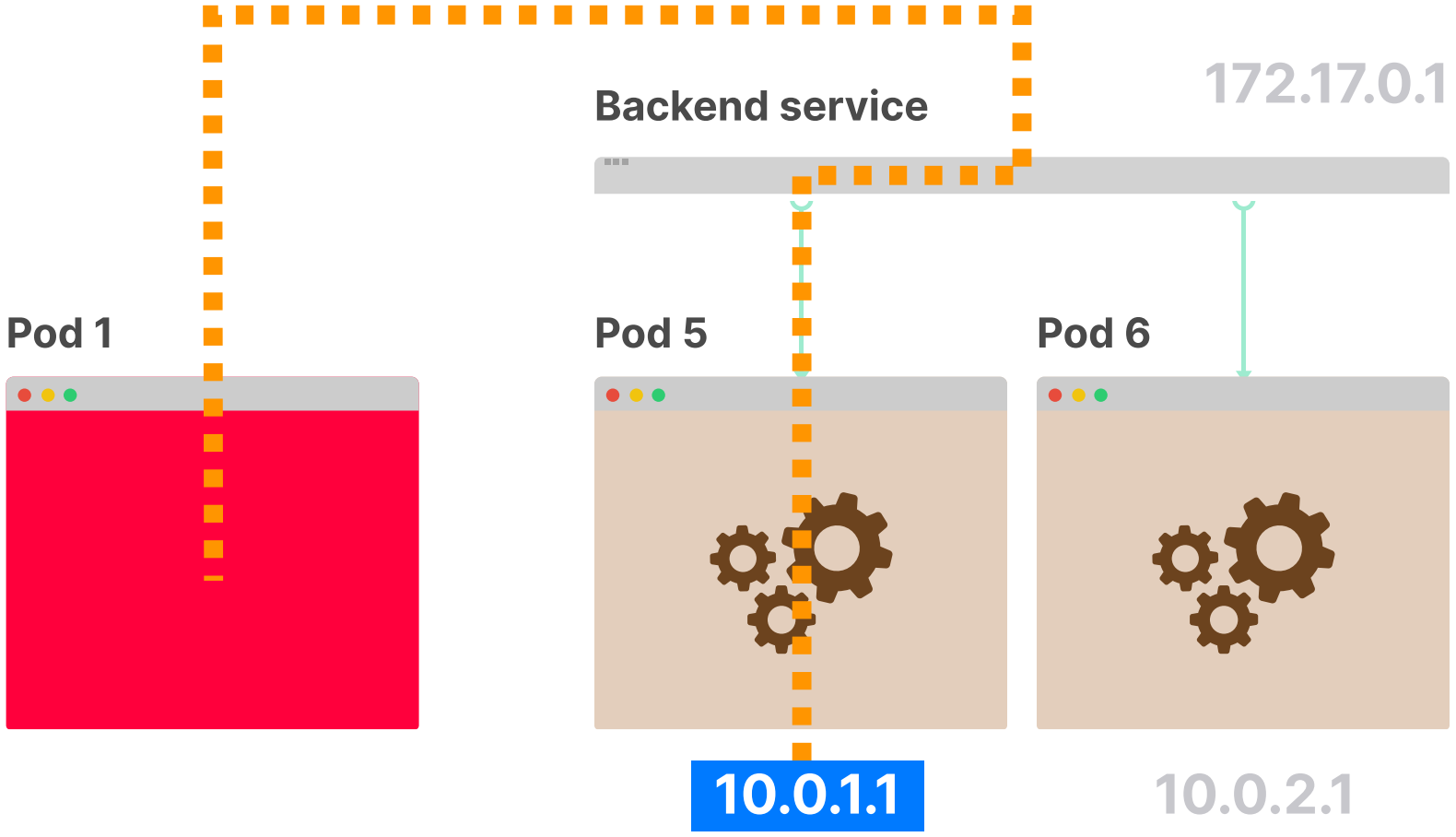
Headless Service & DNS caveats

Cache indefinitely

Not respecting TTLs 

Faking a load balancer with a Service


Service
172.17.0.1
Pod 1
Pod 5
Pod 6
10.0.1.1
10.0.2.1


Backend service
172.17.0.1
Pod 1
Pod 5
Pod 6
10.0.1.1
10.0.2.1

Faking a load balancer with a Service

Virtual IP address

Fixed allocation

Intercept and rewrite

Faking a load balancer with a Service

Virtual IP address

Fixed allocation

Intercept and rewrite

Faking a load balancer with a Service

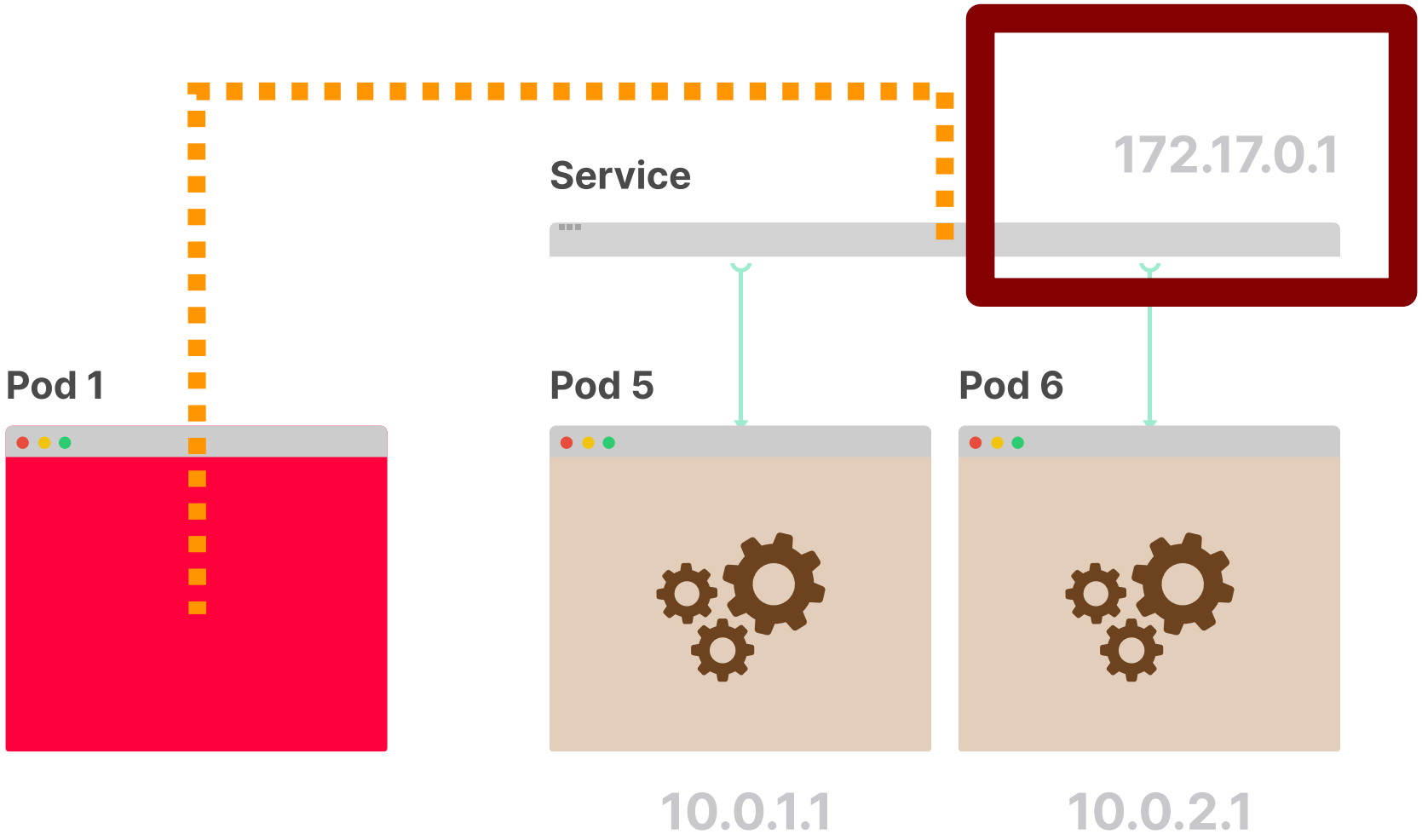
Virtual IP address

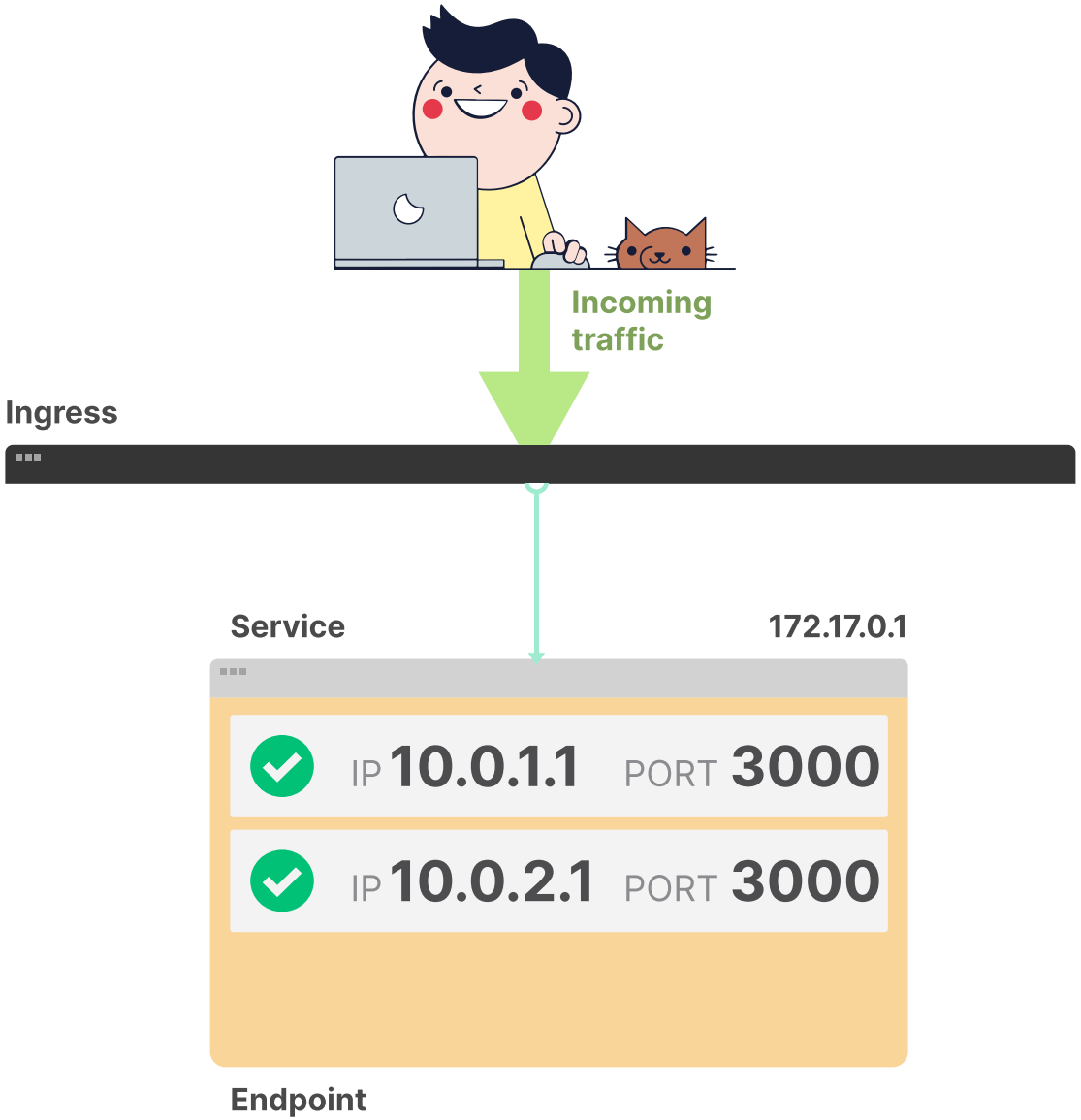
Fixed allocation

Intercept and rewrite

ClusterIP Service


ClusterIP
Headless


172.17.0.1
Service
Pod 1
Pod 5
Pod 6
10.0.1.1
10.0.2.1


Incoming
traffic
Ingress
Service
172.17.0.1
IP 10.0.1.1 PORT 3000
IP 10.0.2.1 PORT 3000
Endpoint

Pod name	Status	Node	IP address
Pod 1	RUNNING	worker1	10.0.1.1
Pod 2	RUNNING	worker1	10.0.1.2
Pod 3	RUNNING	worker2	10.0.2.1
Pod 4	RUNNING	worker2	10.0.2.2

Service name	IP address	Endpoints
Service 1	172.17.0.1	10.0.1.1:3000, 10.0.1.2:3000
Service 2	172.17.0.2	10.0.2.2:8080

Pod name	Status	Node	IP address
Pod 1	RUNNING	worker1	10.0.1.1
Pod 2	RUNNING	worker1	10.0.1.2
Pod 3	RUNNING	worker2	10.0.2.1
Pod 4	RUNNING	worker2	10.0.2.2

Service name	IP address	Endpoints
Service 1	172.17.0.1	1.0.1.1:3000, 10.0.1.2:3000
Service 2	172.17.0.2	10.0.2.2:8080

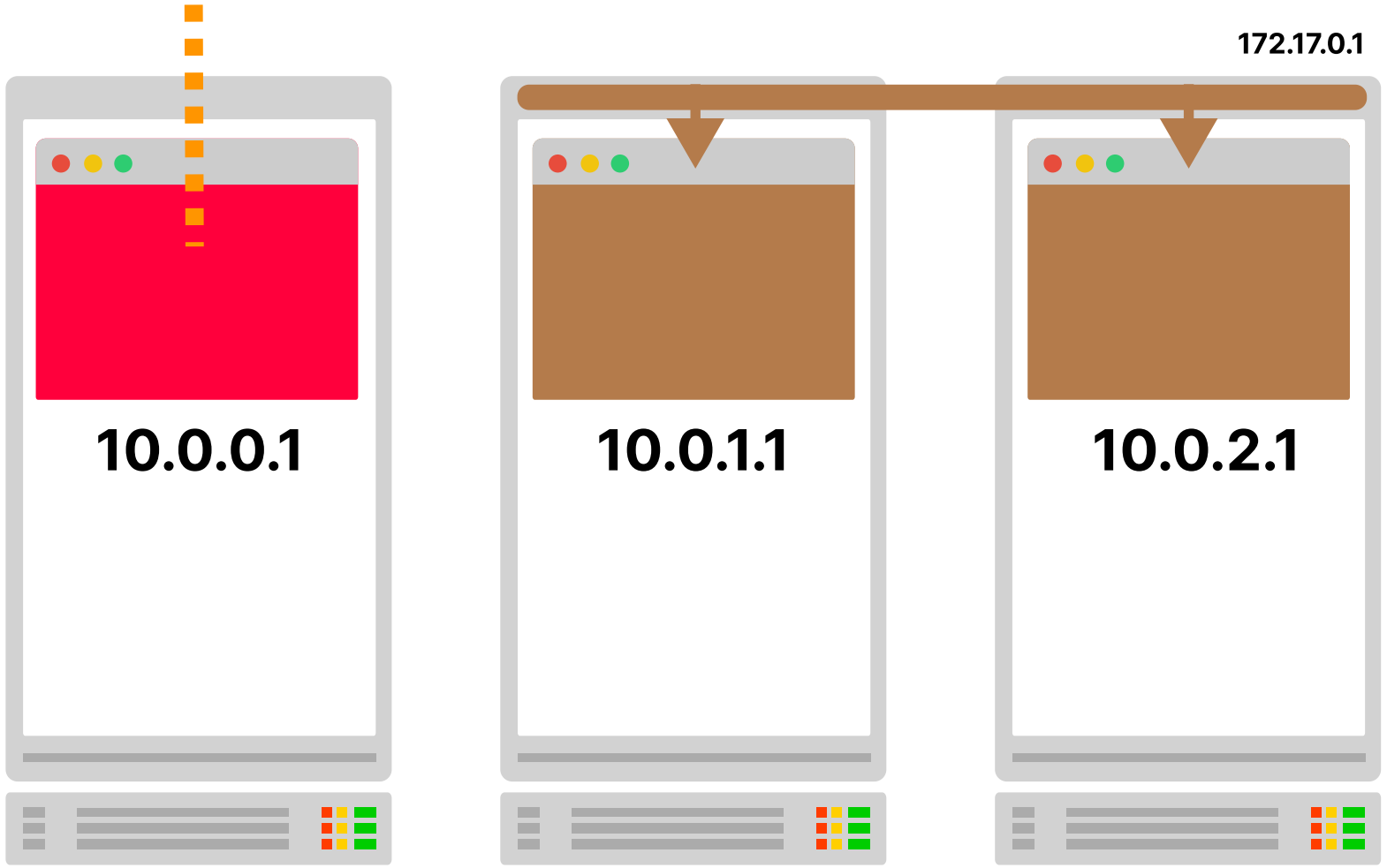
ClusterIP in practice

FROM:

Pod (10.0.0.1)

TO:

Service (172.17.0.1)

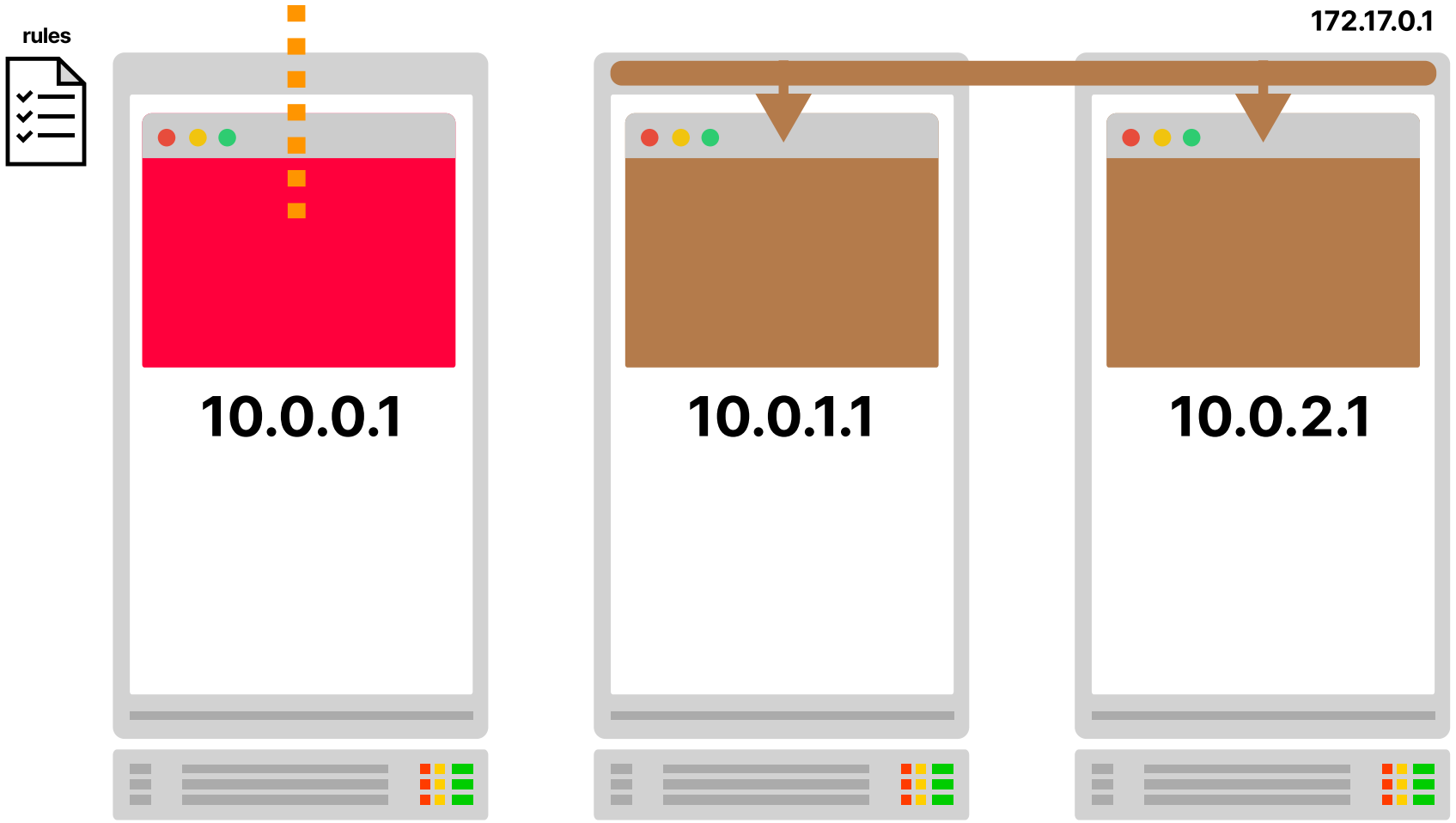


FROM:

Pod (10.0.0.1)

TO:

Service (172.17.0.1)



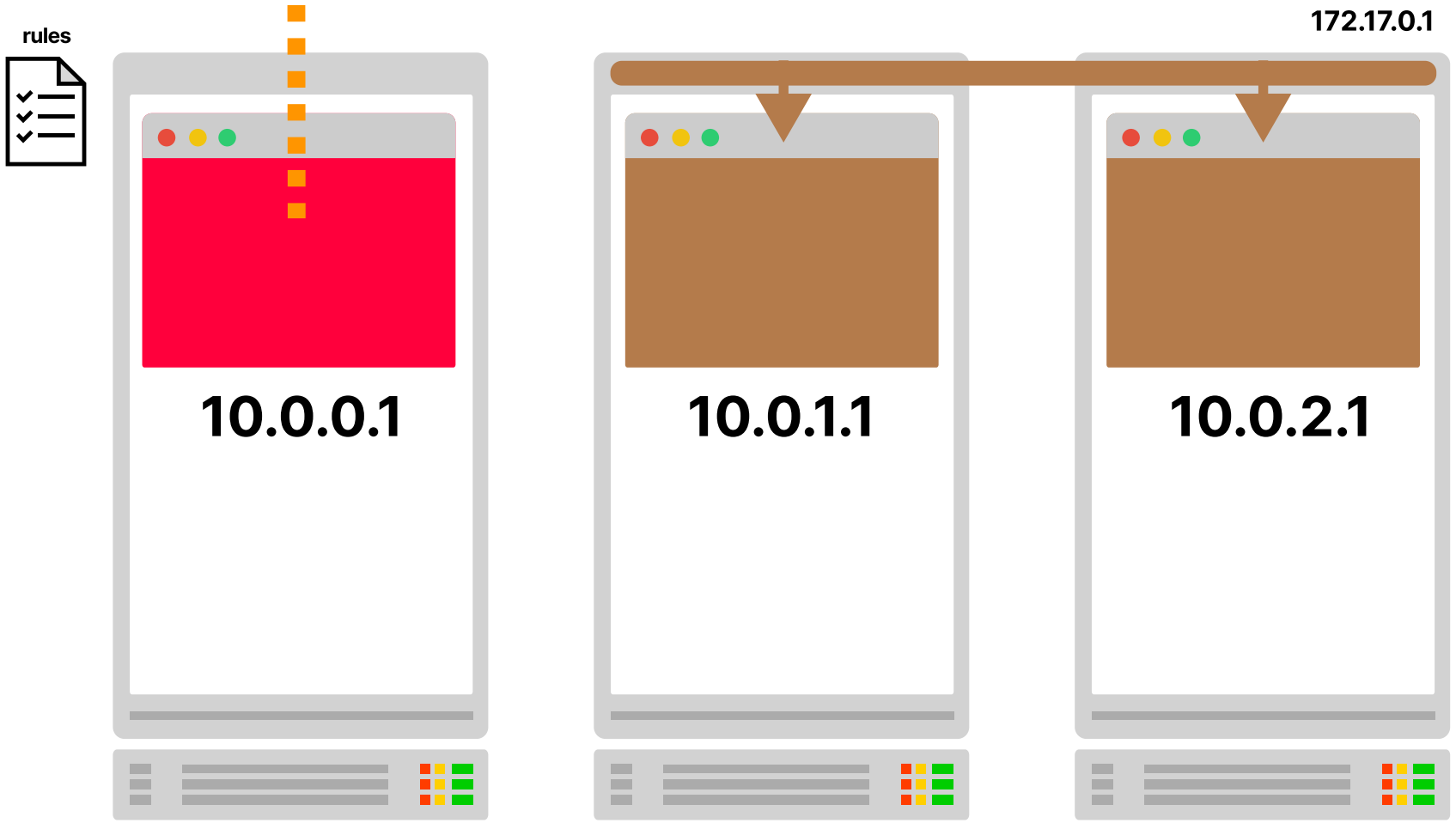
FROM:

Pod (10.0.0.1)

TO:

~~**Service** (172.17.0.1)~~

Pod (10.0.1.1)

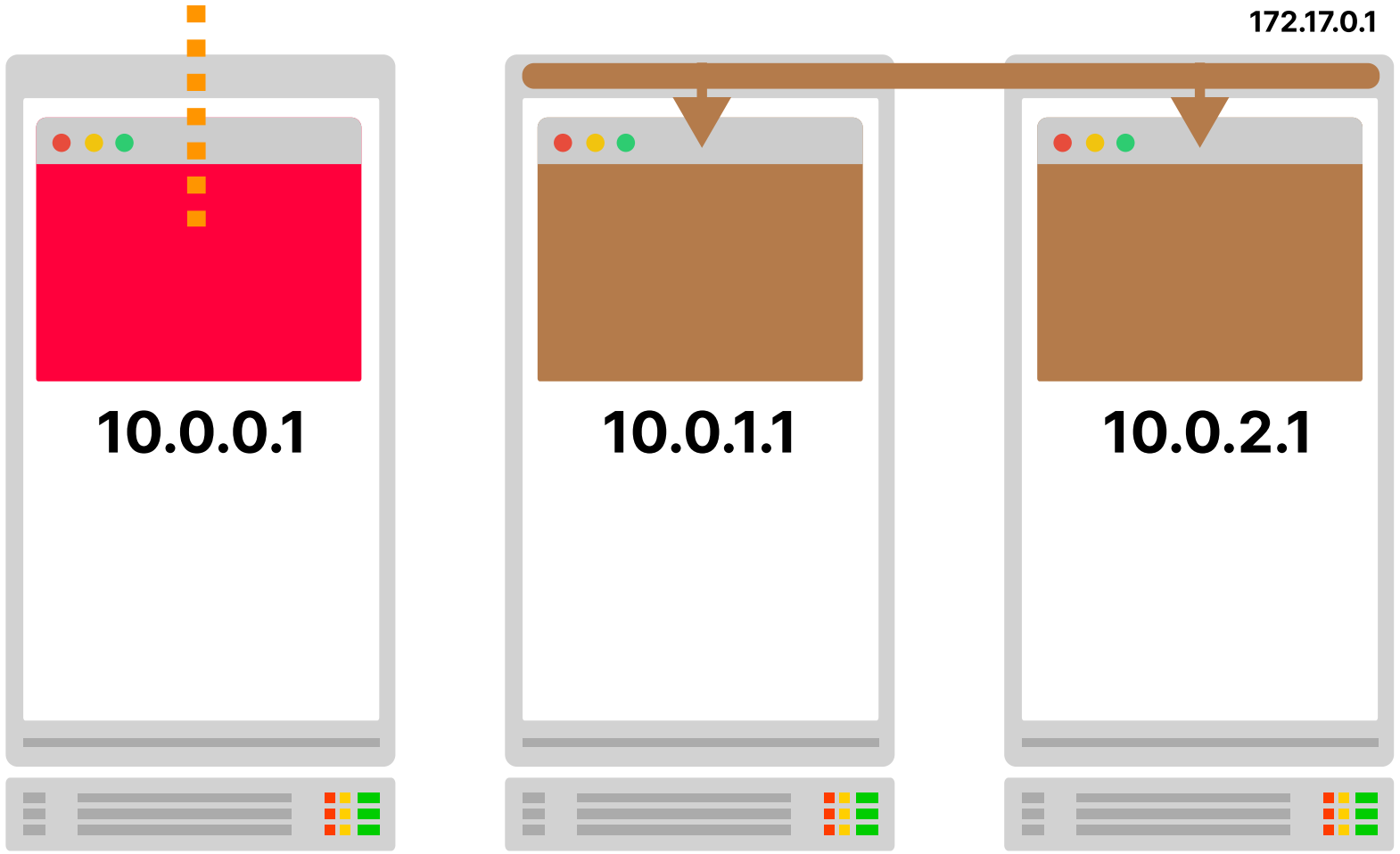


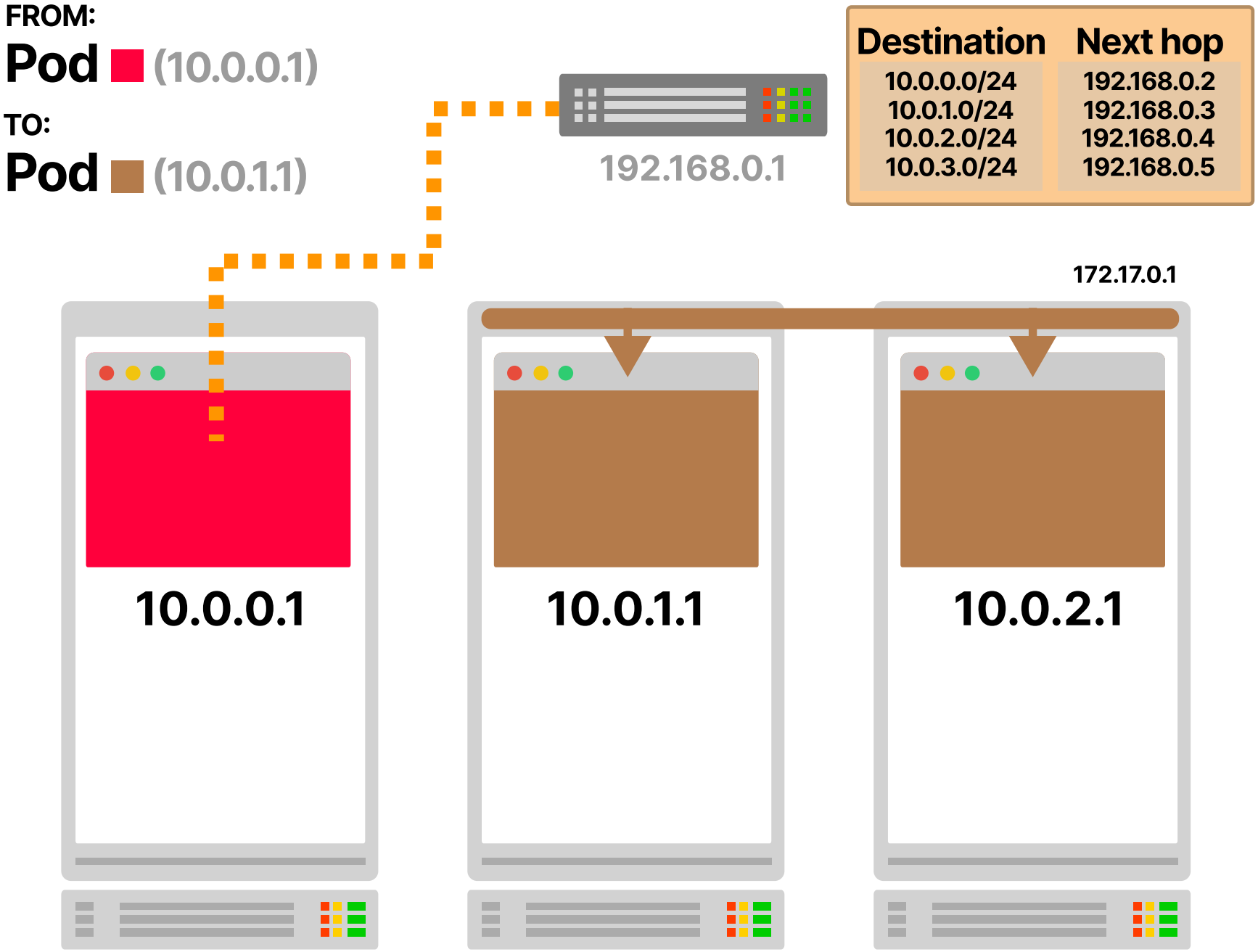
FROM:

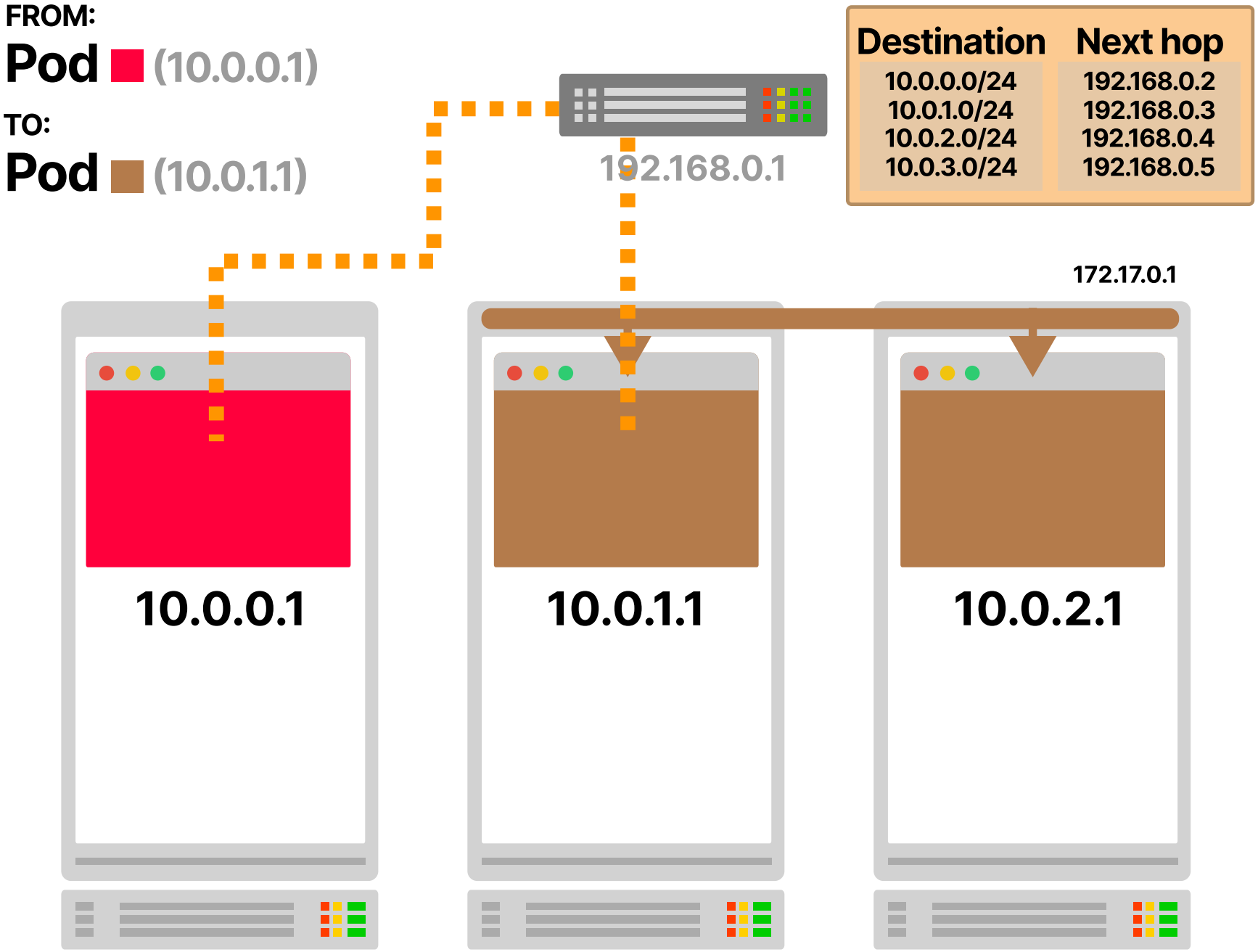
Pod (10.0.0.1)

TO:

Pod (10.0.1.1)




FROM:
Pod (10.0.0.1)
TO:
Pod (10.0.1.1)
192.168.0.1
Destination
Next hop
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5
172.17.0.1
10.0.0.1
10.0.1.1
10.0.2.1


FROM:
Pod (10.0.0.1)
TO:
Pod (10.0.1.1)
Destination
Next hop
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5
192.168.0.1
172.17.0.1
10.0.0.1
10.0.1.1
10.0.2.1

Response

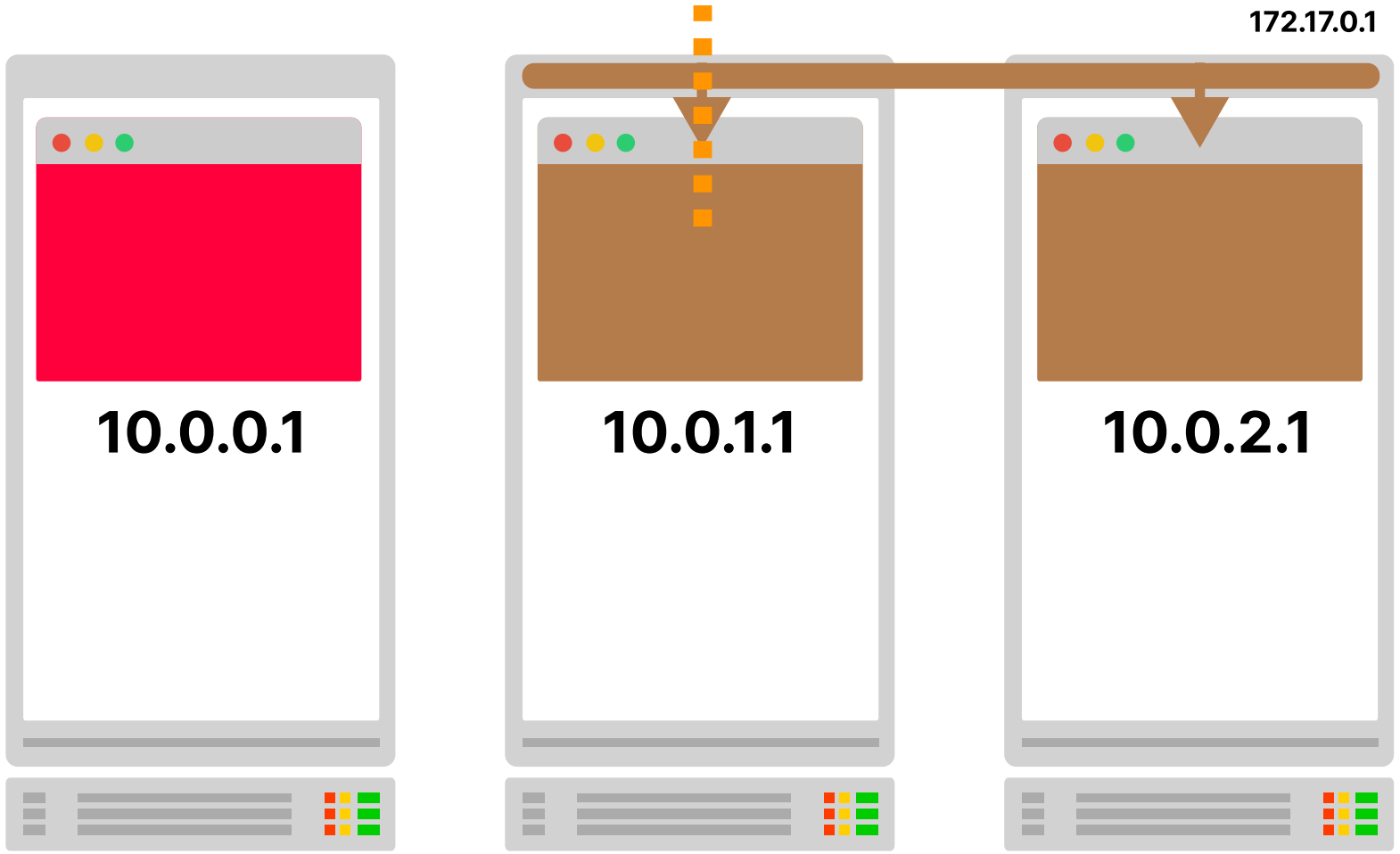
ClusterIP in practice

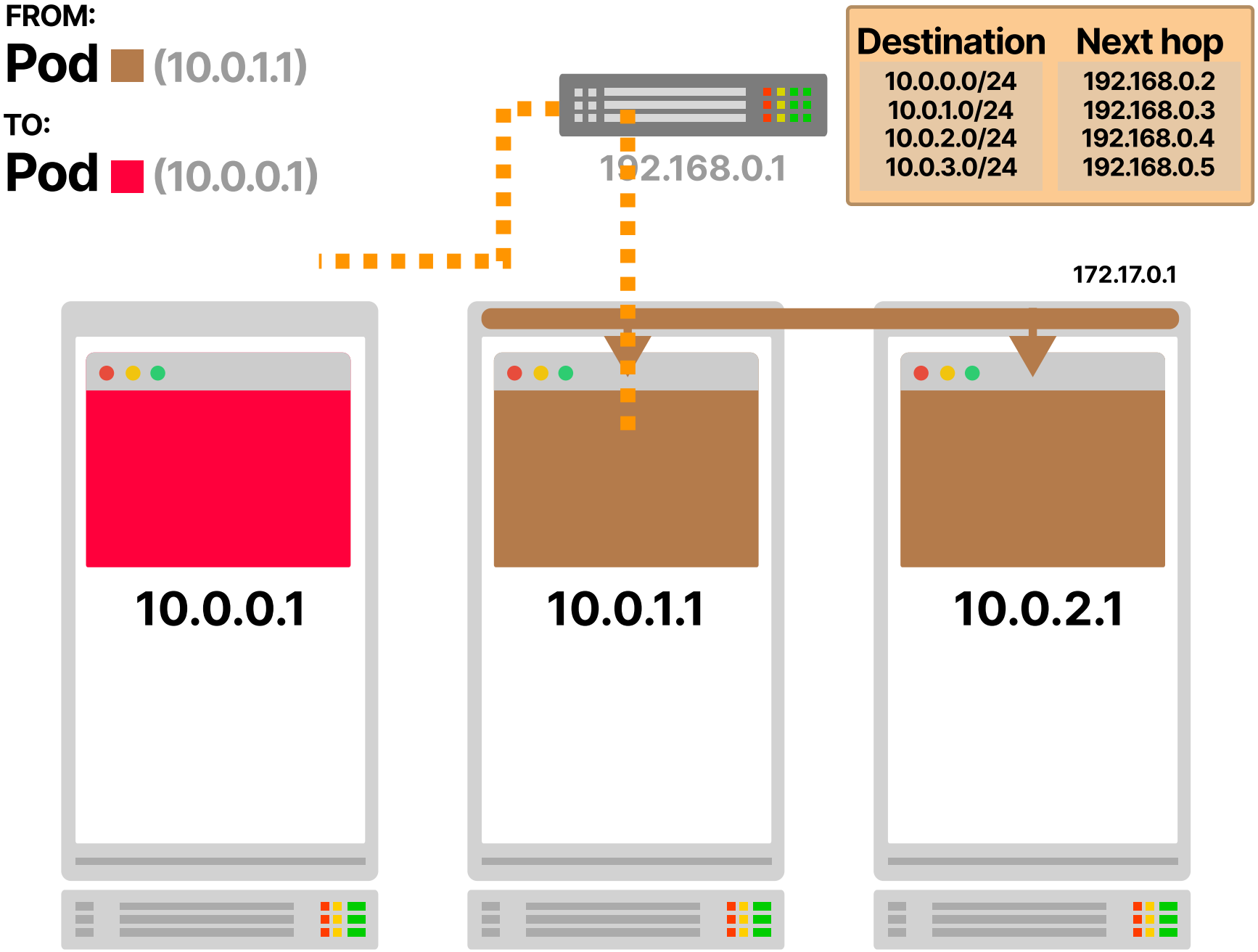
FROM:

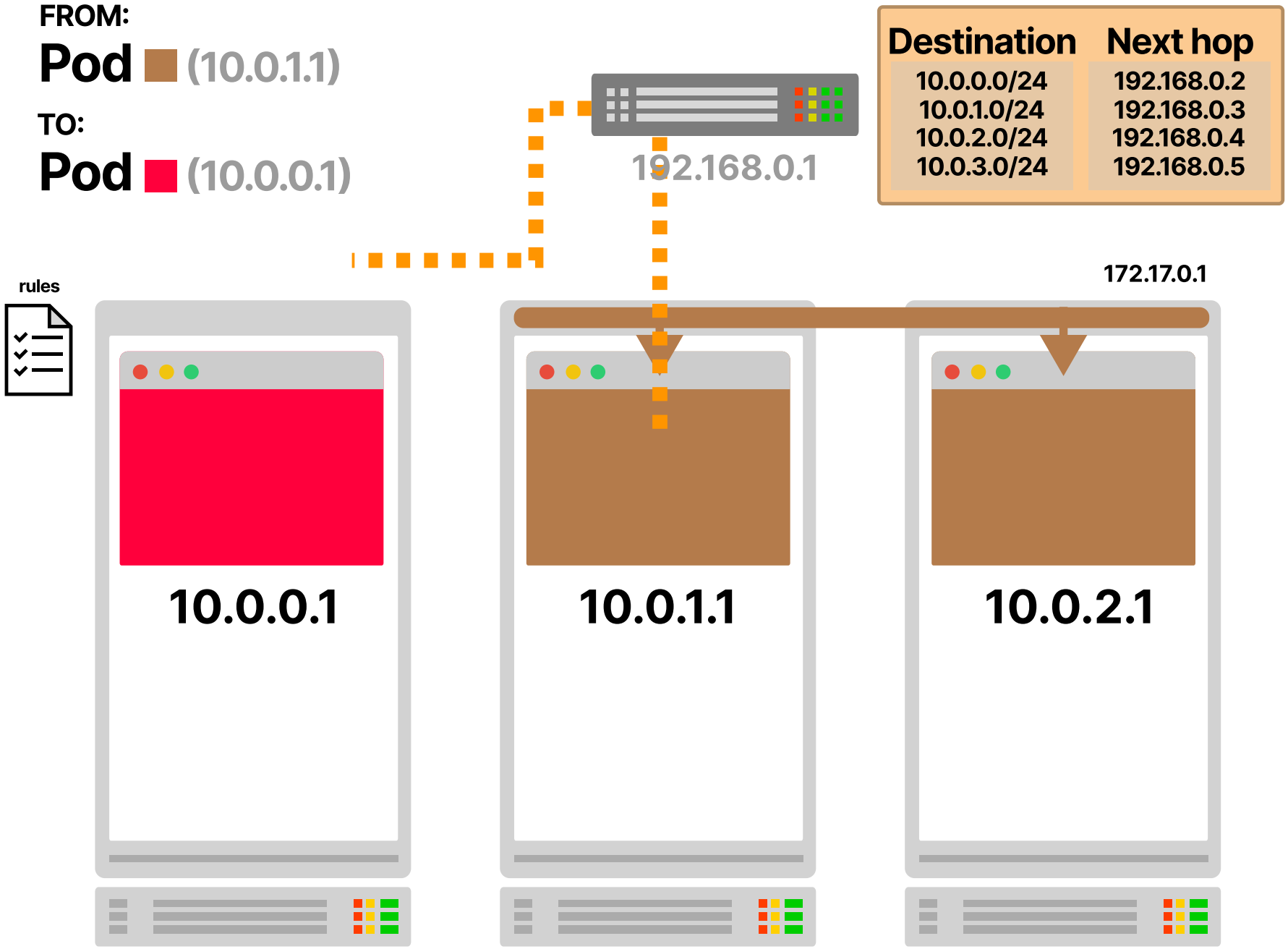
Pod (10.0.1.1)

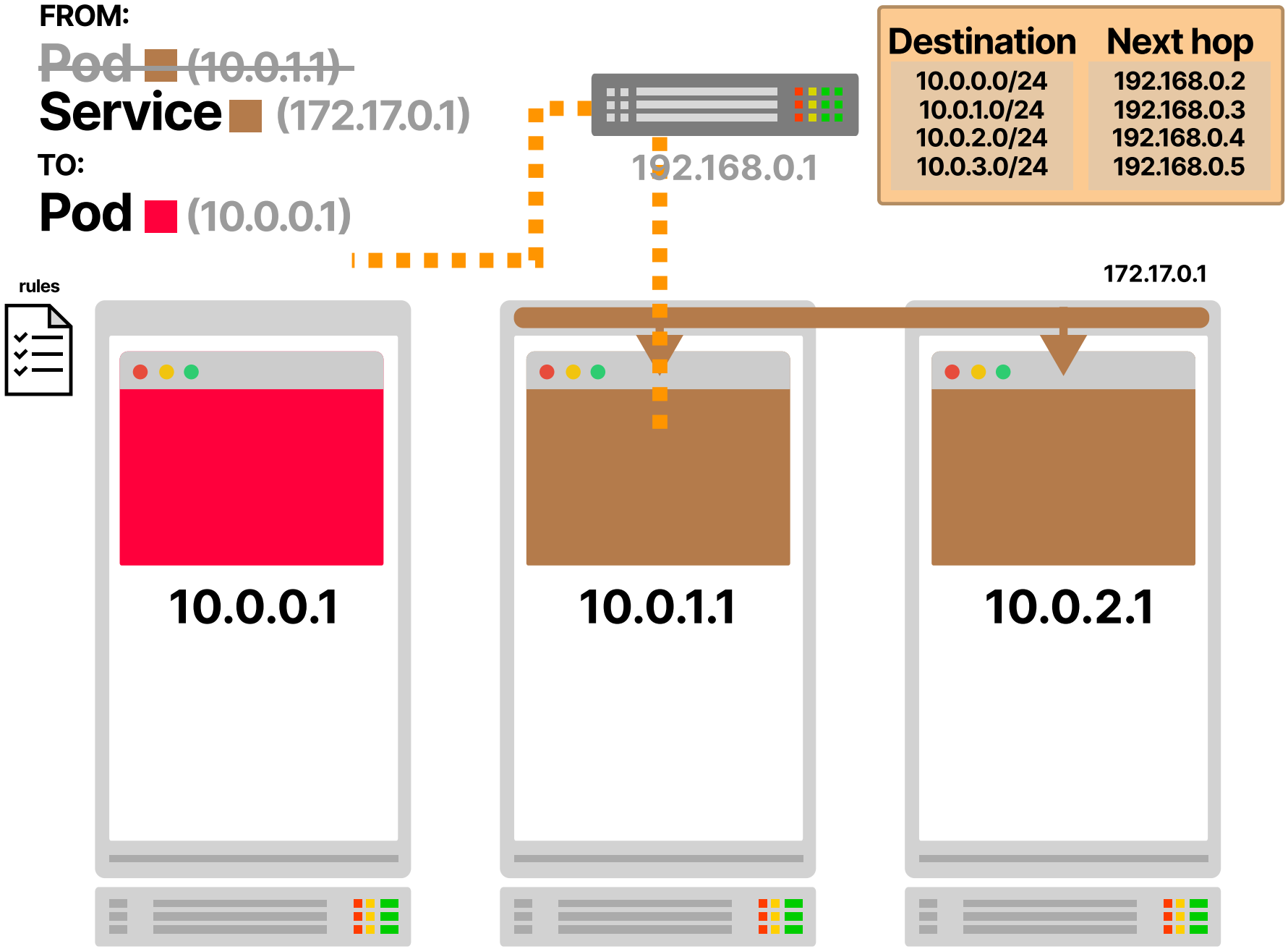
TO:

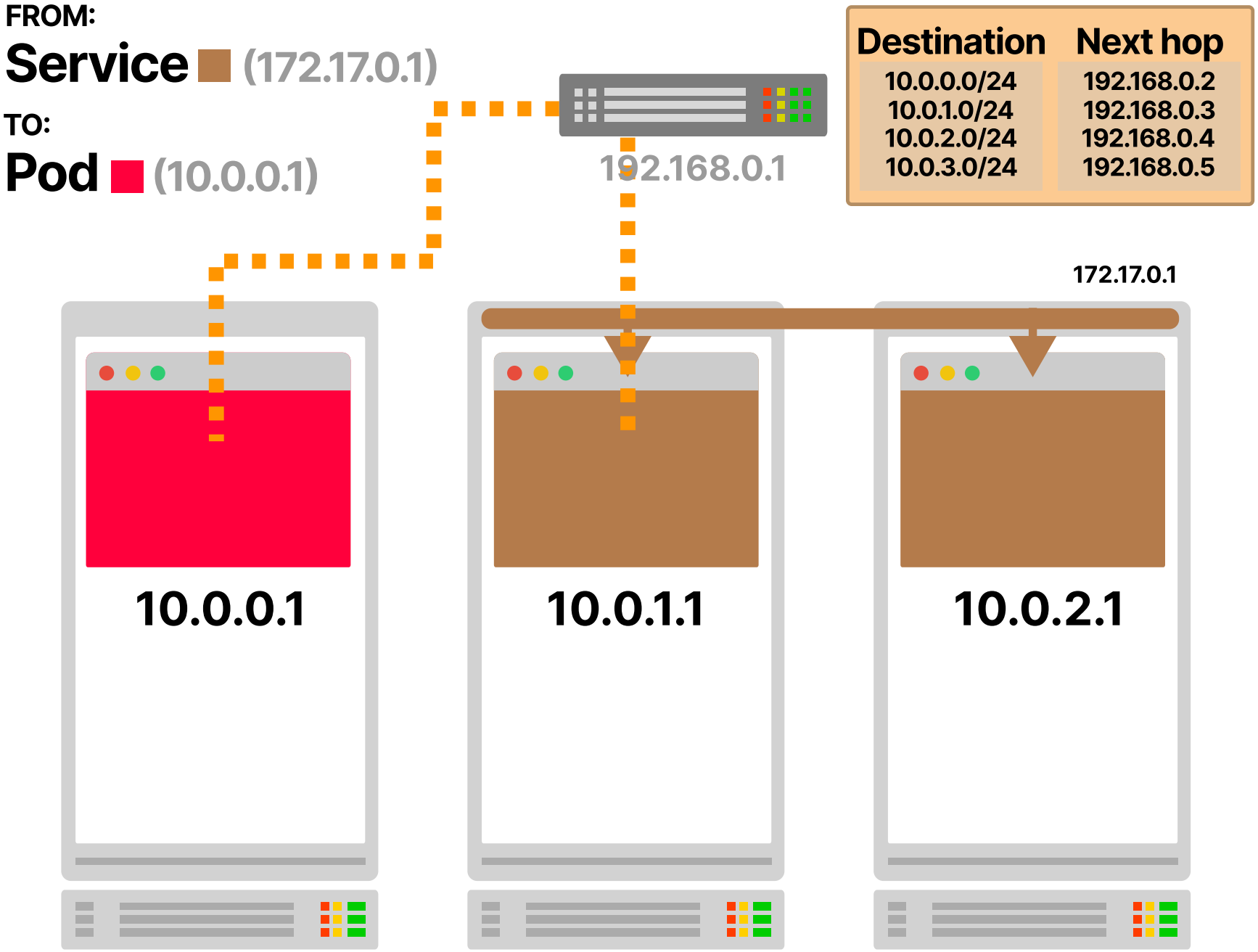
Pod (10.0.0.1)




FROM:
Pod (10.0.1.1)
TO:
Pod (10.0.0.1)
192.168.0.1
Destination
Next hop
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5
172.17.0.1
10.0.0.1
10.0.1.1
10.0.2.1


FROM:
Pod (10.0.1.1)
TO:
Pod (10.0.0.1)
Destination
Next hop
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5
192.168.0.1
172.17.0.1
rules
10.0.0.1
10.0.1.1
10.0.2.1


FROM:
Pod (10.0.1.1)
Service (172.17.0.1)
TO:
Pod (10.0.0.1)
Destination
Next hop
10.0.0.0/24
10.0.1.0/24
10.0.2.0/24
10.0.3.0/24
192.168.0.2
192.168.0.3
192.168.0.4
192.168.0.5
192.168.0.1
172.17.0.1
rules
10.0.0.1
10.0.1.1
10.0.2.1


FROM:
Service (172.17.0.1)
TO:
Pod (10.0.0.1)
192.168.0.1
Destination
Next hop
10.0.0.0/24
192.168.0.2
10.0.1.0/24
192.168.0.3
10.0.2.0/24
192.168.0.4
10.0.3.0/24
192.168.0.5
172.17.0.1
10.0.0.1
10.0.1.1
10.0.2.1

kube-proxy

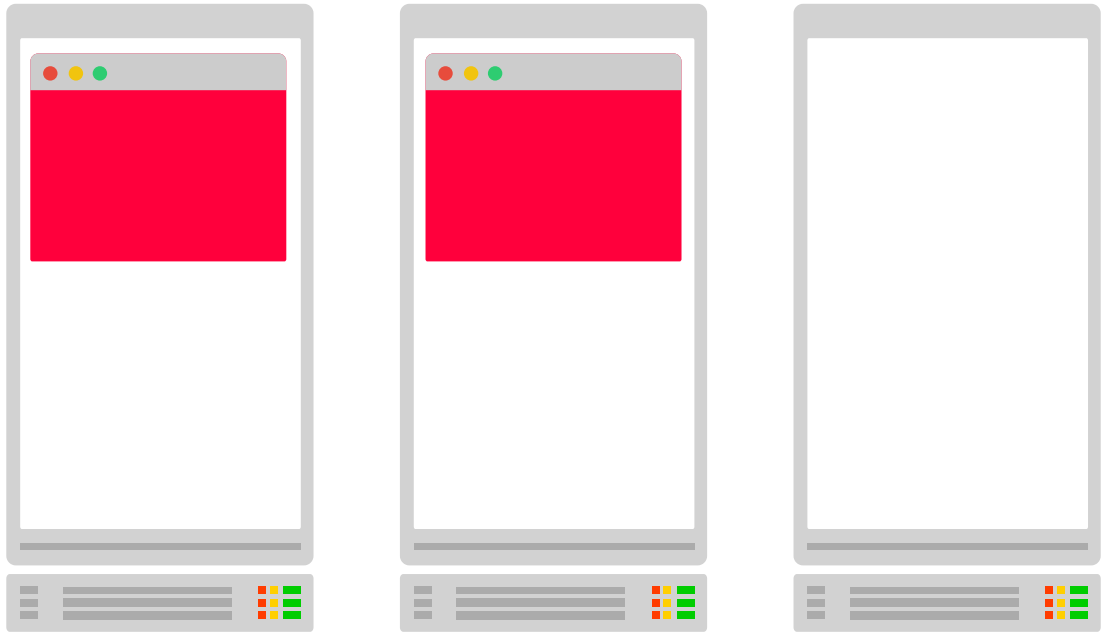


IP address to intercept	Replace with



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints





IP address to intercept	Replace with

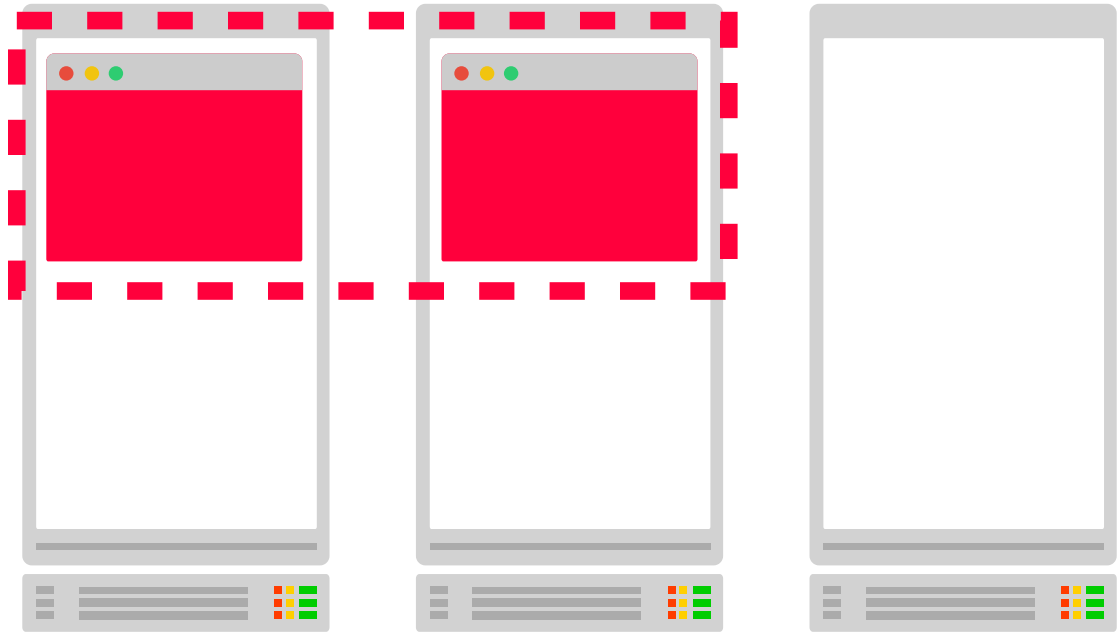


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints

SERVICE

ClusterIP





IP address to intercept	Replace with

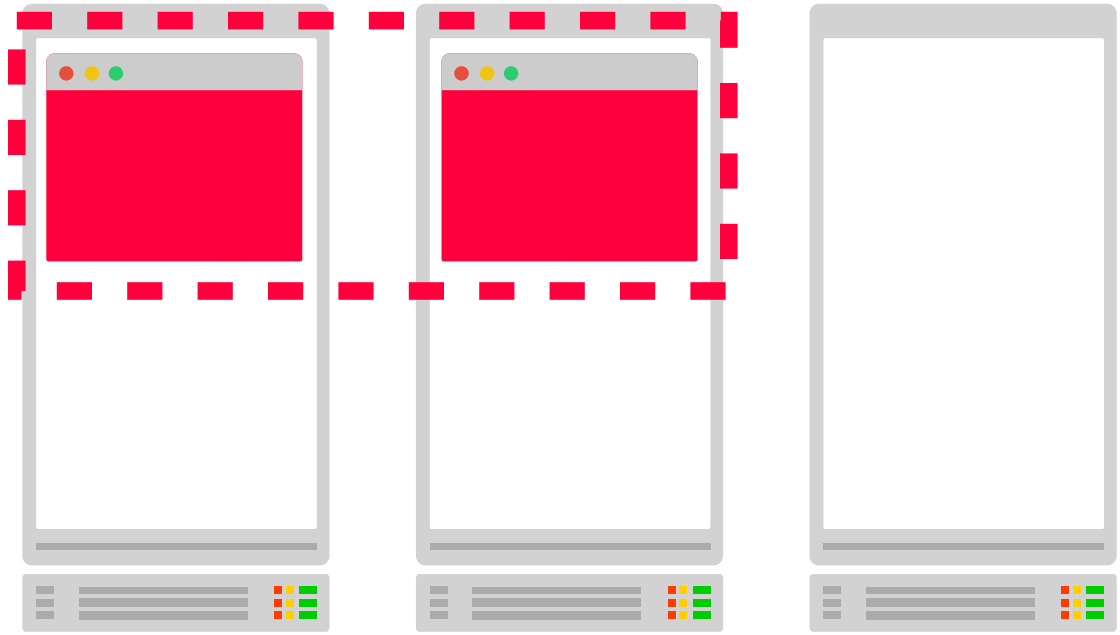


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

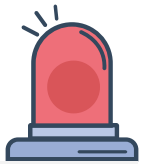
Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000

SERVICE

ClusterIP





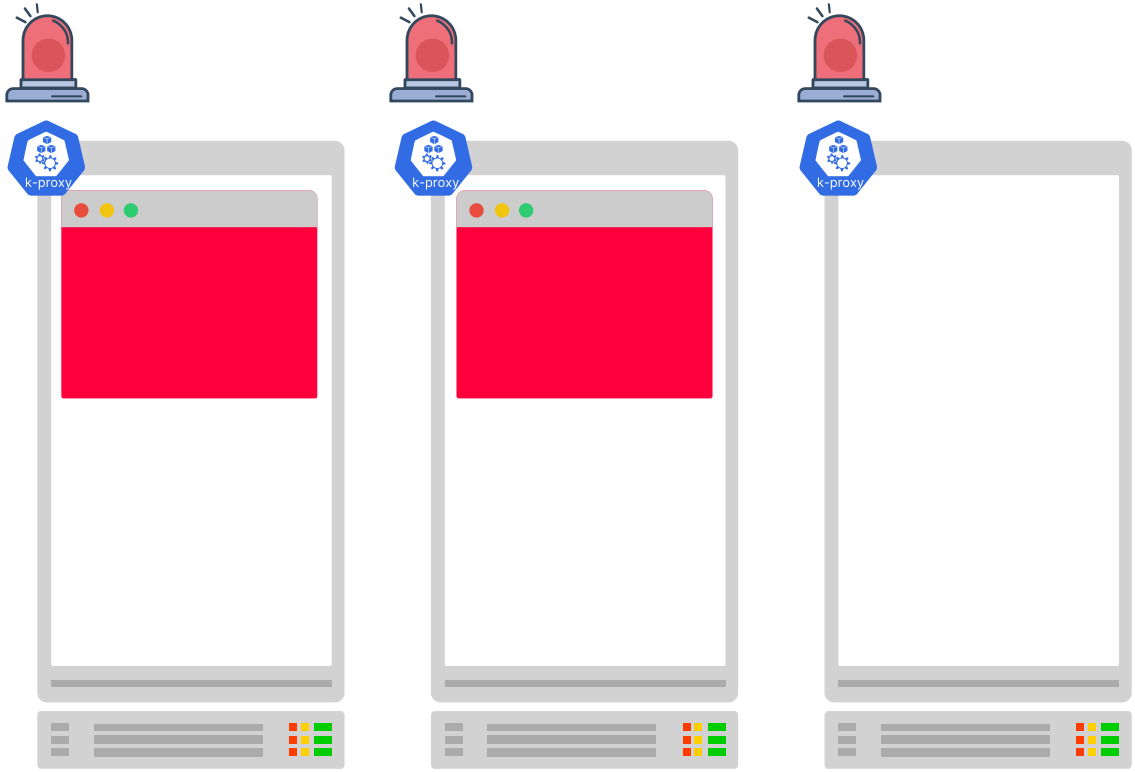


IP address to intercept	Replace with



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000



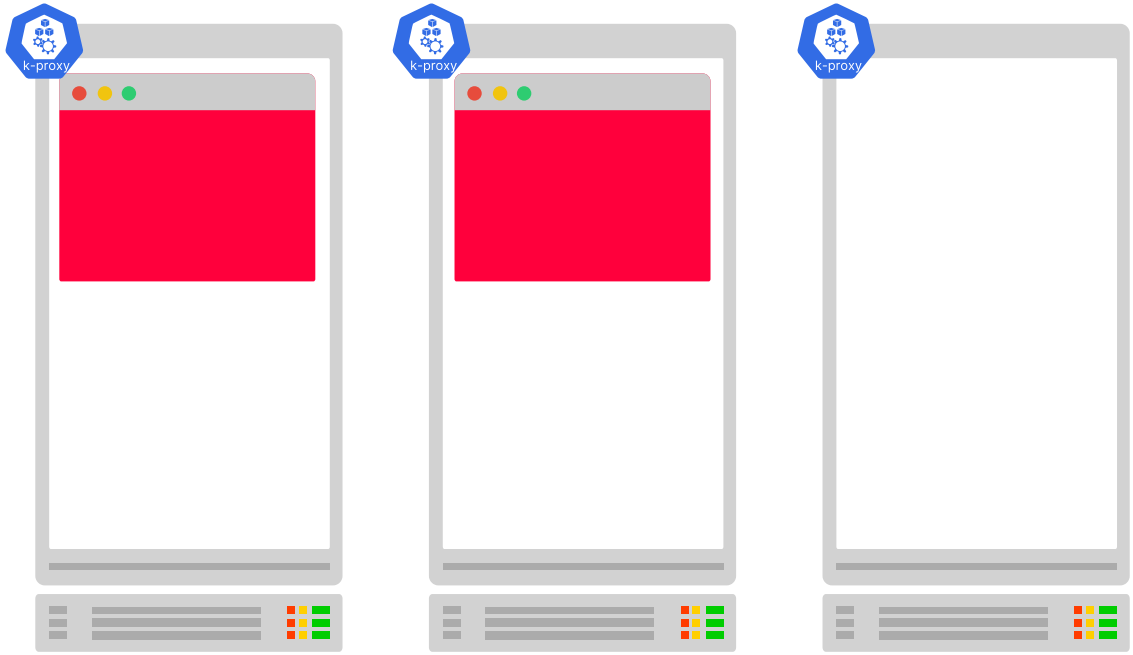


IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000



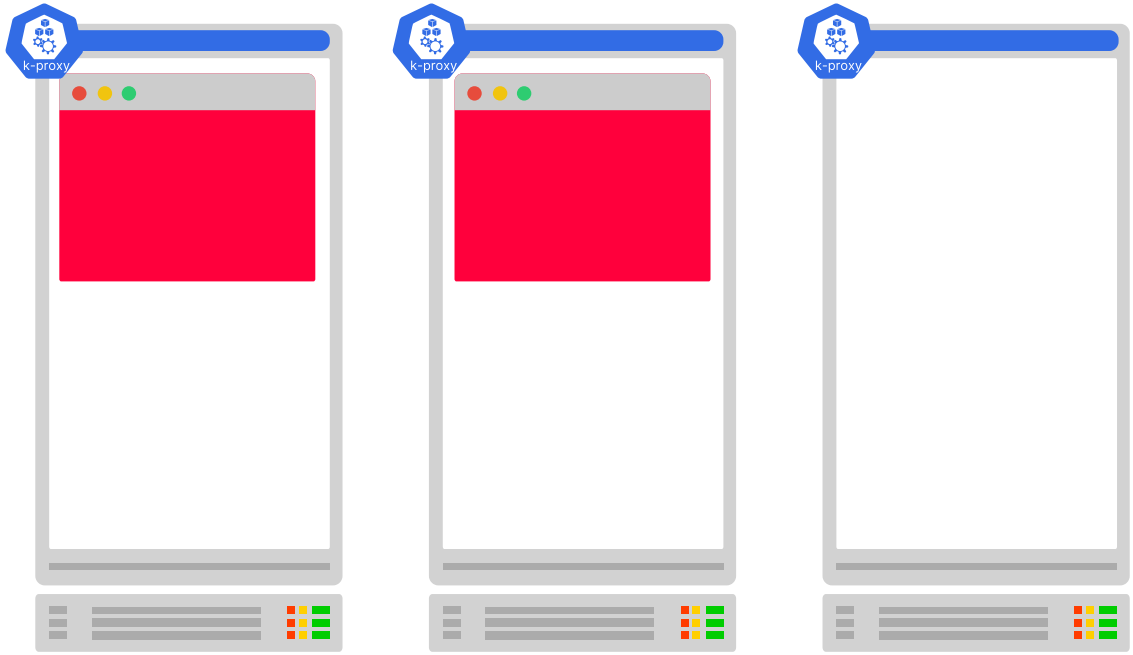


IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000





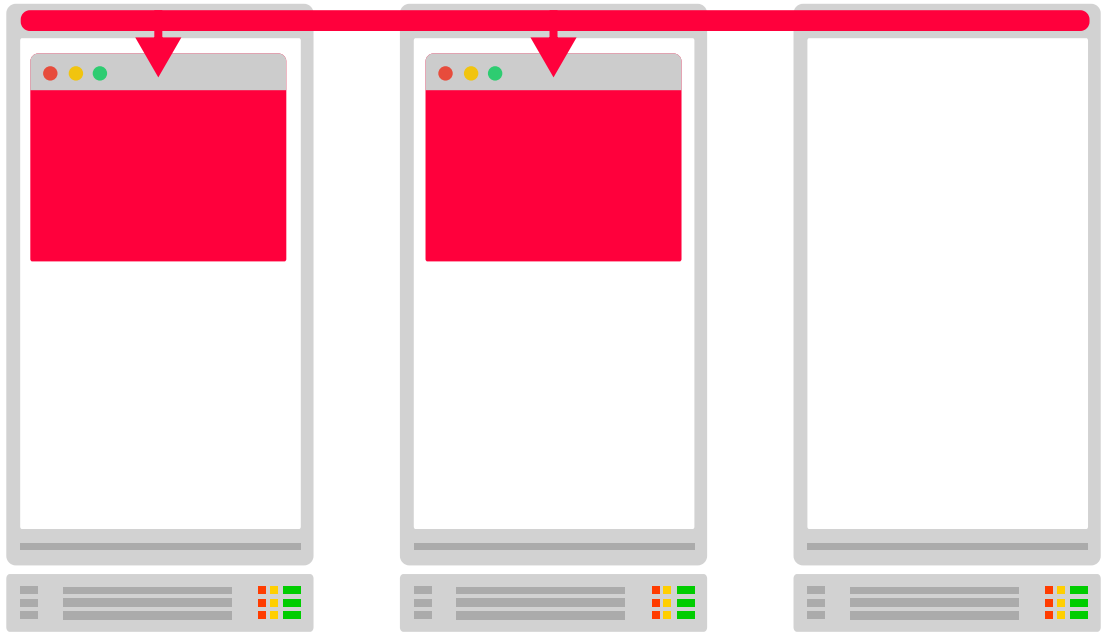
IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000

SERVICE
ClusterIP
172.17.0.1





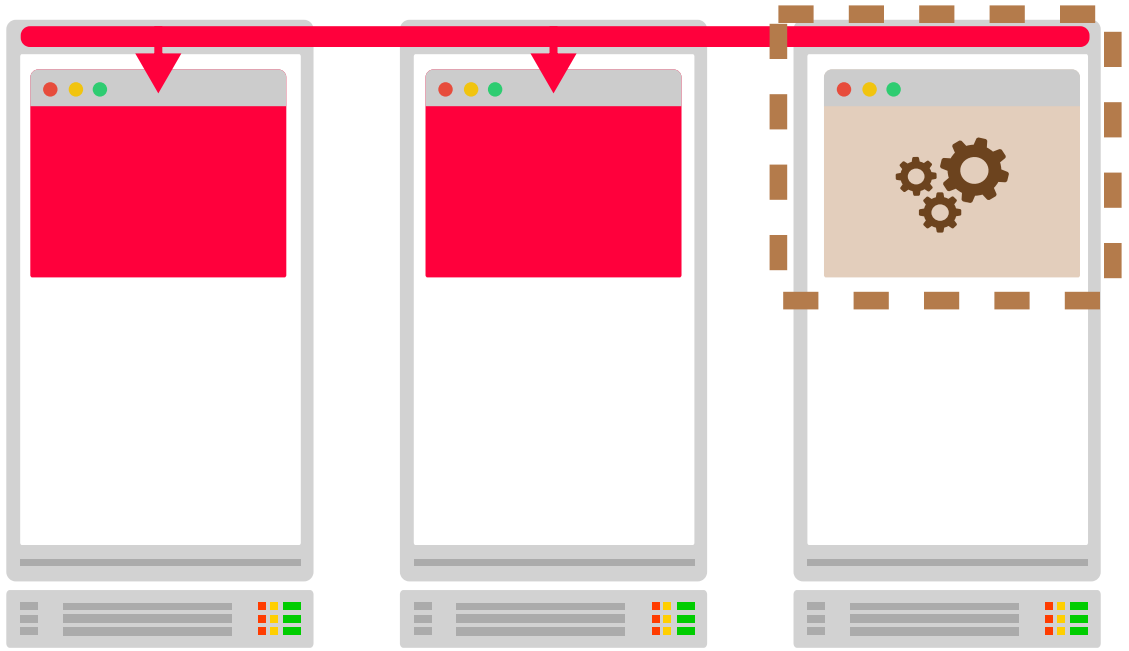
IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000

SERVICE
ClusterIP





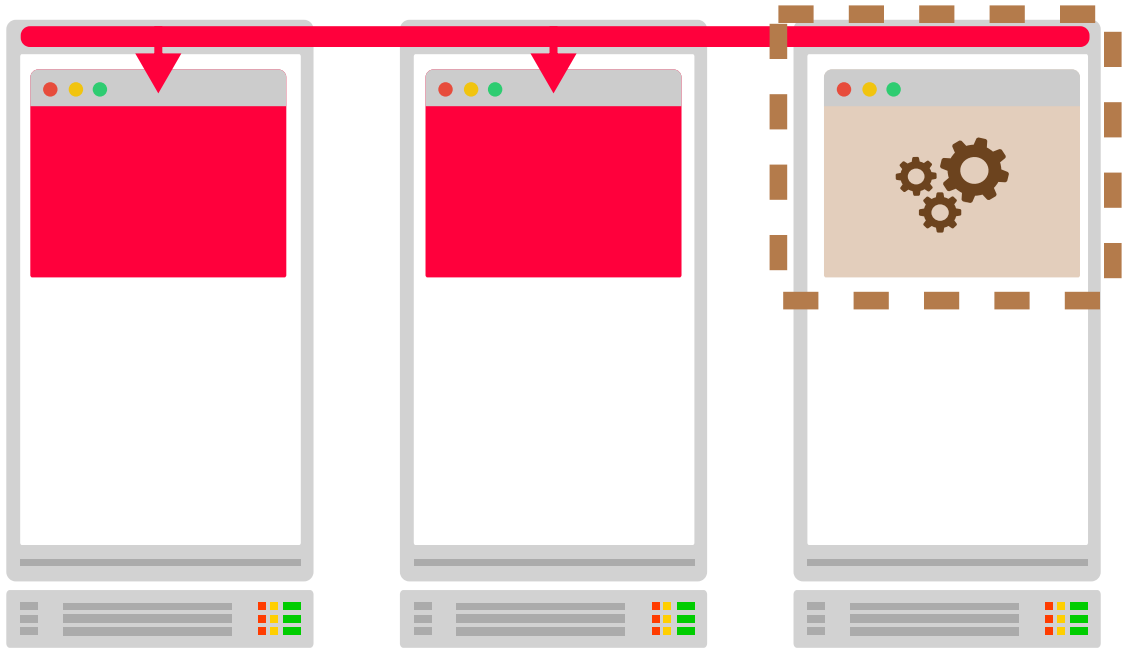
IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1



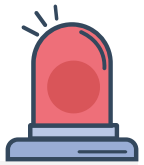
Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000

SERVICE
ClusterIP





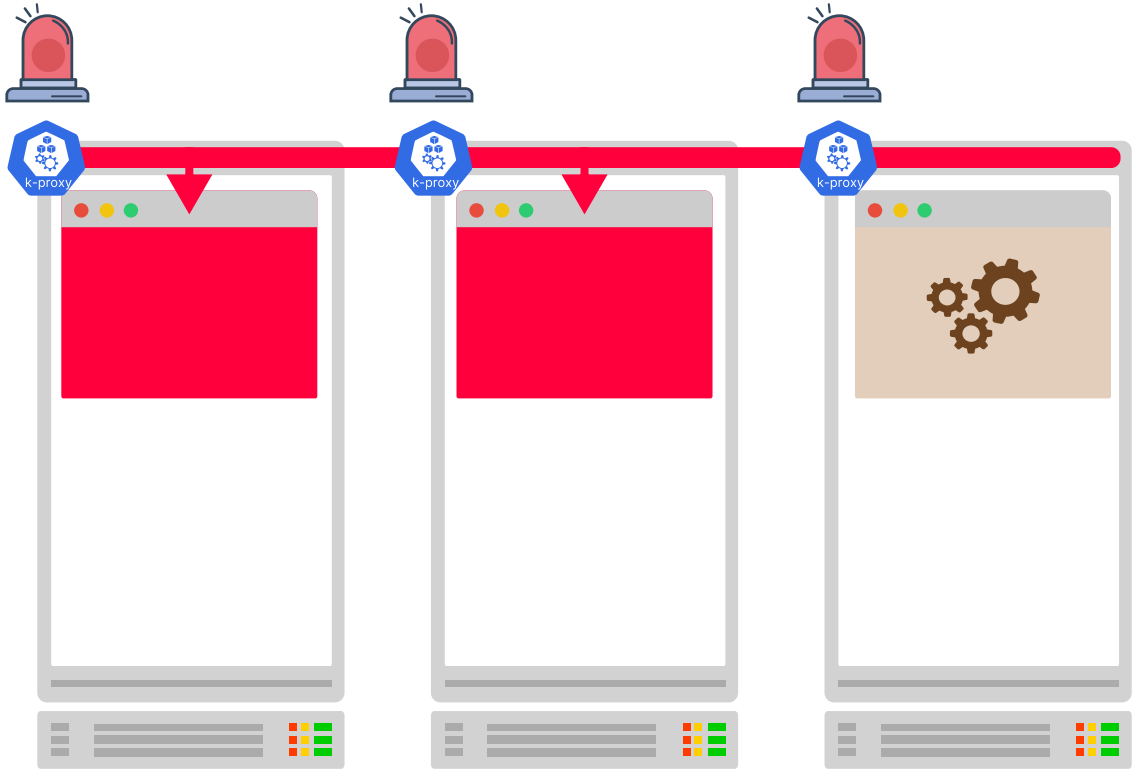


IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000
Beige	172.17.0.2	10.0.2.1:3000



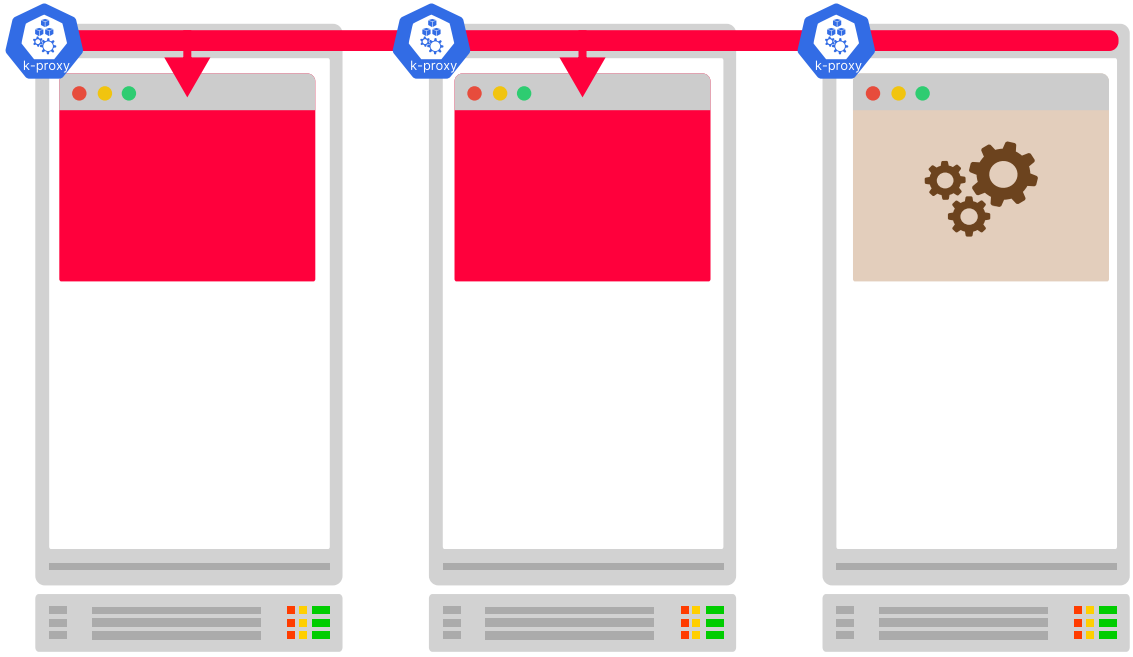


IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1
172.17.0.2	10.0.2.1



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000
Beige	172.17.0.2	10.0.2.1:3000



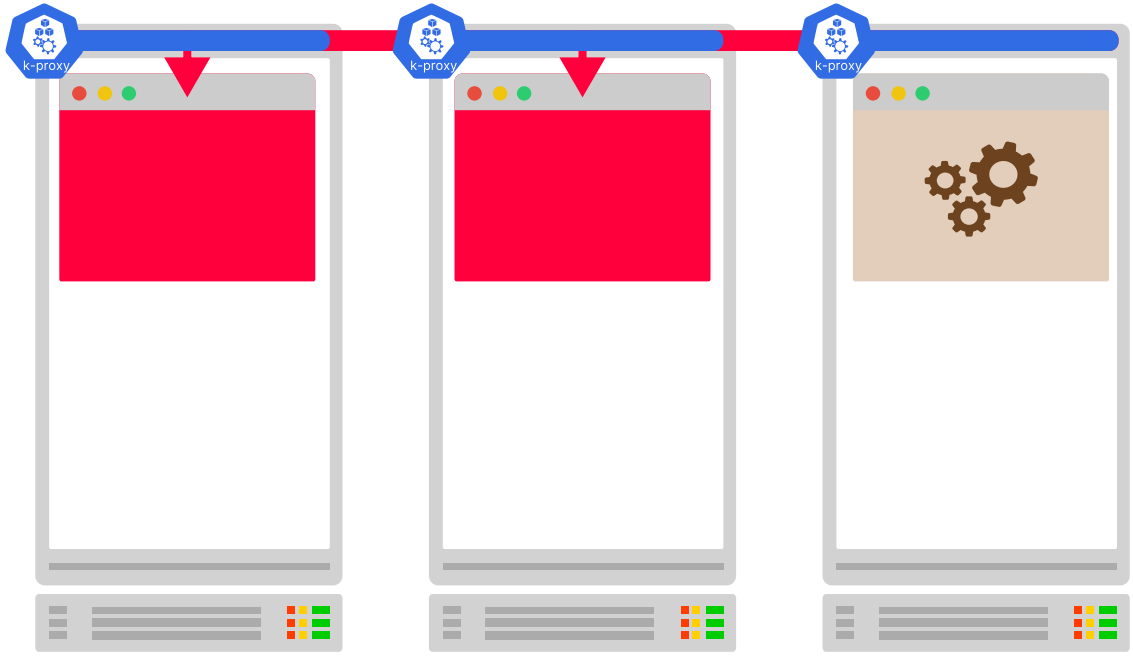


IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1
172.17.0.2	10.0.2.1



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000
Beige	172.17.0.2	10.0.2.1:3000



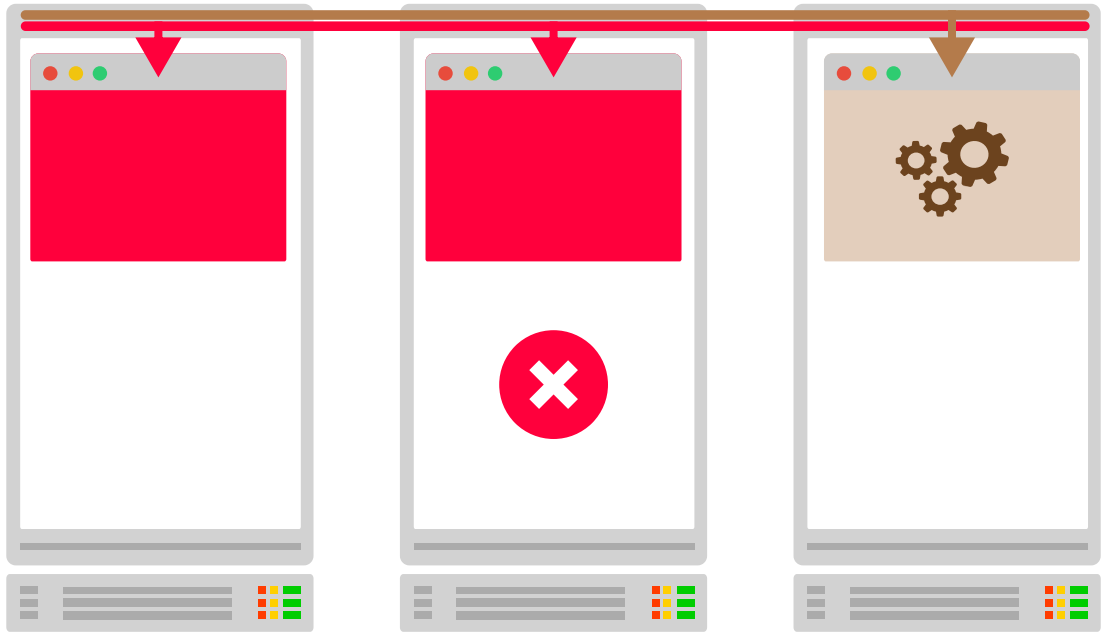


IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1
172.17.0.2	10.0.2.1



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000
Beige	172.17.0.2	10.0.2.1:3000



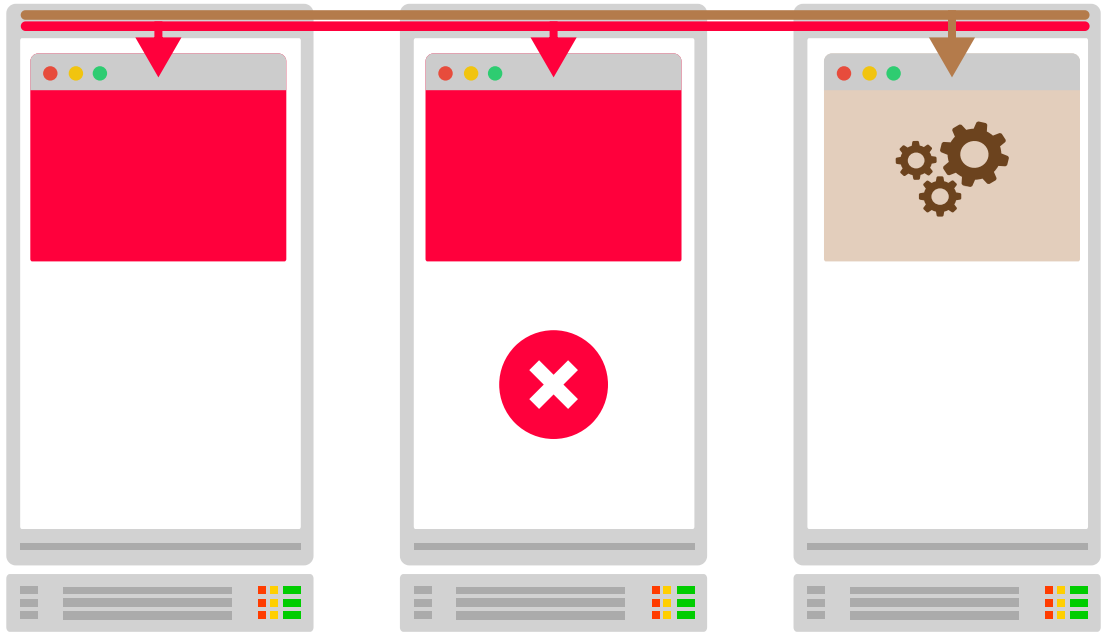


IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1
172.17.0.2	10.0.2.1

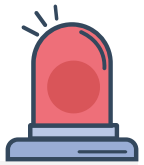


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1
Pod 3	RUNNING	worker3	10.0.2.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000
Beige	172.17.0.2	10.0.2.1:3000





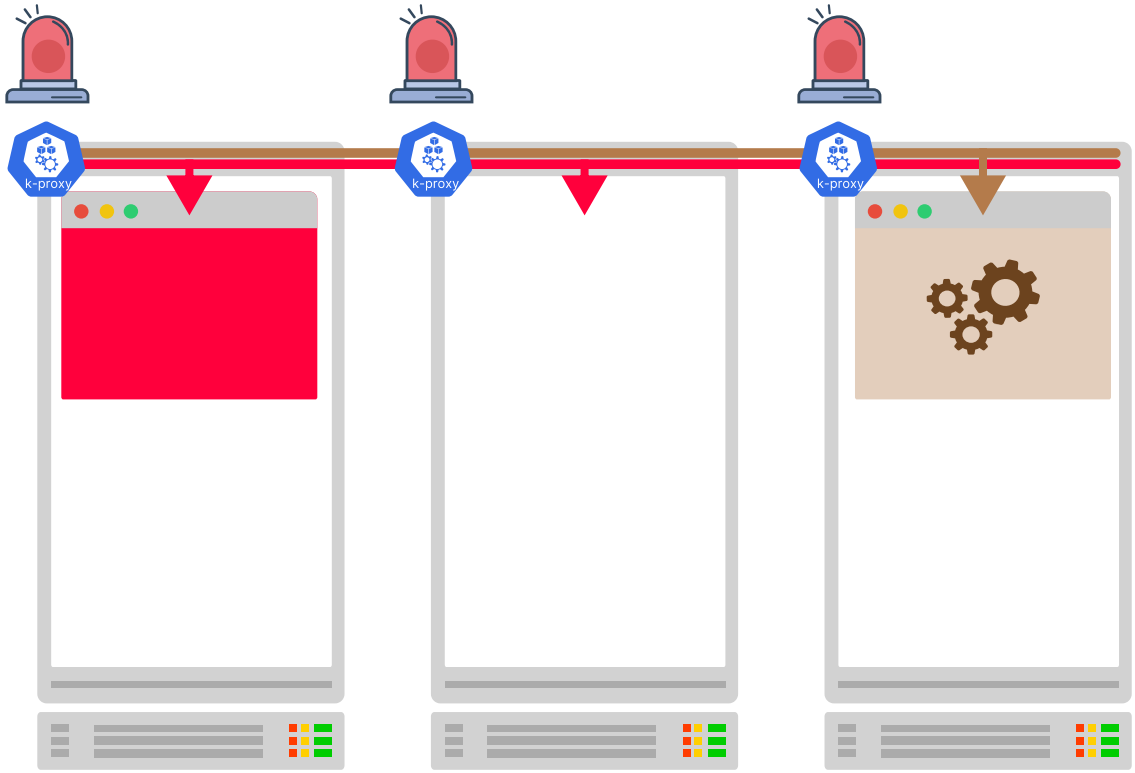


IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1
172.17.0.2	10.0.2.1



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 3	RUNNING	worker3	10.0.2.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000
Beige	172.17.0.2	10.0.2.1:3000



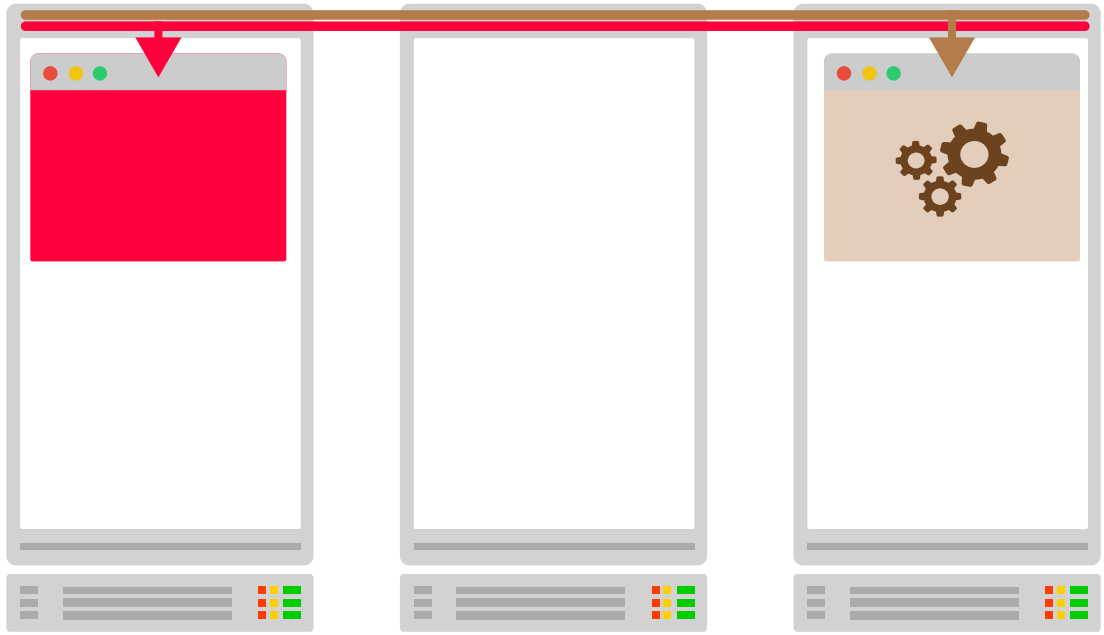


IP address to intercept	Replace with
172.17.0.1	10.0.0.1,
172.17.0.2	10.0.2.1



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 3	RUNNING	worker3	10.0.2.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000
Beige	172.17.0.2	10.0.2.1:3000



ClusterIP and DNS



Service name	IP addresses



IP address to intercept	Replace with

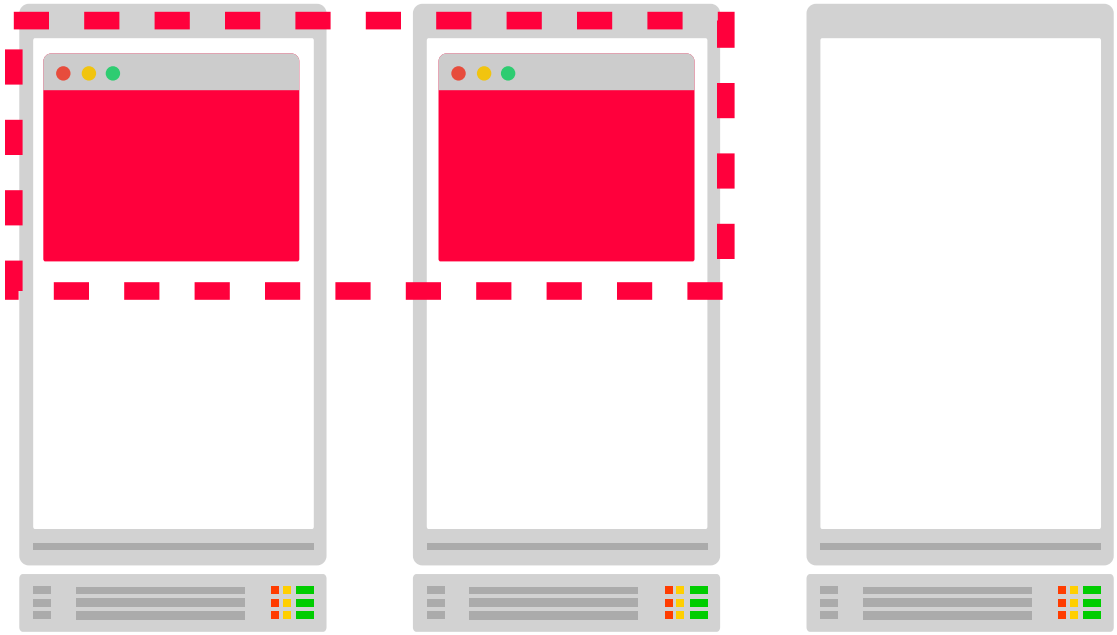


Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

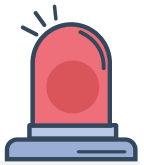
Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000

SERVICE

ClusterIP

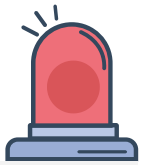






Service name	IP addresses



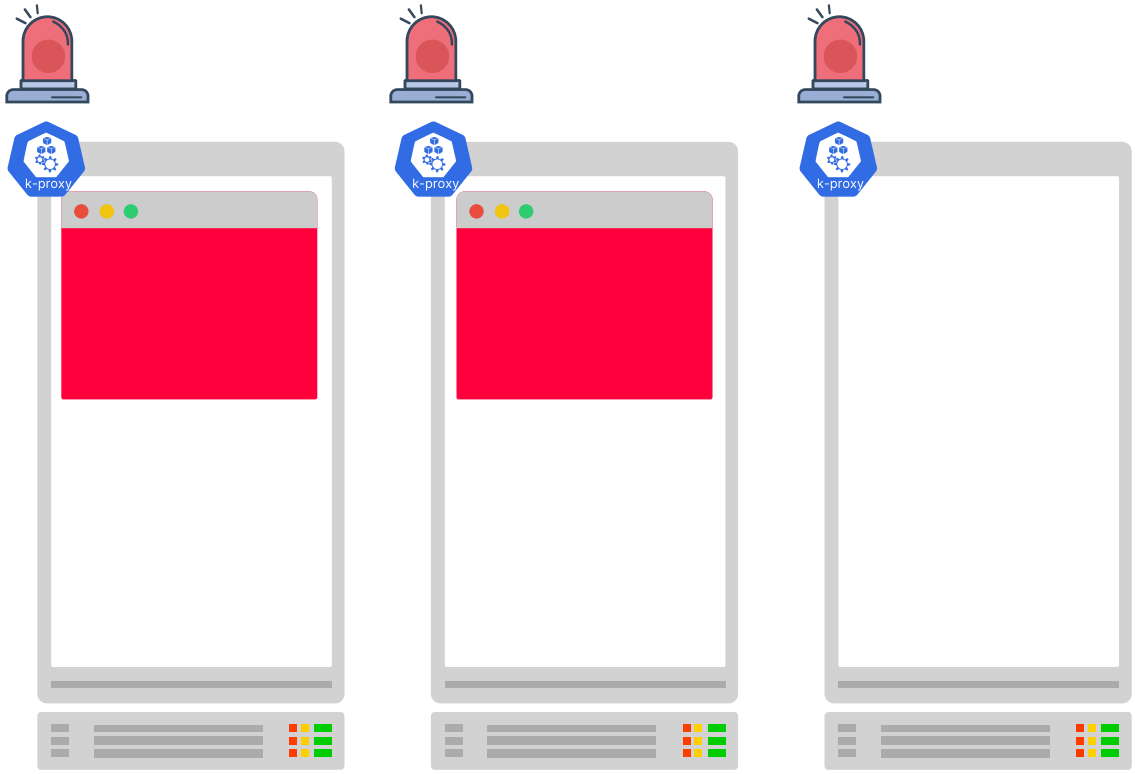


IP address to intercept	Replace with



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000





Service name	IP addresses
red.namespace.svc.cluster.local	172.17.0.1

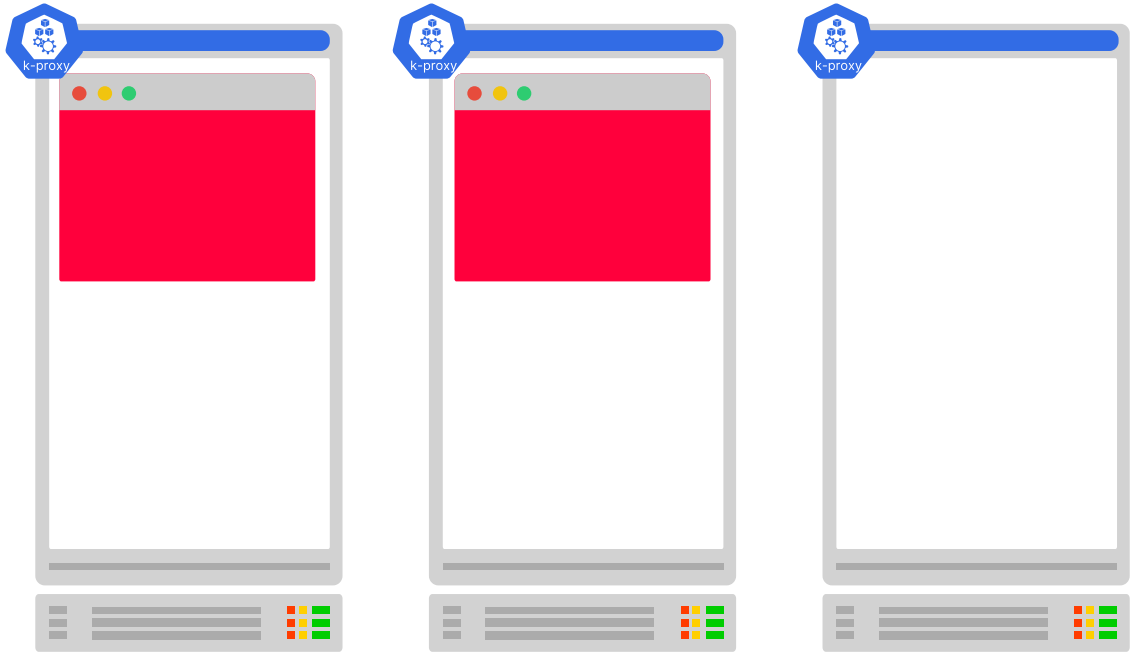


IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000



Service name	IP addresses
red.namespace.svc.cluster.local	172.17.0.1

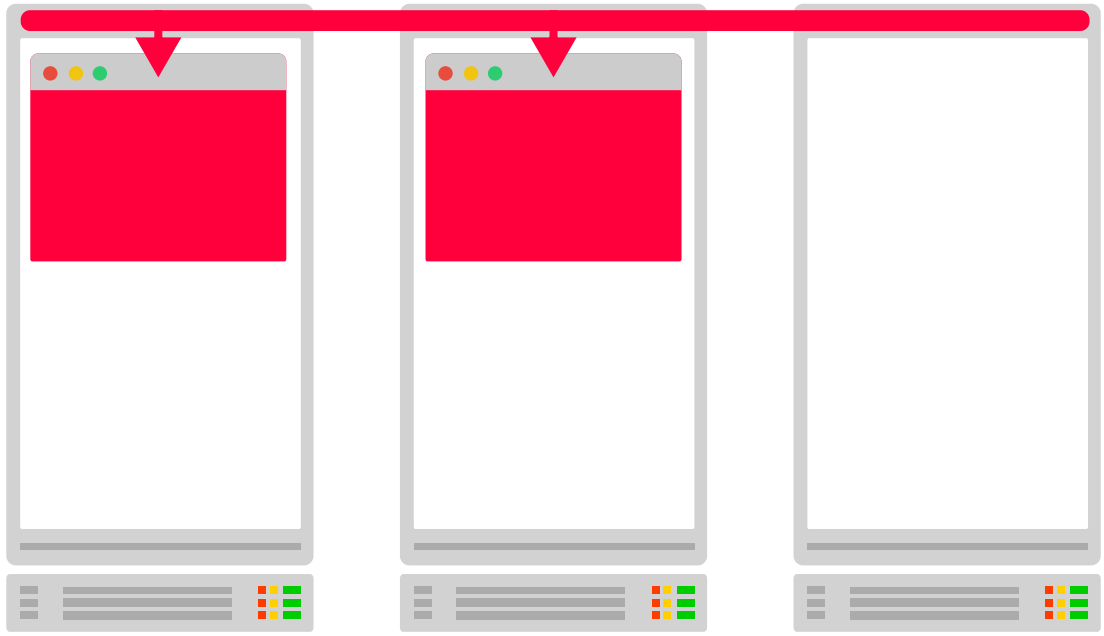
k-proxy

IP address to intercept	Replace with
172.17.0.1	10.0.0.1, 10.0.1.1

Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000

SERVICE
ClusterIP
172.17.0.1



Headless vs ClusterIP

01 IP address


ClusterIP
Headless

ClusterIP

Headless



ClusterIP

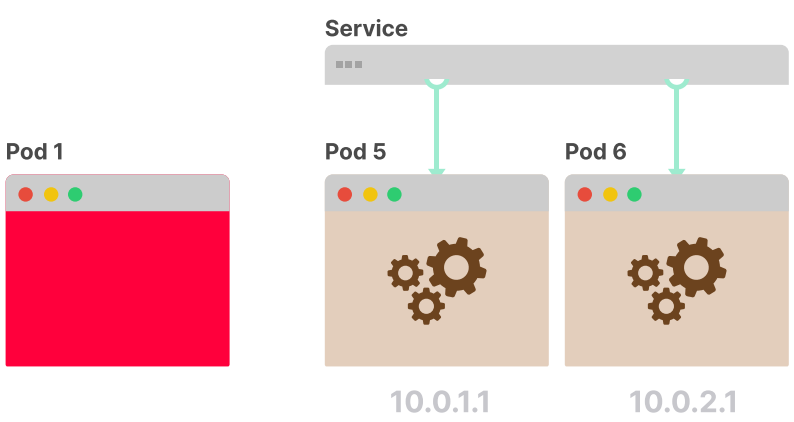
Headless

IP address

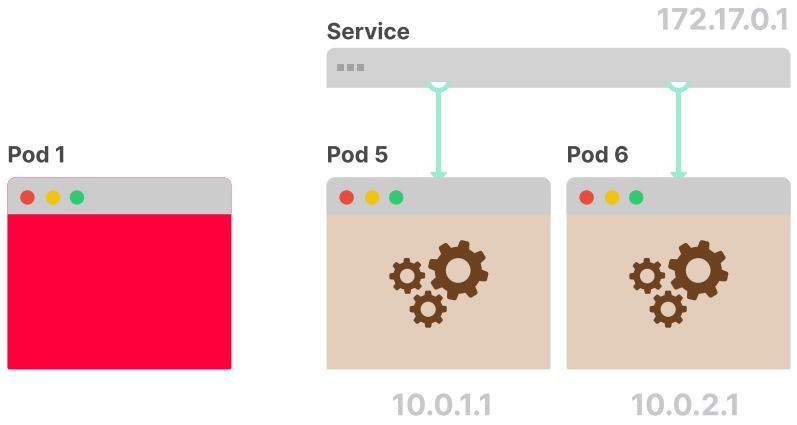




Headless



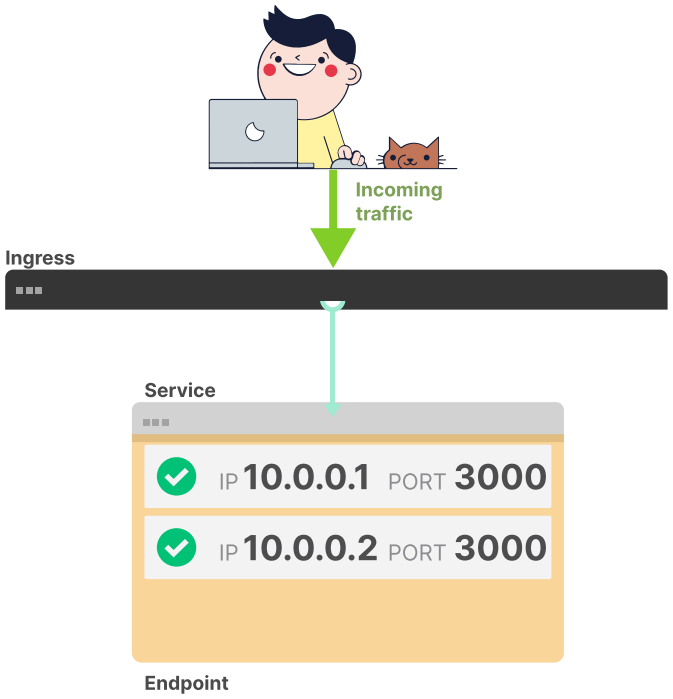
ClusterIP



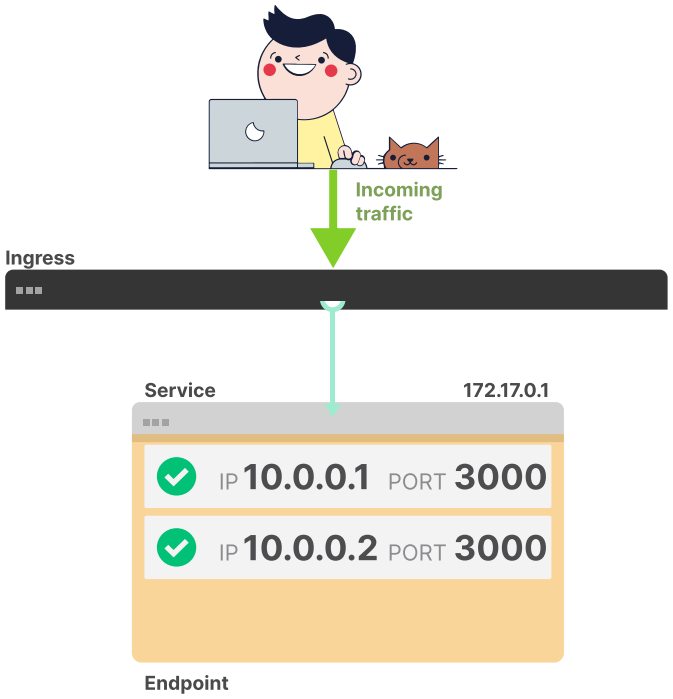
Headless vs ClusterIP

02 Endpoints

Headless



ClusterIP



Headless



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	Endpoints
Red	10.0.0.1:3000,10.0.1.1:3000

ClusterIP



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000

Headless



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	Endpoints
Red	10.0.0.1:3000,10.0.1.1:3000

ClusterIP



Pod name	Status	Node	podIP
Pod 1	RUNNING	worker1	10.0.0.1
Pod 2	RUNNING	worker2	10.0.1.1

Service name	IP address	Endpoints
Red	172.17.0.1	10.0.0.1:3000,10.0.1.1:3000

Headless vs ClusterIP

03 DNS

ClusterIP

```
~$ dig clusterip-service.default.svc.cluster.local
```

ClusterIP

```
~$ dig clusterip-service.default.svc.cluster.local

;; QUESTION SECTION:
;name.default.svc.cluster.local. IN A

;; ANSWER SECTION:
name.default.svc.cluster.local. 30 IN A 172.17.0.1
```

Headless



```
~$ dig headless-service.default.svc.cluster.local
```

Headless

```
~$ dig headless.default.svc.cluster.local

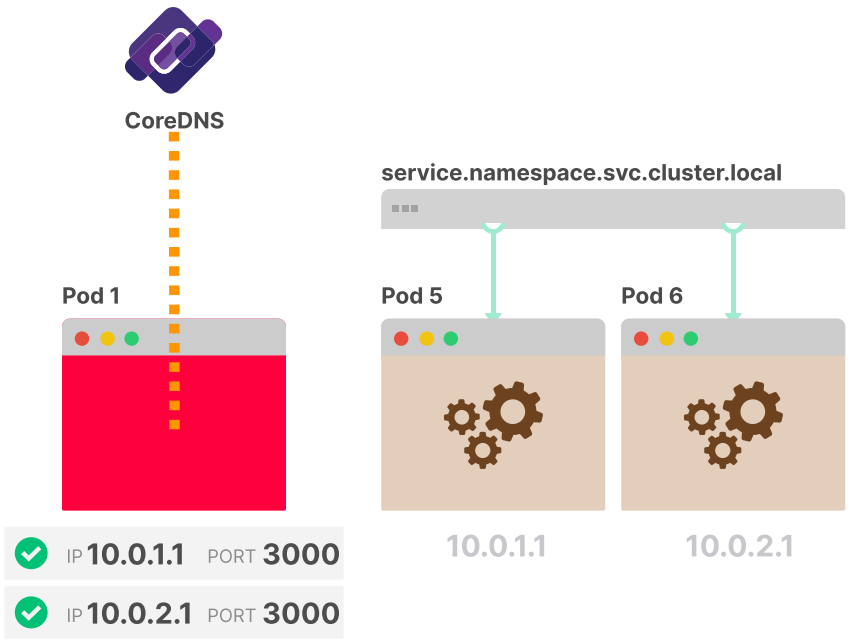
;; QUESTION SECTION:
;name.default.svc.cluster.local. IN A

;; ANSWER SECTION:
name.default.svc.cluster.local. 13 IN
 A 10.0.0.1
name.default.svc.cluster.local. 13 IN
 A 10.0.0.2
```

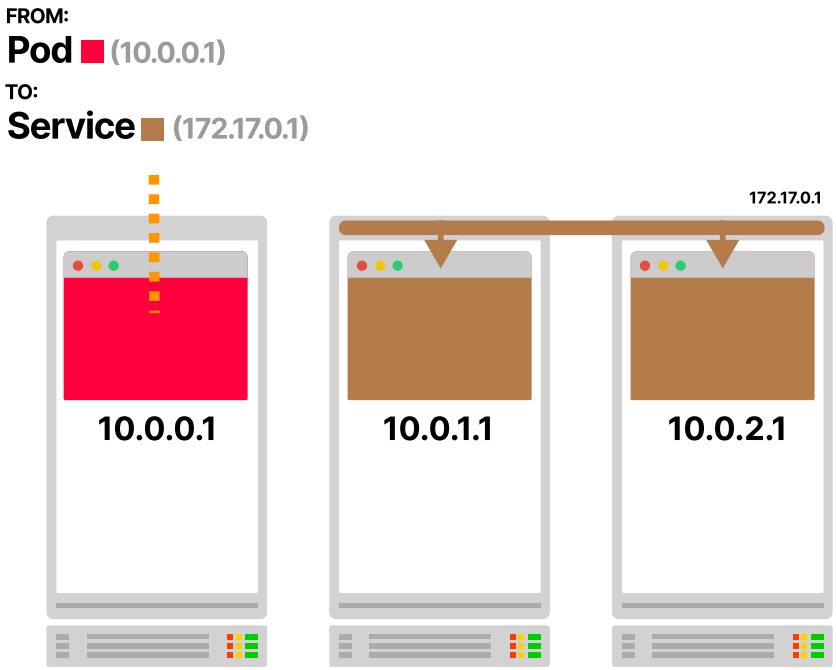
Headless vs ClusterIP

04 Load balancing

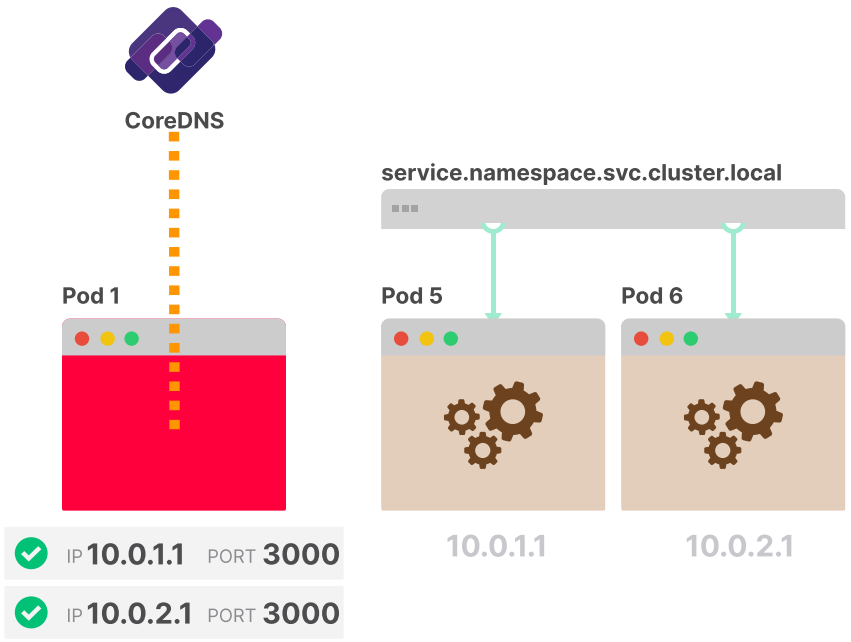
Headless



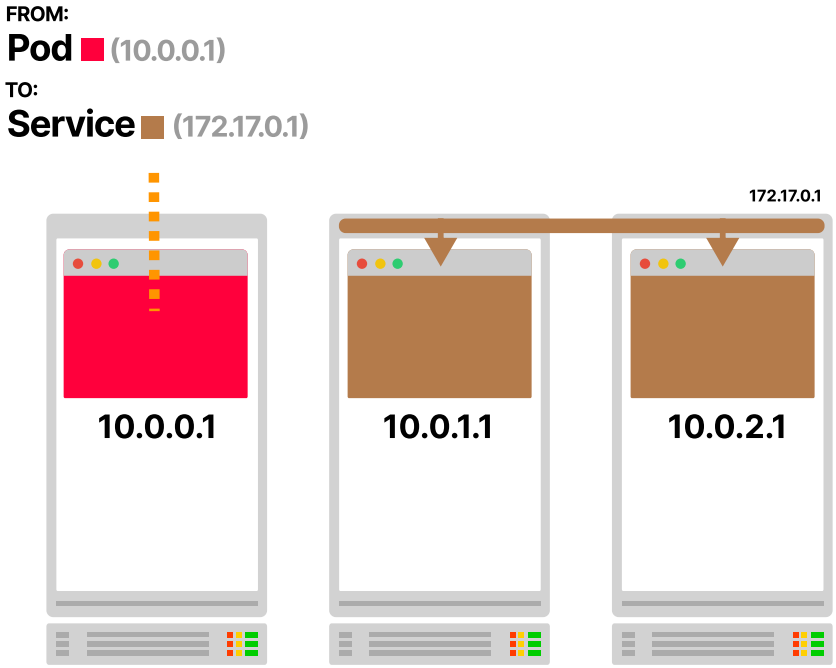
ClusterIP



Headless



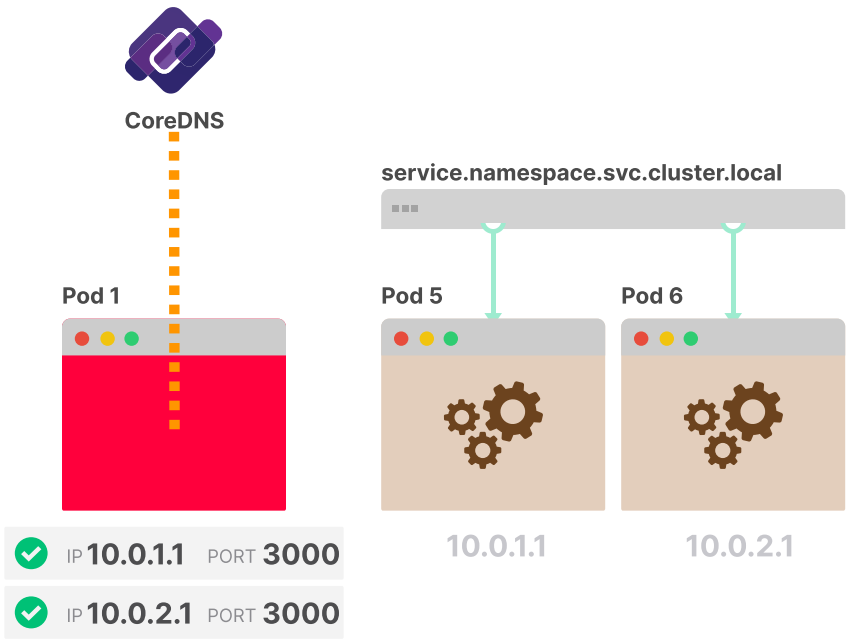
ClusterIP



Client-side

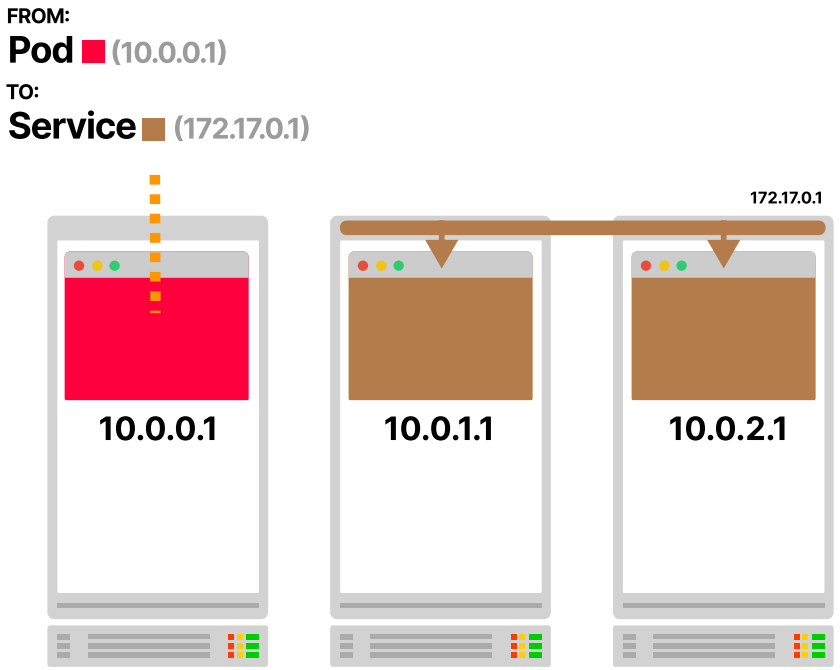
Load balancer-like

Headless



Round-robin

ClusterIP



Probabilistic

Recap

Internal communications

Recap — Internal communications

1. Services don’t exist

2. Endpoint object and endpoints

3. Headless Service

4. CoreDNS

5. ClusterIP Service

Recap — Internal communications

1. Services don’t exist

2. **Endpoint object and endpoints**

3. Headless Service

4. CoreDNS

5. ClusterIP Service

Recap — Internal communications

1. Services don’t exist

2. Endpoint object and endpoints

3. Headless Service

4. CoreDNS

5. ClusterIP Service

Recap — Internal communications

1. Services don’t exist
2. Endpoint object and endpoints
3. Headless Service
4. **CoreDNS**
5. ClusterIP Service

Recap — Internal communications

1. Services don’t exist
2. Endpoint object and endpoints
3. Headless Service
4. CoreDNS
5. ClusterIP Service

Recap — Internal communications

6. kube-proxy

7. Headless vs Cluster IP: DNS

8. Headless vs Cluster IP: Load balancing

Recap — Internal communications

6. kube-proxy

7. Headless vs Cluster IP: DNS

8. Headless vs Cluster IP: Load balancing

Recap — Internal communications

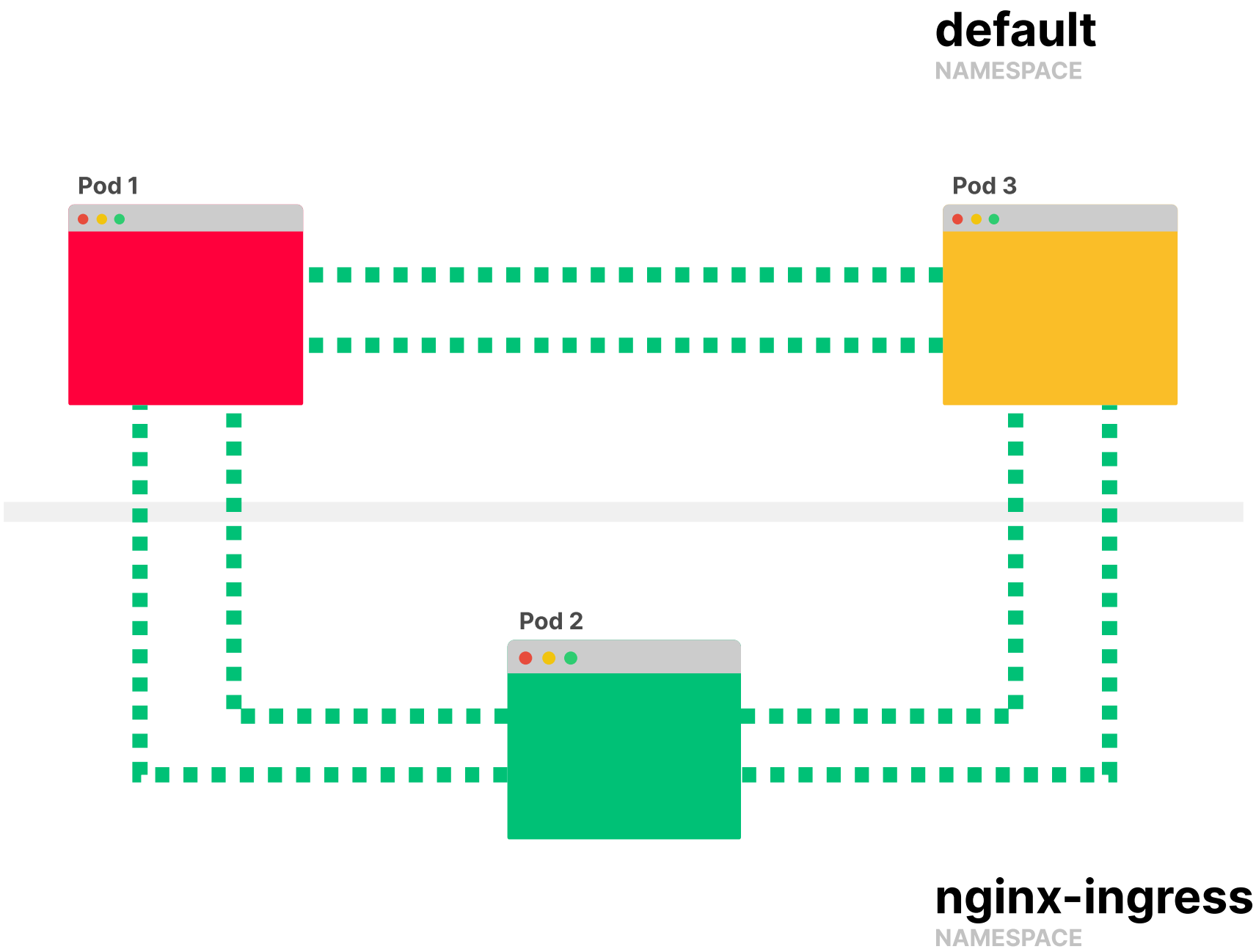
6. kube-proxy

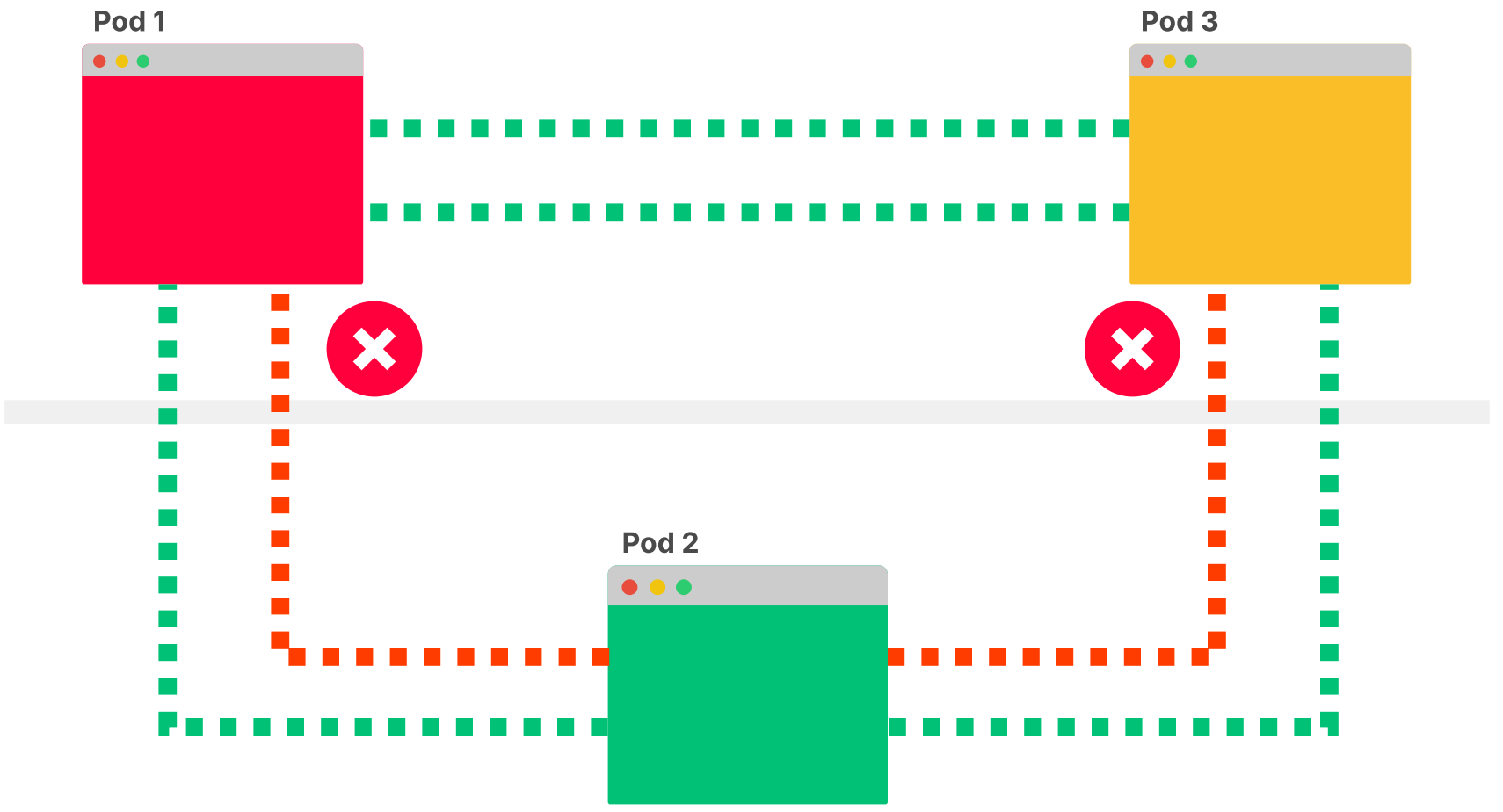
7. Headless vs Cluster IP: DNS

8. Headless vs Cluster IP: Load balancing

Network policies

03


default
NAMESPACE
Pod 1
Pod 3
Pod 2
nginx-ingress
NAMESPACE


default
NAMESPACE
Pod 1
Pod 3
Pod 2
nginx-ingress
NAMESPACE

Policy targets

Policie targets

Pods

Namespaces

IP addresses

Policie targets

Pods

Namespaces

IP addresses

Policie targets

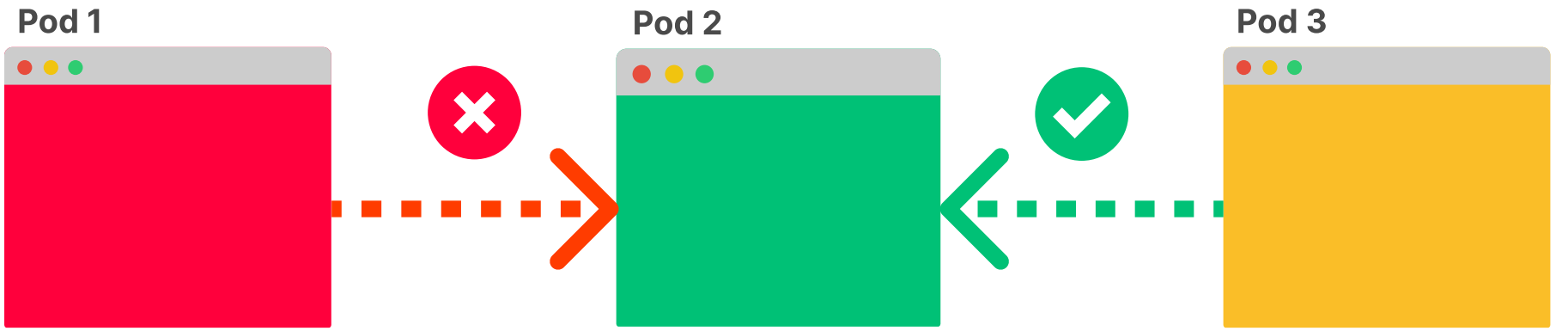
Pods

Namespaces

IP addresses

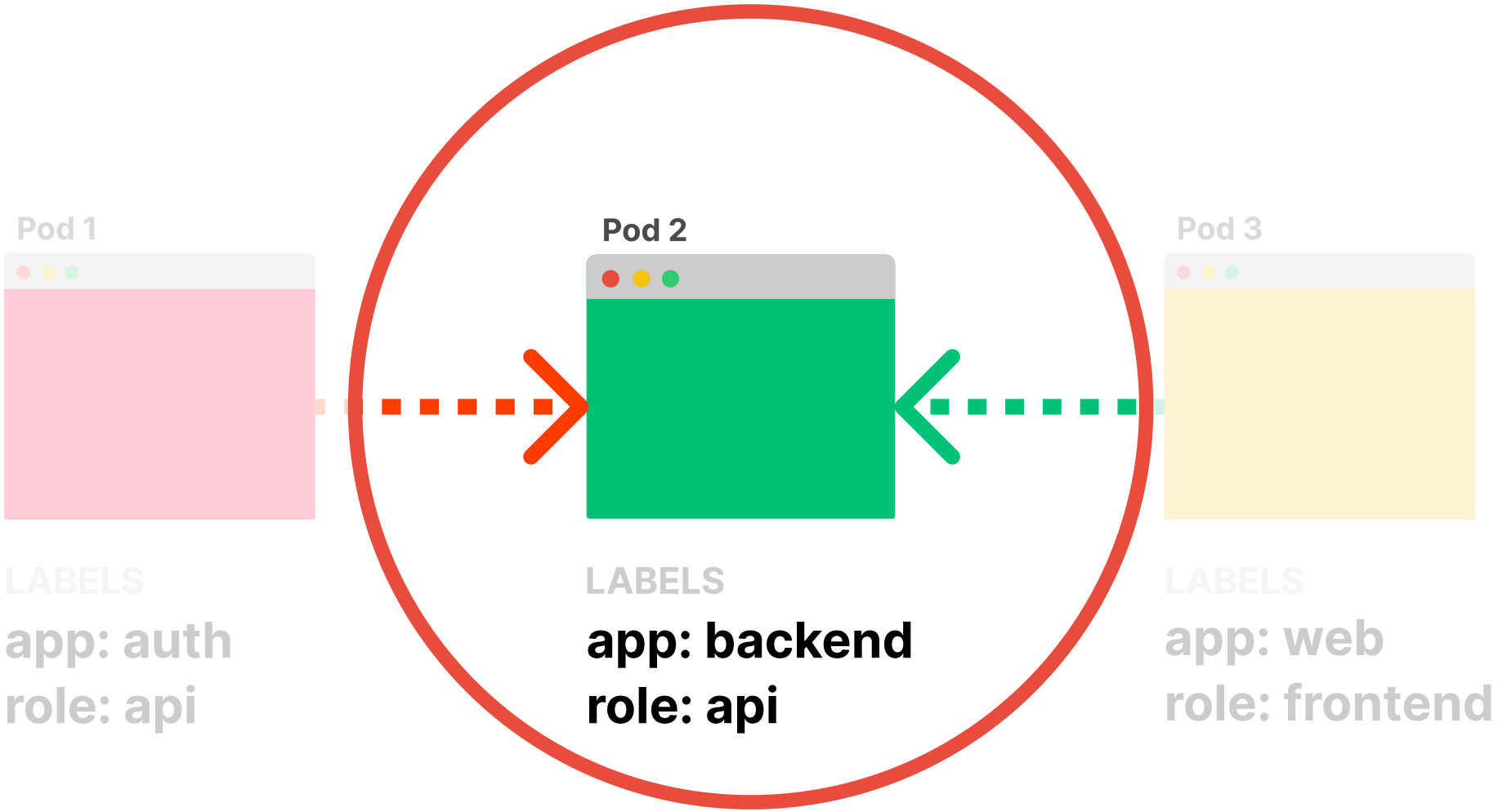
Policies for Pods

Network policies

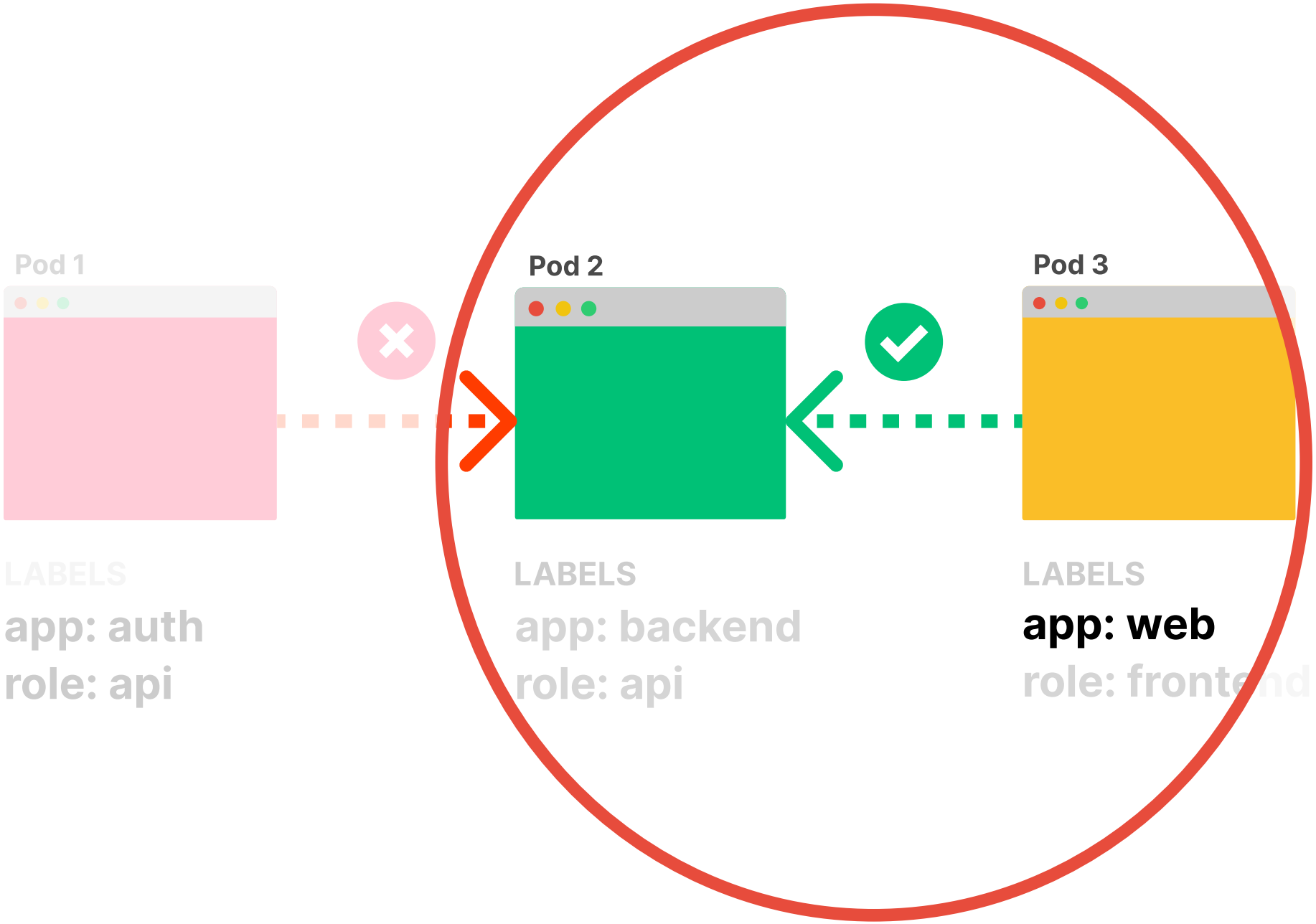

Pod 1
Pod 2
Pod 3
LABELS
app: auth
role: api
LABELS
app: backend
role: api
LABELS
app: web
role: frontend

```
~$ cat pod-policy.yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: api-allow
spec:
  podSelector:
    matchLabels:
      app: backend
      role: api
  ingress:
  - from:
      - podSelector:
          matchLabels:
            app: web
```

```
~$ cat pod-policy.yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: api-allow
spec:
  podSelector:
    matchLabels:
      app: backend
      role: api
  ingress:
  - from:
      - podSelector:
          matchLabels:
            app: web
```

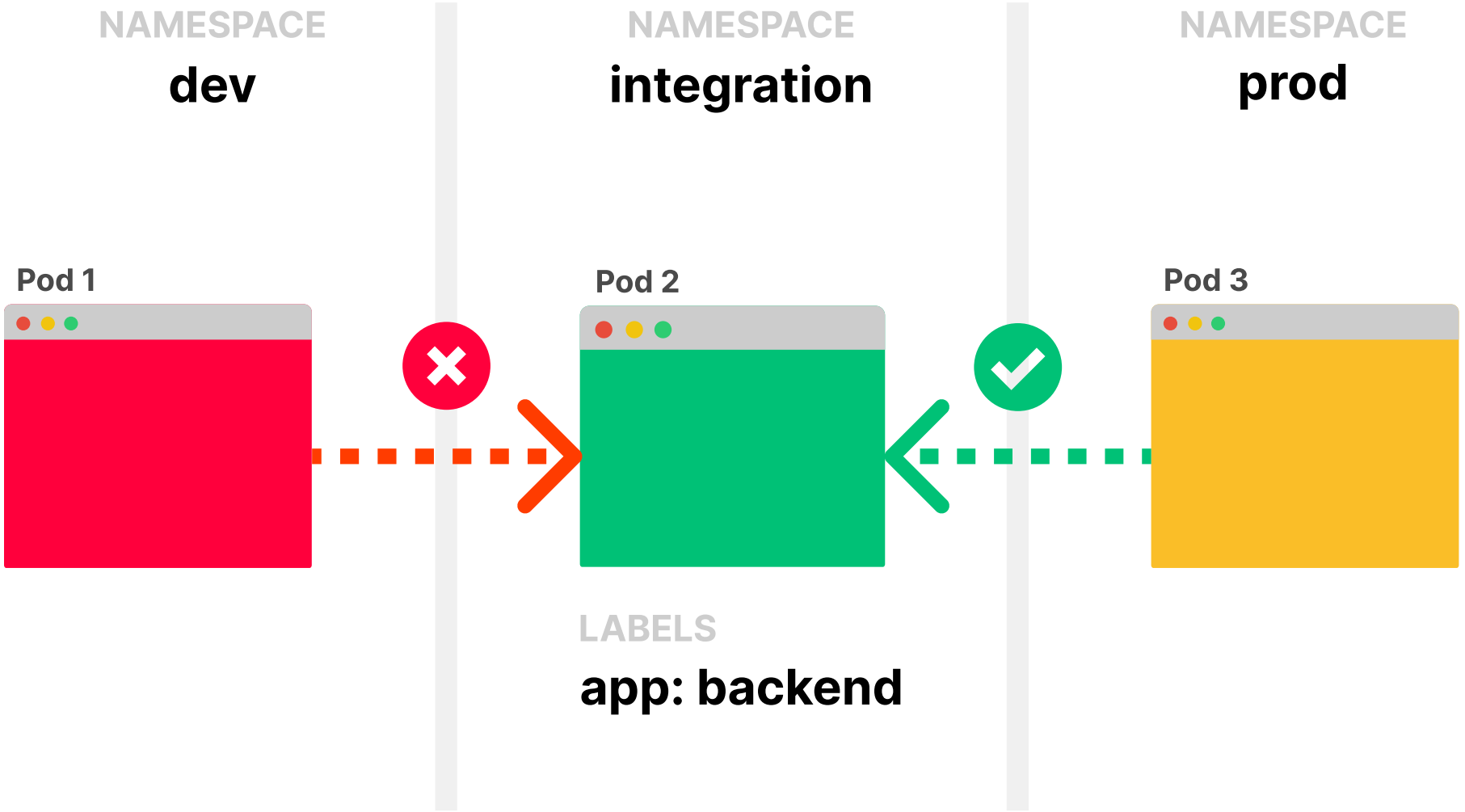

Pod 1
LABELS
app: auth
role: api
Pod 2
LABELS
app: backend
role: api
Pod 3
LABELS
app: web
role: frontend

```
~$ cat pod-policy.yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: api-allow
spec:
  podSelector:
    matchLabels:
      app: backend
      role: api
  ingress:
  - from:
      - podSelector:
          matchLabels:
            app: web
```

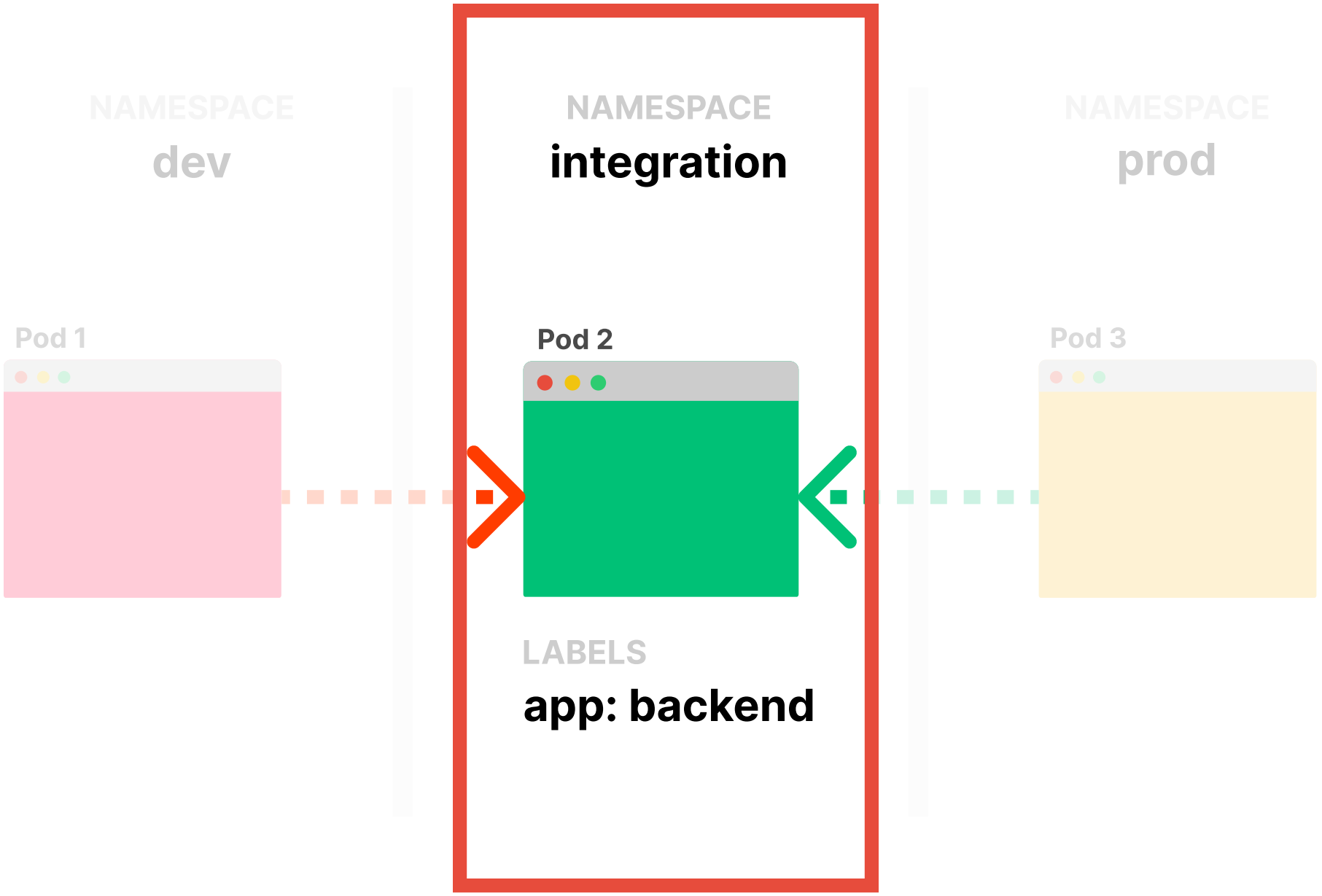

Pod 1
LABELS
app: auth
role: api
Pod 2
LABELS
app: backend
role: api
Pod 3
LABELS
app: web
role: frontend

Policies for Namespaces

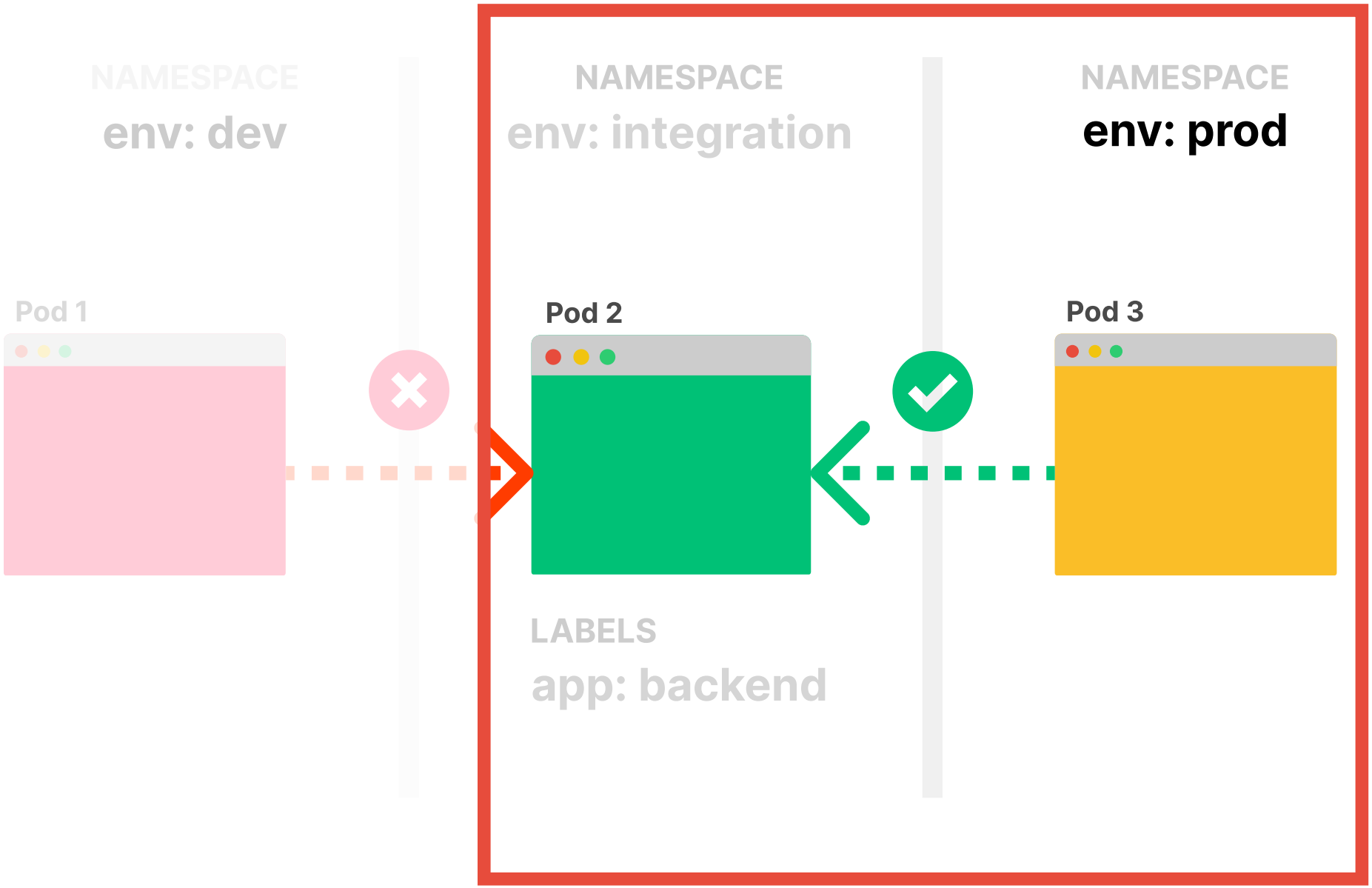
Network policies


NAMESPACE
dev
NAMESPACE
integration
NAMESPACE
prod
Pod 1
Pod 2
Pod 3
LABELS
app: backend

```
~$ cat pod-namespace.yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: backend-allow-prod
  namespace: integration
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          env: prod
```

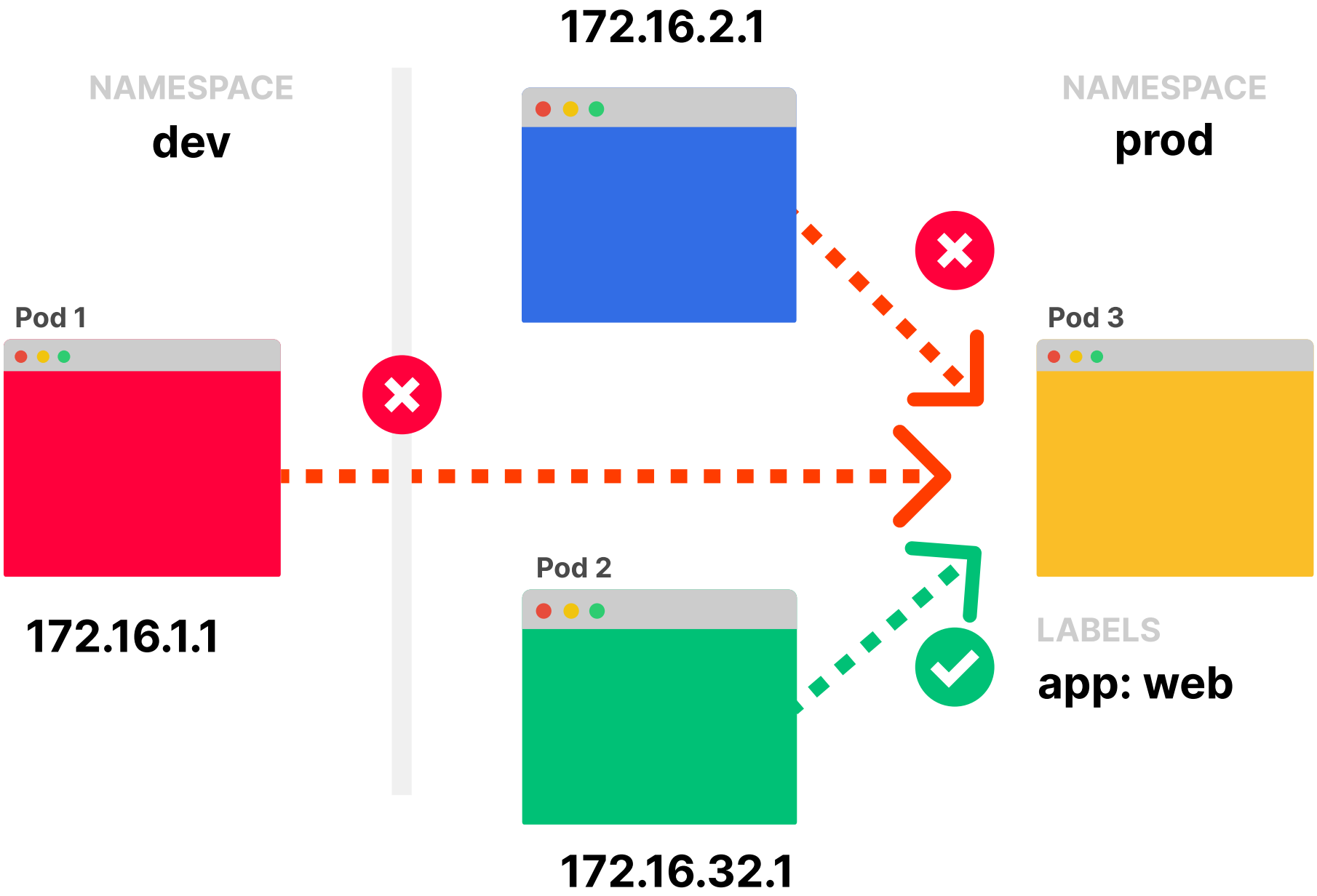

NAMESPACE
dev
NAMESPACE
integration
NAMESPACE
prod
Pod 1
Pod 2
Pod 3
LABELS
app: backend

```
~$ cat pod-namespace.yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: backend-allow-prod
  namespace: integration
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          env: prod
```

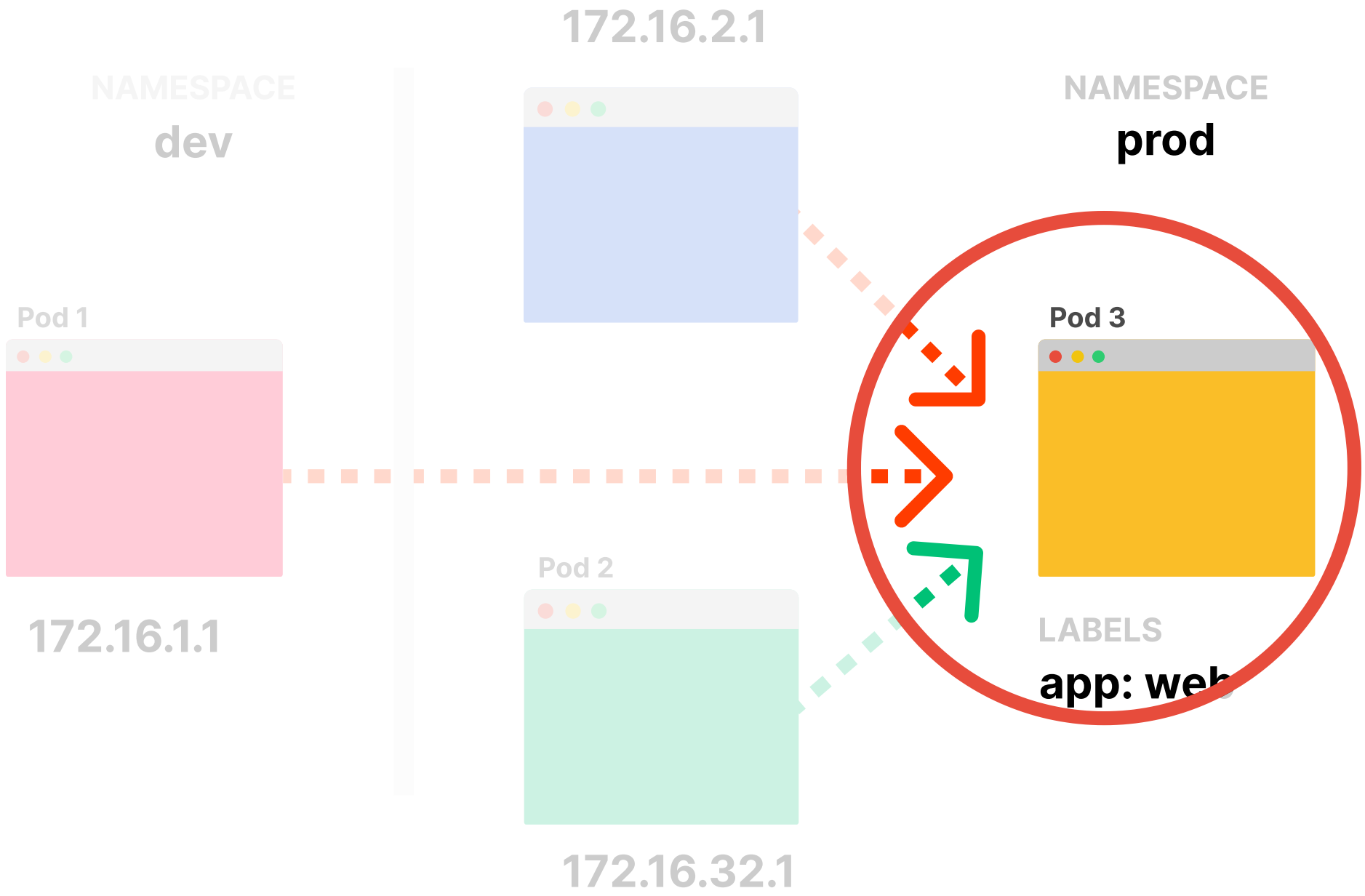

NAMESPACE
env: dev
NAMESPACE
env: integration
NAMESPACE
env: prod
Pod 1
Pod 2
Pod 3
LABELS
app: backend

Policies for CIDR

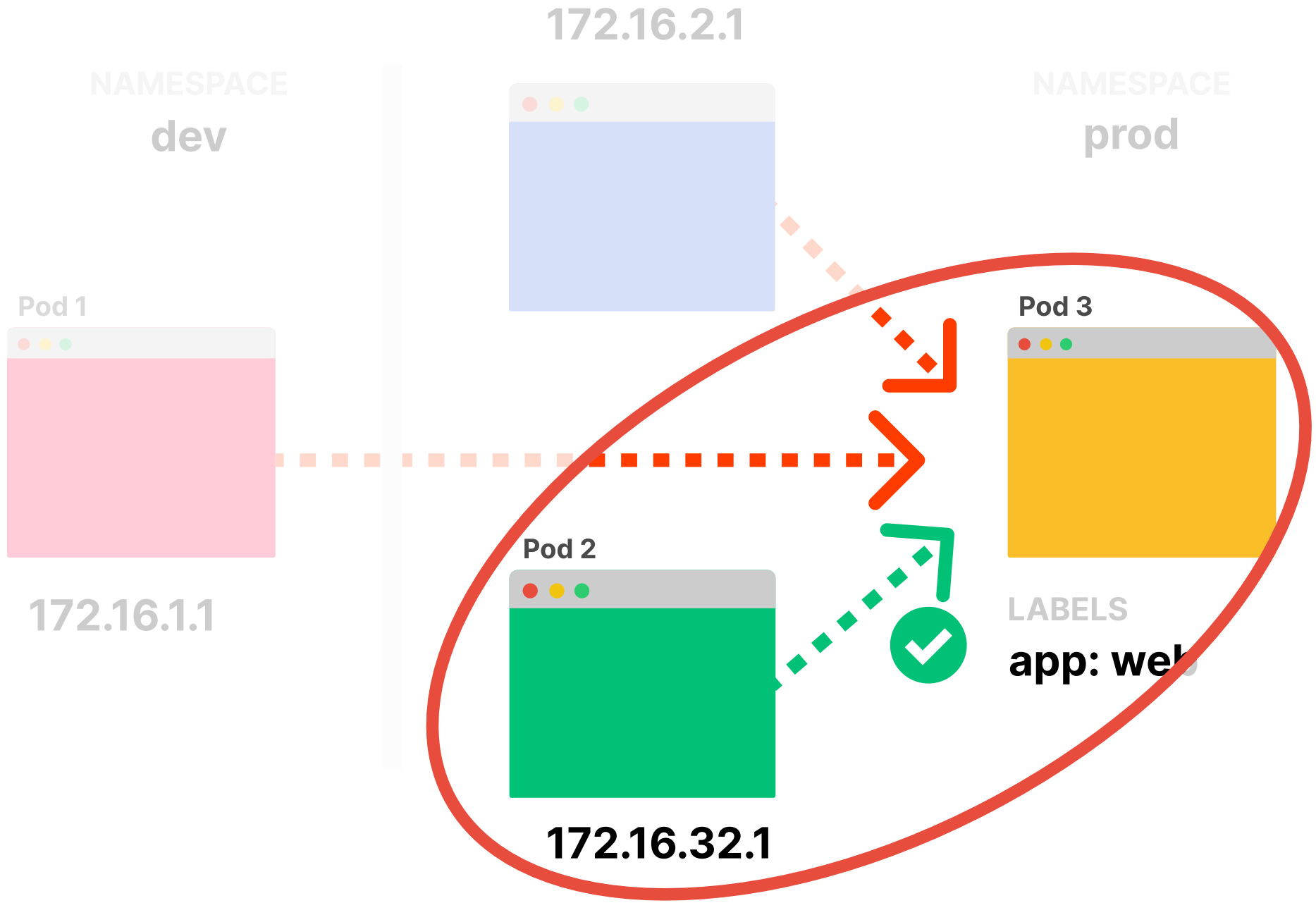
Network policies


172.16.2.1
NAMESPACE
dev
NAMESPACE
prod
Pod 1
Pod 3
Pod 2
172.16.1.1
LABELS
app: web
172.16.32.1

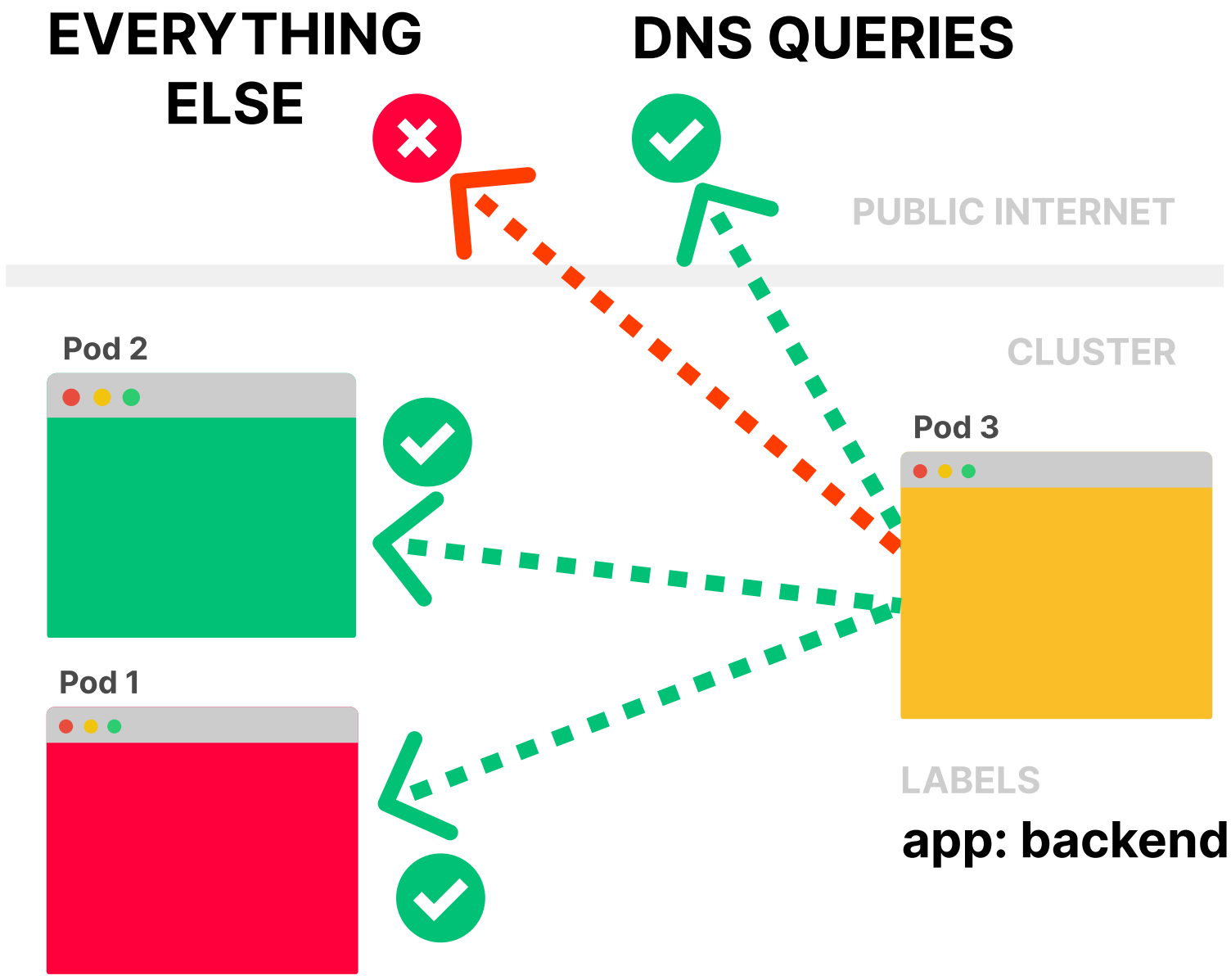
```
~$ cat pod-cidr.yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: customer-api-allow-web
  namespace: prod
spec:
  podSelector:
    matchLabels:
      app: web
  ingress:
  - ports:
    - port: 3306
    to:
    - ipBlock:
        cidr: 172.16.32.0/27
```


172.16.2.1
NAMESPACE
dev
NAMESPACE
prod
Pod 1
Pod 3
Pod 2
172.16.1.1
LABELS
app: web
172.16.32.1

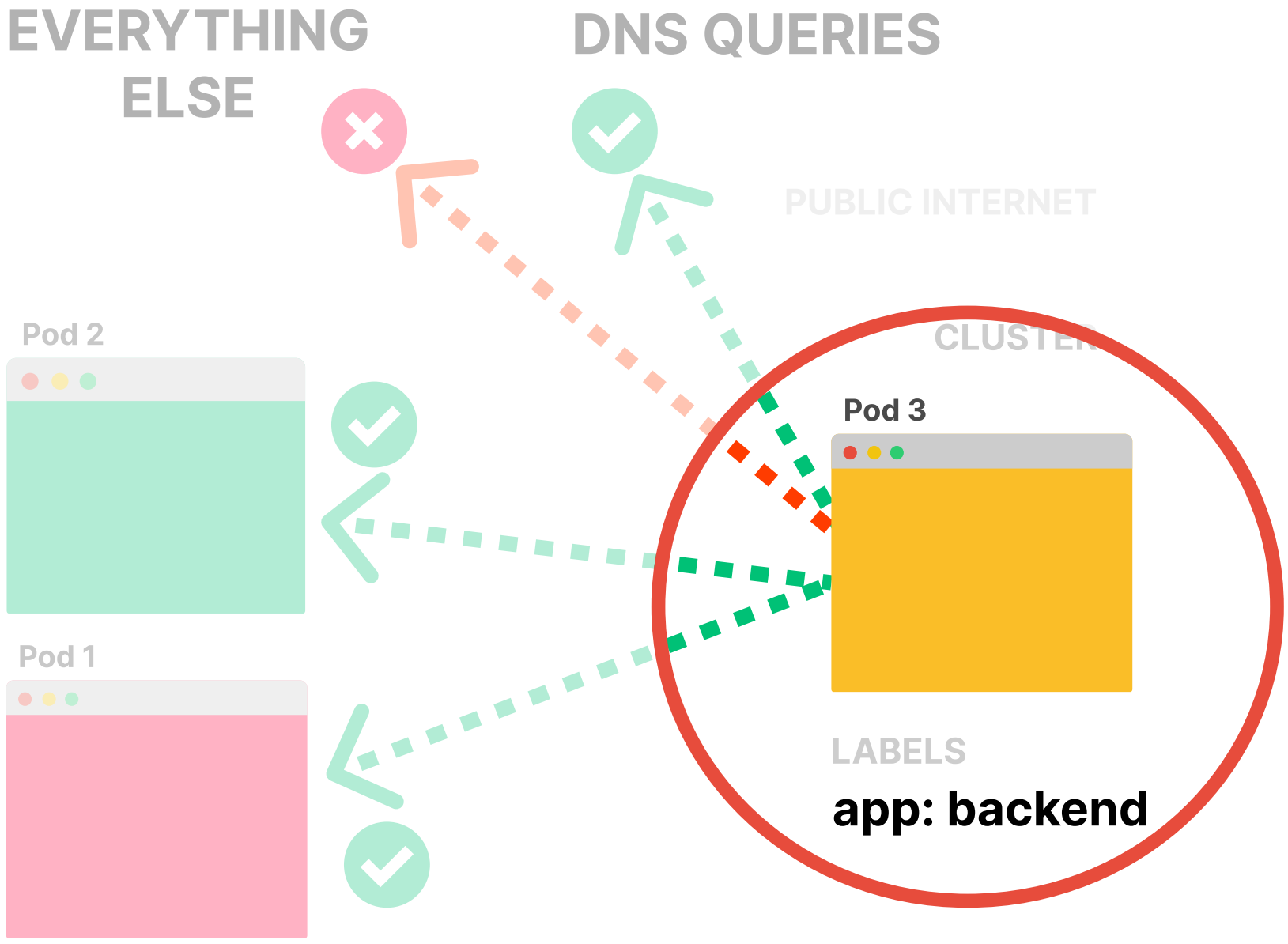
```
~$ cat pod-cidr.yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: customer-api-allow-web
  namespace: prod
spec:
  podSelector:
    matchLabels:
      app: web
  ingress:
  - ports:
    - port: 3306
    to:
    - ipBlock:
        cidr: 172.16.32.0/27
```


172.16.2.1
NAMESPACE
dev
NAMESPACE
prod
Pod 1
172.16.1.1
Pod 3
Pod 2
LABELS
app: web
172.16.32.1

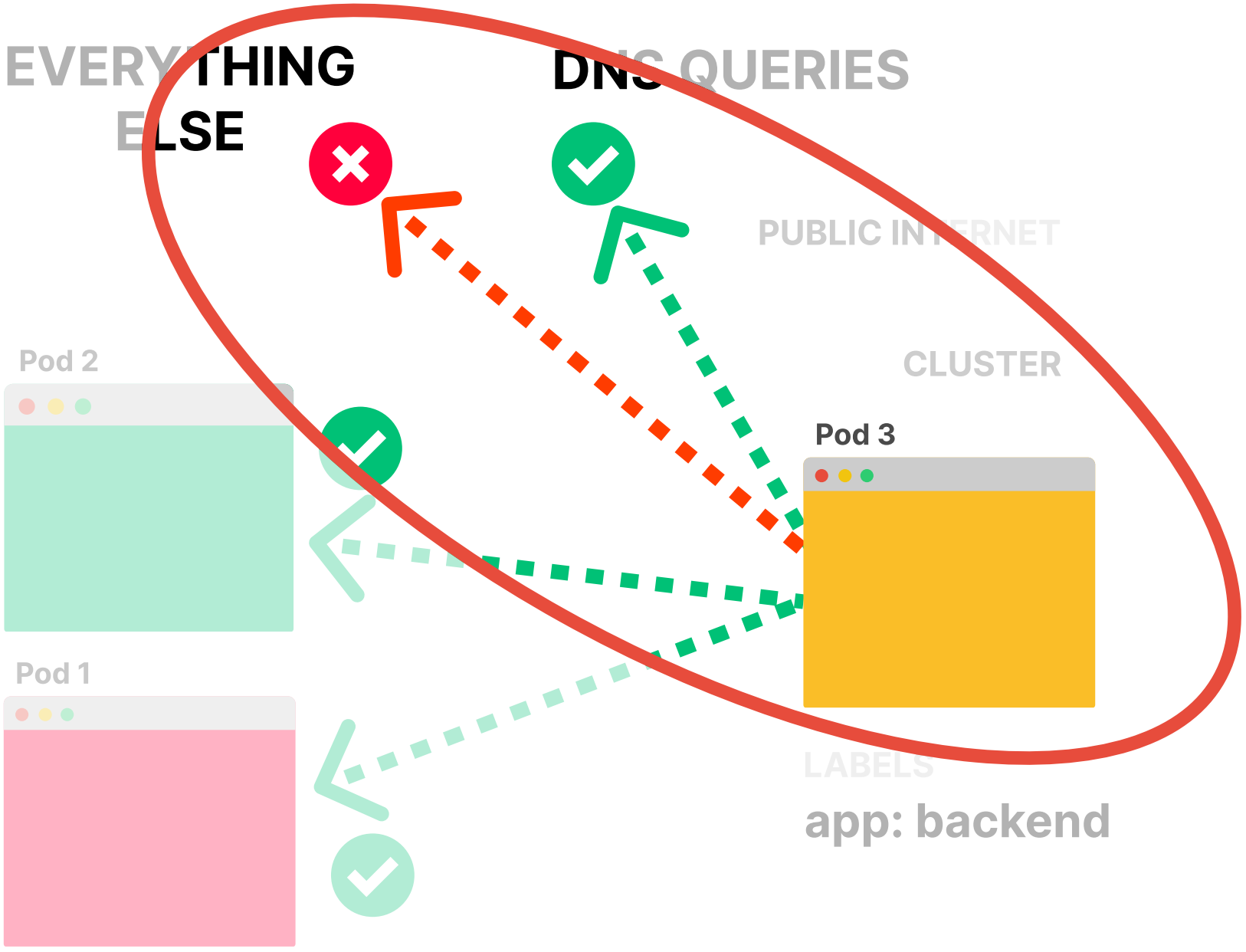
Egress


EVERYTHING ELSE
DNS QUERIES
PUBLIC INTERNET
CLUSTER
Pod 2
Pod 3
Pod 1
LABELS
app: backend

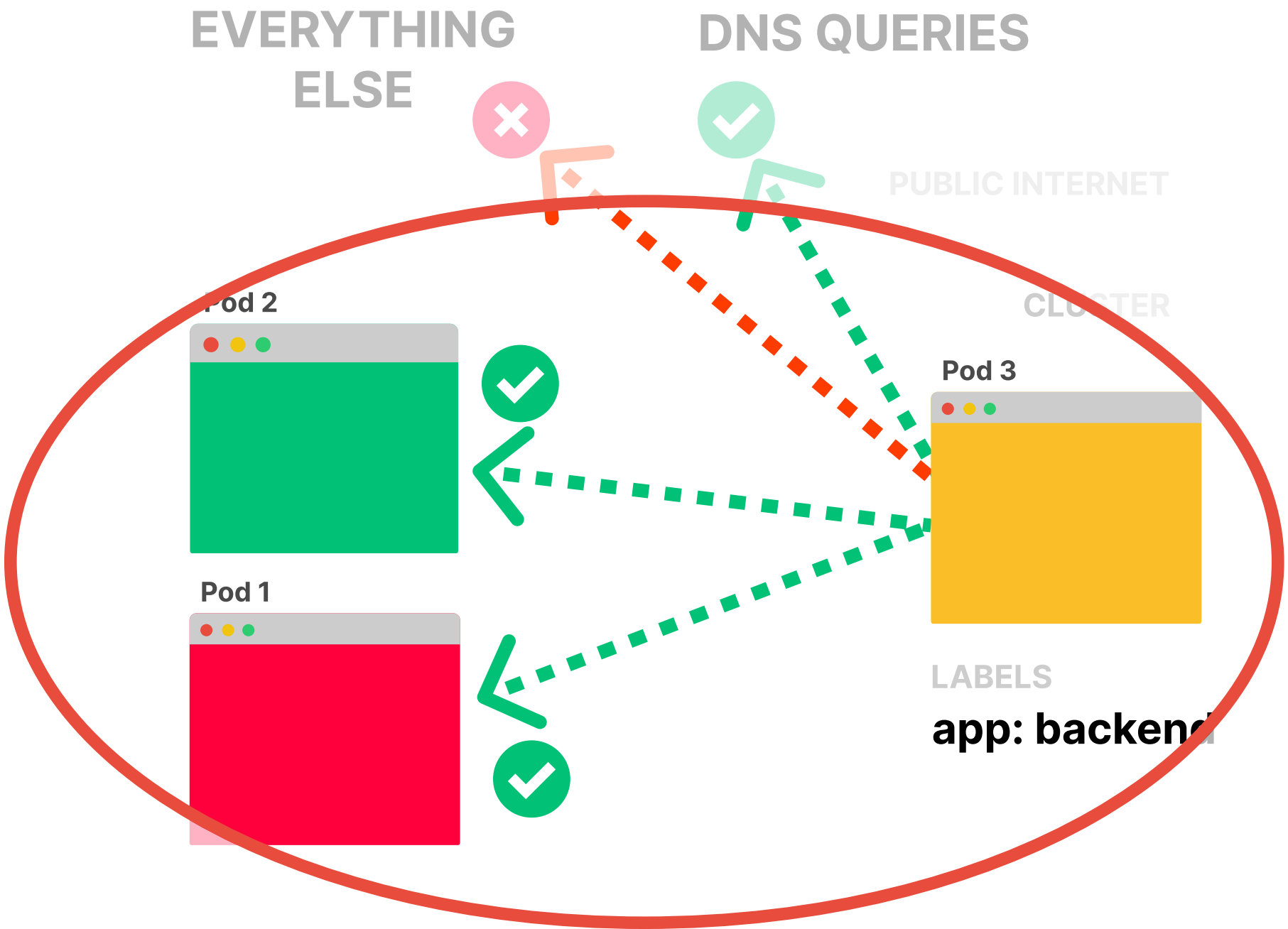
```
~$ cat pod-egress.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: foo-deny-external-egress
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Egress
  egress:
  - ports:
    - port: 53
      protocol: UDP
    - port: 53
      protocol: TCP
  - to:
    - namespaceSelector: {}
```


EVERYTHING ELSE
DNS QUERIES
PUBLIC INTERNET
Pod 2
Pod 1
CLUSTER
Pod 3
LABELS
app: backend

```
~$ cat pod-egress.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: foo-deny-external-egress
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Egress
  egress:
  - ports:
    - port: 53
      protocol: UDP
    - port: 53
      protocol: TCP
  - to:
    - namespaceSelector: {}
```


EVERYTHING ELSE
DNS QUERIES
PUBLIC INTERNET
CLUSTER
Pod 2
Pod 3
Pod 1
LABELS
app: backend

```
~$ cat pod-egress.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: foo-deny-external-egress
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Egress
  egress:
  - ports:
    - port: 53
      protocol: UDP
    - port: 53
      protocol: TCP
  - to:
    - namespaceSelector: {}
```


EVERYTHING ELSE
DNS QUERIES
PUBLIC INTERNET
Pod 2
Pod 3
Pod 1
LABELS
app: backend

Impletementation details

Network plugin

Layer 3 or 4

Policies are additive

Implementation details

Network plugin

Layer 3 or 4

Policies are additive

Implementation details

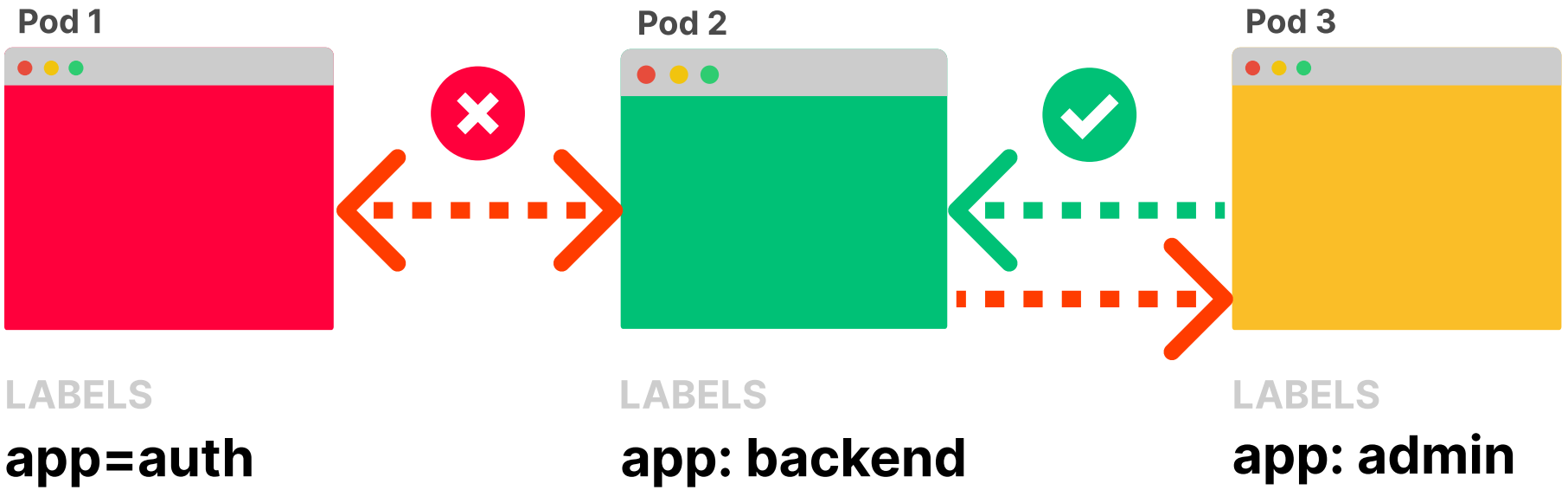
Network plugin
Layer 3 or 4

Policies are additive

```
~$ cat pod-deny-all.yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: default-deny-all
  namespace: default
spec:
  podSelector: {}
  ingress: []
```


Pod 1
Pod 2
Pod 3
LABELS
app=auth
LABELS
app: backend
LABELS
app: admin

```
~$ cat pod-policy.yaml
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: api-allow
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
  - from:
      - podSelector:
          matchLabels:
            app: admin
```


Pod 1
Pod 2
Pod 3
LABELS
app=auth
LABELS
app: backend
LABELS
app: admin

Network policies editor





https://editor.cilium.io/

Recap

Network policies

Recap — Network policies

1. Programmable firewall rules
2. Pods, namespaces, CIDR
3. Ingress and Egress
4. CNI specific

Recap — Network policies

1. Programmable firewall rules

2. Pods, namespaces, CIDR

3. Ingress and Egress

4. CNI specific

Recap — Network policies

1. Programmable firewall rules

2. Pods, namespaces, CIDR

3. Ingress and Egress

4. CNI specific

Recap — Network policies

1. Programmable firewall rules

2. Pods, namespaces, CIDR

3. Ingress and Egress

4. CNI specific

Thanks.